Java Handbook — Vol 1

the language — JVM, types, control flow, methods and arrays, OOP, core APIs, exceptions, generics, collections, lambdas, streams (Ch 1–12)

12 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — Java and the JVM**
   - 1.1 What Java is
   - 1.2 The Java ecosystem: SE, EE, OpenJDK and vendors
   - 1.3 JDK, JRE and JVM
   - 1.4 Installing a JDK and checking it works
   - 1.5 Write → compile → run
   - 1.6 Bytecode and class files
   - 1.7 How the JVM runs your code: class loading, interpreter, JIT
   - 1.8 Memory and garbage collection — an overview
   - 1.9 jshell — the Java REPL
   - 1.10 Launching source files directly
   - 1.11 IDEs, editors and build tools
   - 1.12 Java releases, LTS and JEPs
   - Chapter TL;DR
2. **Chapter 2 — Basics: Types, Variables and Operators**
   - 2.1 Statements, identifiers, keywords and comments
   - 2.2 Variables
   - 2.3 Primitive types
   - 2.4 Literals
   - 2.5 Reference types and `null`
   - 2.6 `var` — local variable type inference (Java 10)
   - 2.7 Type conversion: widening, narrowing, casting, promotion
   - 2.8 Operators
   - 2.9 Operator precedence and associativity
   - 2.10 Integer overflow
   - 2.11 Floating-point numbers
   - 2.12 Strings and text blocks
   - 2.13 Console output and input
   - Chapter TL;DR
3. **Chapter 3 — Control Flow**
   - 3.1 `if`, `else if`, `else`
   - 3.2 The classic `switch` statement
   - 3.3 Arrow labels and `switch` expressions (Java 14)
   - 3.4 Choosing between `if`, ternary and `switch`
   - 3.5 Pattern matching for `instanceof` (Java 16)
   - 3.6 Pattern matching for `switch` (Java 21)
   - 3.7 Record patterns (Java 21) and unnamed patterns (Java 22)
   - 3.8 Loops
   - 3.9 `break`, `continue` and labels
   - 3.10 Loop pitfalls
   - Chapter TL;DR
4. **Chapter 4 — Methods and Arrays**
   - 4.1 Methods — declaring and calling
   - 4.2 Parameters — Java is always pass-by-value
   - 4.3 Overloading
   - 4.4 Varargs
   - 4.5 Recursion
   - 4.6 Arrays
   - 4.7 Multidimensional and jagged arrays
   - 4.8 The `java.util.Arrays` toolbox
   - 4.9 `main`, command-line arguments, and simpler `main` (Java 25)
   - Chapter TL;DR
5. **Chapter 5 — OOP: Classes and Objects**
   - 5.1 Classes and objects
   - 5.2 Fields, instance methods and `this`
   - 5.3 Constructors
   - 5.4 Packages and imports
   - 5.5 Static members
   - 5.6 Access modifiers
   - 5.7 Encapsulation
   - 5.8 The `final` keyword
   - 5.9 Immutability basics
   - 5.10 Initialisation order
   - Chapter TL;DR
6. **Chapter 6 — OOP: Inheritance, Interfaces and More**
   - 6.1 Inheritance and `super`
   - 6.2 What a subclass gets — members, hiding and `Object`
   - 6.3 Overriding
   - 6.4 Polymorphism and casting
   - 6.5 Abstract classes
   - 6.6 Interfaces
   - 6.7 Sealed classes and interfaces (Java 17)
   - 6.8 Records
   - 6.9 Enums
   - 6.10 Nested classes: static nested and inner
   - 6.11 Local classes, anonymous classes and lambdas
   - 6.12 Choosing: abstract class, interface, record, enum or plain class
   - 6.13 Object methods: equals, hashCode and toString
   - Chapter TL;DR
7. **Chapter 7 — Core APIs: Strings, Numbers, Dates and Randomness**
   - 7.1 Strings: immutability, the pool and memory
   - 7.2 The `String` API toolkit
   - 7.3 `StringBuilder`, `StringBuffer` and `StringJoiner`
   - 7.4 Formatting text and numbers
   - 7.5 `Math`
   - 7.6 Wrapper classes and autoboxing
   - 7.7 `BigInteger` and `BigDecimal`
   - 7.8 The `java.time` model

- 7.9 Local dates, times, adjusters and `Period`
- 7.10 Zones, instants, `Duration` and daylight saving
- 7.11 Formatting and parsing dates
- 7.12 Random numbers
- Chapter TL;DR

8. **Chapter 8 — Exceptions**
   - 8.1 What an exception is
   - 8.2 The `Throwable` hierarchy: checked, unchecked and errors
   - 8.3 Handle or declare: checked exceptions and `throws`
   - 8.4 `try` / `catch` / `finally` and control flow
   - 8.5 Multi-catch, catch order and rethrowing
   - 8.6 try-with-resources
   - 8.7 Throwing exceptions and writing custom ones
   - 8.8 Chaining and exception translation
   - 8.9 Reading stack traces
   - 8.10 Best practices and anti-patterns
   - Chapter TL;DR
9. **Chapter 9 — Generics**
   - 9.1 Why generics
   - 9.2 Generic classes and records
   - 9.3 Generic interfaces
   - 9.4 Generic methods and type inference
   - 9.5 Bounded type parameters
   - 9.6 Invariance, wildcards and PECS
   - 9.7 Type erasure: how generics really work
   - 9.8 Restrictions and workarounds (consequences of erasure)
   - 9.9 Designing generic APIs
   - Chapter TL;DR
10. **Chapter 10 — Collections**
    - 10.1 The Collections Framework and its hierarchy
    - 10.2 Lists: `ArrayList` and `LinkedList`
    - 10.3 Sets: `HashSet`, `LinkedHashSet`, `TreeSet`
    - 10.4 Maps: `HashMap`, `LinkedHashMap`, `TreeMap`
    - 10.5 `HashMap` internals
    - 10.6 Queues, deques and priority queues
    - 10.7 Iterators and fail-fast behaviour
    - 10.8 Ordering: `Comparable` and `Comparator`
    - 10.9 The `Collections` utility class
    - 10.10 Unmodifiable and immutable collections
    - 10.11 Sequenced collections (Java 21)
    - 10.12 Big-O cheat sheet and a measured demo
    - 10.13 Choosing a collection
    - Chapter TL;DR
11. **Chapter 11 — Lambdas and Functional Programming**
    - 11.1 Why lambdas: passing behaviour
    - 11.2 Functional interfaces and lambda syntax
    - 11.3 The built-in functional interfaces (`java.util.function`)
    - 11.4 Method references
    - 11.5 Variable capture and "effectively final"
    - 11.6 Composing functions
    - 11.7 `Optional`
    - 11.8 Functional style in everyday Java
    - Chapter TL;DR
12. **Chapter 12 — Streams**
    - 12.1 What a stream is
    - 12.2 Laziness: how a pipeline really executes
    - 12.3 Intermediate operations
    - 12.4 Terminal operations and `reduce`
    - 12.5 Collectors
    - 12.6 Primitive streams
    - 12.7 Streams, `Optional` and nulls
    - 12.8 Parallel streams
    - 12.9 Stream gatherers (Java 24)
    - 12.10 Common mistakes
    - 12.11 Streams vs loops, and a real-world pipeline
    - Chapter TL;DR

Chapter 1 — Java and the JVM

What happens between typing `javac` (the Java compiler) and seeing output: setting up a JDK (Java Development Kit — compiler plus developer tools), three ways to **compile** (translate source code into a form a machine can execute) and run programs, what is inside a `.class` file, the Just-In-Time compiler and the garbage collector at work, `jshell`, and how Java versions and long-term-support releases work.

All code lives in `code/`, compiled with `javac --release 25` (the book targets **Java 25 LTS**) and run on JDK 26; machine-dependent output (timings, memory sizes) is flagged. Exam-level traps (class structure, `main` signatures, packages, GC eligibility): the OCP repo's Chapter 1 — Building Blocks.

1.1 What Java is

What it is

Java is a **general-purpose, statically typed, object-oriented programming language** plus a **runtime platform** (the Java Virtual Machine) that runs compiled Java programs. A **runtime** is the software that must be present while a program runs, executing it and providing services such as memory management.

- *General-purpose*: not tied to one domain — banking back-ends, Android apps (historically), big-data engines, command-line tools.
- *Statically typed*: every variable has a type known at compile time, so the compiler rejects `int n = "five";` before the program runs.
- *Object-oriented*: code is organised into **classes** (blueprints) and **objects** (instances built from them). Modern Java also supports functional style — lambdas (short unnamed functions passed as values) and streams (pipelines that filter and transform data), Chapters 11–12.

Compiled Java runs on any machine with a JVM (Windows, macOS, Linux, containers — isolated packages such as Docker images that bundle an app with everything it needs): *"write once, run anywhere"*.

A short history

Year	Event
1991–96	James Gosling's team at Sun starts the "Green Project" (language: **Oak**); renamed **Java** and announced May 1995 (browser "applets" make it famous); JDK 1.0 January 1996.
2004	J2SE 5.0: generics, enhanced `for`, autoboxing, enums, annotations, varargs — the biggest language update until Java 8.
2006–10	Sun open-sources the JDK as **OpenJDK** (GPL v2 + Classpath Exception); Oracle acquires Sun (2010).
2014	Java 8: lambdas, streams, `java.time`. Still widely deployed.
2017	Java 9: the module system (JPMS — groups packages into named, sealed-off modules) and `jshell`.
2018–21	Java 10 starts the **six-month cadence** (March/September); Java 17 LTS (2021), LTS now every **two years**.
2023	Java 21 LTS: virtual threads, pattern matching for `switch`, record patterns.
2025	**Java 25 LTS** (16 Sept 2025): compact source files and instance `main`, flexible constructor bodies, module imports, scoped values.
2026	Java 26 (March) and Java 27 (15 Sept 2026) — non-LTS.

Design traits

Trait	In practice	Caveat
Platform independence	Bytecode (the JVM's portable instruction format) runs on any OS/CPU with a JVM.	Target still needs a JVM or a native-image build (pre-compiled executable).
Static typing	Compiler catches type errors; IDEs refactor safely.	More declarations than Python/JS (softened by `var`, records).
Automatic memory management	A garbage collector frees unused objects; no `free()` / `delete`.	GC costs CPU/memory; you can still leak by keeping references (handles to objects).
Backward compatibility	Java 8 bytecode usually runs on Java 25.	Old APIs linger (`Date`, `Vector`).
Rich standard library	Collections, concurrency, networking, HTTP client, `java.time`, crypto.	Large; finding the modern API takes time.
Performance	The JIT turns hot (frequently run) code into optimised machine code.	Slower start-up/warm-up (mitigated by AOT — ahead-of-time — caches, native images).
Safety	No pointer arithmetic, bounds-checked arrays, verified bytecode.	Small cost.

Where Java is used

Enterprise back-ends (banks, airlines, retailers on Spring Boot / Jakarta EE — **frameworks**, large reusable libraries that structure an app); **big data** (Kafka, Cassandra, Elasticsearch, Hadoop, much of Flink); **developer tools** (IntelliJ IDEA, Eclipse, Maven, Gradle, Jenkins); **Android** (historically the Java *language* and much of its library on its own runtime, ART, not the JVM; Kotlin-first since 2019); Minecraft: Java Edition; Java Card on SIM and bank cards. **Kotlin, Scala, Groovy, Clojure** also compile to JVM bytecode and interoperate with Java libraries.

> **TL;DR**
>
> Java = statically typed, object-oriented language + portable runtime (the JVM). Mature (1996), backward compatible, fast after warm-up, dominant in server-side and data infrastructure.

1.2 The Java ecosystem: SE, EE, OpenJDK and vendors

What it is

Name	What it is
Java SE (Standard Edition)	The *specification* of the language, JVM and core library (`java.lang`, `java.util`, …). Each version is defined through the Java Community Process (the formal process for agreeing changes).
JDK (Java Development Kit)	A concrete *implementation* of Java SE plus tools (`javac`, `jshell`, `jar`, `javadoc`, `jlink` …).
OpenJDK	The open-source project (openjdk.org) where the reference JDK is developed. Almost every downloadable JDK is built from it.
Vendor builds	Oracle JDK, Eclipse Temurin (Adoptium), Amazon Corretto, Azul Zulu, Microsoft Build of OpenJDK, Red Hat, BellSoft Liberica, SAP Machine. Same code base; different support, licensing, extras.
Jakarta EE (formerly Java EE)	Enterprise API specs on top of Java SE (servlets — handle web requests; JPA — persist objects to databases; CDI — wire objects together). Moved from Oracle to the Eclipse Foundation (2017) and renamed.
Java ME / Java Card	Small-device and smart-card editions.
GraalVM	Oracle JDK distribution with the Graal compiler and **Native Image**, which compiles Java ahead of time into a standalone native executable (no JVM needed).

Which vendor should I install?

For learning, any OpenJDK 25 build; **Eclipse Temurin** is a neutral choice. Companies choose by support contract, cloud (Corretto on AWS, Microsoft's build on Azure) or licensing. Oracle JDK 17+ uses the *Oracle No-Fee Terms and Conditions*: free **production** use for a limited period (JDK 25 updates free until September 2028), then later updates need a paid licence. Other vendors' OpenJDK builds are GPL.

Common mistakes

- Thinking OpenJDK is lower quality than Oracle JDK — same source.
- Confusing Java with **JavaScript**; they share only a name and C-like syntax (JavaScript Handbook).

1.3 JDK, JRE and JVM

What it is

Three nested layers:



Layer	Contains	Who needs it
JVM (Java Virtual Machine)	Abstract machine that loads, verifies and executes bytecode (interpreting and JIT-compiling) and manages memory. **HotSpot** is the JVM in OpenJDK builds.	Every Java program at run time
JRE (Java Runtime Environment)	JVM + standard class library — enough to *run*, not compile.	End users (historically)
JDK	JRE + development tools (`javac` , `jshell` , `jar` , `javadoc` , debuggers, profilers).	Developers — **install this**

Virtual machine here means a program that acts as a CPU with its own instruction set (bytecode) — not a VirtualBox-style whole-computer emulator.

Why the JRE "disappeared"

Since JDK 11, Oracle ships no separate JRE. Java 9's module system introduced `jlink`, which builds a **custom runtime** containing only the modules (named groups of packages, such as `java.base`) your app needs. Some vendors (e.g. Temurin) still publish JRE-only builds, mainly for containers.

A runtime with only `java.base` is enough for `HelloWorld`:

```
jlink --add-modules java.base --strip-debug --no-header-files --no-man-pages --output rt
du -sh rt                      # 44M   (the full JDK folder here is 377M)
rt/bin/java -cp out ch01.hello.HelloWorld
```

Output:

```
Hello, World!
Arguments received: 0
```

In the interview

Q: Difference between JDK, JRE and JVM? The JVM executes bytecode and manages memory; the JRE is JVM + standard library (enough to run); the JDK is JRE + tools like `javac`. Developers install a JDK; production images often use a `jlink`-trimmed runtime.

- *Is the JVM platform-independent?* No — the **bytecode** is portable; each OS/CPU needs its own JVM build.
- *What is HotSpot?* OpenJDK's JVM, named for finding "hot" code and compiling it.
- *Is a JRE enough to compile?* No — you need the JDK (`javac`).

1.4 Installing a JDK and checking it works

How it works

1. **Download** an OpenJDK 25 build (e.g. Temurin from adoptium.net), or use **SDKMAN!** (`sdk list java` , `sdk install java <identifier>`) or an OS package manager (`apt` , Homebrew).
2. **Set** `JAVA_HOME` to the JDK folder and put `$JAVA_HOME/bin` on `PATH` (build tools and IDEs read `JAVA_HOME`). An **environment variable** is a named setting the OS passes to every program; `PATH` lists folders searched for commands. macOS: `export JAVA_HOME=$(/usr/libexec/java_home -v 25)` ; Linux: `export JAVA_HOME=/usr/lib/jvm/<jdk-folder>` ; Windows: *System Properties → Environment Variables*, add `%JAVA_HOME%\bin` to `Path` .
3. **Verify** in a new terminal:

```
java -version
javac -version
```

Output (on the machine used for this book):

```
openjdk version "26" 2026-03-17
OpenJDK Runtime Environment (build 26+35-2893)
OpenJDK 64-Bit Server VM (build 26+35-2893, mixed mode, sharing)
javac 26
```

"mixed mode" = the JVM both interprets and JIT-compiles; *"sharing"* = it uses Class Data Sharing (a pre-parsed archive of core classes that speeds start-up).

Compiling for an older version: `--release`

`javac --release 25 …` checks code against the Java 25 language **and** API, so a newer JDK cannot let a Java 26 method slip in. That is how this book targets 25 on JDK 26. The older `-source` / `-target` pair does not check the API — prefer `--release`.

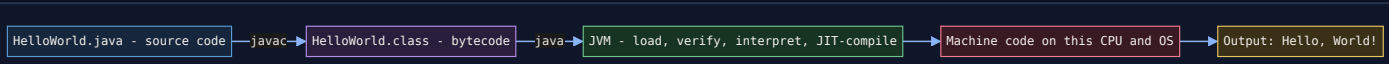
Common mistakes

- Several JDKs installed, with `java` and `javac` on `PATH` from different ones — check both versions.
- `JAVA_HOME` pointing at `bin` instead of the JDK root.
- Installing only a JRE, then `javac`: "command not found".

1.5 Write → compile → run

What it is

Java is **compiled in two stages**: `javac` turns source (`.java`) into platform-neutral **bytecode** (`.class` files); the `java` launcher starts a JVM that loads and runs it.



The program

`code/HelloWorld.java` :

```java
package ch01.hello;

/** The classic first program: one public class, one entry point. */
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
        System.out.println("Arguments received: " + args.length);
    }
}
```

Part	Meaning
`package ch01.hello;`	The **package** (namespace); must be the first statement. Fully qualified name: `ch01.hello.HelloWorld`. See Chapter 5.
`/** … */`	**Javadoc comment**, turned into HTML API docs by `javadoc`. `//` is a line comment, `/* … */` a block comment.
`public class HelloWorld`	At most one **public** top-level class per file, and the file must be named after it.
`public static void main(String[] args)`	The **entry point**. `public` = callable from anywhere; `static` = no object needed; `void` = returns nothing; `args` = command-line arguments (Chapter 4).
`System.out.println(...)`	Prints a line to **standard output** (the terminal by default). `System` is in `java.lang` (imported automatically — an *import* lets you use a class by its short name); `out` is its static `PrintStream` field (a class-level variable holding a text-writing object).
`;` and `{ }`	Statements end with a semicolon; braces group blocks. Java ignores indentation.

Since Java 25 a class may instead declare `void main()` (no `static`, no parameters) — see 1.10. `public static void main(String[] args)` remains the standard and works in every version.

How to compile and run

```
javac -d out HelloWorld.java            # compile; -d puts classes in out/ch01/hello/
java -cp out ch01.hello.HelloWorld       # run: class path + fully qualified class name
java -cp out ch01.hello.HelloWorld a b c
```

Output:

```
Hello, World!
Arguments received: 0
Hello, World!
Arguments received: 3
```

1. `javac` parses the file, checks types and writes `out/ch01/hello/HelloWorld.class` (folders mirror the package).
2. `java` starts a JVM; `-cp` (class path) says where to look for classes.
3. The JVM loads `ch01.hello.HelloWorld`, finds `main`, and calls it with the arguments.

Reading errors: compile-time vs run-time

`javac` reports compile errors with file, line and a caret:

```
Hello.java:3: error: ';' expected
        System.out.println("hi")
                                ^
```

```
Hello.java:1: error: class Greeting is public, should be declared in a file named Greeting.java
```

```
Hello.java:3: error: incompatible types: String cannot be converted to int
        int n = "five";
                ^
```

Launcher errors happen when `java` cannot find the class:

Command	Error	Fix
`java hello`	`NoClassDefFoundError: hello (wrong name: Hello)` on macOS/Windows (case-insensitive file systems); `ClassNotFoundException: hello` on Linux	Class names are case-sensitive.
`java Hello.class`	`ClassNotFoundException: Hello.class`	Pass the class **name**, not the file name.
`java HelloWorld` (class is in a package)	`Could not find or load main class HelloWorld`	Use the fully qualified name and the right `-cp`.

Packaging a JAR

A **JAR** (Java ARchive) is a ZIP of classes plus a manifest (metadata file); a `Main-Class` entry makes it runnable:

```
jar --create --file hello.jar --main-class ch01.hello.HelloWorld -C out .
java -jar hello.jar one two
```

Output:

```
Hello, World!
Arguments received: 2
```

Real projects let Maven or Gradle build JARs (Chapter 18).

Common mistakes

- File name ≠ public class name (case matters: `helloworld.java` fails for `HelloWorld`).
- Missing/wrong `-cp`, or running from inside the package folder.
- Misspelt `main` signature (e.g. `String args` without `[]`) in a traditional class — compiles, but the launcher reports no suitable main method.

TL;DR

`javac` → bytecode; `java -cp <dir> <package.Class>` → JVM runs it. `-d out` puts `ch01.hello.HelloWorld` at `out/ch01/hello/HelloWorld.class`. One public top-level class per file, named like the file.

1.6 Bytecode and class files

What it is

Bytecode is the JVM's instruction set: compact one-byte opcodes (operation codes such as `iload`, `iadd`, `invokevirtual`) for a **stack machine** — instructions push values onto an operand stack and pop them to compute. A `.class` file holds one class's bytecode plus a **constant pool** (names, strings, method references) and metadata.

Why it matters

- It is the unit of portability: the same `.class` runs on every JVM.
- The JVM verifies it before running (no forged pointers, no operand-stack overflow/underflow, no jumps into the middle of an instruction).
- Tools — profilers (measure time/memory), mocking libraries (fake objects for tests), Spring, Hibernate — read or generate bytecode at run time.

Looking inside with `javap`

code/Calculator.java :

```java
package ch01.bytecode;

public class Calculator {
    static int add(int a, int b) {
        return a + b;
    }

    public static void main(String[] args) {
        int total = add(2, 3);
        System.out.println("2 + 3 = " + total);
    }
}
```

`javap -c -cp out ch01.bytecode.Calculator` (the JDK's disassembler):

```
public class ch01.bytecode.Calculator {
  public ch01.bytecode.Calculator();
    Code:
         0: aload_0
         1: invokespecial #1                  // Method java/lang/Object."<init>":()V
         4: return

  static int add(int, int);
    Code:
         0: iload_0
         1: iload_1
         2: iadd
         3: ireturn

  public static void main(java.lang.String[]);
    Code:
         0: iconst_2
         1: iconst_3
         2: invokestatic  #7                  // Method add:(II)I
         5: istore_1
         6: getstatic     #13                 // Field java/lang/System.out:Ljava/io/PrintStream;
         9: iload_1
        10: invokedynamic #19,  0             // InvokeDynamic #0:makeConcatWithConstants:(I)Ljava/lang/String;
        15: invokevirtual #23                 // Method java/io/PrintStream.println:(Ljava/lang/String;)V
        18: return
}
```

1. `javac` added a **default constructor** `Calculator()` (a constructor sets up a new object) that calls `Object`'s constructor — you did not write it.
2. `add`: `iload_0` / `iload_1` push the two `int` parameters, `iadd` pops both and pushes the sum, `ireturn` returns it (`i` = int).
3. `main`: pushes 2 and 3, calls `add` (`(II)I` = two ints in, int out), stores the result in local slot 1.
4. String concatenation compiles to `invokedynamic … makeConcatWithConstants` (since Java 9), letting the JVM choose the fastest strategy at run time.

Class-file version

Every class file starts with the magic number `CAFEBABE` (fixed bytes identifying the file type), then a **major version**: major = 44 + Java version.

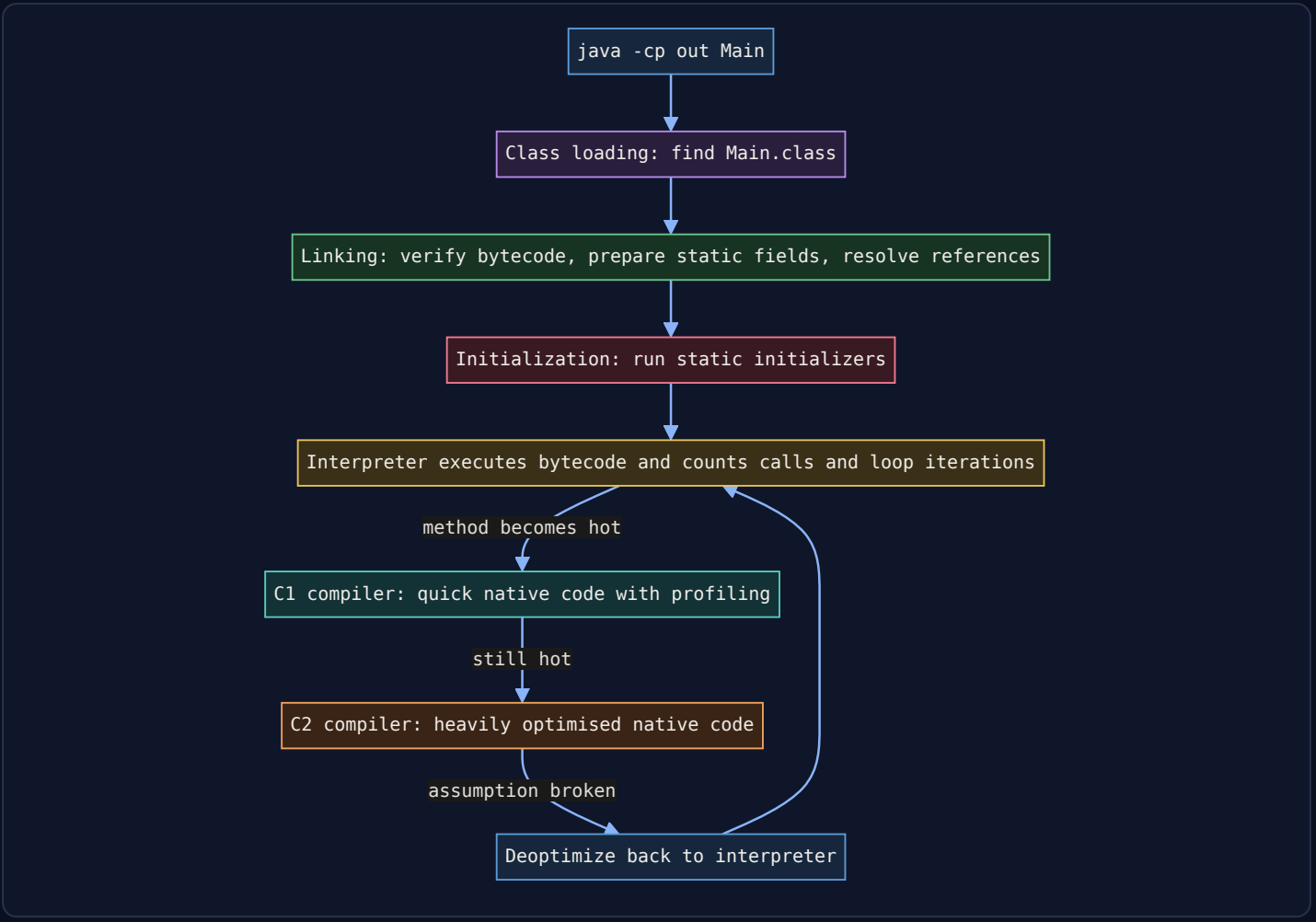
Java	8	11	17	21	25	26
Major version	52	55	61	65	**69**	70

```
javap -v -cp out ch01.bytecode.Calculator | grep major   # major version: 69
xxd out/ch01/bytecode/Calculator.class | head -1         # cafe babe 0000 0045 ...  (0x45 = 69)
```

An older JVM refuses newer class files with `UnsupportedClassVersionError` ("compiled by a more recent version of the Java Runtime (class file version 69.0), this version … only recognizes class file versions up to 65.0" — i.e. compiled by Java 25, run on 21). Fix: run on a newer JVM or compile with `--release <older>`.

1.7 How the JVM runs your code: class loading, interpreter, JIT

How it works



1. **Class loading.** *Class loaders* turn `.class` bytes into classes, lazily (the first time a class is actively used). Three built-in loaders: **bootstrap** (core `java.base`), **platform** (other JDK modules), **application/system** (your class path). Each asks its parent first (*parent delegation*), so you cannot replace `java.lang.String`.
2. **Linking.** *Verification* checks the bytecode is type-safe and well-formed; *preparation* allocates static fields with default values; *resolution* turns symbolic names (e.g. `java/io/PrintStream.println`) into direct references (lazily).
3. **Initialization.** Static field initializers and `static { }` blocks run, once per class.
4. **Execution.** The **interpreter** runs bytecode instruction by instruction — instant start, slow — counting how often each method and loop runs.
5. **JIT compilation.** *Just-In-Time* compilation converts **hot** methods to native code while the program runs. HotSpot uses **tiered compilation** (default since Java 8):

Tier	Who	What
0	Interpreter	Runs bytecode, gathers counts
1–3	**C1** ("client") compiler	Compiles fast, light optimisation; tier 3 adds profiling (which types and branches actually occur)
4	**C2** ("server") compiler	Uses the profile for aggressive optimisation

C2's optimisations: **inlining** (copy a small method's body into the caller), **loop unrolling** (repeat the body several times per iteration to cut loop overhead), **escape analysis** (objects that never leave a

method need not live on the heap), **dead-code removal** (delete work whose result is unused).

6. **Speculation and deoptimization.** C2 optimises for what it has *seen* (e.g. "this call always hits `ArrayList`"). If that breaks, the JVM discards the compiled code ("uncommon trap"), returns to the interpreter, and may recompile. This is why a JIT can beat a static compiler: it optimises for the types and branches actually seen.

Watching the JIT

`code/HotLoop.java` calls a tiny method five million times:

```
static long square(long x) { return x * x; }

public static void main(String[] args) {
    long sum = 0;
    for (long i = 0; i < 5_000_000; i++) {
        sum += square(i % 1000);
    }
    System.out.println("sum = " + sum);
}
```

`java -XX:+PrintCompilation -cp out ch01.jit.HotLoop | grep -E "HotLoop|sum ="` (real output; timestamps vary):

```
15    9       3       ch01.jit.HotLoop::square (4 bytes)
15   10       4       ch01.jit.HotLoop::square (4 bytes)
15    9       3       ch01.jit.HotLoop::square (4 bytes)   made not entrant: not used
16   11 %     3       ch01.jit.HotLoop::main @ 4 (43 bytes)
16   12       3       ch01.jit.HotLoop::main (43 bytes)
16   13 %     4       ch01.jit.HotLoop::main @ 4 (43 bytes)
16   11 %     3       ch01.jit.HotLoop::main @ 4 (43 bytes)   made not entrant: OSR invalidation of lower level
19   13 %     4       ch01.jit.HotLoop::main @ 4 (43 bytes)   made not entrant: uncommon trap
sum = 1664167500000
```

Columns: ms since start, compile ID, flags, **tier**, method. `square` was compiled by C1 (tier 3) then C2 (tier 4) in the same millisecond, and the C1 version discarded ("made not entrant"). `%` = **on-stack replacement (OSR)**: `main` runs once but its loop is hot, so the JVM compiles the loop and switches to it *while the method is running*. The last line is a deoptimization.

Start-up and warm-up: AOT caching

Starting in the interpreter makes a Java service slower for its first seconds (**warm-up**). Recent JDKs record an **ahead-of-time (AOT) cache** from a training run:

- JDK 24 (JEP 483 — a *JEP* is a numbered JDK Enhancement Proposal, 1.12): classes loaded and linked ahead of time into a cache.
- JDK 25 (JEP 514): one-step creation — `java -XX:AOTCacheOutput=app.aot …` on a training run, then `java -XX:AOTCache=app.aot …` in production.
- JDK 25 (JEP 515): the cache also stores method profiles, so the JIT optimises sooner.

GraalVM Native Image instead compiles the whole app to a native executable before it runs: very fast start-up and small memory, at the cost of long builds and restrictions on reflection (inspecting classes and calling methods by name at run time) and dynamic class loading.

Trade-offs: interpreted vs JIT vs AOT

Approach	Start-up	Peak speed	Portability	Example
Pure interpretation	Instant	Slow	High	Early JVMs, CPython
JIT (HotSpot)	Slower (warm-up)	Very high — uses live profiles	High (bytecode)	Normal `java`
AOT cache (JDK 24+)	Faster	Same as JIT	High	`-XX:AOTCache`
Native image	Very fast	Good, often a bit lower without profile data	One binary per OS/CPU	GraalVM `native-image`

In the interview

Q: Is Java compiled or interpreted? Both. `javac` compiles source to bytecode ahead of time; the JVM interprets it and JIT-compiles hot methods to native code using profiling data — so long-running services get faster after warm-up.

- *Tiered compilation?* Interpreter → C1 (fast compile, profiling) → C2 (optimising compile using the profile).
- *Deoptimization?* Discarding compiled code whose speculative assumption became false and returning to the interpreter.
- *Why is a simple timing-loop microbenchmark misleading?* It mixes interpreted, C1 and C2 phases and the JIT may remove "useless" work; use JMH (Java Microbenchmark Harness; Chapter 16).

1.8 Memory and garbage collection — an overview

The picture you need now; algorithms, tuning and profiling are in Chapter 16.

What it is

> **Key term — thread:** one independent sequence of instructions inside a program. A program can run many at once (e.g. one per web request); each has its own call stack but shares the heap. Covered in the concurrency chapters.

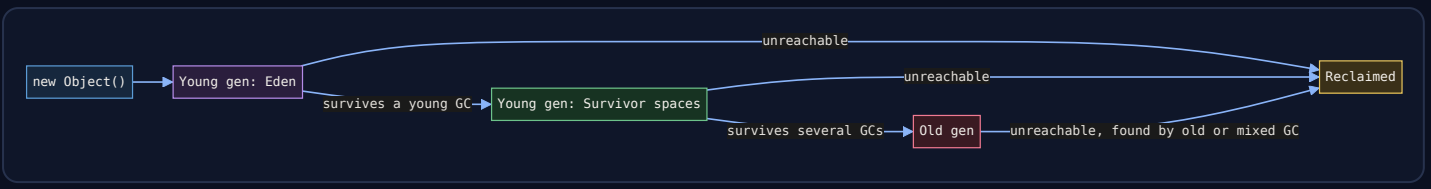
Area	Holds	Lifetime
Stack (one per thread)	*Frames* per method call: locals, parameters, partial results. Primitive locals live here; for objects only the **reference** (pointer-like handle).	Frame disappears when the method returns
Heap (shared)	All objects and arrays (`new …`).	Until the GC reclaims them
Metaspace	Class metadata (methods, field layouts), in native memory (outside the Java heap).	While the class loader is alive
Code cache	JIT-compiled machine code.	Until evicted/deoptimized

So in a method, `int x = 5;` lives on the stack; `new int[5]` lives on the heap with its reference on the stack.

Garbage collection (GC) automatically reclaims heap objects that are no longer **reachable** — no chain of references leads to them from a **GC root** (a local in an active frame, a static field, an active thread, a few JVM-internal references). Unreachable objects are garbage even if they point to each other, so reference cycles are collected.

How it works (the common core)

1. **Mark**: from the roots, follow references and mark every reachable object.
2. **Sweep / compact / copy**: free unmarked memory; many collectors also move live objects together to avoid fragmentation (free memory scattered in gaps too small to use).
3. **Generations**: most objects die young (the *weak generational hypothesis* — e.g. temporary strings in a request). Generational collectors split the heap into a **young generation**, collected often and cheaply (most of it is dead; only the few live objects are copied), and an **old generation** for survivors, collected rarely.
4. **Pauses**: some work stops application threads (**stop-the-world** pause); modern collectors do most work **concurrently** with your threads.



The collectors in JDK 25

Collector	Flag	Design	Good for
G1 (Garbage-First)	`-XX:+UseG1GC`	Generational, region-based (heap split into equal chunks), mostly concurrent marking; pause-time goal (default 200 ms)	**The default** — general servers and desktops
ZGC	`-XX:+UseZGC`	Concurrent, generational (the only mode since JDK 24); pauses typically under 1 ms regardless of heap size	Large heaps, latency-sensitive services (per-request time matters)
Shenandoah	`-XX:+UseShenandoahGC`	Concurrent compacting; generational mode became a product feature in JDK 25	Low latency; in many OpenJDK builds (not Oracle JDK)
Parallel	`-XX:+UseParallelGC`	Stop-the-world, many threads; maximises throughput (work done per second)	Batch jobs where pauses do not matter
Serial	`-XX:+UseSerialGC`	Single-threaded, stop-the-world	Tiny heaps, small containers
Epsilon	experimental	Allocates, never collects	Testing, very short-lived jobs

The JVM picks G1 unless the machine looks "small" (fewer than 2 CPUs or under 1792 MB), where it picks Serial; JDK 27 (JEP 523) makes G1 the default everywhere. Default max heap = ¼ of physical memory — `WhereAmI` below reports 12 288 MiB on a 48 GiB machine (1 MiB = 1024 × 1024 bytes, 1 GiB = 1024 MiB). **Latency** and **throughput** recur in System Design Fundamentals.

Watching the GC

`code/GarbageMaker.java` creates 200 000 short-lived 1 KiB arrays and keeps ten 1 MiB arrays:

```
List<byte[]> keep = new ArrayList<>();          // long-lived: stays reachable
for (int i = 0; i < 200_000; i++) {
    byte[] temp = new byte[1024];                // short-lived: garbage after this iteration
    temp[0] = (byte) i;
    created++;
    if (i % 20_000 == 0) keep.add(new byte[1024 * 1024]);
}
```

`java -Xlog:gc -Xmx256m -cp out ch01.gc.GarbageMaker` (selected lines from one run; timings and sequence vary):

```
[0.004s][info][gc] Using G1
[0.017s][info][gc] GC(0) Pause Young (Concurrent Start) (G1 Humongous Allocation) 1M->1M(10M) 0.407ms
[0.017s][info][gc] GC(1) Concurrent Mark Cycle
[0.018s][info][gc] GC(2) Pause Young (Normal) (G1 Evacuation Pause) 6M->3M(10M) 0.195ms
[0.022s][info][gc] GC(4) Pause Young (Prepare Mixed) (G1 Evacuation Pause) 22M->5M(30M) 0.580ms
[0.024s][info][gc] GC(5) Pause Young (Mixed) (G1 Evacuation Pause) 22M->5M(30M) 0.305ms
...
temporary arrays created: 200000
arrays still reachable  : 10
```

`22M->5M(30M) 0.580ms` = heap used before → after (current heap size), pause length. Young collections free most memory because the temporary arrays are dead. "Humongous" = an object at least half a G1 region (the 1 MiB arrays), allocated specially. With `-XX:+UseZGC` the first line reads `Using The Z Garbage Collector`.

Ask the JVM about itself with `code/WhereAmI.java`:

```
Runtime.Version v = Runtime.version();
System.out.println("Feature release : " + v.feature());
System.out.println("class file ver. : " + System.getProperty("java.class.version"));
System.out.println("Max heap (MiB)  : " + Runtime.getRuntime().maxMemory() / (1024 * 1024));
```

Output (this machine; yours will differ):

```
Feature release : 26
java.vendor     : Oracle Corporation
java.vm.name    : OpenJDK 64-Bit Server VM
class file ver. : 70.0
os.name         : Mac OS X
CPU cores seen  : 14
Max heap (MiB)  : 12288
```

Compiled for Java 25, running on JDK 26 — newer JVMs run older class files.

Common mistakes

- Believing Java cannot leak: a static `Map` cache that only grows keeps every entry reachable forever — a **memory leak**.
- Calling `System.gc()` to "free memory": only a request, usually harmful.
- Relying on `finalize()`: deprecated for removal (JEP 421, Java 18). Use try-with-resources (Chapter 8).
- `-Xmx` (max heap) larger than the container's memory limit (the JVM reads container limits automatically; let it, or size explicitly).

In the interview

Q: How does Java GC decide what to free? By reachability, not reference counting (counting references to each object): anything not reachable from GC roots (thread stacks, static fields, JNI handles — references held by native code via the Java Native Interface) is garbage, so cycles are collected. Most collectors are generational because most objects die young.

- *Default GC?* G1 since Java 9 on server-class machines.
- *When ZGC?* Large heaps or strict latency targets, where sub-millisecond pauses matter more than a little throughput.
- *Stack vs heap?* Stack: per-thread frames with locals, freed on return. Heap: shared objects, freed by GC.

TL;DR

Stack = method frames; heap = objects. GC frees unreachable objects; generational collectors exploit "most objects die young". Default G1; ZGC/Shenandoah for low latency; Parallel for throughput.

1.9 jshell — the Java REPL

What it is

`jshell` (Java 9, JEP 222) is a **REPL** — *Read-Evaluate-Print Loop*: type a snippet, see the result, no class or `main` needed. A scratchpad for trying APIs; anything you want to keep, test or share belongs in a source file.

A session

```
|  Welcome to JShell -- Version 26
|  For an introduction type: /help intro

jshell> int x = 21
x ==> 21

jshell> x * 2
$2 ==> 42

jshell> "Java".length()
$3 ==> 4

jshell> String shout(String s) { return s.toUpperCase() + "!"; }
|  created method shout(String)

jshell> shout("hi")
$5 ==> "HI!"

jshell> /vars
|    int x = 21
|    int $2 = 42
|    int $3 = 4
|    String $5 = "HI!"
```

- Unassigned expressions get reusable scratch names (`$2` , `$3` …).
- Semicolons are optional for single statements; methods can be declared at top level and redefined.

Command	Does
`/help` , `/help intro`	Help
`/vars` , `/methods` , `/types` , `/imports`	List definitions/imports
`/list`	Show your snippets
`/edit <name>`	Open a snippet in an editor
`/save file.jsh` , `/open file.jsh`	Save/load a session
`/reset` , `/exit`	Clear state, quit
Tab	Completion and method signatures

On JDK 26, `/imports` shows `import java.base` , so every public class in `java.base` is usable without imports.

1.10 Launching source files directly

Single-file and multi-file source launch

Since **Java 11** (JEP 330) `java Hello.java` compiles a single file in memory and runs it; no `.class` files are written. Since **Java 22** (JEP 458) the program may use other source files in the same directory tree, compiled on demand.

`code/launch/Main.java` uses `code/launch/Greeter.java` :

```
// Main.java
public class Main {
    public static void main(String[] args) {
        System.out.println(Greeter.greet("launcher"));
    }
}
// Greeter.java
public class Greeter {
    static String greet(String who) {
        return "Hello from " + who + ", compiled in memory";
    }
}
```

```
cd code/launch && java Main.java
```

Output:

```
Hello from launcher, compiled in memory
```

On Linux/macOS a single-file program can start with a **shebang** line (`#!` first line naming the program to run the file), e.g. `#!/usr/bin/env -S java --source 25` , in a file *without* the `.java` extension, and run as a script.

Compact source files and instance `main` — Java 25

JEP 512 (final in **Java 25**, after previews — opt-in trial versions, 1.12 — in 21–24):

- A **compact source file** may declare methods and fields with no enclosing class; the compiler wraps them in an implicit `final` class (cannot be extended).
- `main` may be a parameterless **instance method** (runs on an object, not the class): `void main()` .
- It automatically imports every public class of `java.base` (as if it began with `import module java.base;`), so `List` , `Map` , `Path` need no imports.
- New class `java.lang.IO` : `IO.println(…)` , `IO.print(…)` , `IO.println()` , `IO.readln(prompt)` / `IO.readln()` for simple console I/O (built on `System.out` and `System.in`).

`code/compact/HelloCompact.java` — the whole file:

```
void main() {
    List<String> langs = List.of("Java", "Kotlin", "Scala");   // java.util.List without an import
    IO.println("Hello from a compact source file!");
    IO.println("JVM languages: " + langs);
}
```

```
cd code/compact && java HelloCompact.java
```

Output:

```
Hello from a compact source file!
JVM languages: [Java, Kotlin, Scala]
```

How the launcher chooses `main` : `main(String[])` if present, else a no-argument `main()` . If the chosen method is not `static` , it first creates an instance with the no-argument constructor.

Trade-offs

Style	Best for	Limits
`javac` + `java`	Real applications, packages, dependencies	Two steps
`java File.java`	Scripts, examples, teaching, small tools	Compiles every run; no dependency management
Compact source file	First programs, quick utilities	No `package`; cannot be referenced by name from other code; becomes a normal class as it grows

Common mistakes

- Adding a `package` declaration to a compact source file (not allowed).
- Expecting instance `main` on Java 21 without `--enable-preview` — preview until Java 25.

1.11 IDEs, editors and build tools

What they are

An **IDE** (Integrated Development Environment) combines editor, compiler integration, debugger (pauses a running program to step through and inspect variables), refactoring and test runner: **IntelliJ IDEA** (JetBrains; most popular, strongest refactoring; free and paid tiers), **Eclipse** (free, open source, its own incremental compiler), **VS Code** + *Extension Pack for Java* (lightweight, mixed-language work), **Apache NetBeans** (free, open source).

Build tools automate compiling, downloading **dependencies** (external libraries your project uses, at the right versions), testing and packaging: **Maven** (XML `pom.xml`, fixed lifecycle) and **Gradle** (Groovy/Kotlin DSL — a small build-script language; flexible and incremental, rebuilding only what changed). Both read `JAVA_HOME` and support *toolchains* to pin a JDK version. Details: Chapter 18.

Suggested path: plain `javac` / `java` / `jshell` for Chapters 1–3; an IDE from Chapter 5 (multiple classes, packages); Maven/Gradle for anything with dependencies.

Coding conventions (used throughout this book)

Kind	Convention	Example
Class, interface, record, enum	`UpperCamelCase` noun	`OrderService`, `Shape`
Method, variable	`lowerCamelCase`; methods are verbs	`computeTotal()`, `itemCount`
Constant (`static final`)	`UPPER_SNAKE_CASE`	`MAX_RETRIES`
Package	all lowercase, reversed domain	`com.example.billing`
Indentation	4 spaces; opening brace on the same line	`if (x) {`

1.12 Java releases, LTS and JEPs

How the release model works

- A **feature release** ships every six months, **March** and **September** (since Java 10, 2018). Version numbers are counters: 25, 26, 27…
- Each gets a couple of quarterly **update releases** (25.0.1, 25.0.2) from Oracle's OpenJDK builds; vendors keep patching the **LTS** ones.
- **Long-Term Support (LTS)** releases: **8, 11, 17, 21, 25** — every two years since 17; next expected September 2027 (Java 29). Oracle offers at least eight years of LTS support (five Premier + three

Extended).

- Non-LTS releases (22, 23, 24, 26, 27…) are production-quality but supported only until the next release.



JEPs and feature maturity

New features are proposed in **JEPs** (JDK Enhancement Proposals, openjdk.org/jeps), e.g. *JEP 512: Compact Source Files and Instance Main Methods*. Stages:

Stage	Meaning	How to use it
Incubator	Experimental API in a `jdk.incubator.*` module (e.g. the Vector API, still incubating in 27)	`--add-modules jdk.incubator.vector`
Preview	Complete language/VM/API feature that may still change	`javac --release 25 --enable-preview …` and `java --enable-preview …`
Final / permanent	Standard, stable part of Java	Just use it
Withdrawn	Removed before final (e.g. *string templates*, previewed 21–22, dropped in 23)	—

This book marks each modern feature with the version that **finalised** it (e.g. *records — Java 16*). Full tour: Chapter 17.

Which version should I use?

Situation	Pick
New project, or learning	**Java 25 LTS**
Existing Java 17/21 service	Stay while supported; plan an upgrade to 25 (mostly smooth — run tests on the new JDK first)
Legacy Java 8	8 → 11 → 17 → 21/25 needs attention to removed Java EE modules and strong encapsulation of JDK internals (code can no longer reach the JDK's private implementation classes)
Library you publish	Compile with `--release` of the *oldest* Java you support

In the interview

Q: What is an LTS release and why does it matter? A feature release vendors support for many years (8, 11, 17, 21, 25); companies standardise on them to avoid upgrading every six months.

- *How often does Java release?* Every six months (March/September); LTS every two years. Java 26 is not LTS.
- *Preview feature?* Finished-but-not-final; opt in with `--enable-preview` on both `javac` and `java`; may change or be withdrawn.
- *Name three Java 21 features.* Virtual threads, pattern matching for `switch`, record patterns, sequenced collections.

Chapter TL;DR

- **Java** = statically typed, object-oriented language + portable runtime. Current LTS **Java 25** (Sept 2025); releases every March/September, LTS every two years.
- **JDK ⊃ runtime (JRE) ⊃ JVM.** Install a JDK; ship a `jlink` runtime or container image.

- `javac` compiles `.java` → `.class` bytecode (major version 69 = Java 25); `java` runs it. `--release N` targets an older Java safely.
- The JVM **loads lazily, verifies, interprets, then JIT-compiles** hot code (C1 → C2) using profiles; AOT caches (JDK 24–25) and native images cut warm-up.
- **Stack** = frames and locals; **heap** = objects; **GC** frees unreachable objects. Default G1; ZGC/Shenandoah for low latency.
- **jshell** for experiments; `java File.java` runs sources directly (multi-file since 22); **compact source files +** `void main()` **+** `IO` final in **Java 25**.
- Exam traps: OCP Chapter 1. Next: Chapter 2 — Basics: types, variables and operators.

Chapter 2 — Basics: Types, Variables and Operators

Programs are built from **variables** holding **values** of a **type** (the kind of data — whole number, decimal, text, true/false — which decides what you can do with it), **operators** that combine them, and **statements** executed in order. This chapter covers them precisely, including Java's silent surprises: integer overflow (a too-big number wrapping round), integer division (drops the remainder), floating-point rounding (decimals stored approximately) and `==` on strings. It ends with text blocks, `String` basics and console I/O.

Code: `code/`, compiled with `javac --release 25`, output from real runs. Exam traps: OCP Chapter 1 — Building Blocks (types, literals, `var`, text blocks, scope) and Chapter 2 — Operators (promotion, casting, precedence).

2.1 Statements, identifiers, keywords and comments

What it is

- A **statement** is one complete instruction ending in `;` (`int x = 5;`). A **block** `{ … }` groups statements and can appear wherever a statement can.
- An **expression** produces a value (`x + 1`, `name.length()`, `a > b`); an *expression statement* uses one as a statement (`x++;`).
- An **identifier** is a name you choose for a variable, method (named, callable block of code), class (blueprint for a kind of object) or package (named group of classes, like a folder).

Identifier rules

Rule	Legal	Illegal
Starts with a letter, `$` or `_`	`total`, `$price`, `_temp`	`2fast`
Then letters, digits, `$`, `_`	`item2`, `max_value`	`my-name`, `a b`
Not a reserved keyword or `true` / `false` / `null`	`klass`, `clazz`	`class`, `null`
A single `_` is a keyword (*unnamed variables*, Chapter 3)	`__`	`_`
Case-sensitive	`count` ≠ `Count`	—

Unicode letters are allowed (`größe`); `$` is legal but by convention reserved for generated code.

Keywords

Java 25 has **51 reserved keywords** that can never be identifiers — e.g. `class`, `int`, `if`, `for`, `new`, `static`, `final`, `return`, `try`, plus `const` and `goto` (reserved, unused), `strictfp` (obsolete) and `_`. It has **17 contextual keywords**, keywords only in certain positions so old code using them as names still compiles: `var`, `yield`, `record`, `sealed`, `non-sealed`, `permits`, `when`, and the module words `module`, `open`, `opens`, `exports`, `requires`, `transitive`, `uses`, `provides`, `to`, `with`. `true`, `false` and `null` are *literals* (values written directly in code), not keywords, but are still unusable as names.

Comments

```
// line comment: to the end of the line
/* block comment:
   can span lines (cannot be nested) */
/** Javadoc comment: documents the next class, method or field. @param, @return tags… */
```

Comment **why**, not what the code already says.

2.2 Variables

What it is

A **variable** is a named storage location with a fixed **type**. You **declare** it (`type name;`), **initialise** it (`name = value;` , often together: `int age = 30;`), then read or reassign it with values of that type. It exists within its **scope** — from the declaration to the closing brace of its block.

code/Variables.java :

```
static int counter;          // field: gets a default value (0) automatically

public static void main(String[] args) {
    int age;                 // declaration
    age = 30;                // initialisation (assignment)
    int year = 2025, month = 9;   // declare + initialise several of the same type
    age = age + 1;           // reassignment
    final double TAX_RATE = 0.18; // final: can be assigned exactly once
    // TAX_RATE = 0.2;       // compile error: cannot assign a value to final variable TAX_RATE

    System.out.println("age=" + age + " year=" + year + " month=" + month);
    System.out.println("TAX_RATE=" + TAX_RATE + " counter (field default)=" + counter);

    {                                // an inner block
        int inner = 5;               // exists only inside these braces
        System.out.println("inner=" + inner + ", age visible here too: " + age);
    }
    // System.out.println(inner);    // compile error: cannot find symbol

    int count;
    if (age > 18) count = 1; else count = 0;   // definitely assigned on every path
    System.out.println("count=" + count);
}
```

Output:

```
age=31 year=2025 month=9
TAX_RATE=0.18 counter (field default)=0
inner=5, age visible here too: 31
count=1
```

Local variables vs fields

	Local variable (in a method/block)	Field (in a class)
Default value	**None** — must be assigned before use, else *"variable x might not have been initialized"*	`0` , `0.0` , `false` , `'\u0000'` , or `null` for references
Lifetime	Until the block ends	As long as the object (instance field — one per object) or class (`static` field — one shared copy)
Modifiers	Only `final`	Access modifiers (`public` , `private` … — who may use it), `static` , `final` , … (Chapter 5)

Definite assignment analysis checks that every path assigns a local before it is read — so `count` above compiles, while `int n; System.out.println(n);` does not.

`final`

`final` = *assigned once*. A `final` primitive never changes; a `final` reference cannot point elsewhere but the **object can still change** (`final List<String> xs = new ArrayList<>(); xs.add("ok");` is legal — lists: Chapter 10). Use it for constants (`static final int MAX_RETRIES = 3;`) and to clarify intent.

Common mistakes

- Reading a local before assigning it on every branch.
- Redeclaring a name in scope (`int x = 1; { int x = 2; }` — locals cannot shadow, i.e. hide, other locals).
- Thinking `final` makes an object immutable (unchangeable after creation).

2.3 Primitive types

What it is

Java has exactly **eight primitive types** — simple values stored directly, not as objects; everything else is a reference type. Sizes and ranges are fixed on every platform, unlike C. Sizes are in **bits** (binary digits); 8 bits = one **byte**.

`code/Primitives.java` prints the real values from the wrapper classes (2.5):

```
System.out.printf("%-8s %5d  %-22d %d%n", "int", Integer.SIZE, Integer.MIN_VALUE, Integer.MAX_VALUE);
// ... one line per type
```

Output:

```
type      bits  min                   max
byte         8  -128                  127
short       16  -32768                 32767
int         32  -2147483648            2147483647
long        64  -9223372036854775808   9223372036854775807
char        16  0                     65535
float       32  1.4E-45               3.4028235E38
double      64  4.9E-324              1.7976931348623157E308
boolean  true or false (size not specified by the language)
```

Type	Kind	Size	Range / precision	Default (fields)	Use it for
`byte`	signed integer	8 bits	−128 … 127	`0`	raw binary data, I/O buffers (memory holding data being read/written)
`short`	signed integer	16 bits	−32 768 … 32 767	`0`	rarely — large arrays of small numbers
`int`	signed integer	32 bits	about ±2.1 billion (-2^{31} … $2^{31}-1$ = 2 147 483 647)	`0`	**the default integer** — counts, indexes
`long`	signed integer	64 bits	about ±9.2 × 10^{18}	`0L`	epoch milliseconds (ms since 1 Jan 1970 UTC), IDs, cents, big counters
`float`	IEEE 754 binary floating point	32 bits	~6–7 significant digits, up to ±3.4 × 10^{38}	`0.0f`	graphics, large arrays where memory matters
`double`	IEEE 754 binary floating point	64 bits	~15–16 significant digits, up to ±1.8 × 10^{308}	`0.0`	**the default decimal** — measurements, science
`char`	unsigned UTF-16 code unit	16 bits	`'\u0000'` … `'￿'` (0 … 65 535)	`'\u0000'`	single characters
`boolean`	logical	not specified	`true` , `false`	`false`	conditions, flags

- *Signed* types hold negatives; *unsigned* only zero and positives. **IEEE 754** is the standard for storing floating-point numbers; **UTF-16** stores characters as 16-bit units.
- `float` / `double` `MIN_VALUE` is the smallest **positive** (subnormal — below the normal range, with reduced precision) value; the most negative is `-Double.MAX_VALUE`.
- Characters outside the Basic Multilingual Plane (the first 65 536 Unicode characters; many emoji lie beyond) need two `char`s (a *surrogate pair*); see Chapter 7.
- `boolean` is not a number: `if (1)` does not compile, and you cannot cast between `boolean` and `int`.
- Beyond the table: money and exact decimals → `long` cents or `BigDecimal` (exact decimal class), never `double` (2.11); arbitrary-size integers → `BigInteger` (Chapter 7); file sizes → `long`.

2.4 Literals

What it is

A **literal** is a value written directly in source; its form decides its type. Integer bases: **hexadecimal** (base 16, `0x`), **octal** (base 8, `0`), **binary** (base 2, `0b`).

`code/Literals.java`:

```
int decimal = 255, hex = 0xFF, octal = 0377, binary = 0b1111_1111;
long big = 9_000_000_000L;              // L suffix: needed beyond int range
float price = 19.99f;                   // f suffix: literal is double otherwise
double sci = 6.022e23, plain = 3.14;
char letter = 'A', unicode = 'é', tab = '\t';
boolean ready = true;
String text = "He said \"hi\"\\n";      // escaped quote and backslash

System.out.println(decimal + " " + hex + " " + octal + " " + binary);
System.out.println(big + " " + price + " " + sci + " " + plain);
System.out.println(letter + " " + unicode + " [" + tab + "] " + ready);
System.out.println(text);
System.out.println("char 'A' as a number: " + (int) letter + ", 'A' + 1 = " + (letter + 1)
        + ", (char) ('A' + 1) = " + (char) (letter + 1));
```

Output:

```
255 255 255 255
9000000000 19.99 6.022E23 3.14
A é [   ] true
He said "hi"\n
char 'A' as a number: 65, 'A' + 1 = 66, (char) ('A' + 1) = B
Integer.toBinaryString(10) = 1010, toHexString(255) = ff
```

Literal	Form	Type
Decimal integer	`42`, `1_000_000`	`int`
Long	`42L` (capital `L`; `l` looks like `1`)	`long`
Hexadecimal	`0xFF`, `0xCAFE_BABE`	`int` (or `long` with `L`)
Octal	leading `0`: `0377` = 255	`int` — beware: `010` is **8**
Binary	`0b1010` (Java 7)	`int`
Floating point	`3.14`, `1e-9`, `.5`, `2.`	`double`
Float	`3.14f`	`float`
Character	`'A'`, `'\n'`, `'é'`	`char`
Boolean	`true`, `false`	`boolean`
String	`"text"`, text blocks `"""…"""`	`String`
Null	`null`	the null type (assignable to any reference)

Underscores (Java 7) separate digits (`1_000_000`, `0b1111_0000`) but not at the start or end of the digits, next to the decimal point, next to an `L`/`f` suffix, or right after `0x`/`0b`.

Escape sequences: `\n` newline, `\t` tab, `\r` carriage return, `\b` backspace, `\f` form feed, `\"`, `\'`, `\\`, `\s` space (Java 15), octal `\0`–`\377`, Unicode `\uXXXX`.

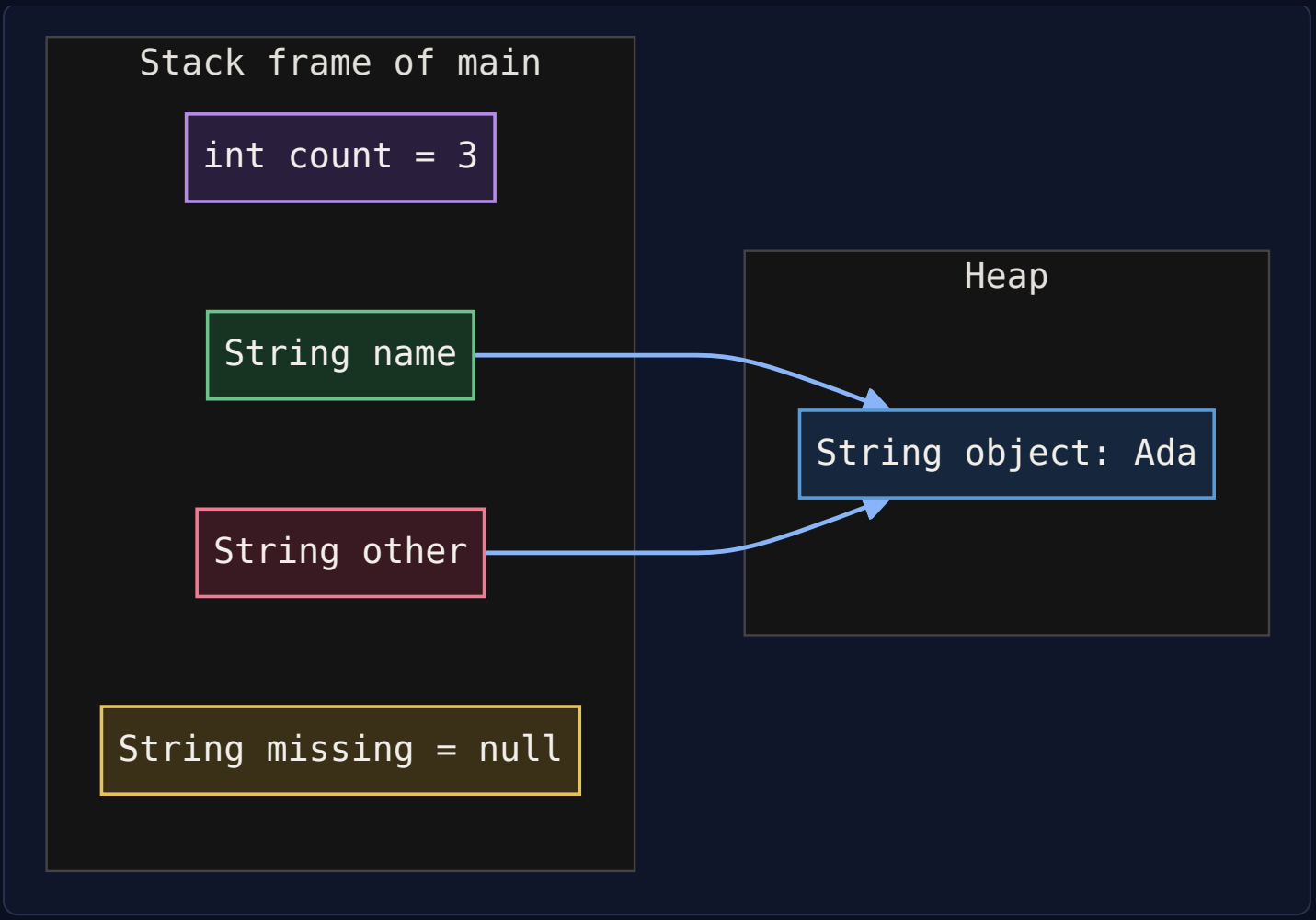
Common mistakes

- `long big = 9000000000;` → *"integer number too large"*: the literal is an `int` until you add `L`.
- `float f = 1.5;` → *possible lossy conversion from double to float*; write `1.5f`.
- Leading zeros for alignment: `int code = 010;` is 8.

2.5 Reference types and `null`

What it is

Every non-primitive type is a **reference type**: classes (`String`, `Scanner`), interfaces (`List`), arrays (`int[]`), enums and records (Chapters 4–6). A reference variable holds a **reference** — a handle to an object on the heap (1.8) — or `null`, "points to nothing".



- `other = name` copies the **arrow**, not the object: both point to the same `String`.
- Calling a method through `null` throws `NullPointerException`. *Throwing* an **exception** signals an error that stops normal execution unless caught (Chapter 8). Since Java 14 (JEP 358, default from 15) *helpful NullPointerException* messages name what was null: *Cannot invoke "String.length()" because "name" is null*.

	Primitive	Reference
Holds	the value itself	a pointer to an object, or `null`
Default (field)	`0` / `false` / `'\u0000'`	`null`
`==` compares	values	**identity** (same object?)
Has methods	no	yes (`name.length()`)

Every primitive has a **wrapper class** (`Integer` , `Double` , `Character` , `Boolean` …) for use where an object is required, e.g. `List<Integer>` (`<…>` is a *generic* type argument, Chapter 9); Java converts automatically (*autoboxing/unboxing*). Details and the `Integer` cache trap: Chapter 7.

2.6 `var` — local variable type inference (Java 10)

What it is

`var` (Java 10, JEP 286) lets the compiler **infer** a local variable's type from its initialiser (the `= value` part). The variable is still statically typed — fixed at compile time, just not written.

`code/VarDemo.java` :

```java
var count = 10;                          // int
var ratio = 10 / 4.0;                    // double
var name = "Ada";                        // String
var scores = new HashMap<String, List<Integer>>();   // HashMap<String, List<Integer>>
scores.put(name, new ArrayList<>(List.of(90, 85)));
for (var entry : scores.entrySet()) {    // Map.Entry<String, List<Integer>>
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}
// count = "ten";                        // compile error: the type is fixed at int
var items = new ArrayList<>();           // careful: ArrayList<Object>
items.add(1); items.add("two");
System.out.println(count + " " + ratio + " " + items);
```

Output:

```
Ada -> [90, 85]
10 2.5 [1, two]
{a=1}
```

Where `var` is allowed

Allowed	Not allowed
Local variables with an initialiser	Fields, method parameters, return types
`for` and for-each loop variables	No initialiser: `var x;`
try-with-resources variables (Chapter 8)	`var x = null;` (no type to infer)
Lambda parameters (Java 11): `(var a, var b) -> a + b` — lets you add annotations (`@` -tags like `@Override`)	Array initialiser alone: `var a = {1, 2};`
	Multiple variables: `var a = 1, b = 2;`

`var` is a contextual keyword: `var var = 1;` compiles, but no *type* may be named `var`.

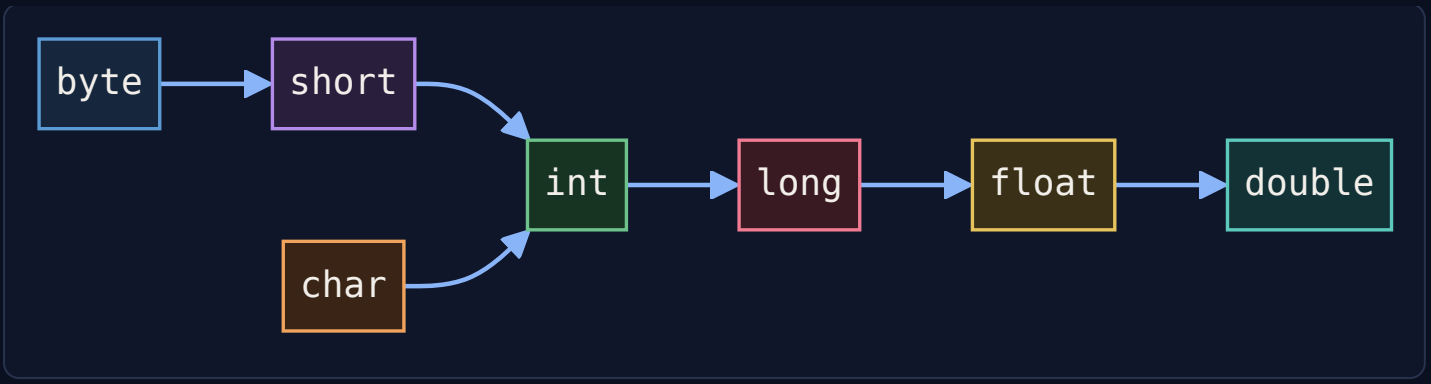
Trade-offs and style

Use `var` when the type is obvious from the right side (`new`, a well-named factory method) or long and noisy (`Map<String, List<Integer>>`). Avoid it when the reader cannot see the type (`var result = service.process();`) or the inferred type is not what you want (`new ArrayList<>()` → `ArrayList<Object>` ; `var x = 1;` is `int`, not `long`). OpenJDK's *Local Variable Type Inference Style Guidelines* agree: clarity for the reader.

2.7 Type conversion: widening, narrowing, casting, promotion

What it is

Java does **safe** conversions automatically and needs an explicit **cast** — `(type) value` — for risky ones.



Arrows are **widening primitive conversions**, automatic (`int i = 'A'; long l = i; double d = l;`). Going against them is **narrowing** and needs a cast.

How it works

1. **Widening** is automatic but not always lossless: `int → float` , `long → float` and `long → double` keep magnitude but can lose **precision** — `float` has a 24-bit mantissa (the part storing significant digits), `double` 53 bits vs `long` 's 64.
2. **Narrowing** (cast): `(int) 3.99` truncates toward zero to `3` (`(int) -3.99` is `-3`); `(byte) 300` keeps the low 8 bits (44); `(byte) 200` is −56 (bits `11001000` read as signed). Floating → integer clamps out-of-range values to MIN/MAX and turns NaN ("Not a Number", e.g. 0.0/0) into 0.
3. **Numeric promotion**: before a binary arithmetic operator runs, `byte` , `short` and `char` operands become `int` ; if either operand is `long` , `float` or `double` , both widen to the wider type. So `byte + byte` is `int` , and `'a' + 1` is the `int` 98.
4. **Compile-time constants** (literals, or `final` variables set from literals) may be assigned to a narrower type if they fit: `byte b = 10 + 20;` is fine; `byte b = x + y;` is not.
5. **Compound assignment** (`+=` , `-=` , `*=` …) hides a cast: `x += 5` means `x = (T) (x + 5)` .

`code/Conversions.java` (key lines):

```
double pi = 3.99;
int truncated = (int) pi;          // narrowing needs a cast; drops the fraction
long rounded = Math.round(pi);     // rounding is a method, not a cast

byte b = (byte) 300;               // keeps only the low 8 bits: 300 - 256 = 44
int precise = 123_456_789;
float f = precise;                 // widening but LOSSY: float has 24 bits of precision

byte x = 10, y = 20;
// byte sum = x + y;               // compile error: x + y is an int
byte sum = (byte) (x + y);
x += 5;                            // compound assignment hides a cast: x = (byte) (x + 5)

long wrong = 1_000_000 * 1_000_000;       // int overflow BEFORE widening to long
long right = 1_000_000L * 1_000_000;
```

Output:

```
widened: 100 100.0
(int) 3.99 = 3, Math.round(3.99) = 4
(byte) 300 = 44, (byte) 200 = -56
(int) 1e20 = 2147483647, (int) Double.NaN = 0
int 123456789 -> float -> int: 123456792
sum=30 sum2=30 x=15
constant=30 'a'+1=98 7/2=3 7/2.0=3.5
wrong=-727379968 right=1000000000000
```

The last line is the classic bug: the multiplication runs in `int` , overflows, and only then widens. Put `L` on the **first** operand (`24L * 60 * 60 * 1000 * 365`).

Common mistakes

- Expecting `(int)` to round — use `Math.round`, `Math.floor`, `Math.ceil`.
- `7 / 2` is `3` (integer division). Write `7 / 2.0` or `(double) 7 / 2`; `(double) (7 / 2)` is `3.0` — cast too late.
- Assuming widening is lossless.

In the interview

Q: Why doesn't `short s = 1; s = s + 1;` compile, but `s += 1;` does? `s + 1` is promoted to `int`, and assigning `int` to `short` needs a cast; compound assignment includes it implicitly.

- *`(byte) 200`?* −56.
- *Is `long → double` safe?* Magnitude yes, precision no (53-bit vs 64-bit).
- *`(int) 7.9 + (int) -7.9`?* 7 + (−7) = 0.

2.8 Operators

What it is

An **operator** computes a value from one (*unary*), two (*binary*) or three (*ternary*) operands.

Arithmetic: `+ - * / %` and `++ --`

code/Arithmetic.java :

```
System.out.println("17 / 5 = " + 17 / 5 + ", 17 % 5 = " + 17 % 5);
System.out.println("-17 / 5 = " + -17 / 5 + ", -17 % 5 = " + -17 % 5
        + ", Math.floorMod(-17, 5) = " + Math.floorMod(-17, 5));
System.out.println("17.0 / 5 = " + 17.0 / 5 + ", 5.5 % 2 = " + 5.5 % 2);
System.out.println("1.0 / 0 = " + 1.0 / 0 + ", 0.0 / 0 = " + 0.0 / 0);
try { System.out.println(1 / 0); }
catch (ArithmeticException e) { System.out.println("1 / 0 -> ArithmeticException: " + e.getMessage()); }

int a = 5;
int b = a++;     // post-increment: b gets the old value, then a becomes 6
int c = ++a;     // pre-increment: a becomes 7, then c gets 7
int n = 3;
int result = n++ + ++n;   // 3 + 5
```

Output:

```
17 / 5 = 3, 17 % 5 = 2
-17 / 5 = -3, -17 % 5 = -2, Math.floorMod(-17, 5) = 3
17.0 / 5 = 3.4, 5.5 % 2 = 1.5
1.0 / 0 = Infinity, 0.0 / 0 = NaN
1 / 0 -> ArithmeticException: / by zero
a=7 b=5 c=7
n++ + ++n with n=3 -> 8, n=5
compound ops -> x=1
-(-5) = 5, +3 = 3
```

Rule	Detail
Integer `/` truncates toward zero	`-17 / 5` is `-3`; `-7 / 2` is `-3`
`%` takes the sign of the **dividend**	`-17 % 5` is `-2`, `-7 % 2` is `-1`. For a non-negative result (circular buffers, day of week) use `Math.floorMod`
Integer division by zero	throws `ArithmeticException`
Floating division by zero	no exception: `Infinity`, `-Infinity` or `NaN`
`x++` vs `++x`	both add 1; the expression's value is the old (postfix) or new (prefix) value — `int x = 5; int y = x++ * 2;` gives `y = 10`, `x = 6`
`+` with a `String` operand	concatenation (2.12)

Use `++` / `--` as standalone statements; `n++ + ++n` is deterministic (operands evaluate left to right) but unreadable.

Assignment operators

`=` and compound `+= -= *= /= %= &= |= ^= <<= >>= >>>=`. Assignment is an expression whose value is the assigned value, right-associative: `a = b = c = 7` sets all three.

Relational and equality: `< <= > >= == !=`, `instanceof`

All return `boolean`. For **primitives** `==` compares values; for **references** it compares **identity** (same object). Compare contents with `equals`.

`code/Comparison.java`:

```
String s1 = "java";
String s2 = new String("java");
System.out.println("s1 == s2: " + (s1 == s2) + ", s1.equals(s2): " + s1.equals(s2));
```

Output:

```
s1 == s2: false, s1.equals(s2): true
```

`x instanceof Type` tests the object's type (false for `null`); the pattern form `x instanceof String s` is in Chapter 3.

Logical: `&& || !` and `& | ^`

Operator	Name	Evaluates right side?
`&&`	conditional AND	only if left is `true` (**short-circuit**)
`||`	conditional OR	only if left is `false` (short-circuit)
`!`	NOT	—
`&`, `|`	logical AND / OR on booleans	**always** both sides
`^`	XOR (exactly one true)	always both

```
static boolean check(String label, boolean value) {
    System.out.println("  evaluated " + label);
    return value;
}
boolean r1 = check("left", false) && check("right", true);
boolean r2 = check("left", false) & check("right", true);

String name = null;
if (name != null && name.length() > 3) System.out.println("long name");   // safe: && stops at null
```

Output:

```
&& short-circuits:
  evaluated left
& evaluates both:
  evaluated left
  evaluated right
r1=false r2=false true ^ true=false !true=false
null-safe check with && worked
```

Short-circuiting makes the *null guard* `x != null && x.foo()` safe; with `&` it throws `NullPointerException`.

Ternary `?:`

`condition ? valueIfTrue : valueIfFalse` is an **expression**, unlike `if`:

```
String category = age < 13 ? "child" : age < 20 ? "teen" : "adult";   // "adult" for 20
```

Keep to one level; otherwise use `if` or a `switch` expression (Chapter 3).

Bitwise and shift: `& | ^ ~ << >> >>>`

These work on individual bits of integers. A *flag* is a yes/no setting in one bit; a *mask* is a value used with `&` to pick out bits. `code/Bitwise.java`:

```
static final int READ = 1, WRITE = 2, EXEC = 4;   // 001, 010, 100
int a = 0b1100, b = 0b1010;
System.out.println("a & b = " + bin(a & b) + ", a | b = " + bin(a | b)
        + ", a ^ b = " + bin(a ^ b) + ", ~a = " + (~a));
System.out.println("1 << 4 = " + (1 << 4) + ", 64 >> 2 = " + (64 >> 2));
System.out.println("-16 >> 2 = " + (-16 >> 2) + ", -16 >>> 28 = " + (-16 >>> 28));
int perms = READ | WRITE;                // set two flags
perms &= ~WRITE;                         // clear a flag
```

Output:

```
a & b = 1000, a | b = 1110, a ^ b = 110, ~a = -13
1 << 4 = 16, 64 >> 2 = 16
-16 >> 2 = -4, -16 >>> 28 = 15
1 << 32 = 1  (shift distance is taken mod 32 for int)
can write? true, can exec? false
after clearing WRITE: 1
is 12 even? true
```

Operator	Meaning	Example
`&`	1 where both bits are 1	test a flag: `(perms & WRITE) != 0`
`|`	1 where either is 1	set a flag
`^`	1 where bits differ	toggle; `a ^ a == 0`
`~`	flip every bit; `~x == -x - 1`	clear with `& ~FLAG`
`<<`	shift left, fill with 0; `x << n` = x·2ⁿ (until overflow)	`1 << k` = k-th bit
`>>`	*arithmetic* shift right, copies the sign bit (top bit; 1 = negative)	`-16 >> 2 = -4`
`>>>`	*logical* shift right, fills with 0	unsigned view of negatives

The shift distance uses only the low 5 bits for `int` (6 for `long`), so `1 << 32 == 1`. Uses: permission flags, hash functions (`HashMap` mixes `h ^ (h >>> 16)`), `EnumSet`'s bit vector, network protocols, interview bit tricks (Chapter 19).

2.9 Operator precedence and associativity

What it is

Precedence decides which operator binds first (`*` before `+`); **associativity** orders equal-precedence operators (mostly left-to-right; assignment and ternary right-to-left). Operands are always **evaluated left to right**.

Level (high → low)	Operators	Associativity
1 Postfix	`expr++` `expr--`	left
2 Unary	`++expr` `--expr` `+` `-` `~` `!` `(type)` cast	right
3 Multiplicative	`*` `/` `%`	left
4 Additive	`+` `-` (and string `+`)	left
5 Shift	`<<` `>>` `>>>`	left
6 Relational	`<` `>` `<=` `>=` `instanceof`	left
7 Equality	`==` `!=`	left
8 Bitwise AND	`&`	left
9 Bitwise XOR	`^`	left
10 Bitwise OR	`|`	left
11 Conditional AND	`&&`	left
12 Conditional OR	`||`	left
13 Ternary	`? :`	right
14 Assignment, lambda	`=` `+=` `-=` … `->`	right

`code/Precedence.java` :

```
System.out.println("2 + 3 * 4 = " + (2 + 3 * 4));
System.out.println("(2 + 3) * 4 = " + ((2 + 3) * 4));
System.out.println("10 - 4 - 3 = " + (10 - 4 - 3) + "  (left to right)");
int a, b, c;
a = b = c = 7;                                  // assignment is right-associative
System.out.println("a=" + a + " b=" + b + " c=" + c);
System.out.println(1 + 2 + "3" + 4 + 5);        // left to right: 3, then "33", "334", "3345"
System.out.println("true || false && false = " + (true || false && false));
int x = 6;
System.out.println("x & 1 == 0 needs parentheses: " + ((x & 1) == 0));
```

Output:

```
2 + 3 * 4 = 14
(2 + 3) * 4 = 20
10 - 4 - 3 = 3  (left to right)
a=7 b=7 c=7
3345
true || false && false = true
5 > 3 == true: true
x & 1 == 0 needs parentheses: true
```

- `&&` binds tighter than `||` : `true || (false && false)` .
- `==` binds tighter than `&` : `x & 1 == 0` means `x & (1 == 0)` — a compile error for `int x` . Parenthesise bitwise tests.
- `+` is left-associative: `1 + 2 + "3"` adds first, then concatenates.

Rule of thumb: unary → multiplicative → additive → comparison → equality → logical → ternary → assignment; parenthesise whenever a reader might hesitate.

2.10 Integer overflow

What it is

Integer arithmetic is modulo 2^{32} (`int`) or 2^{64} (`long`): a result outside the range silently **wraps around** — no exception, no warning. This is **overflow**.

> **Key term — two's complement:** how computers store negative integers. The top bit is the sign bit; −n is stored as the bit pattern of $2^{32} - n$ (for `int`). One adding circuit handles both signs, and adding 1 to the largest `int` yields the most negative one.

Why it matters

The binary-search (repeatedly halving a sorted range) midpoint bug `(low + high) / 2` sat in the JDK's own `Arrays.binarySearch` for years (fixed 2006); timestamps multiplied in `int` give negative durations.

`code/Overflow.java` :

```
int max = Integer.MAX_VALUE;
System.out.println("MAX_VALUE + 1 = " + (max + 1));
System.out.println("Math.abs(Integer.MIN_VALUE) = " + Math.abs(Integer.MIN_VALUE));

int low = 2_000_000_000, high = 2_100_000_000;
System.out.println("(low + high) / 2 = " + (low + high) / 2 + "   <- overflow bug");
System.out.println("low + (high - low) / 2 = " + (low + (high - low) / 2));
System.out.println("(low + high) >>> 1 = " + ((low + high) >>> 1));

try { Math.addExact(max, 1); }
catch (ArithmeticException e) { System.out.println("Math.addExact(MAX, 1) -> " + e.getMessage()); }
long seconds = 30L * 24 * 60 * 60 * 1000;     // L on the first operand: all long math
```

Output:

```
MAX_VALUE + 1 = -2147483648
Math.abs(Integer.MIN_VALUE) = -2147483648
-Integer.MIN_VALUE = -2147483648
(low + high) / 2 = -97483648   <- overflow bug
low + (high - low) / 2 = 2050000000
(low + high) >>> 1 = 2050000000
Math.addExact(MAX, 1) -> integer overflow
Math.toIntExact(3e9) -> integer overflow
30 days in ms (long) = 2592000000, as int math = -1702967296
```

`Math.abs(Integer.MIN_VALUE)` stays negative because +2 147 483 648 does not fit in an `int`.

How to defend

Technique	Example
Use a wider type early	`long total = (long) a * b;` or `30L * …`
Throw instead of wrapping	`Math.addExact` , `subtractExact` , `multiplyExact` , `incrementExact` , `negateExact` , `toIntExact` (Java 8), `absExact` (Java 15)
Rearrange the formula	`low + (high - low) / 2`
Unlimited size	`BigInteger` (Chapter 7)

In the interview

Q: What happens when an `int` overflows? It wraps silently via two's complement (`MAX_VALUE + 1 == MIN_VALUE`). Use `long` , `Math.*Exact` (throws `ArithmeticException`), or `BigInteger` .

- *Where in practice?* Binary-search midpoints, hash-code arithmetic (intentional), millisecond calculations, summing large `int` arrays.

2.11 Floating-point numbers

What it is

`float` and `double` follow **IEEE 754** binary floating point: sign × binary fraction (mantissa) × 2^exponent. Most decimal fractions (0.1, 0.2) have no exact binary form — as 1/3 has no exact decimal form — so the nearest representable value is stored.

code/FloatingPoint.java :

```
System.out.println("0.1 + 0.2 = " + (0.1 + 0.2));
System.out.println("0.1 + 0.2 == 0.3: " + (0.1 + 0.2 == 0.3));
System.out.println("|diff| < 1e-9: " + (Math.abs((0.1 + 0.2) - 0.3) < 1e-9));
double nan = 0.0 / 0;
System.out.println("NaN == NaN: " + (nan == nan) + ", Double.isNaN: " + Double.isNaN(nan));
System.out.println("-0.0 == 0.0: " + (-0.0 == 0.0) + ", Double.compare(-0.0, 0.0): " + Double.compare(-0.0, 0.0));
double total = 0;
for (int i = 0; i < 10; i++) total += 0.1;
BigDecimal exact = new BigDecimal("0.1").add(new BigDecimal("0.2"));
```

Output:

```
0.1 + 0.2 = 0.30000000000000004
0.1 + 0.2 == 0.3: false
|diff| < 1e-9: true
0.1f + 0.2f = 0.3
1.0 - 0.9 = 0.09999999999999998
NaN == NaN: false, Double.isNaN: true
-0.0 == 0.0: true, Double.compare(-0.0, 0.0): -1
1e308 * 10 = Infinity, Math.sqrt(-1) = NaN
16777217 as float = 1.6777216E7  (2^24 + 1 not representable)
0.1 added ten times = 0.9999999999999999
BigDecimal 0.1 + 0.2 = 0.3, new BigDecimal(0.1) = 0.1000000000000000055511151231257827021181583404541015625
Math.round(2.5)=3 Math.round(-2.5)=-2 Math.floor(-2.5)=-3.0 Math.ceil(-2.5)=-2.0
```

(`0.1f + 0.2f` *prints* `0.3` only because `float`'s shorter printed form hides the error.)

Special values

Value	Produced by	Test with
`Infinity`, `-Infinity`	overflow (`1e308 * 10`), `1.0 / 0`	`Double.isInfinite(x)`
`NaN`	`0.0 / 0`, `Math.sqrt(-1)`	`Double.isNaN(x)` (or `x != x`) — **never** `x == Double.NaN` (NaN equals nothing, not even itself)
`-0.0`	`-1.0 / Double.POSITIVE_INFINITY`, rounding tiny negatives	`== 0.0` is true, but `Double.compare` / `Double.equals` distinguish it

How to handle it

1. **Compare with a tolerance**: `Math.abs(a - b) < 1e-9` (choose it for your domain).
2. **Money**: `long` minor units (cents) or `BigDecimal` from a **String** (`new BigDecimal("0.1")`) or `BigDecimal.valueOf(0.1)` — `new BigDecimal(0.1)` captures the binary error exactly, as shown.
3. **Rounding**: `Math.round` rounds half **up toward positive infinity** (`-2.5 → -2`, `-0.5 → 0`); `Math.floor` / `ceil` return `double`. For decimal rounding modes (half-even — round a trailing 5 to the nearest even digit, as banks do) use `BigDecimal.setScale`.
4. **Loops**: never compare a `double` loop counter with `==` — ten additions of 0.1 give 0.9999999999999999.
5. Since Java 17 (JEP 306) all floating-point arithmetic is strict IEEE 754 everywhere, so results are reproducible; `strictfp` is obsolete.

In the interview

Q: Why is `0.1 + 0.2 != 0.3`? Neither is exactly representable in binary; the approximations sum to 0.30000000000000004. Compare with a tolerance, or use `BigDecimal` for exact decimals such as money.

- *Check for NaN?* `Double.isNaN(x)`; `x != x` also works.
- *Why not* `new BigDecimal(0.1)`*?* It converts the already-inexact double; use the String constructor or `BigDecimal.valueOf`.

2.12 Strings and text blocks

What it is

A **`String`** is an object holding an immutable sequence of characters, with special support: literals (`"hi"`), `+` concatenation, and a pool of shared literals. Full API, `StringBuilder` and formatting: Chapter 7.

`code/StringBasics.java` :

```
String message = "Hello" + ", " + "Ada" + "!";
System.out.println(message + " length=" + message.length());
System.out.println("charAt(0)=" + message.charAt(0) + " indexOf(\"Ada\")=" + message.indexOf("Ada")
        + " substring(7, 10)=" + message.substring(7, 10));

String s = "java";
s.toUpperCase();                        // result thrown away: Strings are immutable
System.out.println("after s.toUpperCase(): " + s);
s = s.toUpperCase();                    // reassign the variable to the new String
System.out.println("after s = s.toUpperCase(): " + s);

String a = "hi", b = "hi", c = new String("hi");
System.out.println("a == b: " + (a == b) + " (same pooled literal), a == c: " + (a == c)
        + ", a.equals(c): " + a.equals(c) + ", \"HI\".equalsIgnoreCase(a): " + "HI".equalsIgnoreCase(a));
System.out.println("String.valueOf(42) + 1 = " + String.valueOf(42) + 1
        + ", Integer.parseInt(\"42\") + 1 = " + (Integer.parseInt("42") + 1));
System.out.println("'a' + 'b' = " + ('a' + 'b') + ", \"\" + 'a' + 'b' = " + ("" + 'a' + 'b'));
```

Output:

```
Hello, Ada! length=11
charAt(0)=H indexOf("Ada")=7 substring(7, 10)=Ada
upper=HELLO, ADA! contains Ada? true
after s.toUpperCase(): java
after s = s.toUpperCase(): JAVA
a == b: true (same pooled literal), a == c: false, a.equals(c): true, "HI".equalsIgnoreCase(a): true
compareTo: -1, isBlank: true, strip: [pad], repeat: ababab
String.valueOf(42) + 1 = 421, Integer.parseInt("42") + 1 = 43
'a' + 'b' = 195, "" + 'a' + 'b' = ab
null concatenated: null
```

Key facts

Fact	Consequence
Immutable	Methods return a *new* string — assign the result. Safe to share between threads and as `HashMap` keys.
String pool — identical literals share one object	`"hi" == "hi"` is `true`; `new String("hi")` is a distinct object. Use `equals`, never `==`.
Indexes are **0-based**; `substring(begin, end)` excludes `end`	`"Hello".substring(1, 3)` is `"el"`, `"abc".substring(1)` is `"bc"`; out of range throws `StringIndexOutOfBoundsException`.
`+` with any `String` operand concatenates	Values go through `String.valueOf`; `null` becomes `"null"`. Two `char`s add **numerically** (`'a' + 'b' == 195`).
Concatenating in a loop creates many objects	Use `StringBuilder` (mutable, growable string) in loops (Chapter 7).
Converting	`String.valueOf(x)` / `"" + x` to text; `Integer.parseInt`, `Double.parseDouble` from text (throw `NumberFormatException` on bad input).

Most-used methods (version for newer ones): `length`, `charAt`, `indexOf`, `substring`, `contains`, `startsWith` / `endsWith`, `toUpperCase` / `toLowerCase`, `trim`, `strip` (11), `isEmpty`, `isBlank` (11), `repeat` (11), `lines` (11), `replace`, `split`, `equals`, `equalsIgnoreCase`, `compareTo`, `String.join`, `formatted` (15).

Text blocks (Java 15)

A **text block** (JEP 378, final in Java 15) is a multi-line literal opened by `"""` plus a line break and closed by `"""`. It is an ordinary `String`.

`code/TextBlocks.java`:

```
String json = """
        {
          "name": "Ada",
          "langs": ["Java", "SQL"]
        }
        """;
System.out.print(json);

String sql = """
    SELECT id, name
    FROM users
    WHERE active = true\
    """;                                        // \ at line end: no newline there
System.out.println("[" + sql + "]");

String spaces = """
    trailing kept:\s
    quote: \""" inside
    """;
System.out.print(spaces.replace(' ', '.'));

String html = """
    <p>Hello, %s!</p>
    """.formatted("World");
```

Output:

```
{
  "name": "Ada",
  "langs": ["Java", "SQL"]
}
[SELECT id, name
FROM users
WHERE active = true]
trailing.kept:.
quote:."""".inside
<p>Hello, World!</p>
lines in json: 4
```

How the compiler processes it:

1. Content starts on the line **after** the opening `"""` (text on that line is a compile error).
2. **Incidental indentation** is removed: the smallest indentation among non-blank lines *and the closing `"""` line* is stripped from every line. Move the closing delimiter left to keep more indentation.
3. Trailing spaces are stripped; line endings become `\n` on every OS.
4. Escapes are processed last. Two new ones: `\<line-end>` joins lines (no newline), `\s` is a space that survives stripping.
5. Closing `"""` on its own line → the string ends with `\n`; directly after text → no trailing newline.

Use for embedded SQL, JSON, HTML and test fixtures. Indentation traps: OCP text blocks section.

Common mistakes

- `if (input == "yes")` compares identity; use `"yes".equals(input)` (also null-safe, unlike `input.equals("yes")`).
- `s.trim();` without assigning the result.
- `"Total: " + a + b` with numbers concatenates both (`"Total: 12"`); write `"Total: " + (a + b)`.

2.13 Console output and input

Output: `print`, `println`, `printf`

`System.out` is a `PrintStream` connected to **standard output** (terminal by default); `System.err` to **standard error**, a separate stream (shown in red by many IDEs) for diagnostics that should not mix with data

output.

code/Formatting.java :

```
System.out.print("print has no newline; ");
System.out.println("println adds one.");
System.out.printf("%-10s|%5d|%8.2f|%n", "Coffee", 3, 4.5);
System.out.printf("%-10s|%05d|%8.3e|%n", "Padded", 42, 12345.678);
System.out.printf("%s %b %c %x %o %%%n", "str", true, 'J', 255, 8);
System.out.printf(Locale.GERMANY, "German: %,.2f%n", 1234567.891);
System.out.printf(Locale.US, "US:     %,.2f%n", 1234567.891);
String line = String.format("Total: %.2f", qty * price);
String line2 = "Items: %d".formatted(qty);         // Java 15
System.err.println("(this line goes to standard error)");
```

Output:

```
print has no newline; println adds one.
Coffee    |    3|    4.50|
Padded    |00042|1.235e+04|
str true J ff 10 %
German: 1.234.567,89
US:     1,234,567.89
Total: 13.50 / Items: 3
(this line goes to standard error)
```

Specifier	Meaning	Flags/width examples
`%s`	any value via `toString()` (every object's text form)	`%-10s` left-align in 10 chars
`%d`	integer (`int` , `long` , `BigInteger`)	`%5d` width 5, `%05d` zero-pad, `%,d` grouping
`%f` , `%e`	decimal, scientific	`%.2f` two decimals (rounds half-up)
`%b` , `%c`	boolean, char	
`%x` , `%o`	hex, octal	`%08X`
`%n`	platform line separator (`\n` Linux/macOS, `\r\n` Windows)	prefer over `\n` in `printf`
`%%`	a literal `%`	

Formatting is **locale-sensitive**: the default locale (language-and-region setting) decides decimal and grouping separators, so tests and machine-readable output should pass an explicit `Locale` (`Locale.ROOT` , `Locale.US`). A mismatched type (`%d` with a `double`) throws `IllegalFormatConversionException` at run time.

Input: four options

Option	Best for	Notes
`Scanner` (`java.util`)	Beginner programs, parsing numbers and words	Splits input into whitespace-separated tokens; `nextInt` , `nextDouble` , `next` , `nextLine` , `hasNextInt` … Slow for very large input.
`IO.readln(prompt)` (`java.lang.IO` , **Java 25**)	Simplest line input	Returns a line (`null` at end of input); pairs with `IO.println` .
`BufferedReader` over `InputStreamReader(System.in)`	Fast line reading (competitive programming, large input)	`readLine()` returns `null` at end; parse numbers yourself; throws `IOException` .
`System.console()` → `java.io.Console`	Passwords (`readPassword` does not echo)	`null` when there is no interactive terminal (IDE consoles, input redirected from a file with `<`) — always check; a `null` in your IDE is not a bug.

One reader per program: each buffers (reads ahead from) `System.in` , so a `BufferedReader` created after a `Scanner` misses what the `Scanner` already consumed.

Deterministic `Scanner` demo reading from a `String` (code/ScannerDemo.java); a real program uses `new Scanner(System.in)` :

```
String fakeKeyboard = """
        Ada Lovelace
        36
        1.5 2.5 3.0
        done
        """;
Scanner in = new Scanner(fakeKeyboard);
String fullName = in.nextLine();          // whole line
int age = in.nextInt();                   // one token -> int
in.nextLine();                            // consume the rest of the "36" line!
double sum = 0;
while (in.hasNextDouble()) sum += in.nextDouble();
String word = in.next();                  // next token

Scanner trap = new Scanner("42\nAlan Turing\n");
int n = trap.nextInt();
String afterInt = trap.nextLine();        // returns the EMPTY rest of the first line

Scanner bad = new Scanner("forty-two");
if (!bad.hasNextInt()) System.out.println("not an int: " + bad.next());
```

Output:

```
name=Ada Lovelace, age=36, sum=7.0, word=done
n=42, nextLine after nextInt=[], then=[Alan Turing]
not an int: forty-two
```

The `nextInt` + `nextLine` trap: `nextInt()` leaves the line break, so the next `nextLine()` returns the empty remainder. Call an extra `nextLine()` after `nextInt()` , or read every line and `Integer.parseInt` it. `nextInt()` on non-numeric input throws `InputMismatchException` — guard with `hasNextInt()` .

code/Interactive.java takes the reading style as an argument (`java -cp out ch02.io.Interactive scanner|io|reader|console`). The `console` mode, run in a terminal:

```java
Console console = System.console();          // null if not an interactive terminal
if (console == null) {
    System.out.println("No interactive console (IDE or redirected input).");
    return;
}
char[] secret = console.readPassword("Password (not echoed): ");
if (secret == null) return;                  // end of input (Ctrl+D)
System.out.println("Password length: " + secret.length);
Arrays.fill(secret, ' ');                    // wipe it from memory
```

```
Password (not echoed):
Password length: 7
```

With redirected input (`… console < file.txt`) it prints `No interactive console (IDE or redirected input).` Passwords come as `char[]`, not `String`, so you can wipe them (a `String` lingers until garbage-collected). (JDK 22–24 `System.console()` could return a `Console` even when redirected; JDK 25 restored the older default, and JDK 26 made `null` the rule whenever input or output is redirected.)

`java.lang.IO` also works in ordinary classes (`code/IOClassDemo.java`):

```java
IO.print("IO.print, ");
IO.println("IO.println");
IO.println(42);
IO.println();
IO.println("done");
```

Output:

```
IO.print, IO.println
42

done
```

Command-line arguments (`String[] args`) are the other common input channel — Chapter 4.

Chapter TL;DR

- Eight primitives: `byte short int long` (8/16/32/64-bit signed), `float double` (IEEE 754), `char` (16-bit unsigned UTF-16 unit), `boolean`. Everything else is a reference (or `null`).
- Locals must be assigned before use; fields get defaults. `final` = assign once (the object can still change). `var` (Java 10) infers local types; the type is still fixed.
- Literals: `L` long, `f` float, `0x` / `0b` /leading-`0` hex/binary/octal, `_` separators.
- Widening is automatic (but `int/long → float` can lose precision); narrowing needs a cast and truncates/wraps. `byte/short/char` promote to `int`; compound assignment hides a cast.
- Integer `/` truncates; `%` follows the dividend's sign (`Math.floorMod` otherwise); integer `/0` throws, floating `/0` gives Infinity/NaN.
- `&&` / `||` short-circuit; `&` / `|` do not. `==` on references compares identity — use `equals`.
- Precedence: unary → `* / %` → `+ -` → shifts → relational → equality → `&` → `^` → `|` → `&&` → `||` → `?:` → assignment.
- Overflow wraps silently: `long`, `Math.*Exact`, `BigInteger`. Floating point is approximate: tolerance compares; money in `BigDecimal` /cents.
- `String` is immutable and pooled; text blocks (Java 15) strip incidental indentation.
- Output with `println` / `printf` (locale-aware); input with `Scanner` (mind `nextLine`), `IO.readln` (Java 25), `BufferedReader`, or `Console` for passwords — one reader per program.
- Exam traps: OCP Chapter 1, OCP Chapter 2. Next: Chapter 3 — Control flow.

Chapter 3 — Control Flow

Statements run top to bottom, once each, unless **control flow** changes that: *choosing* a path (`if` , `switch`), *repeating* (`for` , `while` , `do`) or *jumping* (`break` , `continue` , `return`). Modern Java (16–21) adds **pattern matching**: one construct tests a value's shape and extracts its data, turning `if` /cast chains (a cast is an explicit conversion, `(Type) value`) into short, compiler-checked `switch` expressions.

Code: `code/` , compiled with `javac --release 25` , output from real runs. Records (compact data-carrier classes) and sealed interfaces (an *interface* declares methods implementing classes must provide; a *sealed* one lists every type allowed to implement it) appear here only as data for patterns; they are taught in Chapter 6. Exam traps (accepted `switch` types, pattern-scope puzzles, labels, unreachable code): OCP Chapter 3 — Making Decisions.

3.1 `if` , `else if` , `else`

What it is

`if (condition) statement` runs the statement only when the **boolean** condition is `true` ; an optional `else` runs otherwise. An `else if` chain tests conditions in order and runs only the **first** matching branch.

The condition must be `boolean` (or `Boolean`). Unlike C or JavaScript, `if (x)` with an `int x` is a compile error: *incompatible types: int cannot be converted to boolean*.

`code/IfElse.java` :

```
static String grade(int score) {
    if (score < 0 || score > 100) {
        return "invalid";
    } else if (score >= 90) {
        return "A";
    } else if (score >= 80) {
        return "B";
    } else if (score >= 70) {
        return "C";
    } else {
        return "F";
    }
}

// Dangling else: the else belongs to the NEAREST if, whatever the indentation says.
int x = 5, y = -1;
if (x > 0)
    if (y > 0)
        System.out.println("both positive");
else
    System.out.println("printed! this else pairs with 'if (y > 0)'");
```

Output:

```
95 -> A
85 -> B
72 -> C
40 -> F
101 -> invalid
Beach day
printed! this else pairs with 'if (y > 0)'
minor
```

Because the chain stops at the first match, test narrow or invalid cases first (`score >= 90` before `score >= 80`).

Common mistakes

- **Dangling else** (above): Java ignores indentation; an `else` attaches to the closest unmatched `if`. Brace every nested `if`.
- Omitting braces, then adding a second line you think is inside the `if`:

```
if (loggedIn)
    showDashboard();
    audit();          // ALWAYS runs — only the first statement is controlled by the if
```

- `if (flag = true)` — assignment, always true for `boolean`. Write `if (flag)`.
- Comparing strings with `==` (Chapter 2).
- Deep nesting — prefer **guard clauses** (return early on invalid input so the main logic stays flat).

3.2 The classic `switch` statement

What it is

A `switch` compares one value (the **selector**) against constant **case labels** and jumps to the match — replacing an `if (x == 1) … else if (x == 2) …` chain on one variable.

How it works (colon form, since Java 1.0)

1. Evaluate the selector once.
2. Jump to the `case` whose constant equals it, else to `default` (no `default` → nothing runs).
3. Execute **downwards** until `break` (or `return` / `throw`) or the end. Running into the next case's statements is **fall-through**.

`code/SwitchStatement.java`:

```
int month = 4;
int days;
switch (month) {
    case 2:
        days = 28;
        break;
    case 4: case 6: case 9: case 11:        // several labels share one body
        days = 30;
        break;
    default:
        days = 31;
}

int level = 2;
switch (level) {
    case 1:
        System.out.println("  unlock level 1 perks");
    case 2:
        System.out.println("  unlock level 2 perks");
    case 3:
        System.out.println("  unlock level 3 perks");   // runs too: no break above
        break;
    case 4:
        System.out.println("  not reached");
}
```

Output:

```
month 4 has 30 days
stopping
Fall-through without break (level 2):
  unlock level 2 perks
  unlock level 3 perks
SAT is weekend
b is a consonant
```

What can you switch on?

Selector type	Since	Case labels
`char`, `byte`, `short`, `int` and wrappers (`Integer` …)	Java 1.0 / 5	compile-time constants that fit the type (`'a'`, `42`, `final` constants)
`enum` (fixed set of named constants)	Java 5	constant names (`case SAT`); qualified `case Day.SAT` also since Java 21
`String`	Java 7	string literals; compared with `equals` (case-sensitive) after a `hashCode` jump
Any reference type with patterns	Java 21	type/record patterns, `null` (3.6)
`long`, `float`, `double`, `boolean`	not allowed	primitive patterns are only a *preview* (opt-in trial; JEP 507 in Java 25, still preview in 27)

Labels must be unique. A `null` `String`/boxed selector throws `NullPointerException` (Chapter 8) unless there is a `case null` (Java 21).

Intentional fall-through (shared code, cumulative steps like the perks) is rare; a forgotten `break` is a common bug — hence the arrow form. Comment deliberate fall-through (`// fall through`).

In the interview

Q: What is fall-through? In colon-style `switch`, execution continues from the matched label through following labels until a `break`. Arrow labels (`case X ->`) never fall through.

- *Switch on a `String`?* Yes (Java 7); `hashCode` (number computed from the characters) narrows candidates, then `equals` confirms.
- *`null` selector?* `NullPointerException`, unless `case null` is present (Java 21+).

3.3 Arrow labels and `switch` expressions (Java 14)

What it is

Java 14 (JEP 361) finalised:

- **Arrow labels** `case A, B -> …` — several constants per label; the right side is one expression, block `{ }` or `throw`; **no fall-through**. Usable in `switch` statements too.
- `switch` **expressions** — a `switch` that **produces a value**: `int x = switch (…) { … };`, `return switch (…) { … };`.

Why it matters

Computing a value with a classic `switch` needs a pre-declared variable, an assignment and `break` in every case, and a `default` to satisfy definite assignment — each a chance for a bug. The expression form removes them, and the compiler checks **every possible value is handled** (*exhaustiveness*).

How it works

1. `case … -> expression;` — the expression's value is the switch's value.
2. `case … -> { … }` — the block must end with `yield value;` (or throw). `yield` gives the switch's result; `return` would leave the whole method (and is a compile error here).

3. The expression must be **exhaustive**: `default` for `int` / `String`; all constants for an `enum`; all permitted subtypes for a sealed type (3.6).

code/SwitchExpression.java:

```
enum Size { S, M, L, XL }

static int priceCents(Size size) {
    return switch (size) {                      // no default needed: all enum constants covered
        case S -> 250;
        case M -> 300;
        case L, XL -> 350;
    };
}

static String describe(int code) {
    return switch (code) {
        case 200, 201, 204 -> "success";
        case 301, 302 -> "redirect";
        case 404 -> "not found";
        default -> {
            String family = code >= 500 ? "server error" : "other";
            yield family + " (" + code + ")";    // yield returns a value from a block
        }
    };
}

// yield also works with old-style colon labels inside a switch expression
String kind = switch (letters % 2) {
    case 0:
        yield "even";
    default:
        yield "odd";
};
```

Output:

```
S=250 M=300 L=350 XL=350
200 -> success
302 -> redirect
404 -> not found
503 -> server error (503)
418 -> other (418)
WEDNESDAY has 9 letters
9 is odd
```

A missing enum constant fails compilation:

```
B.java:4: error: the switch expression does not cover all possible input values
        return switch (s) {
               ^
```

Add `XXL` to `Size` later and the compiler flags every `switch` that must handle it. (If a switch compiled against the old enum meets the new constant at run time, the JVM throws `MatchException`.)

Statement vs expression, colon vs arrow

	Colon labels `case X:`	Arrow labels `case X ->`
Statement	Classic; fall-through; `break` needed	No fall-through; one expression/block/throw per label
Expression	Allowed; must `yield`; fall-through possible	The usual modern form
Exhaustive?	Statement: only if it uses patterns or `null` (Java 21 rule). Expression: **always**	same

Colon and arrow labels cannot be mixed in one `switch`.

Common mistakes

- Missing `;` after a switch expression used in an assignment (`};`).
- `return` instead of `yield` inside a switch-expression block.
- Adding `default` to an enum switch expression "just in case": it silences the exhaustiveness check that would catch a new constant.

3.4 Choosing between `if`, ternary and `switch`

Situation	Best choice	Why
One condition, two actions	`if` / `else`	Clearest
One condition, pick one of two **values**	ternary `c ? a : b`	An expression; short
Ranges or unrelated conditions (`age < 13`, `isVip && total > 100`)	`if` / `else if` (or guarded patterns)	`switch` labels are constants/patterns, not ranges
One variable vs many constants	arrow `switch`	Readable, no fall-through
Compute a value from many cases	`switch` expression	Exhaustive, no temporary variable
Branch on an object's **type/shape**	pattern `switch` (Java 21)	Replaces `instanceof` chains; checked for sealed types
Per-subtype behaviour that grows over time	**Polymorphism** (each class supplies its own method version)	The OOP alternative; Chapter 6, LLD handbook

The last two rows are a real design choice (the *expression problem*): a pattern `switch` over a sealed hierarchy keeps an operation in one place, making new **operations** easy; virtual methods (version chosen at run time from the object's class) keep each type's code together, making new **types** easy.

3.5 Pattern matching for `instanceof` (Java 16)

What it is

A **pattern** combines a *test* ("is this a `String`?") with *binding* ("…if so, call it `s`"). `o instanceof String s` is a **type pattern** (JEP 394, Java 16): on success the **pattern variable** `s` is already cast. It replaces the old idiom that repeats the type three times and invites wrong casts:

```
if (o instanceof String) {
    String s = (String) o;
    ...
}
```

How it works — flow scoping

A pattern variable is in scope **only where the compiler can prove the match succeeded**:

1. Inside the `if` body for `o instanceof T t`.
2. On the right of `&&` (`o instanceof String s && !s.isEmpty()`).
3. **After** an `if` that tests the *negation* and exits (`if (!(o instanceof String s)) return 0; … s.length()`).
4. **Not** with `||` — `o instanceof String s || s.isEmpty()` fails with *cannot find symbol*, since the right side runs exactly when the match failed.

`code/InstanceofPattern.java`:

```java
static String newStyle(Object o) {
    if (o instanceof String s && !s.isEmpty()) {     // s is in scope on the right of &&
        return "String of length " + s.length();
    }
    if (o instanceof Integer i) return "Integer, doubled " + (i * 2);
    return "something else: " + o;
}

static int lengthOrZero(Object o) {
    if (!(o instanceof String s)) {
        return 0;                                    // leave early when it is NOT a String
    }
    return s.length();                               // flow scoping: s is definitely a String here
}

record Money(long cents, String currency) {
    @Override public boolean equals(Object other) {  // a classic use: equals()
        return other instanceof Money m && cents == m.cents && currency.equals(m.currency);
    }
}
```

Output:

```
String of length 5
String of length 5 | Integer, doubled 42 | something else:  | something else: null
lengthOrZero: 3, 0
true
```

- `null instanceof X` is always `false`, so a pattern never binds `null`.
- The pattern variable is an ordinary local, not `final` unless declared so.
- An *unconditional* pattern (testing a `String` variable with `instanceof String s`) was an error in Java 16–20; allowed since 21.

Typical uses: `equals(Object)` (above), mixed-type JSON nodes (`if (node instanceof ObjectNode obj)`), inspecting exception causes (`if (e.getCause() instanceof SQLException sql)` ; Chapter 8).

3.6 Pattern matching for `switch` (Java 21)

What it is

Java 21 (JEP 441) allows **patterns** in `case` labels: one `switch` can test any object's type, bind variables, add conditions and handle `null`.

How it works

1. **Type pattern** — `case Integer i ->` matches an `Integer` and binds `i`.
2. **Guard** — `case Integer i when i < 0 ->` matches only if the pattern matches **and** the condition holds (`when` is a contextual keyword).
3. **`case null`** — without it a `null` selector throws `NullPointerException` (backward compatible). `case null, default ->` combines both.
4. **Dominance** — cases are tried top to bottom; a label already fully matched by an earlier one is a compile error (e.g. `CharSequence cs` before `String s`). Put **specific before general**, guarded before unguarded of the same type.
5. **Exhaustiveness** — a pattern `switch` (expression *or* statement) must cover every value: `default`, or all permitted subtypes of a **sealed** type.

`code/SwitchPatterns.java` :

```java
sealed interface Shape permits Circle, Square, Rectangle {}
record Circle(double radius) implements Shape {}
record Square(double side) implements Shape {}
record Rectangle(double width, double height) implements Shape {}

static double area(Shape shape) {
    return switch (shape) {                          // exhaustive: every permitted subtype covered
        case Circle c -> Math.PI * c.radius() * c.radius();
        case Square s -> s.side() * s.side();
        case Rectangle r -> r.width() * r.height();
    };
}

static String classify(Object o) {
    return switch (o) {
        case null -> "null!";                        // without this case, switch throws NPE
        case Integer i when i < 0 -> "negative int " + i;   // guard
        case Integer i -> "int " + i;
        case String s when s.isBlank() -> "blank string";
        case String s -> "string of " + s.length();
        case int[] arr -> "int array of " + arr.length;
        case Shape sh -> "shape with area " + String.format("%.2f", area(sh));
        default -> "other: " + o.getClass().getSimpleName();
    };
}
```

Output:

```
negative int -7
int 42
blank string
string of 5
int array of 3
shape with area 3.14
shape with area 6.00
other: Double
null!
```

`Shape` is **sealed** (Java 17) — `permits` lists all implementations — so `area`'s three cases are complete without `default`. Add `Triangle` to `permits` and every such `switch` fails to compile until it handles it.

Compiler errors you will meet:

```
A.java:12: error: this case label is dominated by a preceding case label
            case String s -> "string";
                 ^
```

```
B.java:10: error: the switch statement does not cover all possible input values
        switch (o) {
        ^
```

Trade-offs

Approach	Pros	Cons
`if` / `instanceof` chain	Works on Java ≥ 16	Easy to miss a type; order-sensitive; verbose
Pattern `switch` + sealed types	Exhaustive, compact, value-producing	Every `switch` changes when a subtype is added (the point)
Polymorphic method	Adding a type touches one place	Adding an operation touches every class

For an open hierarchy (plug-ins, user subclasses) you cannot list cases — use `default` or polymorphism. For equality against constants, a constant `switch` is clearer.

In the interview

Q: What does pattern matching for `switch` give over `instanceof` chains? Test, cast and bind in one label; `when` guards; explicit `null`; compile-time exhaustiveness and dominance checks — strongest with sealed records, where a new subtype flags every switch to update.

- *Guard?* A `when` condition on a pattern label.
- *Why does order matter?* First match wins; a dominated label is a compile error — so `case String s when s.isEmpty()` goes before `case String s`.
- *`null`?* Only with `case null`; otherwise `NullPointerException`.
- *Must a pattern switch* statement *be exhaustive?* Yes (Java 21 rule for switches using patterns or `null`).

3.7 Record patterns (Java 21) and unnamed patterns (Java 22)

What it is

A **record** (Java 16; Chapter 6) is a compact data class: `record Point(int x, int y) {}` automatically gets a constructor, accessors `x()` / `y()`, `equals`, `hashCode` and `toString`. A **record pattern** (JEP 440, Java 21) matches a record **and deconstructs** its components in one step: `o instanceof Point(int x, int y)`.

The **unnamed pattern** `_` (JEP 456, Java 22) matches anything and binds nothing. `_` also names unused variables: `catch (Exception _)`, `for (var _ : list)`, lambda parameters `(_, v) -> v` (Chapter 11).

How it works

1. The value is tested against the record type (`null` never matches).
2. Each component is extracted via its accessor and matched against a nested pattern: a type (`int x`), `var x` (inferred), another record pattern, or `_`.
3. Works in `instanceof` and `switch`, with guards.

`code/RecordPatterns.java` — a tiny expression evaluator:

```
record Point(int x, int y) {}
sealed interface Expr permits Num, Add, Mul, Neg {}
record Num(int value) implements Expr {}
record Add(Expr left, Expr right) implements Expr {}
record Mul(Expr left, Expr right) implements Expr {}
record Neg(Expr operand) implements Expr {}

static int eval(Expr e) {
    return switch (e) {
        case Num(int v) -> v;
        case Add(Expr l, Expr r) -> eval(l) + eval(r);
        case Mul(Num(var v), _) when v == 0 -> 0;          // nested pattern + unnamed _ + guard
        case Mul(Expr l, Expr r) -> eval(l) * eval(r);
        case Neg(var inner) -> -eval(inner);
    };
}

Object o = new Point(3, 4);
if (o instanceof Point(int x, int y)) {              // deconstruct in one step
    System.out.println("x=" + x + " y=" + y + " distance=" + Math.sqrt(x * x + y * y));
}
if (o instanceof Point(var x, _)) System.out.println("only care about x: " + x);
```

Output:

```
x=3 y=4 distance=5.0
only care about x: 3
(2 + 3 * -4) = -10
0 * (1 + 2) = 0
```

`Mul(Num(var v), _) when v == 0` short-circuits multiplication by zero without evaluating the right side. Sealed interface + records + exhaustive pattern switch is how modern Java models **algebraic data types** (a type defined as a fixed set of alternative shapes, each with its own data): compiler ASTs (abstract syntax trees), JSON values, commands/events, `Success | Failure` results.

Still in preview

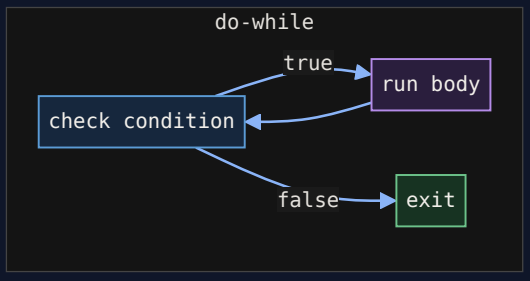
Primitive types in patterns, `instanceof` **and** `switch` (e.g. switching on a `long`, `case int i` matching a `double` that fits) is preview (JEP 507 in Java 25; still preview in 26 and 27). Without `--enable-preview`, switching on a `long` fails: *primitive patterns are a preview feature and are disabled by default*.

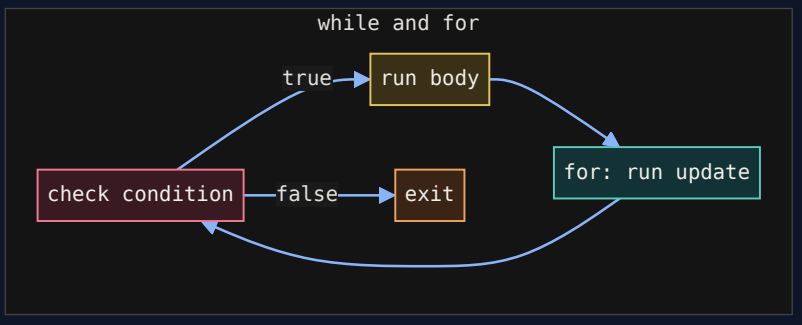
3.8 Loops

What it is

A **loop** repeats a body while a condition holds, or once per element of a collection; each run is an **iteration**.

Loop	Checks condition	Runs at least once?	Typical use
`while (cond) body`	before each iteration	no	unknown count: read until end of input, wait for a state
`do body while (cond);`	after each iteration	**yes**	menus, retry-until-success, "ask until valid"
`for (init; cond; update) body`	before each iteration	no	counting, index access
`for (T x : iterable) body` (enhanced for, Java 5)	—	no	every element of an array or `Iterable`





code/Loops.java :

```java
int n = 9075, digitSum = 0;
while (n > 0) {                         // condition checked BEFORE each iteration
    digitSum += n % 10;
    n /= 10;
}

int attempts = 0;
int[] fakeRolls = {2, 5, 6, 3};
int roll;
do {                                    // body runs at least once; condition checked AFTER
    roll = fakeRolls[attempts++];
    System.out.print("rolled " + roll + "; ");
} while (roll != 6);

long factorial = 1;
for (int i = 1; i <= 10; i++) {         // init; condition; update
    factorial *= i;
}

for (int i = 0, j = 10; i < j; i += 3, j -= 3) {   // several variables of one type
    System.out.print("(" + i + "," + j + ") ");
}
for (int i = 10; i > 0; i /= 2) System.out.print(i + " ");   // any update expression

int[] temps = {21, 25, 19, 30};
int max = Integer.MIN_VALUE;
for (int t : temps) max = Math.max(max, t);   // enhanced for over an array

Map<String, Integer> stock = new TreeMap<>(Map.of("apple", 3, "pear", 0, "fig", 7));
for (Map.Entry<String, Integer> e : stock.entrySet()) {
    System.out.print(e.getKey() + ":" + e.getValue() + " ");
}
```

Output:

```
digit sum of 9075 = 21
rolled 2; rolled 5; rolled 6; got a 6 after 3 attempts
10! = 3628800
(0,10) (3,7)
10 5 2 1
max temp = 30
ADA GRACE LINUS
apple:3 fig:7 pear:0
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

The `for` loop in detail

1. **Initialisation** runs once; it may declare several variables *of one type* (`int i = 0, j = 10` — `int i = 0, long j = 0` does not compile), scoped to the loop.
2. **Condition** is checked before every iteration; omitted means `true` (`for (;;)` is infinite).
3. **Body** runs, then the **update** (comma-separated expressions allowed), then back to 2. E.g. `for (int i = 0; i < 10; i += 3)` runs 4 times (0, 3, 6, 9).

The enhanced `for` in detail

`for (T x : source)` works on **arrays** and any `Iterable` (all collections). It compiles to an index loop for arrays and an `Iterator` loop (`hasNext()` / `next()`) for collections. Limits:

- No index (use a classic `for` or `IntStream.range` — Chapter 12).
- For primitives the loop variable is a **copy**: assigning it does not change the array.
- No adding/removing elements of the collection being iterated (3.10).
- A `Map` is not `Iterable` — iterate `entrySet()`, `keySet()` or `values()`.

Loops vs streams

Loops that filter, transform and aggregate can be streams (Chapter 12): `Arrays.stream(temps).max()`. Loops stay better for early exits with complex state, index arithmetic, or peak performance on primitive arrays; streams read better for transformation pipelines.

In the interview

Q: `while` vs `do-while`? `while` tests first and may run zero times; `do-while` tests after, so it runs at least once — good for "prompt until valid".

- *When the enhanced `for`?* When you need only elements, not indexes, and don't change the collection's structure.
- *How is it compiled for a `List`?* Into an `Iterator` loop.

3.9 `break`, `continue` and labels

What it is

- `break` exits the innermost loop or `switch` (inside a `switch` in a loop, it exits only the `switch`).
- `continue` skips to the next iteration (in a `for`, the update still runs).
- A **label** (`name:` before a loop) lets `break name;` / `continue name;` target an **outer** loop. `return` exits the whole method — often the cleanest way out of nested loops.

code/Branching.java:

```
for (int v : data) {
    if (v < 0) {
        System.out.println("found a negative, stopping");
        break;                                  // leave the loop entirely
    }
    System.out.println("processing " + v);
}

for (int i = 1; i <= 10; i++) {
    if (i % 3 != 0) continue;                   // skip the rest of this iteration
    System.out.print(i + " ");
}

search:                                         // a label names the outer loop
for (int r = 0; r < grid.length; r++) {
    for (int c = 0; c < grid[r].length; c++) {
        if (grid[r][c] == target) {
            foundRow = r;
            foundCol = c;
            break search;                       // exits BOTH loops
        }
    }
}

outer:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == i) continue outer;             // jump to the next i
        System.out.print(i + "" + j + " ");
    }
}

while (true) {                                   // loop-and-a-half: exit in the middle
    String line = lines[idx++];
    if (line.isEmpty()) break;
    System.out.print(line + " ");
}
```

Output:

```
processing 4
processing 8
found a negative, stopping
3 6 9 <- multiples of 3
42 found at (1,1)
21 31 32 <- pairs with j < i
alpha beta <- read until blank line
```

Trade-offs

Technique	Good for	Watch out
`break` / `continue`	Early exit, skipping invalid items	Many in one loop hide the flow
Labelled `break`	Leaving nested loops (grid search)	Often clearer to extract a method and `return`
`while (true)` + `break`	"Loop-and-a-half": read, test, process	Every path must eventually break
`return`	Leaving everything once the answer is found	—

Java has no `goto` (reserved, unused); labels work only with `break` / `continue` on enclosing statements. Code directly after an unconditional `break` / `continue` / `return` in the same block is **unreachable** — a compile error.

3.10 Loop pitfalls

code/LoopPitfalls.java — each compiles fine:

```
int[] a = {10, 20, 30};
for (int i = 0; i <= a.length; i++) System.out.print(a[i] + " ");   // off-by-one: <= should be <

List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
for (Integer x : nums) if (x % 2 == 0) nums.remove(x);   // structural change during for-each

for (Iterator<Integer> it = nums.iterator(); it.hasNext(); ) {
    if (it.next() % 2 == 0) it.remove();                      // fix 1: Iterator.remove
}
more.removeIf(x -> x % 2 == 0);                               // fix 2: removeIf (Java 8)

for (double d = 0.0; d != 1.0 && count < 20; d += 0.1) count++; // d never equals 1.0 exactly

for (int v : values) v *= 10;                  // changes the copy, not the array

for (int i = 0; i < 5; i++); { total++; }      // stray ';' = empty loop body
```

Output:

```
10 20 30
off-by-one -> Index 3 out of bounds for length 3
remove in for-each -> ConcurrentModificationException
after Iterator.remove: [1, 3, 5]
after removeIf: [1, 3, 5]
double counter ran 20 times (guard stopped it)
values after 'v *= 10' in for-each: [1, 2, 3]
stray semicolon: block ran 1 time(s)
```

Pitfall	Why it happens	Fix
Off-by-one (`<=` with `length`)	Indexes run 0 … length−1	`i < a.length` ; prefer enhanced `for`
`ConcurrentModificationException`	`ArrayList` etc. are *fail-fast*: the iterator detects structural changes (adds/removes) made behind its back (Chapter 10). Best-effort only — removing the second-to-last element can end the loop silently with a wrong result.	`Iterator.remove()` , `removeIf` , or build a new list
Floating-point counter	0.1 is inexact, so `d` never equals 1.0 (Chapter 2)	Loop over an `int` , compute `i * 0.1`
Assigning the for-each variable	It is a copy	Index loop: `values[i] *= 10`
Stray semicolon after `for (…)` / `while (…)`	`;` is an empty statement that becomes the body	Always use braces; IDEs warn
Infinite loop	Condition never false (forgot `i++` ; `while (x != 10)` when `x` jumps 9 → 11)	Update the variable; prefer `<` / `>` over `!=`
Counter overflow	`for (int i = 0; i <= Integer.MAX_VALUE; i++)` never ends — `i` wraps negative	Use `<` or a `long`
Expensive condition	`get(i)` in an index loop over a `LinkedList` (chain of nodes, slow to index) is O(n²) — time grows with the square of the size	Enhanced `for` /iterator

---

Chapter TL;DR

- `if` / `else if` / `else` : first true branch wins; conditions must be `boolean` ; braces avoid dangling-else and "second statement" traps; prefer guard clauses.
- Classic `switch` (colon labels) falls through until `break` ; works on `int` -like types, wrappers, `enum` , `String` — never `long` / `float` / `double` / `boolean` (primitive patterns still preview).
- Arrow labels (Java 14) never fall through; **switch expressions** return a value, `yield` from blocks, and must be exhaustive.
- `instanceof` patterns (Java 16) test + cast + bind; the variable is in scope only where the match is certain (`&&` , negated early exit), not with `||` .
- Pattern `switch` (Java 21): type patterns, `when` guards, `case null` , dominance (specific first), exhaustiveness — sealed types need no `default` , and a new subtype breaks every incomplete switch at compile time.
- Record patterns (Java 21) deconstruct and nest, with `var` ; `_` (Java 22) ignores parts. Sealed interface + records + pattern switch = algebraic data types.
- Loops: `while` (0+), `do-while` (1+), `for` (counting), enhanced `for` (elements, no index, no structural changes).
- `break` exits, `continue` skips, labels target outer loops; a method + `return` is often cleaner.
- Classic bugs: off-by-one, `ConcurrentModificationException` , floating counters, assigning the for-each variable, stray `;` .
- Exam traps: OCP Chapter 3. Next: Chapter 4 — Methods and arrays.

Chapter 4 — Methods and Arrays

Real programs are built from small named pieces of behaviour — **methods** — that take inputs, do one job and return a result, and they handle many values at once with Java's most basic container, the **array**. This chapter covers both, plus where they meet: passing arrays to methods, and the `String[] args` every classic `main` receives.

Code: `code/` (package `ch04.*`), compiled with `javac --release 25`, run on JDK 26. Exam traps on declarations, access modifiers, varargs, overloading and pass-by-value: OCP Chapter 5 — Methods.

4.1 Methods — declaring and calling

What it is

A **method** is a named block of code belonging to a class (Chapter 5). It takes zero or more inputs (**parameters**), runs its body, and may hand back one **return value**. Other languages say *function*; in Java every function lives in a class.

Why: **reuse** (write once, call anywhere), **naming** (`isEven(n)` explains itself), **testing** (small methods with clear inputs/outputs are easy to test, Chapter 18), **change** (fix a bug in one place).

How it works — anatomy of a declaration

```
public static double area(double width, double height) throws IllegalArgumentException {
//  ^       ^     ^      ^    ^-----------------------^   ^-----------------------^
//  |       |     |      |    parameter list                optional throws clause
//  |       |     |      method name
//  |       |     return type (void = nothing)
//  |       optional specifier(s): static, final, abstract, synchronized ...
//  access modifier (public / protected / private / none)
    return width * height;       // method body
}
```

Part	Required?	Notes
Access modifier	No	`public`, `protected`, `private`, or none (= package-private, same package only). Chapter 5.
Specifiers	No	`static` (belongs to the class), `final` (cannot be overridden), `abstract` (no body; subclasses supply one), `synchronized` (one thread at a time), `native` (implemented in C etc.), `strictfp` (no effect since Java 17).
Return type	**Yes**	Any type, or `void`. Only constructors (which set up new objects) have none.
Name	**Yes**	A lowerCamelCase verb: `calculateTotal`, `isEmpty`.
Parameter list	**Yes** (parentheses)	May be empty `()`; each parameter has a type and name.
`throws` clause	No	Checked exceptions (error types callers are forced to handle) the method may throw (Chapter 8).
Body	Unless `abstract` / `native`	The code in `{ }`.

The **method signature** is the *name plus the ordered parameter types* — `area(double, double)`. Return type, parameter names and modifiers are **not** part of it, and no two methods in a class may share a signature (§4.3).

When a method is called:

1. The caller evaluates the arguments left to right.

2. The JVM creates a new **stack frame** — a private workspace for the method's parameters and locals.
3. Each argument value is **copied** into its parameter.
4. The body runs until `return` or the closing brace.
5. The frame is discarded; the return value replaces the call expression.

A *parameter* is the declared variable (`width`); an *argument* is the value passed at the call site (`3`).

`return` and `void`

- A non-`void` method must `return` a compatible value on **every** path ("missing return statement").
- A `void` method returns nothing; a bare `return;` leaves early.
- The caller may ignore a return value (`area(1, 1);` is legal).
- One return value only: for several, return a record (Chapter 6) — e.g. `record MinMax(int min, int max)` — or an array of same-typed values.

Static vs instance methods

- A `static` **method** belongs to the class: `ClassName.method()` (or `method()` inside the class). It has no `this` (current object), so it cannot touch instance fields directly. Utilities: `Math.max`, `Integer.parseInt`.
- An **instance method** runs *on an object* and reads/writes its fields: `counter.countCall()`, `list.add`. Full treatment: §5.5.

```java
// code/ch04/methods/MethodBasics.java (key parts)
static double area(double width, double height) {
    return width * height;
}
static void greet(String name) {
    if (name == null || name.isBlank()) {
        System.out.println("Hello, stranger");
        return;                          // early exit from a void method
    }
    System.out.println("Hello, " + name);
}
static boolean isEven(int n) { return n % 2 == 0; }
static int max(int a, int b) { return a >= b ? a : b; }

private int calls;                       // instance field
int countCall() { return ++calls; }      // instance method

public static void main(String[] args) {
    double a = area(3, 4.5);             // int argument 3 is widened to double
    System.out.println("area = " + a);
    greet("Ada");
    greet("  ");
    System.out.println("isEven(7) = " + isEven(7) + ", max(3, 9) = " + max(3, 9));
    area(1, 1);                          // result ignored: legal
    MethodBasics counter = new MethodBasics();
    counter.countCall();
    System.out.println("calls = " + counter.countCall());
}
```

Output:

```
area = 13.5
Hello, Ada
Hello, stranger
isEven(7) = false, max(3, 9) = 9
calls = 2
```

Local variables and scope

- Variables declared in a method (including parameters) are **local**: they live while the method runs and are visible only in their block.

- Locals have **no default value** (reading one first is a compile error); fields and array elements do get defaults.
- Each call gets fresh locals — which makes recursion (§4.5) work.
- `final` on a parameter (`void log(final String msg)`) stops reassignment; documentation, not a performance trick.

Common mistakes

- **Discarding the result**: `name.toUpperCase();` alone does nothing (strings are immutable). Write `name = name.toUpperCase();` .
- **Calling an instance method from** `static main` without an object: "non-static method cannot be referenced from a static context". Create an object or make the helper `static` .
- **Methods that do too much** (`validateAndSaveAndEmail`) — split them (Clean code).
- **Boolean flag parameters** (`print(report, true, false)`) — prefer two named methods or an enum.

Frameworks such as Spring and JUnit discover your methods by name and annotations (`@Test`) and call them for you.

TL;DR

Method = modifiers + return type + name + parameters + body; signature = name + parameter types. Non-void methods return on every path. Static methods belong to the class; instance methods run on an object. Locals live in the stack frame and must be initialised before use.

4.2 Parameters — Java is always pass-by-value

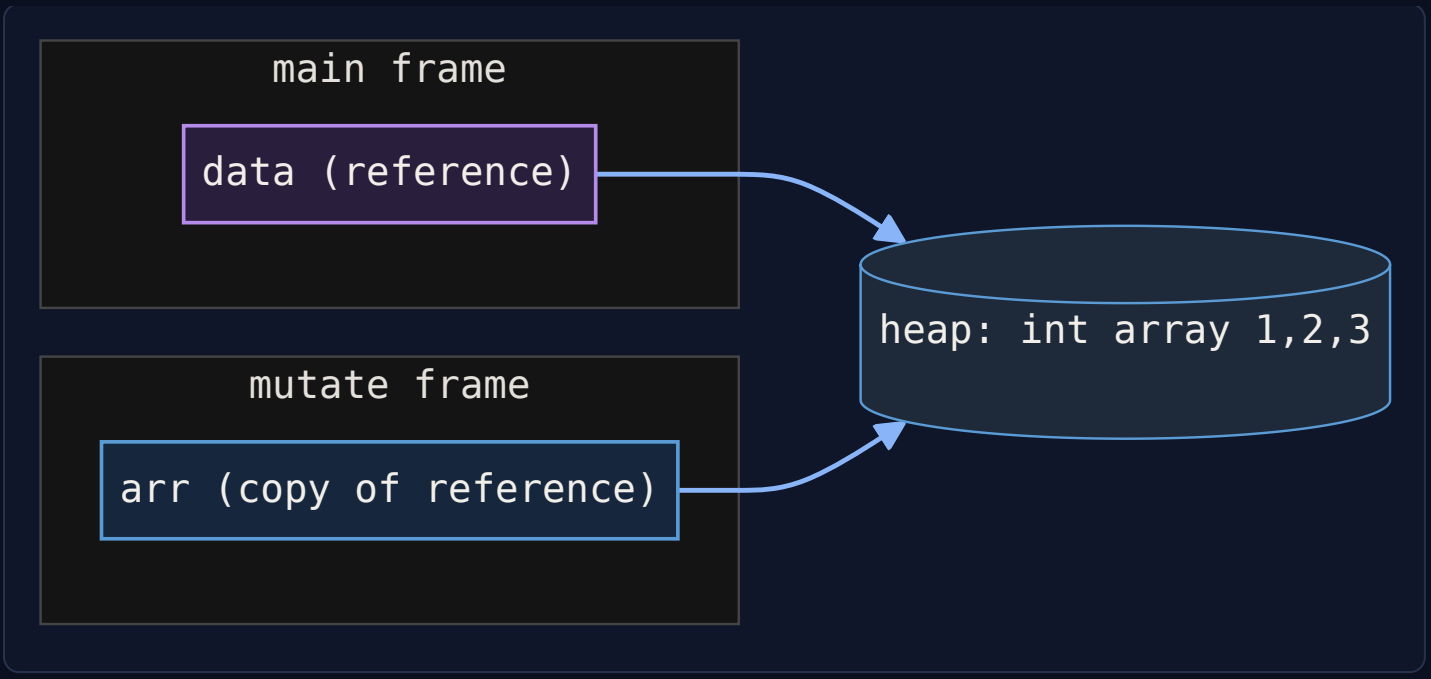
What it is

Java **copies each argument's value into the parameter** — *pass-by-value*, always; there is no pass-by-reference. What differs is what the "value" is:

Variable type	Holds	What gets copied
Primitive (`int` , `double` , `boolean` …)	The value itself	The value — an independent copy; changes are invisible to the caller
Reference (`int[]` , `String` , `List` , any object)	A **reference** (address of an object on the heap)	The reference — the copy still points at the *same object*

So through a copied reference:

- **Mutating** the object (`arr[0] = 99` , `list.add(x)`) is visible to the caller — there is one object.
- **Reassigning** the parameter (`arr = new int[]{…}` , `a = null`) changes only the local copy.




```java
// code/ch04/methods/PassByValue.java (key parts)
static void addOne(int x) { x = x + 1; }                  // changes the local copy only
static void reassign(int[] arr) { arr = new int[] {99}; } // re-points the local copy only
static void mutate(int[] arr) { arr[0] = 99; }            // same array as the caller's
static void appendTo(List<String> list) { list.add("added"); }
static void swap(String a, String b) { String t = a; a = b; b = t; }  // cannot work in Java
```

Output:

```
n after addOne:      5
data after reassign: [1, 2, 3]
data after mutate:   [99, 2, 3]
names after append:  [a, added]
after swap:          left right
```

A C-style `swap(a, b)` is impossible: the method swaps only its copies. Swap elements *inside* an array/object, or return the result.

Why it matters

- A method can sort or fill your array (`Arrays.sort(a)` sorts in place) but never make your variable point elsewhere.
- It is the root of **defensive copying**: a method that stores a list you passed sees your later changes (§5.9).
- Immutable types (`String`, `Integer`, records with immutable fields) are safe to pass — nobody *can* mutate them.

Common mistakes

- Saying "objects are passed by reference" — *references are passed by value*.
- Expecting `s = s.trim()` in a method to change the caller's string — return the new value.
- Returning `null` for "no results" from an array/list method — return an empty one (`new int[0]`, `List.of()`) so callers can loop safely (*Effective Java* — Joshua Bloch's Java best-practice book — Item 54).

In the interview

30-second answer: "Java is strictly pass-by-value. For primitives the value is the number; for objects it is a reference, so the method gets a copy of the reference: it can mutate the shared object, but reassigning the parameter never affects the caller. That's why swap can't work."

- *Is `String` special?* No — it only looks different because it is immutable, so nothing can be mutated through the reference.
- *Return two values?* A small record, or an array for same-typed values.

4.3 Overloading

What it is

Overloading = several methods with the **same name but different parameter lists** in one class (or inherited into it). The compiler picks one at **compile time** from the argument types — *static polymorphism*, unlike overriding (a subclass replacing an inherited method, resolved at run time — Chapter 6). `System.out.println` has ten overloads; `Math.max` has `int` / `long` / `float` / `double` versions.

Rules

Legal overload if parameter lists differ in…	Not an overload (compile error "already defined")
Count: `sum(int,int)` vs `sum(int,int,int)`	Only the return type differs (`int f()` vs `void f()`)
Types: `sum(int,int)` vs `sum(double,double)`	Only parameter *names* differ
Order: `f(int,String)` vs `f(String,int)`	Only modifiers / `throws` differ
	`f(int[])` vs `f(int...)` — same signature
	Generic types that erase to the same type, `f(List<String>)` vs `f(List<Integer>)` (Chapter 9)

How it works — how the compiler chooses

JLS §15.12.2 resolves a call in **three phases**, stopping at the first that finds an applicable method:

1. **Exact match, primitive widening or reference subtyping** — no boxing, no varargs. `byte → short → int → long → float → double`, `char → int`, `String → Object`.
2. **Boxing/unboxing** allowed (`int ↔ Integer`), still no varargs.
3. **Varargs** allowed.

Among several candidates in a phase the **most specific** wins (`int` over `long`, `String` over `Object`); if none is most specific, the call is **ambiguous** — a compile error. Memorise: *widening beats boxing, boxing beats varargs*.

```
// code/ch04/methods/Overloading.java (key parts)
static String describe(int x)      { return "int"; }
static String describe(long x)     { return "long"; }
static String describe(Integer x)  { return "Integer"; }
static String describe(Object x)   { return "Object"; }
static String describe(int... xs)  { return "int..."; }

static String pick(long x)    { return "long (widening)"; }
static String pick(Integer x) { return "Integer (boxing)"; }

static String box(Object o)   { return "Object"; }
static String box(double d)   { return "double"; }
```

Output:

```
describe(5)        -> int
describe(5L)       -> long
describe(b)        -> int
describe(c)        -> int
describe("hi")     -> Object
describe()         -> int...
describe(1, 2)     -> int...
pick(5)            -> long (widening)
box(5)             -> double
sum(1,2) sum(1,2,3) sum(1.5,2) -> 3 6 3.5
```

`describe(b)` (a `byte`) finds `int` in phase 1 (most specific widening). `pick(5)` takes `long` over `Integer`, and `box(5)` takes `double` over `Object`, because phase 1 succeeds before boxing is considered.

Trade-offs

Good use	Bad use
Same operation, different input types (`println(int)` / `println(String)`)	Same name, *different meaning* (`save(User)` saves; `save(String)` emails)
Defaults: `connect(host)` calls `connect(host, 443)` — Java has no default parameter values	Same-count overloads of related types (`f(Object)` vs `f(String)`) that surprise readers

Effective Java Item 52: avoid same-arity overloads a call could plausibly fit both; use different names (`writeInt`, `writeLong`).

Common mistakes

- Thinking a different return type is enough.
- `list.remove(1)` on a `List<Integer>` calls `remove(int index)` (phase 1, no boxing), not `remove(Object)`. Use `list.remove(Integer.valueOf(1))`.
- `null` with two reference overloads (`f(String)`, `f(Integer)`) is ambiguous; cast: `f((String) null)`.

TL;DR

Overloads share a name and differ in parameter types, count or order. Compile-time choice: exact/widening → boxing → varargs, then most specific.

4.4 Varargs

What it is

A **varargs** (variable-arity, Java 5) parameter `Type... name` accepts zero, one or many arguments; inside the method it is an **array** (`int... numbers` is an `int[]`).

How it works

1. The compiler collects the extra arguments into a new array.
2. No extra arguments → an **empty array** (length 0), never `null`.
3. An existing array of the right type is passed as-is.
4. At most **one** varargs parameter, and it must be **last**: `join(String sep, String... parts)` is legal, `f(String... a, int b)` is not.

```java
// code/ch04/methods/Varargs.java (key parts)
static int sum(int... numbers) {
    int total = 0;
    for (int n : numbers) total += n;
    return total;
}
static String join(String separator, String... parts) { return String.join(separator, parts); }
static void inspect(Object... items) { /* prints length and contents */ }

sum();  sum(4);  sum(1, 2, 3, 4);  sum(new int[] {10, 20});
inspect(1, "two");  inspect((Object) null);  inspect((Object[]) null);
```

Output:

```
0
4
10
30
a | b | c
cart has 3 items
inspect(1, "two"):
  length=2 -> [1, two]
inspect((Object) null):
  length=1 -> [null]
inspect((Object[]) null):
  length=null array -> null
```

The subtlety: with `Object...`, `(Object) null` is "one null argument", while `(Object[]) null` makes the whole array null. Code accepting varargs from outside should tolerate a `null` array.

Real examples: `String.format(String, Object...)`, `printf`, `List.of(E...)`, `Arrays.asList(T...)`, `Path.of(String, String...)`, SLF4J's `log.info("{} {}", a, b)`.

Trade-offs

- Each call allocates an array; hot JDK APIs add fixed-arity overloads for small cases — `List.of` has 0–10-element overloads plus varargs (*Effective Java* Item 53).
- For **at least one** argument, declare it: `min(int first, int... rest)` — the compiler enforces it.
- Generic varargs (`T...`) trigger "possible heap pollution" warnings (a generic variable ending up holding the wrong type); `@SafeVarargs` suppresses them for methods that don't store or expose the array (Chapter 9).

TL;DR

`T... x` = an array parameter the compiler fills. Last parameter only, one per method, empty array when no arguments.

4.5 Recursion

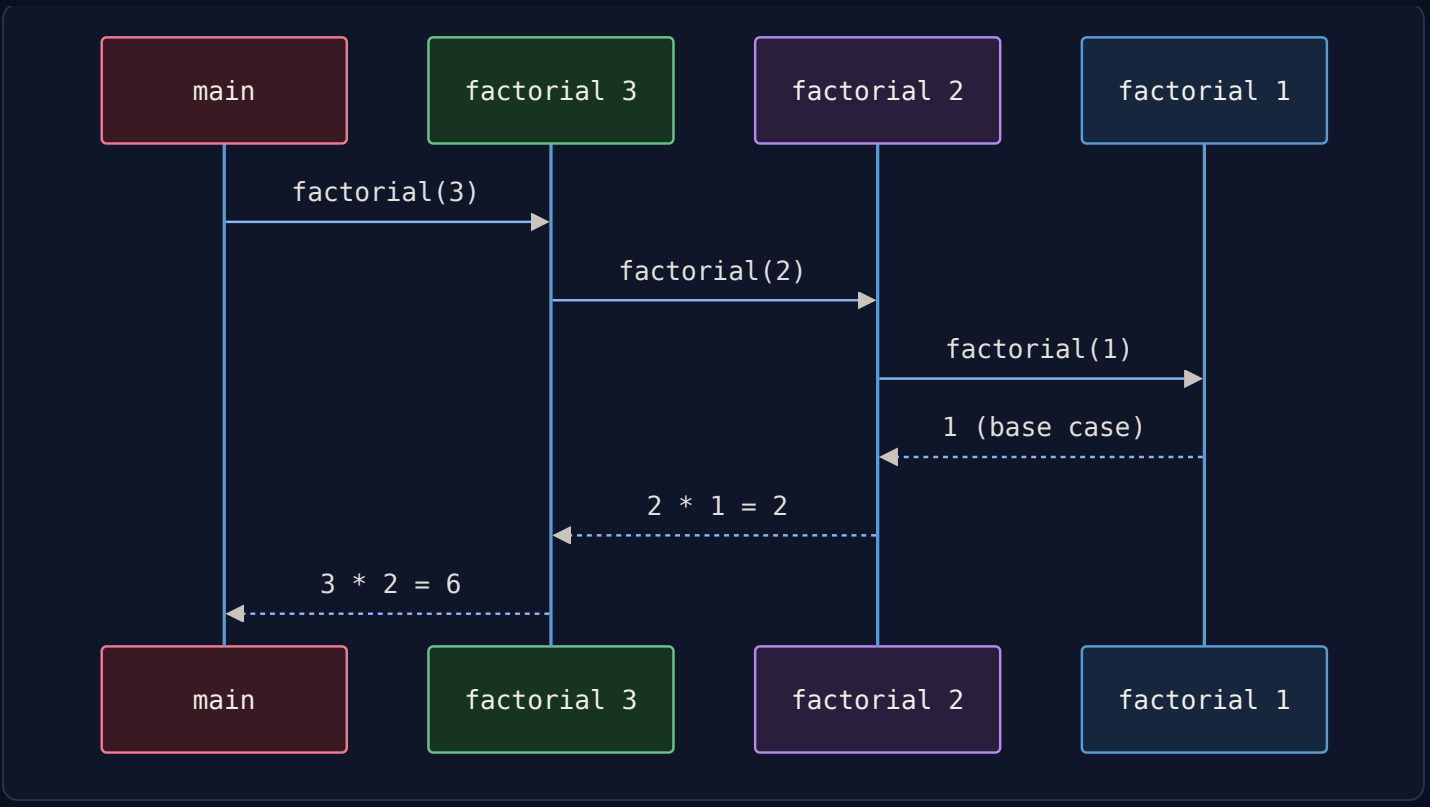
What it is

A method is **recursive** when it calls itself on a smaller version of the problem. It needs:

1. **A base case** answered directly (`factorial(0) = 1`).
2. **A recursive case** calling itself on a *smaller* input, moving towards the base case.

How it works — the call stack

Each call pushes a **stack frame** (own parameters and locals) onto the thread's **call stack** (a *thread* is one independent path of execution); returning pops it and the caller resumes.



The stack has a fixed size per thread (`-Xss`; platform-dependent default, typically about 1 MB on 64-bit). Missing base cases or very deep recursion throw `StackOverflowError`. Java does **not** do tail-call elimination (reusing the frame when the last action is a call), so even tail-recursive methods use one frame per call.

```java
// code/ch04/methods/Recursion.java (key parts)
static long factorial(int n) {
    if (n < 0) throw new IllegalArgumentException("n must be >= 0");
    if (n <= 1) return 1;                // base case
    return n * factorial(n - 1);         // recursive case
}
static long fibNaive(int n) { return n < 2 ? n : fibNaive(n - 1) + fibNaive(n - 2); }

static final Map<Integer, Long> memo = new HashMap<>();
static long fibMemo(int n) {             // memoization: cache each answer
    if (n < 2) return n;
    Long cached = memo.get(n);
    if (cached != null) return cached;
    long value = fibMemo(n - 1) + fibMemo(n - 2);
    memo.put(n, value);
    return value;
}
static long fibLoop(int n) {             // iterative: O(n) time, O(1) space
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) { long t = a + b; a = b; b = t; }
    return a;
}
static int binarySearch(int[] a, int key, int lo, int hi) {
    if (lo > hi) return -1;
    int mid = lo + (hi - lo) / 2;        // avoids int overflow of (lo + hi)
    if (a[mid] == key) return mid;
    return a[mid] < key ? binarySearch(a, key, mid + 1, hi)
                        : binarySearch(a, key, lo, mid - 1);
}
static int depth = 0;
static void dive() { depth++; dive(); }   // no base case
```

Output (the last line's depth varies by run, JVM, platform, `-Xss` and frame size):

```
5! = 120, 20! = 2432902008176640000
fibNaive(30) = 832040
fibMemo(90)  = 2880067194370816120
fibLoop(90)  = 2880067194370816120
index of 23 = 5
StackOverflowError after roughly 38000+ calls
```

- `20!` is the largest factorial fitting a `long`; `21!` overflows silently (Chapter 2) — use `BigInteger` (Chapter 7).
- `fibNaive(n)` makes ~1.6^n calls (about 2.7 million for n = 30) because subproblems repeat. **Memoization** (caching earlier results) cuts it to O(n) — the heart of *dynamic programming* (combining stored answers to overlapping subproblems).

Key term — Big-O notation: how an algorithm's time or memory grows with input size *n*, ignoring constants. O(1) constant; O(n) proportional; $O(n^2)$ square; O(log n) one extra step per doubling; $O(2^n)$ exponential.

Trade-offs — recursion vs iteration

	Recursion	Iteration (loops)
Fits naturally	Trees, nested structures, divide-and-conquer (split, solve, combine: merge sort, quicksort), backtracking (try, undo on dead end), grammars	Linear scans, counters, accumulations
Readability	Often shorter, mirrors the definition	May need an explicit stack for tree problems
Memory	One frame per active call; `StackOverflowError` risk	Constant stack
Speed	Call overhead; exponential if subproblems repeat without memoization	Usually fastest

Avoid recursion when depth grows with large inputs (a recursive walk of a million-node linked list overflows — use a loop or an explicit `ArrayDeque` stack), or when a loop is just as clear (factorial, array sum).

Common mistakes

- Missing/unreachable base case → `StackOverflowError`.
- A step that doesn't shrink the problem, or skips past the base case (`n - 2` jumping over `n == 1`).
- Catching `StackOverflowError` to "recover" — it is an `Error` (serious problems not meant to be caught), signalling a bug or unsuitable algorithm.

Real examples: `Files.walkFileTree` visits directory trees depth-first; JSON parsers are recursive-descent (one method per grammar rule); compilers walk syntax trees; `Arrays.sort` for objects uses TimSort, divide-and-conquer at heart.

In the interview

30-second answer: "A recursive method calls itself on a smaller input until a base case. Each call costs a stack frame and Java has no tail-call optimization, so very deep recursion throws StackOverflowError. For overlapping subproblems like Fibonacci I memoize, turning exponential time into linear."

- *Make deep recursion safe?* Rewrite iteratively with an `ArrayDeque` stack, or bound the depth; raising `-Xss` is a last resort.
- *Naive Fibonacci complexity?* $O(\varphi^n) \approx O(1.618^n)$, commonly quoted as $O(2^n)$; memoized O(n) time and space.
- *Why `lo + (hi - lo) / 2`?* `(lo + hi)` can overflow `int` for huge arrays.

4.6 Arrays

What it is

An **array** is a fixed-size, ordered container of same-typed elements in one contiguous memory block, accessed by an integer **index** from 0. An array is an **object**: it lives on the heap, array variables hold references, and it has a `length` field.

Why it matters

- **Speed**: `a[i]` is O(1) — the address is computed from the index.
- **Foundation**: `ArrayList`, `HashMap`, `ArrayDeque` and `String` (internally a byte array) are built on arrays.
- **No boxing**: `int[]` stores raw ints; `List<Integer>` stores objects. For large numeric data arrays are several times smaller and faster.

How it works

1. **Declare**: `int[] scores;` (`int scores[]` compiles but is discouraged).
2. **Create** with a size: `scores = new int[4];` — any runtime `int` expression, fixed afterwards.
3. Or **initialise**: `double[] prices = {9.99, 4.5, 12.0};` (declarations only) or `new char[] {'a','e','i'}` (anywhere).
4. New arrays hold **default values**: `0` for integer types, `0.0` for `float` / `double`, `'\u0000'` for `char`, `false` for `boolean`, `null` for references (`new String[3]` is three `null`s).
5. **Access** `a[i]` with `0 ≤ i < a.length`; anything else throws `ArrayIndexOutOfBoundsException` (bounds are checked on every access — no silent memory corruption as in C).
6. `length` is a field: `a.length` (vs `s.length()` for strings, `list.size()` for collections).

```java
// code/ch04/arrays/ArrayBasics.java (key parts)
int[] scores = new int[4];
String[] names = new String[2];
boolean[] flags = new boolean[2];
double[] prices = {9.99, 4.5, 12.0};
char[] vowels = new char[] {'a', 'e', 'i'};

scores[0] = 90; scores[3] = 75;
for (int i = 0; i < prices.length; i++) prices[i] *= 2;   // index loop: can modify
double total = 0;
for (double p : prices) total += p;                       // enhanced for: read each element

try { scores[4] = 1; }
catch (ArrayIndexOutOfBoundsException e) { System.out.println("error: " + e.getMessage()); }

int[] alias = scores;          // copies the REFERENCE: one array, two names
int[] copy = scores.clone();   // a NEW array with the same elements
alias[1] = 50;
```

Output:

```
defaults: [0, 0, 0, 0] [null, null] [false, false]
scores=[90, 0, 0, 75] length=4
last price = 12.0, vowels = aei
doubled total = 52.980000000000004
error: Index 4 out of bounds for length 4
scores=[90, 50, 0, 75] copy=[90, 0, 0, 75]
scores == alias? true, scores == copy? false, Arrays.equals(scores, copy)? false
println(array) prints an identity string, e.g. starts with: [I@
empty array length = 0
```

- `52.980000000000004` is binary floating-point rounding (Chapter 2).
- `println(a)` prints `[I@1b6d3586` — type code `[I` ("array of int") plus an identity hash. Use `Arrays.toString(a)` (`deepToString` for nested).
- `==` and `a.equals(b)` compare references; use `Arrays.equals` for contents.

- The enhanced `for` variable is a *copy*; modify through an index loop.

Copying arrays

Technique	Result	Notes
`b = a`	Same array, two names	No copy
`a.clone()`	New array, same length	**Shallow**: object elements are shared (after `b = a.clone(); b[0] = 9;`, `a[0]` is unchanged)
`Arrays.copyOf(a, n)`	New array of length `n`	Truncates or pads with defaults
`Arrays.copyOfRange(a, from, to)`	New array of `[from, to)`	`to` exclusive
`System.arraycopy(src, srcPos, dst, dstPos, len)`	Into an **existing** array	Fastest; handles overlapping ranges

Arrays are covariant (and why that can bite)

A `String[]` may be treated as an `Object[]`, but the array remembers its real element type and checks every store at run time:

```
// code/ch04/arrays/Covariance.java (key parts)
String[] strings = {"a", "b"};
Object[] objects = strings;          // allowed: arrays are covariant
objects[0] = 42;                     // compiles, fails at runtime
```

Output:

```
ArrayStoreException: java.lang.Integer
min=-2 max=17
[] is an empty array, never return null for 'no results'
```

Generic collections are *invariant* (`List<String>` is not a `List<Object>`), turning this runtime failure into a compile error — one reason *Effective Java* Item 28 prefers lists to arrays in APIs (Chapter 9).

Trade-offs — array vs `ArrayList`

	`Array ( int[] , String[] )`	`ArrayList<E>` (Chapter 10)
Size	Fixed at creation	Grows automatically
Primitives	Stored directly	Must box (`Integer`)
Syntax	`a[i]` , `a.length`	`list.get(i)` , `list.size()`
Methods	Few (use `Arrays.*`)	Many (`add` , `remove` , `contains` , `sort` …)
Type safety	Covariant, runtime store checks	Invariant generics, compile-time checks
`toString` / `equals`	Identity-based	Content-based
Best for	Fixed-size numeric data, performance-critical code, buffers, `main(String[])`	Almost everything else

Common mistakes

- Off-by-one `i <= a.length` ; printing with `println(a)` ; comparing with `==` / `equals` ; assuming `b = a` copies; returning `null` instead of `new int[0]` .

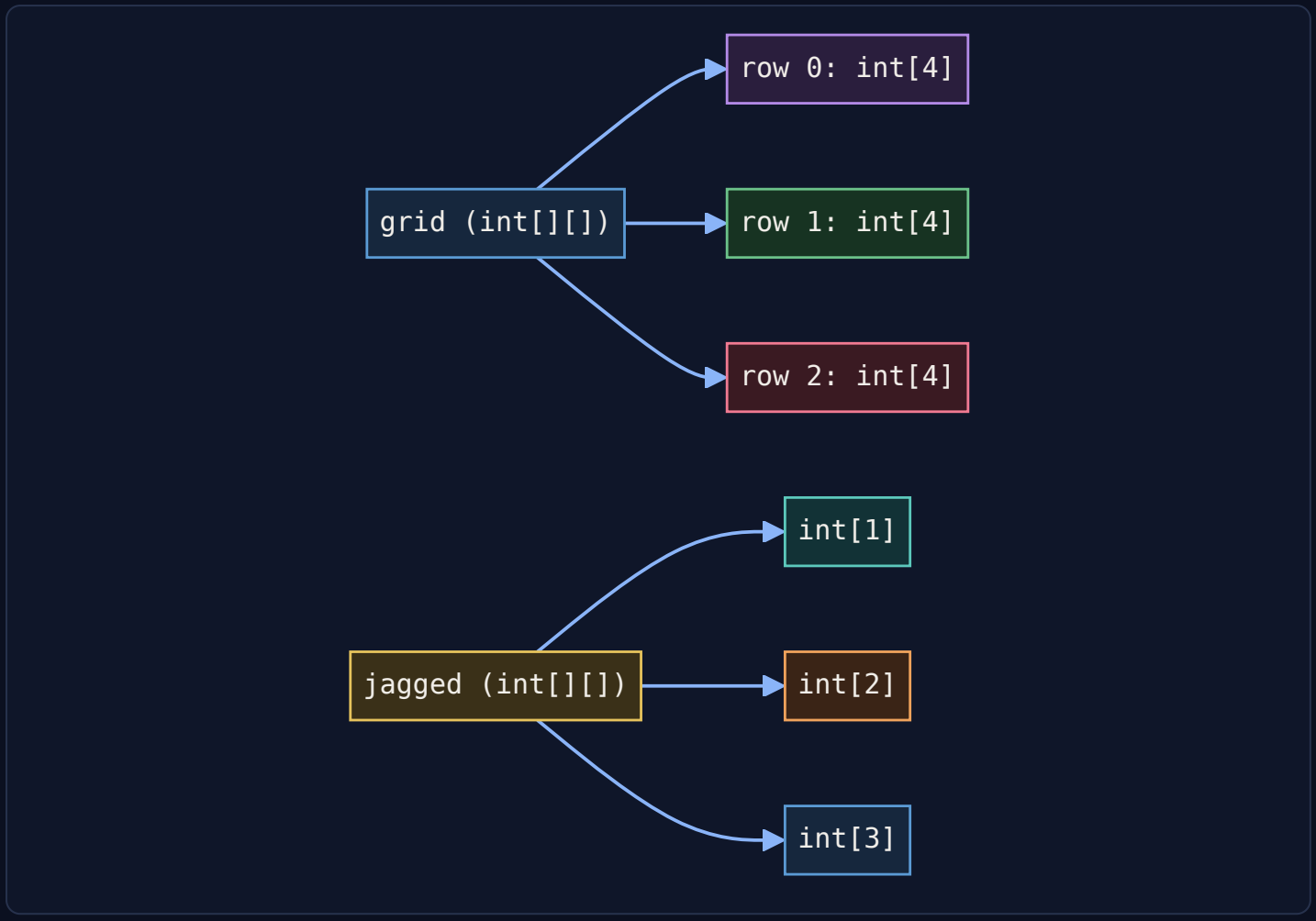
TL;DR

Arrays are fixed-size, zero-indexed, bounds-checked objects with a `length` field and default-filled elements. Assignment copies the reference; `clone` / `copyOf` / `arraycopy` copy; `Arrays.toString` / `Arrays.equals` print and compare.

4.7 Multidimensional and jagged arrays

What it is

Java has **arrays of arrays**, not true 2-D arrays: `int[][] grid` holds `int[]` references, each row its own object. Rows can differ in length — a **jagged** (ragged) array.



How it works

1. `new int[3][4]` creates an outer array of 3 references plus 3 inner `int[4]` rows.
2. `new int[3][]` creates only the outer array; rows are `null` until assigned (`jagged[0] = new int[1]`).
3. `grid.length` = rows; `grid[r].length` = length of row `r`. Access `grid[row][col]`.
4. Nested initializers work: `char[][] board = {{'X','O','X'}, …};`.

```java
// code/ch04/arrays/MultiDim.java (key parts)
int[][] grid = new int[3][4];
for (int r = 0; r < grid.length; r++)
    for (int c = 0; c < grid[r].length; c++)
        grid[r][c] = r * 10 + c;
System.out.println(Arrays.deepToString(grid));

int[][] jagged = new int[3][];
jagged[0] = new int[] {1};
jagged[1] = new int[] {1, 1};
jagged[2] = new int[] {1, 2, 1};

int n = 6;                                      // Pascal's triangle
int[][] pascal = new int[n][];
for (int i = 0; i < n; i++) {
    pascal[i] = new int[i + 1];
    pascal[i][0] = pascal[i][i] = 1;
    for (int j = 1; j < i; j++) pascal[i][j] = pascal[i - 1][j - 1] + pascal[i - 1][j];
}

char[][] board = { {'X','O','X'}, {' ','X','O'}, {'O',' ','X'} };
int[][] a = {{1, 2}, {3, 4}}, b = {{1, 2}, {3, 4}};
System.out.println("Arrays.equals: " + Arrays.equals(a, b) + ", Arrays.deepEquals: " + Arrays.deepEquals(a, b));
```

Output:

```
[[0, 1, 2, 3], [10, 11, 12, 13], [20, 21, 22, 23]]
[1] len=1
[1, 1] len=2
[1, 2, 1] len=3
row 5 of Pascal: [1, 5, 10, 10, 5, 1]
X wins on the diagonal? true
unallocated row is null
Arrays.equals: false, Arrays.deepEquals: true
```

`Arrays.equals` on 2-D arrays compares *row references*; `Arrays.deepEquals` compares contents recursively.

Uses: game boards, pixel grids, matrices, dynamic-programming tables (`dp[i][j]` for edit distance), adjacency matrices for small graphs.

Common mistakes

- `grid.length` as the column count — use `grid[r].length`.
- `NullPointerException` from unallocated jagged rows.
- Column-first loops over big arrays: rows are separate arrays, so row-major (`for r … for c …`) is far more cache-friendly (the CPU loads neighbouring memory in blocks).
- `Arrays.fill(grid, 0)` on `int[][]` compiles (`fill(Object[], Object)`) but throws `ArrayStoreException`; fill each row: `for (int[] row : grid) Arrays.fill(row, 0);`.

TL;DR

A 2-D array is an array of row arrays; rows can be jagged. Use `deepToString` / `deepEquals`; iterate row-major.

4.8 The `java.util.Arrays` toolbox

What it is

`java.util.Arrays` holds static helpers for sorting, searching, filling, copying, comparing and converting arrays.

Method	What it does	Notes
`toString(a)` / `deepToString(a)`	Readable `[1, 2, 3]`	`deep…` for nested
`sort(a)` / `sort(a, from, to)`	Sort in place, ascending	Primitives: Dual-Pivot Quicksort, O(n log n) typical. Objects: TimSort — **stable** (equal elements keep their order), O(n log n) worst case
`sort(a, comparator)`	Sort objects by a `Comparator` (decides which of two comes first)	Chapter 10, Chapter 11
`parallelSort(a)`	Multi-threaded sort	Pays off only for large arrays
`binarySearch(a, key)`	O(log n) search of a **sorted** array	Index, or `-(insertionPoint) - 1` if absent; unsorted → undefined
`fill(a, v)`	Every element = `v`	Objects: every slot gets the *same* reference
`setAll(a, i -> …)`	Each element from its index	Java 8
`copyOf` , `copyOfRange`	New array (truncate/pad, or slice)	
`equals` / `deepEquals` , `hashCode` / `deepHashCode`	Content equality and matching hash	
`compare(a, b)`	Lexicographic (element-by-element) comparison	Java 9
`mismatch(a, b)`	Index of first difference, or -1	Java 9
`asList(T...)`	Fixed-size `List` view **backed by** the array	Writes go through; `add` / `remove` throw
`stream(a)`	Stream over the array (`IntStream` for `int[]`)	Chapter 12

```
// code/ch04/arrays/ArraysUtil.java (key parts)
int[] nums = {42, 7, 19, 3, 25};
Arrays.sort(nums);
Arrays.binarySearch(nums, 19);   Arrays.binarySearch(nums, 20);
String[] words = {"pear", "Fig", "apple", "kiwi"};
Arrays.sort(words);
Arrays.sort(words, String.CASE_INSENSITIVE_ORDER);
Arrays.sort(words, Comparator.comparing(String::length).thenComparing(Comparator.reverseOrder()));
Arrays.fill(filled, -1);
Arrays.setAll(squares, i -> i * i);
System.arraycopy(nums, 0, dest, 2, nums.length);
List<String> view = Arrays.asList(backing);   view.set(0, "z");   view.add("d"); // throws
List<String> independent = List.of(backing);  backing[1] = "changed";
```

Output:

```
sort:          [3, 7, 19, 25, 42]
binarySearch(19) = 2, binarySearch(20) = -4
sort strings:  [Fig, apple, kiwi, pear]
ignore case:   [apple, Fig, kiwi, pear]
by length:     [Fig, pear, kiwi, apple]
fill:          [-1, -1, -1, -1, -1]
setAll:        [0, 1, 4, 9, 16, 25]
copyOf(3):     [3, 7, 19]
copyOf(7):     [3, 7, 19, 25, 42, 0, 0]
copyOfRange:   [7, 19, 25]
arraycopy:     [0, 0, 3, 7, 19, 25, 42]
equals=false compare=-1 mismatch=2 hashCode equal? true
stream sum:    96, max: 42
asList view:   [z, b, c] array now [z, b, c]
asList add ->  UnsupportedOperationException
List.of copy:  [z, b, c]
```

- `binarySearch(nums, 20)` = `-4`: insertion point 3 → `-(3) - 1`. Negative always means "not found".
- String natural order (a type's default ordering) is by UTF-16 code unit, so `F` (70) precedes `a` (97). Use `CASE_INSENSITIVE_ORDER` or a `Collator` (language-aware comparer) for human order.
- `Arrays.asList` is a fixed-size *view* (changes flow both ways); `List.of(array)` is an unmodifiable *copy* (unaffected by `backing[1] = "changed"`). Use `new ArrayList<>(Arrays.asList(a))` for a growable list.

Common mistakes

- `binarySearch` on an unsorted array.
- `Arrays.asList(intArray)` gives a `List<int[]>` of **one** element; use `Arrays.stream(intArray).boxed().toList()`.
- `null` elements in `Arrays.sort` on objects throw `NullPointerException` unless the comparator handles them (`Comparator.nullsFirst(...)`).

4.9 `main`, command-line arguments, and simpler `main` (Java 25)

What it is

The classic **entry point**:

```
public static void main(String[] args)
```

- `public` — the launcher, outside your class, must call it.
- `static` — no object exists at start-up.
- `void` — the exit status (a number for the OS; 0 = success) is set with `System.exit(code)`, not returned.
- `String[] args` — the **command-line arguments** after the class name; `String... args` also works.

How it works — arguments

1. `java ch04.args.CommandLineArgs 3 7 --verbose` → `args` = `{"3", "7", "--verbose"}`.
2. No arguments → an **empty array**, never `null`.
3. Every argument is a `String`; convert with `Integer.parseInt` etc. and handle `NumberFormatException`.
4. The shell splits on spaces; quote to keep them: `java App "New York"` is one argument.
5. The program name is **not** in `args` (unlike C's `argv[0]`).

```java
// code/ch04/args/CommandLineArgs.java (key parts)
public static void main(String[] args) {
    if (args.length == 0) {
        args = new String[] {"3", "7", "x", "--verbose"};
        System.out.println("(no args given, using demo args)");
    }
    System.out.println("args = " + Arrays.toString(args) + ", count = " + args.length);
    boolean verbose = false;
    int sum = 0;
    for (String a : args) {
        if (a.equals("--verbose")) { verbose = true; continue; }
        try { sum += Integer.parseInt(a); }
        catch (NumberFormatException e) { System.out.println("skipping non-number: " + a); }
    }
    System.out.println("sum = " + sum + (verbose ? " (verbose mode)" : ""));
}
```

Output (no arguments, so demo arguments are used):

```
(no args given, using demo args)
args = [3, 7, x, --verbose], count = 4
skipping non-number: x
sum = 10 (verbose mode)
```

Real CLI tools use a parser library (picocli, JCommander) once they have more than a couple of options.

Simpler `main` — compact source files and instance `main` (Java 25, JEP 512)

Introduced in §1.10. For `main`:

- It need not be `public` or `static`, and `String[]` is optional. If both exist the launcher picks `main(String[])`; for an instance `main` it first creates an object with the non-private no-argument constructor.
- A **compact source file** holds just methods and fields, wrapped in an implicit unnameable `final` class, with every exported package of `java.base` (`java.lang`, `java.util`…) auto-imported; `IO.println` / `IO.readln` handle console I/O.

```java
// code/HelloCompact.java — the entire file
void main() {
    String name = "compact world";
    IO.println("Hello, " + name + "!");
    IO.println("Lists are auto-imported: " + List.of(1, 2, 3));
}
```

Output (`java HelloCompact.java`):

```
Hello, compact world!
Lists are auto-imported: [1, 2, 3]
```

`java HelloCompact.java` compiles and runs in one step (source-file mode, Java 11). Compact files suit scripts and learning; when the code grows, wrap the members in `class HelloCompact { … }` and add imports.

Common mistakes

- `public void main(String[] args)` without `static` — runs as an instance main on Java 25, but fails on Java 21 or earlier ("main method is not static"). Keep the classic form for older JDKs.
- Reading `args[0]` without checking `args.length`.
- Returning an exit code from `main` — use `System.exit(1)`.

Chapter TL;DR

Topic	Remember
Method	modifiers + return type + name + params + body; signature = name + param types
`return`	every path of a non-void method; bare `return;` in void methods
Static vs instance	static = belongs to class, no `this`; instance = runs on an object
Pass-by-value	always; references are copied, so mutation is shared but reassignment is not
Overloading	same name, different params; compile time: widening → boxing → varargs → most specific
Varargs	`T... x` is an array; last param only; empty array when no args
Recursion	base case + smaller call; stack frames; no tail-call optimisation; memoize overlaps
Arrays	fixed size, 0-based, bounds-checked, default-filled, `length` field, covariant
2-D arrays	arrays of arrays; can be jagged; `deepToString`, `deepEquals`
`Arrays`	`sort`, `binarySearch` (sorted!), `fill`, `setAll`, `copyOf`, `equals`, `asList` (fixed-size view)
`main`	classic `public static void main(String[] args)`; Java 25: instance/no-arg main, compact source files, `IO`

Next: objects and classes (Chapter 5). Exam traps: OCP Chapter 5.

Chapter 5 — OOP: Classes and Objects

Java is **object-oriented**: programs are built from *objects* that bundle data with the code that works on it. This chapter covers the mechanics — writing classes, creating and constructing objects, sharing state with `static` (class-level members shared by all objects), hiding details with access modifiers (keywords like `private` controlling visibility), packages, immutability (unchangeable after creation), and Java's exact initialisation order.

OOP *design* — relationships, SOLID (five classic OO design principles), patterns (named reusable solutions) — lives in the LLD (low-level design) handbook: LLD Chapter 2 — OOP fundamentals.

Code: `code/` (package `ch05.*`), `javac --release 25`, run on JDK 26. Exam traps: constructors, init-order puzzles, `this` / `super`, immutability in OCP Chapter 6 — Class design; access and `static` in OCP Chapter 5 — Methods.

5.1 Classes and objects

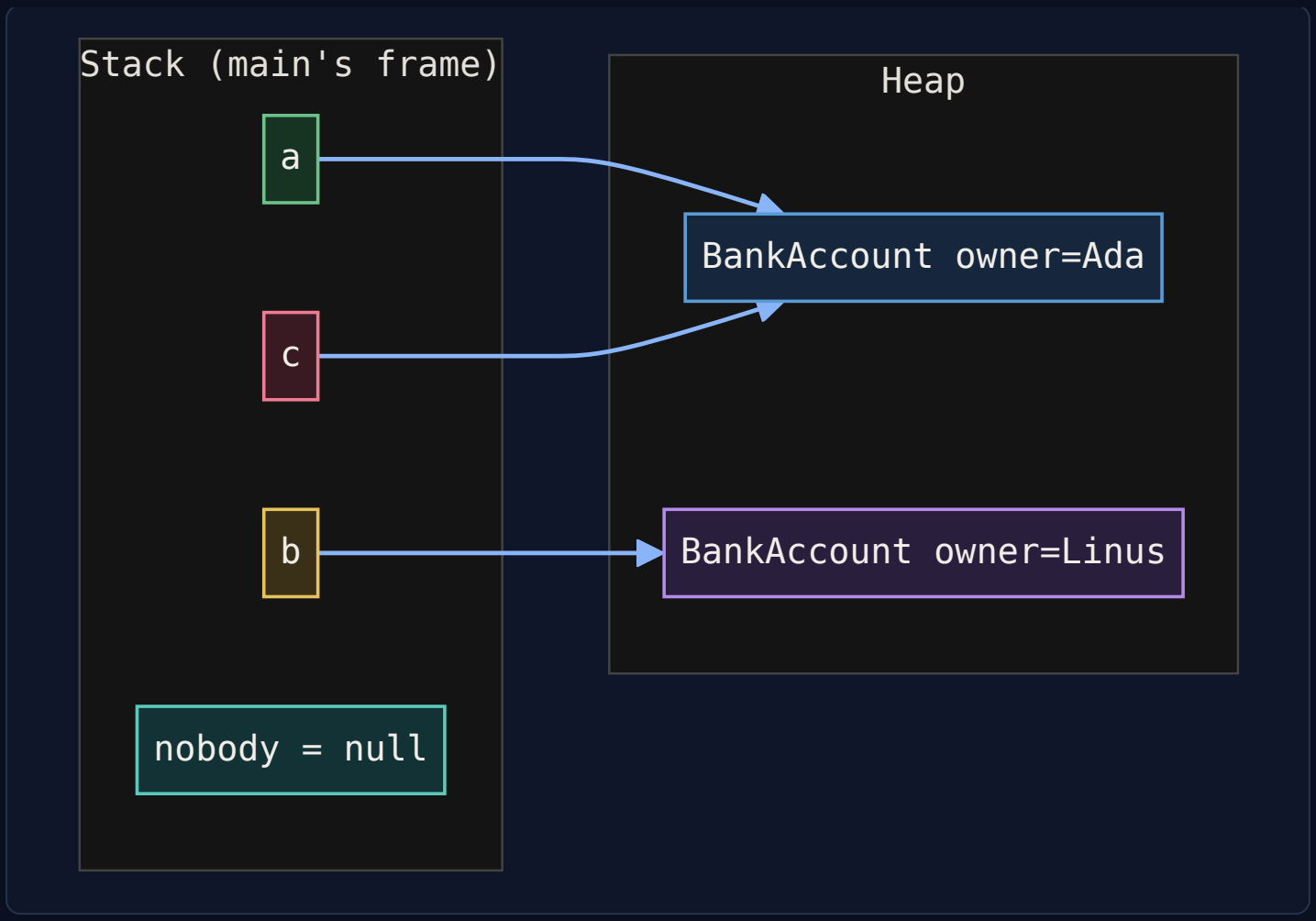
What it is

- A **class** is a blueprint declaring what data each object holds (**fields**) and what it can do (**methods**). It is also a **type**.
- An **object** (**instance**) is one concrete thing built from it, with its own copy of the fields.
- **Instantiation** creates an object: `new BankAccount()`.

Grouping data with the behaviour that protects it keeps rules in one place: a `BankAccount` that knows how to `withdraw` can refuse to go negative; a bare `long balance` cannot.

How it works

1. **Declare** the class (one public top-level class per file, named after it).
2. **Create** with `new ClassName(args)`: allocates memory on the **heap**, runs the constructor (§5.3), returns a **reference**.
3. **Store** the reference in a variable — the variable *points to* the object; it is not the object.
4. **Use** it with the dot operator: `a.deposit(100)`, `a.owner`.
5. Once no live reference reaches an object (from thread stacks, static fields, …) it is **unreachable** and the **garbage collector** (GC) eventually reclaims it. No `delete` / `free` (Chapter 16).




```java
// code/ch05/classes/BankAccount.java
public class BankAccount {
    String owner;          // fields: each object gets its own copy
    long balanceCents;     // defaults: null and 0 until assigned

    void deposit(long cents) { balanceCents += cents; }

    boolean withdraw(long cents) {
        if (cents > balanceCents) return false;
        balanceCents -= cents;
        return true;
    }

    String summary() {
        return owner + ": " + balanceCents / 100 + "." + String.format("%02d", balanceCents % 100);
    }
}
```

```java
// code/ch05/classes/ClassesDemo.java (key parts)
BankAccount a = new BankAccount();
a.owner = "Ada";
a.deposit(10_050);
BankAccount b = new BankAccount();
b.owner = "Linus";
b.deposit(2_000);
System.out.println(a.summary() + " | " + b.summary());
System.out.println("withdraw 50.00 from Linus: " + b.withdraw(5_000) + " -> " + b.summary());

BankAccount c = a;                // copies the reference, not the object
c.deposit(1);
System.out.println("via alias: " + a.summary() + ", same object? " + (a == c));

BankAccount fresh = new BankAccount();
System.out.println("defaults: owner=" + fresh.owner + ", balance=" + fresh.balanceCents);

BankAccount nobody = null;
nobody.deposit(1);                // NullPointerException (caught in the file)
```

Output:

```
Ada: 100.50 | Linus: 20.00
withdraw 50.00 from Linus: false -> Linus: 20.00
via alias: Ada: 100.51, same object? true
defaults: owner=null, balance=0
NPE: Cannot invoke "ch05.classes.BankAccount.deposit(long)" because "<local5>" is null
```

- `c = a` made a second name for the *same* object (one object, two references).
- **Fields get defaults** (`null`, `0`, `false`); locals do not.
- `null` = no object; calling through it throws `NullPointerException` (Chapter 8). Helpful NPE messages (Java 14, JEP 358; default since 15) say what was null — `<local5>` here because the class lacks debug info; compile with `javac -g` (IDEs and build tools do) to get `because "nobody" is null`.
- Money is `long` cents — never `double`.

Common mistakes

- `==` on objects compares *references*; content equality is `equals` (Chapter 6).
- Calling a method on an uninitialised object field (NPE).
- Public fields as here — anyone can set `balanceCents = -5` (fixed in §5.7).

TL;DR

Class = blueprint and type; object = instance on the heap; variables hold references. `new` creates, GC reclaims unreachable objects, `null` means no object.

5.2 Fields, instance methods and `this`

What it is

- An **instance field** is per-object state (`owner`, `balanceCents`).
- An **instance method** runs on one object and reads/writes its fields.
- `this` references *the current object*; it exists in instance methods and constructors, not static methods. A bare field name means `this.field` unless a local or parameter shadows it.

The four uses of `this`

Use	Example	Why
Reach a field **shadowed** by a parameter	`this.size = size;`	`size = size` would assign the parameter to itself
Pass the current object	`registry.register(this)`	Others can hold a reference to you
Return it for chaining	`return this;` → `pizza.addTopping("a").addTopping("b")`	Fluent APIs, builders (LLD Builder)
Call another constructor	`this(size, false);`	Constructor chaining (§5.3)

Common mistakes

- `name = name` instead of `this.name = name` — compiles, does nothing; the field stays `null` (IDEs warn).
- `this` in a `static` method — compile error.
- Letting `this` **escape** a constructor (registering a listener, starting a thread with `this`) before construction finishes — others may see a half-built object; especially dangerous with threads (Chapter 14).

5.3 Constructors

What it is

A **constructor** runs when `new` creates an object and puts it into a **valid** state. Unlike a method, its name is exactly the class name, it has **no return type** (adding `void` makes it an ordinary method), and it is **not inherited**.

How it works

1. **Default constructor**: if you write *none*, the compiler adds a no-argument one calling `super()`, with the class's access. Write **any** constructor and no default is generated — `new X()` stops compiling unless you write it.
2. **Overloading**: several constructors with different parameter lists (§4.3).
3. **Chaining with `this(...)`**: put validation and assignments in one "full" constructor; others delegate with defaults.
4. **Every constructor first runs a superclass constructor** (the *superclass* is the parent class; the extender is the *subclass*) — explicit `super(...)` or an implicit `super()` (Chapter 6). A constructor calls *either* `this(...)` *or* `super(...)`, at most once.
5. **Validate** and throw (`IllegalArgumentException`, or `NullPointerException` via `Objects.requireNonNull`) so an invalid object never exists — "fail fast" (LLD design principles).

Flexible constructor bodies (Java 25, JEP 513)

Until Java 24, `this(...)` / `super(...)` had to be the **first statement**, forcing helper static methods to validate or transform arguments first. **JEP 513, final in Java 25** (preview 22–24), allows statements *before* the call — the **prologue**:

Allowed in the prologue	Not allowed
Locals, computations, `if`/loops	Using `this` explicitly or implicitly (reading fields, calling instance methods)
Validating arguments, throwing	`super.x` or `super.m()`
Assigning this class's fields that have no initialiser	Reading those fields

```java
// code/ch05/constructors/Pizza.java (key parts)
public class Pizza {
    private final String size;
    private final boolean extraCheese;
    private final List<String> toppings = new ArrayList<>();

    public Pizza(String size, boolean extraCheese, String... toppings) {   // the "full" constructor
        if (!List.of("S", "M", "L").contains(size))
            throw new IllegalArgumentException("bad size: " + size);
        this.size = size;
        this.extraCheese = extraCheese;
        this.toppings.addAll(List.of(toppings));
    }
    public Pizza(String size) { this(size, false); }   // chaining
    public Pizza()            { this("M"); }

    // Java 25: statements BEFORE this(...) — parse and validate, without touching 'this'
    public Pizza(String order, char separator) {
        String[] parts = order.strip().split(String.valueOf(separator));
        if (parts.length == 0 || parts[0].isEmpty())
            throw new IllegalArgumentException("empty order");
        this(parts[0].toUpperCase(), false, Arrays.copyOfRange(parts, 1, parts.length));
    }

    public Pizza addTopping(String t) { toppings.add(t); return this; }  // fluent: returns this
    @Override public String toString() { /* Pizza[size, extra cheese?, toppings] */ }
}
```

```java
// code/ch05/constructors/ConstructorsDemo.java (key parts)
static class NoConstructorWritten { int x = 7; }           // compiler adds NoConstructorWritten() { super(); }
static class OnlyArgConstructor {
    final int x;
    OnlyArgConstructor(int x) { this.x = x; }              // now 'new OnlyArgConstructor()' will not compile
}

System.out.println(new Pizza());
System.out.println(new Pizza("L"));
System.out.println(new Pizza("S", true, "olives", "basil"));
System.out.println(new Pizza(" m,ham,pineapple ", ','));
System.out.println(new Pizza("M").addTopping("mushroom").addTopping("onion"));
new Pizza("XXL");                                           // throws, caught in the file
```

Output:

```
Pizza[M, []]
Pizza[L, []]
Pizza[S, extra cheese, [olives, basil]]
Pizza[M, [ham, pineapple]]
Pizza[M, [mushroom, onion]]
default ctor gives x=7, arg ctor gives x=42
rejected: bad size: XXL
```

Trade-offs — ways to create objects

Approach	Good for	Weak spot
Telescoping constructors (each adds a parameter, like `Pizza`)	1–3 optional parameters	Same-typed parameters are easy to swap
Static factory method (`Pizza.margherita()`, `List.of`, `Integer.valueOf`)	Descriptive names, caching, returning subtypes (*Effective Java* Item 1)	Less obvious in docs
Builder (`Pizza.builder().size("L").cheese(true).build()`)	Many optional parameters, readable calls (Item 2)	More code — LLD Builder
Records (Chapter 6)	Plain data carriers	Canonical form takes every component

Common mistakes

- `public void Pizza() {}` is a *method*.
- Adding an argument constructor, then `new Pizza()` stops compiling.
- Duplicating validation instead of chaining.
- Calling overridable methods from a constructor — a subclass override can run before the subclass's fields are initialised (*Effective Java* Item 19; demo in Chapter 6).
- Recursive chaining (`A() { this(1); } A(int x) { this(); }`) — "recursive constructor invocation".

In the interview

> **30-second answer:** "A constructor initialises a new object into a valid state. Declare none and Java adds a no-arg default; declare any and it disappears. I chain overloads with `this(...)` so validation lives in one place. Every constructor first runs a superclass constructor via `super(...)`, explicit or implicit. Since Java 25 I can validate before that call, as long as I don't touch `this`."

- *Private constructor?* Yes — singletons, static-factory-only classes, non-instantiable utility classes.
- *Inherited?* No; a subclass declares its own and calls a superclass one.

TL;DR

Constructors: class name, no return type, not inherited. Default only if you write none. Chain with `this(...)`; one constructor validates. Java 25 allows code before `this(...)` / `super(...)`.

5.4 Packages and imports

What it is

A **package** is a named group of related classes that maps to a folder. It provides **namespaces** (`java.util.Date` and `java.sql.Date` coexist), an **access boundary** (package-private members, §5.6) and **organisation** (`com.shop.orders`).

How it works

1. The file's first statement declares it: `package ch05.access.bank;`. None → the **unnamed (default) package**: fine for scratch code, but other packages cannot import from it.
2. Folders must match: `ch05/access/bank/Vault.java` (classes compile to the same structure).
3. Convention: lowercase reversed domain, `com.example.shop.orders`.

4. The **fully qualified name** (package + name, `ch05.access.bank.Vault`) works without imports.
5. **Imports** allow the short name:

Form	Example	Notes
Single-type	`import java.util.List;`	Preferred — explicit
On-demand (wildcard)	`import java.util.*;`	That package only, **not** subpackages (`java.util.function` needs its own)
Static	`import static java.lang.Math.max;` / `…Math.*;`	Static members; sparingly (constants, test assertions)
Module (Java 25, JEP 511)	`import module java.base;`	All public types of every package the module exports; single-type imports win clashes

6. `java.lang` and your own package are imported automatically.
7. Imports are **compile-time only** — no code loaded, no runtime cost.
8. At run time classes are found via the **class path** (`java -cp out ch05.statics.StaticDemo`) or the **module path** (Chapter 16).

Common mistakes

- `java.util.*` + `java.sql.*` then `Date` — ambiguous; import one explicitly or fully qualify.
- Assuming `java.util.*` covers `java.util.concurrent.ConcurrentHashMap`.
- File path not matching the package — compiles, but the class isn't found at run time.

5.5 Static members

What it is

`static` = **belongs to the class, not to any object**.

- A **static field** has **one copy**, shared by all instances (usable with none).
- A **static method** runs without an object; no `this`.
- A **static initialiser block** `static { … }` runs **once**, at class initialisation.
- A **constant** is a `static final` field in `UPPER_SNAKE_CASE`.

How it works

1. Access via the class name: `Ticket.issuedSoFar()`, `Ticket.MAX_PER_ORDER` (via an object compiles but misleads; IDEs warn).
2. Static methods use only static members directly; instance methods use both.
3. **Class initialisation** (static initialisers and blocks, top to bottom) runs once, on first *active use*: creating an instance, calling a static method, reading a non-constant static field.
4. Reading a **compile-time constant** (`static final` primitive/`String` with a constant expression) does **not** initialise the class — the value is copied into the caller.

```java
// code/ch05/statics/Ticket.java
public class Ticket {
    public static final int MAX_PER_ORDER = 10;      // constant
    private static int issued;                       // ONE copy, shared
    private static final String PREFIX;

    static {                                         // runs once, at class initialisation
        PREFIX = "TKT-" + (2000 + 26);
        System.out.println("[Ticket class initialised]");
    }

    private final int number;                        // one copy PER object

    public Ticket() { issued++; number = issued; }
    public String id() { return PREFIX + "-" + number; }
    public static int issuedSoFar() { return issued; }
    public static boolean canBuy(int quantity) { return quantity > 0 && quantity <= MAX_PER_ORDER; }
}
```

```java
// code/ch05/statics/StaticDemo.java (key parts)
import static java.lang.Math.max;
import static java.lang.Math.PI;

System.out.println("before first use");
System.out.println("MAX_PER_ORDER = " + Ticket.MAX_PER_ORDER);  // constant: inlined, no class init
System.out.println("issued = " + Ticket.issuedSoFar());           // first real use: class init
Ticket t1 = new Ticket(), t2 = new Ticket(), t3 = new Ticket();
System.out.println(t1.id() + ", " + t2.id() + ", " + t3.id());
System.out.println("issued = " + Ticket.issuedSoFar() + ", canBuy(12) = " + Ticket.canBuy(12));
System.out.println("max(3, 8) = " + max(3, 8) + ", PI = " + PI);
```

Output:

```
before first use
MAX_PER_ORDER = 10
[Ticket class initialised]
issued = 0
TKT-2026-1, TKT-2026-2, TKT-2026-3
issued = 3, canBuy(12) = false
max(3, 8) = 8, PI = 3.141592653589793
```

Reading `MAX_PER_ORDER` did *not* trigger the static block; `issuedSoFar()` did. Three tickets: one `issued`, three `number`s.

When to use `static` — and when not

Good fit	Poor fit
Constants (`MAX_PER_ORDER` , `Math.PI`)	Mutable global state (`static List<User> users`) — hidden coupling, hard to test, **not thread-safe** without synchronisation
Pure utilities, output depends only on inputs (`Math.max` , `Integer.parseInt`)	Behaviour that should vary by implementation — statics cannot be overridden (Chapter 6)
Static factories (`List.of` , `Integer.valueOf`)	Dependencies to swap in tests (`static Database db`) — pass objects in
Counters/ID generators — `AtomicInteger` if threads are involved	

Utility classes (`Math` , `Collections` , `Arrays`) are `final` with a `private` constructor (*Effective Java* Item 4). Singleton pitfalls: LLD Chapter 8.

Common mistakes

- "non-static variable cannot be referenced from a static context" — calling instance members from `main` .

- `issued++` is a **race condition** (outcome depends on thread timing) if two threads create tickets (Chapter 14).
- Static imports so heavy that readers can't tell where `of(...)` comes from.

In the interview

30-second answer: "Static members belong to the class: one shared copy of a static field; static methods run without an instance, have no `this`, and can't be overridden — only hidden. Static initialisation happens once, lazily, on first active use. I use static for constants, pure utilities and factories, and avoid mutable static state: it's global, hard to test and needs synchronisation."

- *Static method reading an instance field?* Only through an object reference it is given or creates.
- *When does a static block run?* Once, at class initialisation — first instantiation, static method call, or non-constant static field access.

5.6 Access modifiers

What it is

Access modifiers control *who can see* a class, field, method or constructor — Java's compile-time enforcement of encapsulation (§5.7).

Modifier	Same class	Same package	Subclass in another package	Everywhere else
`private`	yes	no	no	no
(none) = **package-private**	yes	yes	no	no
`protected`	yes	yes	yes (through inheritance)	no
`public`	yes	yes	yes	yes

Top-level classes can only be `public` or package-private; members (fields, methods, constructors, nested types) can use all four.

How it works

```java
// code/ch05/access/bank/Vault.java
package ch05.access.bank;
public class Vault {
    public    String name = "Main vault";   // everyone
    protected int    floors = 3;            // same package + subclasses
              int    guards = 4;            // package-private: same package only
    private   String code = "0000";         // this class only
    public boolean open(String attempt) { return code.equals(attempt); }
}
```

```java
// code/ch05/access/bank/Teller.java — same package: sees everything except private
return v.name + ", floors=" + v.floors + ", guards=" + v.guards;
```

```java
// code/ch05/access/client/AccessDemo.java — different package
static class BranchVault extends Vault {
    int floorsPlusOne() { return floors + 1; }     // protected: visible through inheritance
}
Vault v = new Vault();
System.out.println("public name:        " + v.name);
// v.floors  -> compile error: protected, not accessed through a subclass
// v.guards  -> compile error: package-private, different package
// v.code    -> compile error: private
System.out.println("open(\"1234\"):       " + v.open("1234"));
System.out.println("Teller (same pkg):  " + Teller.inspect(v));
System.out.println("subclass protected: " + new BranchVault().floorsPlusOne());
```

Output:

```
public name:        Main vault
open("1234"):       false
Teller (same pkg):  Main vault, floors=3, guards=4
subclass protected: 4
```

The protected subtlety: a subclass in another package may use a protected member only *through inheritance* — on `this` or references of its own subclass type — not on an arbitrary `Vault`. Hence `v.floors` fails while `BranchVault.floorsPlusOne()` works.

Guidelines

1. **Make every field** `private` (constants excepted); expose behaviour, not data.
2. Start members private or package-private; widen only when needed (*Effective Java* Item 15). Every `public` member is a permanent promise.
3. `protected` is part of your API to subclassers — use deliberately.
4. Package-private suits helper classes other packages shouldn't depend on.
5. Module-level hiding (a `public` class other *modules* can't use): JPMS in Chapter 16.

TL;DR

private < package-private < protected < public. Fields private, members as narrow as possible.

5.7 Encapsulation

What it is

Encapsulation bundles data with the methods that operate on it and **hides the data** so it changes only through those methods. The object guards its **invariants** — conditions that must always hold ("5 ≤ target ≤ 30", "balance ≥ 0").

Why: **validity** (no nonsense states), **freedom to change** the internal representation (store Fahrenheit, add caching) without breaking callers, **debuggability** (every change passes through one method — one breakpoint).

How it works

1. Make fields `private`.
2. Prefer **behaviour** methods (`raise`, `withdraw`) over raw setters ("tell, don't ask" — LLD principles).
3. Where accessors are needed, a **getter** returns a value (`getTargetCelsius()`, `isActive()` for booleans) and a **setter** validates before changing.
4. Omit setters for read-only properties; compute derived values.

```java
// code/ch05/encapsulation/Thermostat.java
public class Thermostat {
    public static final double MIN = 5.0, MAX = 30.0;
    private double targetCelsius = 20.0;
    private int changes;

    public double getTargetCelsius() { return targetCelsius; }
    public void setTargetCelsius(double value) {
        if (Double.isNaN(value) || value < MIN || value > MAX)
            throw new IllegalArgumentException("target must be in [" + MIN + ", " + MAX + "]: " + value);
        targetCelsius = value;
        changes++;
    }
    public void raise(double delta) { setTargetCelsius(Math.min(MAX, targetCelsius + delta)); }
    public double getTargetFahrenheit() { return targetCelsius * 9 / 5 + 32; }   // derived
    public int getChanges() { return changes; }                                  // read-only
}
```

```java
// code/ch05/encapsulation/EncapsulationDemo.java (key parts)
static class OpenThermostat { public double targetCelsius; }   // no encapsulation
open.targetCelsius = -273;                                      // nothing stops nonsense
Thermostat t = new Thermostat();
t.setTargetCelsius(22.5);
t.raise(10);                                                    // clamped to MAX
t.setTargetCelsius(-273);                                       // rejected
```

Output:

```
open thermostat: -273.0
target=30.0C = 86.0F, changes=2
rejected: target must be in [5.0, 30.0]: -273.0
```

Trade-offs

Getters/setters for every field ("anaemic" objects) barely beat public fields: they keep room for later validation but express no behaviour. Prefer meaningful operations; for pure data, an immutable **record** (Chapter 6). Encapsulation vs abstraction, "tell don't ask", Law of Demeter: LLD §2.3 Encapsulation.

Common mistakes

- A getter returning a mutable internal (`List` , `Date` , array) — callers can change it. Return an unmodifiable view or copy (§5.9).
- Setters that don't validate (private fields + unvalidated public setters is barely encapsulated).
- Generating accessors for every field by reflex.

In the interview

30-second answer: "Encapsulation hides an object's state behind methods, so the object enforces its own invariants and can change internals without breaking callers. In Java: private fields, behaviour-oriented public methods, validated setters only where needed, no leaking of mutable internals through getters."

5.8 The `final` keyword

`final` = "cannot be changed after it is set"; what is fixed depends on where it appears:

On	Meaning	Example
Local variable / parameter	Assigned exactly once	`final int max = 10;`
Instance field	Assigned exactly once — declaration, instance initialiser, or **every** constructor	`private final String size;`
Static field	Assigned once — declaration or static block	`static final String PREFIX;`
Method	Cannot be **overridden**	`public final void close()`
Class	Cannot be **extended**	`final class String`, `final class Money`

- `final` on a reference stops **reassignment**, not **mutation**: a `final StringBuilder sb` can still `append`; a `final` array's elements can change.
- A never-reassigned variable is **effectively final**; lambdas and inner classes may only capture such variables (Chapter 11).
- `final` fields have a **Java Memory Model** guarantee (the rules for what one thread sees of another's writes): after a constructor finishes without leaking `this`, all threads see correct `final` values without synchronisation (Chapter 15).
- Not to be confused with `finally` (Chapter 8) or deprecated `finalize()`.

5.9 Immutability basics

What it is

An **immutable** object's observable state **never changes after construction** — `String`, `Integer`, `LocalDate`, `BigDecimal`, `Instant`, `UUID`, records with immutable components. To "change" one you create a new one (`s.toUpperCase()`).

Why it matters

Benefit	Why
Simple to reason about	One state, set and validated once
Thread-safe for free	Nothing to race on; share without locks
Safe to share and cache	`Integer.valueOf` caches; `String` literals are pooled (Chapter 7)
Safe as `HashMap` keys / `HashSet` elements	The hash code can't change after insertion
No defensive copies needed	Nobody can modify it

Effective Java Item 17: "minimise mutability". Cost: a new object per change, which matters for large objects updated in tight loops (hence `StringBuilder` as `String`'s mutable companion). Design view: LLD §2.13, LLD principle 14.

How it works — the five rules

1. **No methods that modify state** (no setters).
2. **Make the class `final`** (or constructors private) so a subclass can't add mutable state or override methods to behave mutably.
3. **All fields `private final`.**
4. **Validate in the constructor** — every instance valid from birth.
5. **Exclusive access to mutable components**: **defensive copies** of mutable inputs (lists, arrays, `Date`) in the constructor; never hand out mutable internals — return copies or unmodifiable views.

"Modifiers" return a **new** object (`plus`, `with…`).

```java
// code/ch05/immutable/Money.java
public final class Money {
    private final long cents;
    private final String currency;
    public Money(long cents, String currency) {
        this.cents = cents;
        this.currency = Objects.requireNonNull(currency, "currency");
    }
    public long cents() { return cents; }
    public String currency() { return currency; }
    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(cents + other.cents, currency);   // new object
    }
    @Override public String toString() { return String.format("%d.%02d %s", cents / 100, cents % 100, currency); }
}
```

```java
// code/ch05/immutable/Playlist.java
public final class Playlist {
    private final String name;
    private final List<String> songs;
    public Playlist(String name, List<String> songs) {
        this.name = name;
        this.songs = List.copyOf(songs);          // defensive copy in; also unmodifiable
    }
    public List<String> songs() { return songs; } // safe to hand out
    public Playlist with(String song) {           // "wither": copy + change
        List<String> next = new ArrayList<>(songs);
        next.add(song);
        return new Playlist(name, next);
    }
}
```

```java
// code/ch05/immutable/ImmutabilityDemo.java (key parts)
static final class LeakyPlaylist {                // final fields, but NOT immutable
    private final List<String> songs;
    LeakyPlaylist(List<String> songs) { this.songs = songs; }   // keeps the caller's list
    List<String> songs() { return songs; }                      // hands out the internal list
}

List<String> input = new ArrayList<>(List.of("Intro", "Verse"));
LeakyPlaylist leaky = new LeakyPlaylist(input);
input.add("SNEAKY");
leaky.songs().clear();

final StringBuilder sb = new StringBuilder("final ref");
sb.append(" but mutable object");
```

Output:

```
19.99 EUR + 5.01 EUR = 25.00 EUR (price unchanged: 19.99 EUR)
leaky after outside changes: []
safe after outside change:   Mix[Intro, Verse]
safe.songs().clear() -> UnsupportedOperationException
with("Chorus"): Mix[Intro, Verse, Chorus], original: Mix[Intro, Verse]
final ref but mutable object
```

`LeakyPlaylist` has only `final` fields yet was changed twice from outside — **`final` fields are not enough**; rule 5 makes `Playlist` safe.

Common mistakes

- Thinking `final` = immutable.
- `Collections.unmodifiableList(list)` without copying: a read-only *view* that still reflects the owner's changes. `List.copyOf` makes an independent copy.
- Returning an array field — return `array.clone()`.

- Mutable components like `java.util.Date` — prefer `java.time` (Chapter 7).

In the interview

> **30-second answer:** "An immutable class has fixed state after construction: final class, private final fields, no setters, constructor validation, defensive copies of mutable inputs and outputs. Benefits: thread safety without locks, safe sharing and caching, safe map keys. String, Integer and java.time are examples; records make simple immutable classes a one-liner if their components are immutable."

- *Why final class?* A subclass could add mutable state or override methods, breaking the guarantee for code holding the parent type.
- *Is a record deeply immutable* (nothing reachable can change)? No — fields are final, but a `List` component stays mutable unless copied in the compact constructor (Chapter 6).

TL;DR

Immutable = final class, private final fields, no setters, validate in constructor, copy mutable stuff in and out. Changes return new objects. `final` ≠ immutable.

5.10 Initialisation order

What it is

Java runs initialisation code in a **fixed order** when a class is first used and when an object is created — knowing it prevents "why is this field still null?" bugs, and it's a favourite exam/interview question. The kinds:

- **Static field initialisers** (`static int x = 5;`) and **static blocks** (`static { … }`) — class level.
- **Instance field initialisers** and **instance initialiser blocks** (`{ … }`) — object level.
- **Constructor bodies.**

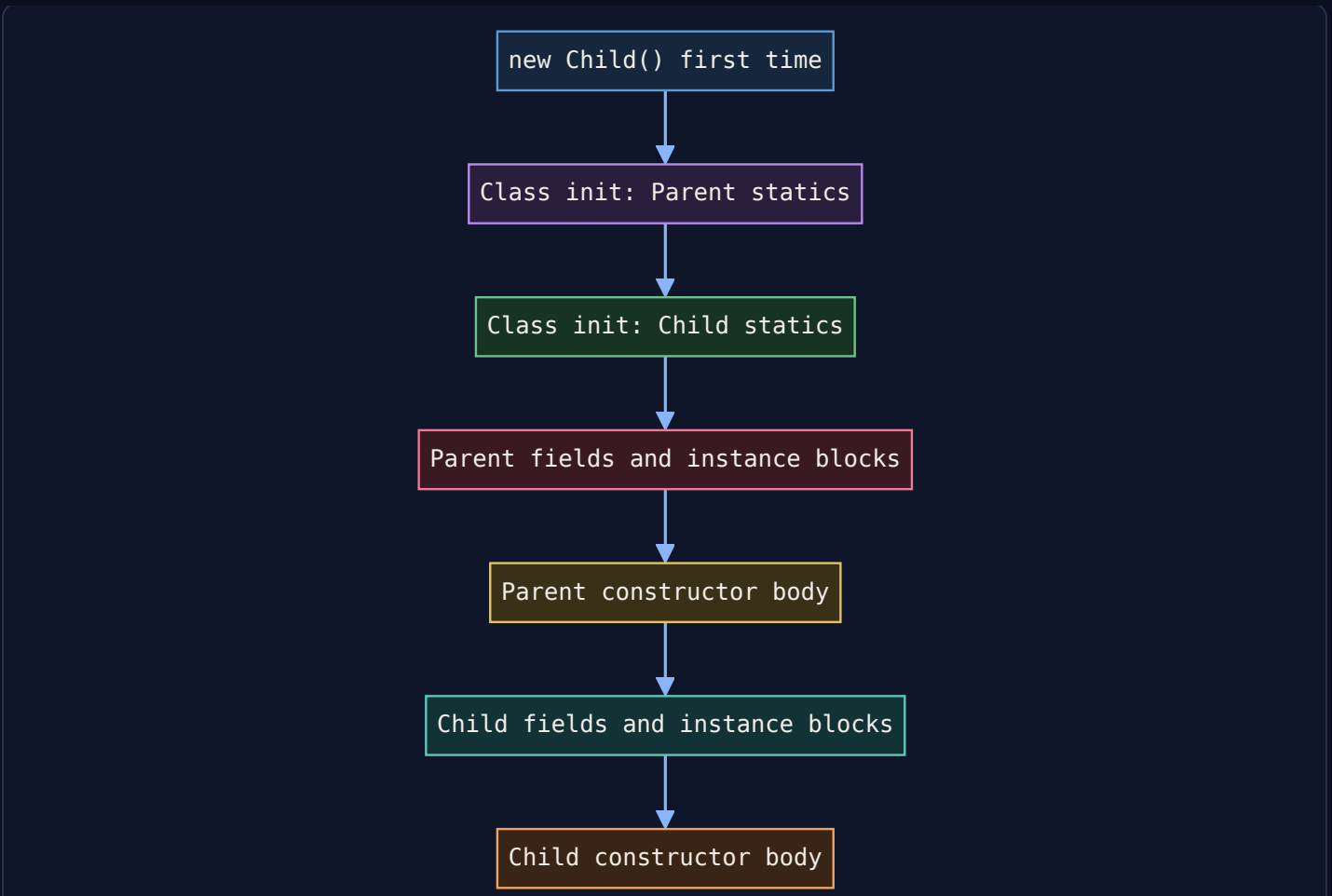
How it works

Class initialisation (once per class per class loader, on first active use):

1. Initialise the **superclass** first (recursively up to `Object`).
2. Run this class's static initialisers and blocks **in source order**.

Object creation (`new Child()`), every time:

1. Memory allocated; all fields get defaults.
2. The `Child` constructor starts by calling the superclass constructor (Java 25: prologue statements run before it).
3. The **superclass** runs its instance initialisers/blocks (source order), then its constructor body — recursively.
4. `Child` 's instance initialisers/blocks run **in source order**.
5. The rest of the `Child` constructor body runs.




```java
// code/ch05/init/InitOrder.java (key parts)
static class Parent {
    static { log("1. Parent static block"); }
    private final String p = log("3. Parent field initializer");
    { log("4. Parent instance block"); }
    Parent() { log("5. Parent constructor body"); }
}
static class Child extends Parent {
    static final String TAG = log("2. Child static field");
    private final String c = log("6. Child field initializer");
    { log("7. Child instance block"); }
    Child() {
        super();                                  // implicit if omitted
        log("8. Child constructor body");
    }
}
public static void main(String[] args) {
    System.out.println("--- first new Child() ---");
    new Child();
    System.out.println("--- second new Child(): static parts do not run again ---");
    new Child();
}
```

Output:

```
--- first new Child() ---
1. Parent static block
2. Child static field
3. Parent field initializer
4. Parent instance block
5. Parent constructor body
6. Child field initializer
7. Child instance block
8. Child constructor body
--- second new Child(): static parts do not run again ---
3. Parent field initializer
4. Parent instance block
5. Parent constructor body
6. Child field initializer
7. Child instance block
8. Child constructor body
```

(`TAG` comes from a method call, so it is not a compile-time constant and runs during class initialisation.) Note the child's static part runs before the parent's instance blocks — all class init precedes object creation.

Common mistakes

- A static field read by a static block declared *above* it still has its default (source order).
- An overridable method called from the parent constructor runs the child's override **before** the child's field initialisers, so it sees defaults (§6.3).
- Heavy work or I/O in static blocks: failures surface as `ExceptionInInitializerError` and later uses get `NoClassDefFoundError`.

TL;DR

Statics once (parent first, source order). Per object: defaults → parent (fields/blocks, constructor body) → child (fields/blocks, constructor body).

Chapter TL;DR

Topic	Remember
Class vs object	Blueprint/type vs instance on the heap; variables hold references
`null` / NPE	No object; helpful NPE messages since Java 14
`this`	Current object: shadowed fields, pass/return self, `this(...)`
Constructors	Name = class, no return type, not inherited; default only if none written; chain with `this(...)` ; Java 25 prologue before `this` / `super`
Packages	Namespace + folder + access boundary; single-type imports; `java.lang` automatic; `import module` (25)
`static`	One per class; no `this` ; init once on first active use; constants are `static final`
Access	private < package-private < protected < public; fields private
Encapsulation	Hide state, expose behaviour, protect invariants
`final`	Variable: assign once. Method: no override. Class: no subclass. Not the same as immutable
Immutability	Final class, private final fields, no setters, validate, defensive copies
Init order	Parent statics → child statics → (per object) parent fields/blocks → parent ctor → child fields/blocks → child ctor

Next: inheritance, interfaces, polymorphism, records, enums and nested classes in Chapter 6. OOP design: LLD handbook.

Chapter 6 — OOP: Inheritance, Interfaces and More

Chapter 5 built single classes; this chapter covers **relationships between types**: extending a class, sharing a contract (a promise of which methods a type provides) through interfaces, and how Java picks the right code at run time (polymorphism). Then the modern type kinds — **sealed** hierarchies (closed list of subtypes), **records** (compact data carriers), **enums** (fixed named constants) — plus nested classes and the `Object` methods `equals`, `hashCode`, `toString`.

This is *language mechanics*. *Design* — inherit vs compose, SOLID, the Liskov Substitution Principle (a subclass must work wherever its parent is expected), patterns — is in the LLD handbook (linked below).

Code: `code/` (package `ch06.*`), `javac --release 25`, run on JDK 26. Exam traps: overriding vs hiding, abstract classes, constructor chains in OCP Chapter 6 — Class design; interfaces, enums, sealed types, records, nested classes, casting in OCP Chapter 7 — Beyond classes.

6.1 Inheritance and `super`

What it is

Inheritance defines a new class (the **subclass** / child / derived class) as an extension of an existing one (the **superclass** / parent / base class). The subclass gets the parent's accessible fields and methods, can add new ones, and can **override** (replace) inherited methods. It models **"is-a"**: a `Manager` *is an* `Employee`.

How it works

1. `class Manager extends Employee` — **one** direct superclass only (single inheritance of classes).
2. A class without `extends` implicitly extends **`java.lang.Object`**, so every class has `toString`, `equals`, `hashCode`, `getClass` …
3. **Constructors are not inherited.** Each subclass constructor first initialises the parent part via **`super(args)`**; if omitted, the compiler inserts `super()` — a compile error if the parent has no no-arg constructor.
4. **`super.method()`** calls the parent's version of an overridden method — to extend rather than replace behaviour.
5. **Accessible:** `public` / `protected` members always; package-private only in the same package. `private` members exist in the object but are reachable only through the parent's methods.
6. A **`final`** class cannot be extended (`String`, `Integer`, records).

```java
// code/ch06/inheritance/InheritanceDemo.java (key parts)
static class Employee {
    private final String name;              // private: not accessible in subclasses
    protected double baseSalary;            // protected: accessible in subclasses
    Employee(String name, double baseSalary) { this.name = name; this.baseSalary = baseSalary; }
    String getName() { return name; }
    double pay() { return baseSalary; }
    String describe() { return getName() + " earns " + pay(); }
}

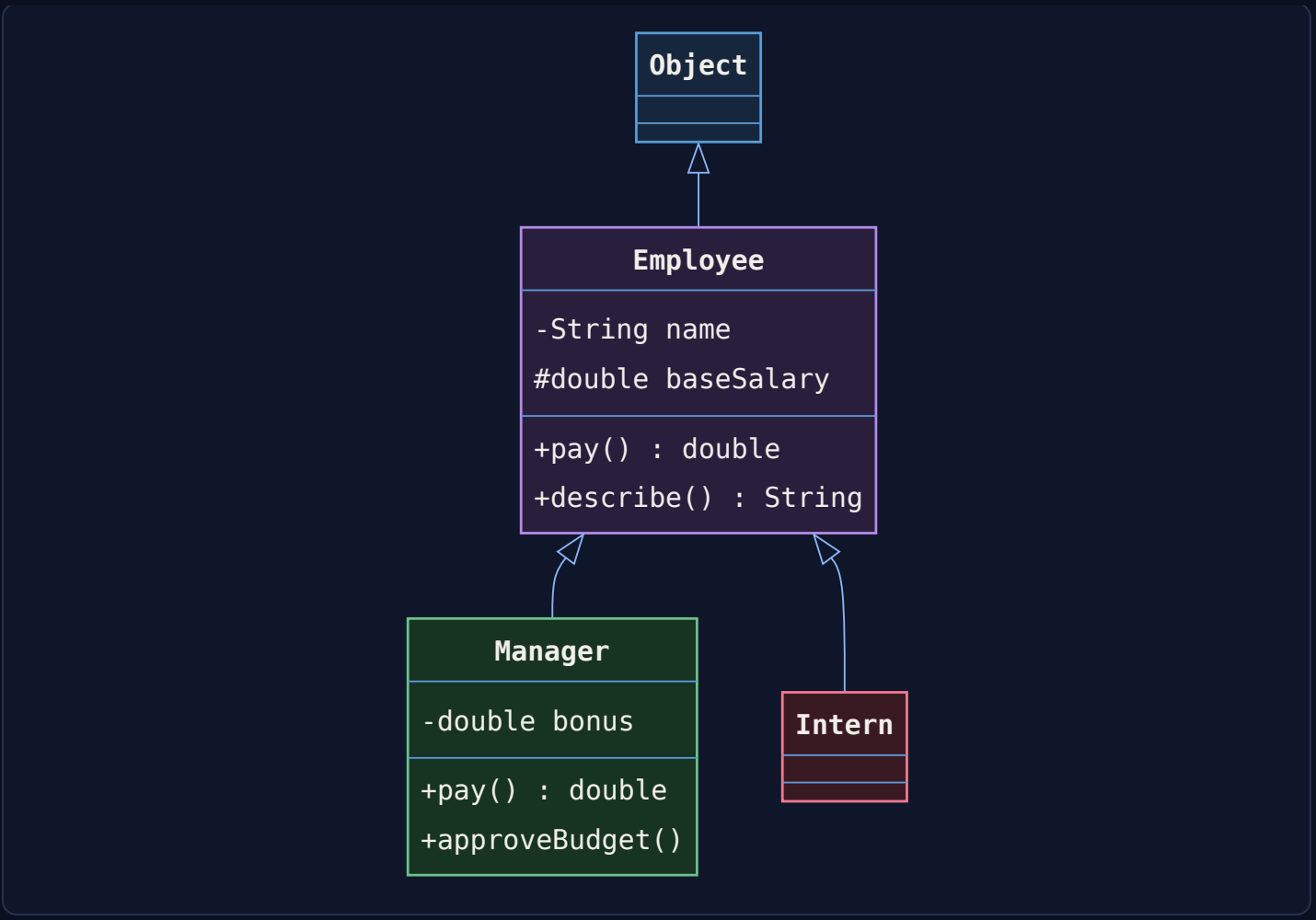
static class Manager extends Employee {     // Manager IS-AN Employee
    private final double bonus;
    Manager(String name, double baseSalary, double bonus) {
        super(name, baseSalary);            // initialise the Employee part first
        this.bonus = bonus;
    }
    @Override double pay() { return super.pay() + bonus; }   // extend the parent's version
    void approveBudget() { System.out.println(getName() + " approved the budget"); }
}

static class Intern extends Employee {
    Intern(String name) { super(name, 1_000); }             // inherits pay() unchanged
}
```

Output:

```
Ana earns 3000.0
Bo earns 6500.0
Cy earns 1000.0
Bo approved the budget
m instanceof Employee? true, superclass of Manager: Employee, superclass of Employee: Object
```

Line 2: `describe()` is written once in `Employee`, yet for a `Manager` it calls `Manager.pay()` — **dynamic dispatch** (§6.3–6.4).



Trade-offs — inheritance vs composition

	Inheritance (extends)	Composition (hold a reference to another object)
Relationship	is-a	has-a / uses-a
Coupling (how strongly code depends on other code)	Tight: depends on the parent's implementation details	Loose: depends only on the other object's public API
Flexibility	Fixed at compile time; one parent	Swap parts at run time; combine many
Risk	**Fragile base class** — parent changes break subclasses	Forwarding boilerplate

Effective Java Item 18 "favour composition over inheritance"; Item 19 "design and document for inheritance or else prohibit it" (make classes `final` unless designed for extension). Full treatment: LLD §2.11, LLD §3.6.

Don't inherit just to reuse code when "is-a" is false (JDK's `Stack extends Vector` regret: a stack exposing `insertElementAt`), when a subclass would *disable* inherited behaviour (throwing `UnsupportedOperationException` violates Liskov — LLD §4.8), or across team boundaries on classes not designed for it.

Common mistakes

- Forgetting `super(...)` when the parent has only `Employee(String, double)` → "constructor Employee in class Employee cannot be applied to given types".
- Expecting access to the parent's `private` field.
- Deep hierarchies (5+ levels); prefer shallow ones plus interfaces.

TL;DR

`extends` = is-a, single parent, root `Object`. `super(...)` builds the parent part first; `super.m()` reuses the parent's method. Prefer composition unless designed for extension.

6.2 What a subclass gets — members, hiding and `Object`

Member in parent	In the subclass
`public` / `protected` instance method	Inherited; may be **overridden**
package-private instance method	Inherited only in the same package
`private` method	Not inherited (a same-named child method is a *new* method)
`final` method	Inherited; cannot be overridden
`static` method	Inherited (callable via the subclass name); a same-signature child static **hides** it
Field (static or instance)	Inherited if accessible; a same-named child field **hides** it (both exist)
Constructor	**Never** inherited
Initialiser blocks	Not inherited; the parent's still run during construction (§5.10)

`java.lang.Object` gives every class:

Method	Purpose	Override?
`toString()`	Text form; default `ClassName@hexHash`	Usually (§6.13)
`equals(Object)`	Logical equality; default identity (`==`)	For value-like classes (§6.13)
`hashCode()`	`int` used by `HashMap` / `HashSet` to place objects; must match `equals`	Whenever you override `equals`
`getClass()`	Runtime class object	`final`
`clone()`	Field-by-field copy (needs the `Cloneable` marker interface)	Rarely; prefer copy constructors/factories (*Effective Java* Item 13)
`wait()`, `notify()`, `notifyAll()`	Low-level thread coordination	`final` ; Chapter 14
`finalize()`	Cleanup before GC	Deprecated for removal (JEP 421, Java 18) — use try-with-resources / `Cleaner`

6.3 Overriding

What it is

Overriding: a subclass re-implements an inherited **instance** method with the **same signature**. At run time the JVM calls the version of the object's **actual (runtime) class**, not the variable's declared type — **dynamic dispatch** / *runtime polymorphism* (overloading, by contrast, is resolved at compile time — §4.3).

The rules

For `Dog.m` to override `Animal.m`:

#	Rule	Example that fails
1	Same name and parameter types	`speak(String)` vs `speak()` is an overload
2	Return type the same or a **subtype** (covariant return) for references; identical for primitives (`Number get()` → `Integer get()` is fine)	Returning `Object` where the parent returns `Animal`
3	Access **same or wider**	`private Dog reproduce()` over a `protected` method; `void run()` over `public void run()`
4	Checked exceptions: none new or broader (fewer/narrower fine); unchecked unrestricted	Parent throws `IOException`, child declares `Exception`
5	Can't override `final` methods	`String dna()` when the parent's is `final`
6	`static` methods are **hidden**, not overridden; `private` methods aren't inherited	—
7	An instance method can't "override" a static one or vice versa	compile error

Always write `@Override`: the compiler then checks you really override something. Without it a typo (`tostring()`) or wrong parameter (`equals(Book)` instead of `equals(Object)`) silently creates a new method.

```java
// code/ch06/overriding/OverridingRules.java (key parts)
static class Animal {
    protected Animal reproduce() { return new Animal(); }
    void speak() throws IOException { System.out.println("..."); }
    static String kind() { return "Animal (static)"; }
    String name = "animal field";
    final String dna() { return "ATGC"; }
}
static class Dog extends Animal {
    @Override public Dog reproduce() { return new Dog(); }    // covariant return + wider access
    @Override void speak() { System.out.println("Woof"); }    // throws fewer checked exceptions
    static String kind() { return "Dog (static)"; }           // HIDES Animal.kind()
    String name = "dog field";                                // HIDES Animal.name
    // String dna() {...}        -> compile error: final
    // private Dog reproduce()   -> compile error: weaker access
}

static class Base {
    Base() { init(); }                    // calls an overridable method: dangerous
    void init() { }
}
static class Derived extends Base {
    private String label = "ready";
    @Override void init() { System.out.println("Derived.init sees label = " + label); }
}

Animal a = new Dog();                     // declared type Animal, runtime type Dog
a.speak();
System.out.println("reproduce() -> " + a.reproduce());
System.out.println("a.kind()    -> " + a.kind() + "   (static method: picked by the declared type)");
System.out.println("Dog.kind()  -> " + Dog.kind());
System.out.println("a.name      -> " + a.name + " (fields use the STATIC type)");
System.out.println("((Dog) a).name -> " + ((Dog) a).name);
new Derived();
```

Output:

```
Woof
reproduce() -> Dog
a.kind()    -> Animal (static)   (static method: picked by the declared type)
Dog.kind()  -> Dog (static)
a.name      -> animal field (fields use the STATIC type)
((Dog) a).name -> dog field
Derived.init sees label = null
```

- **Instance methods** dispatch on the **runtime** type (`Woof`).
- **Static methods and fields** resolve by the **declared** type — *hiding*. `a.kind()` compiles but misleads; call statics via the class name. Field access is never polymorphic — another reason for private fields.
- **Never call overridable methods from a constructor**: `Base()` ran `Derived.init()` *before* `Derived`'s field initialiser, so `label` was `null` (§5.10; *Effective Java* Item 19). Make such methods `private`, `final` or `static`.

Common mistakes

- Accidental overload: `public boolean equals(Book b)` doesn't override `equals(Object)`; `HashSet` never calls it. `@Override` catches this.
- Narrowing access or adding a checked exception in an override.

In the interview

> **30-second answer:** "Overriding is a subclass redefining an inherited instance method with the same signature; the JVM picks the implementation from the object's runtime class. The override may return a subtype, must not reduce visibility or add broader checked exceptions, and can't override final methods. Static methods and fields are hidden, not overridden — resolved by the declared type. I always use @Override."

- *Overloading vs overriding?* Overloading: same name, different parameters, chosen at compile time by argument types. Overriding: same signature in a subclass, chosen at run time by object type.
- *Override a private method?* No — it isn't inherited; a same-named subclass method is unrelated.

6.4 Polymorphism and casting

What it is

Polymorphism ("many forms"): code written against a common supertype works with many classes, each responding its own way — `shape.area()` once, every shape computing its own.

- **Declared (static) type** — the variable's type: `Shape s`.
- **Runtime (dynamic) type** — the object's class: `new Circle(1)`.
- **Upcasting** — subclass object as supertype (`Shape s = new Circle(1)`): always safe, implicit.
- **Downcasting** — supertype reference as subtype (`(Rect) s`): explicit, **checked at run time**, `ClassCastException` if wrong.

How it works

1. The **declared type** decides which methods you may call (`s.side()` won't compile on a `Shape`, even holding a `Square`).
2. The **runtime type** decides which implementation runs.
3. For subtype-only methods use **pattern matching for** `instanceof` (Java 16): `if (s instanceof Square sq) { sq.side(); }` — test and cast in one, with flow-scoped `sq` (§3.5). `null instanceof Foo f` is `false`.
4. Casts that can *never* succeed (unrelated classes, `(String) someInteger`) are compile errors; casts that *might* succeed compile and are checked at run time (`Object o = "hi"; (Integer) o` compiles, then throws).

```java
// code/ch06/polymorphism/Shapes.java (key parts)
static class Shape  { double area() { return 0; } String name() { return getClass().getSimpleName(); } }
static class Circle extends Shape { final double r; ... @Override double area() { return Math.PI * r * r; } }
static class Rect   extends Shape { final double w, h; ... @Override double area() { return w * h; } }
static class Square extends Rect  { Square(double side) { super(side, side); } double side() { return w; } }

List<Shape> shapes = List.of(new Circle(1), new Rect(2, 3), new Square(2));   // upcasting
for (Shape s : shapes) System.out.printf("%-6s area = %.2f%n", s.name(), s.area());

Shape s = shapes.get(2);
// s.side();              -> compile error: Shape has no side()
if (s instanceof Square sq) System.out.println("pattern variable: side = " + sq.side());
Rect r = (Rect) s;                              // downcast: OK, a Square is a Rect
Circle c = (Circle) s;                          // compiles, ClassCastException at runtime
```

Output:

```
Circle area = 3.14
Rect   area = 6.00
Square area = 4.00
total = 13.14
pattern variable: side = 2.0
downcast to Rect ok: w=2.0
ClassCastException: class ch06.polymorphism.Shapes$Square cannot be cast to class ch06.polymorphism.Shapes$Circle
```

Why it matters

Add a `Triangle` and the loop works unchanged — code **open for extension**: the Open/Closed Principle and the basis of Strategy, State, Command… (LLD §2.6, LLD §4.7 OCP).

`instanceof` chains over an *open* hierarchy usually mean the behaviour belongs in an overridden method. Exception: a **sealed** hierarchy with a pattern `switch` (§6.7), checked exhaustive by the compiler.

Common mistakes

- Downcasting without checking.
- Pattern-variable scope: after `if (!(s instanceof Square sq)) return;` `sq` *is* in scope; in the `else` of a positive test it isn't.

TL;DR

Code against the supertype; the runtime type picks the implementation. Upcasts are free; downcasts are checked — prefer `instanceof` patterns, and overriding to type checks.

6.5 Abstract classes

What it is

An **abstract class** is marked `abstract`, **cannot be instantiated**, and may declare **abstract methods** (no body) that concrete subclasses must implement — a *partially built* class holding shared state and code, with the gaps left to subclasses.

How it works

1. `abstract class Report { … }` — `new Report()` is a compile error.
2. `protected abstract String body();` — no braces, ends with `;`.
3. Any abstract method ⇒ the class must be `abstract`; an abstract class may have **zero** abstract methods (just to block instantiation).
4. The first **concrete** subclass implements every inherited abstract method; abstract subclasses may defer.
5. Abstract classes have everything normal classes have: mutable fields, constructors (called via `super(...)`), static members, `final` methods.

6. `abstract` can't combine with `final`, `private` or (on methods) `static`.

The classic use is **Template Method**: a `final` method fixes an algorithm's skeleton and calls abstract "steps" subclasses fill in.

```
// code/ch06/abstracts/AbstractDemo.java (key parts)
static abstract class Report {
    private final String title;
    protected Report(String title) { this.title = title; }
    public final String render() { return header() + "\n" + body() + "\n" + footer(); }   // template method
    protected String header() { return "== " + title + " =="; }   // default step
    protected abstract String body();                             // required step
    protected String footer() { return "-- end --"; }
}
static class SalesReport extends Report {
    private final int units;
    SalesReport(int units) { super("Sales"); this.units = units; }
    @Override protected String body() { return "units sold: " + units; }
}
static class AuditReport extends Report {
    AuditReport() { super("Audit"); }
    @Override protected String body() { return "no issues found"; }
    @Override protected String footer() { return "-- signed: auditor --"; }
}

Report[] reports = { new SalesReport(42), new AuditReport() };
for (Report r : reports) System.out.println(r.render());
Report adHoc = new Report("Ad hoc") {                 // anonymous subclass (§6.11)
    @Override protected String body() { return "one-off content"; }
};
```

Output:

```
== Sales ==
units sold: 42
-- end --
== Audit ==
no issues found
-- signed: auditor --
== Ad hoc ==
one-off content
-- end --
```

Real examples: `java.util.AbstractList` (implement `get` and `size`, get a whole `List`), `java.io.InputStream` (implement `read()`), `HttpServlet` (override `doGet` / `doPost`). Template Method in depth: LLD §11.4.

TL;DR

Abstract class = shared state + shared code + abstract "holes"; not instantiable; subclasses fill the holes. Good for template methods and skeletal implementations.

6.6 Interfaces

What it is

An **interface** is a named **contract**: method signatures implementing classes promise to provide. A class `implements` any number of them, and callers depend on the interface type instead of a concrete class ("program to an interface" — LLD principle 7).

What an interface may contain

Member	Since	Implicit modifiers	Notes
Abstract method	1.0	`public abstract`	The contract
Constant field	1.0	`public static final`	Must be initialised; **no instance fields**
`default` method	Java 8	`public`	Has a body; inherited, overridable. Lets APIs evolve without breaking implementers (`List.sort` , `Collection.stream`)
`static` method	Java 8	`public` (unless `private`)	Called only as `InterfaceName.m()` — **not** inherited by implementers
`private` method (instance or static)	Java 9	—	Helper for default/static methods
Nested types	1.0	`public static`	`Map.Entry`

No constructors, no instance state. An interface can `extend` several interfaces.

How it works

1. `interface Payable { double amount(); }`
2. `class Invoice implements Payable, Printable, Shippable { … }` — implement all abstract methods (or be `abstract`); implementations must be `public` .
3. Use as a type: `List<Payable> bills` .
4. Exactly **one abstract method** = a **functional interface**, implementable by a **lambda** (`() -> value` ; Chapter 11). `@FunctionalInterface` makes the compiler check it.

Multiple inheritance and the diamond rules

When a class inherits the same default method along two paths (the *diamond*), Java resolves it:

1. **Classes win**: a method in the class or any superclass beats any interface default.
2. **More specific interface wins**: if `B extends A` and both define `m()` , `B` 's wins.
3. **Otherwise a compile error** until the class overrides; inside, call a parent with `InterfaceName.super.m()` .

```
// code/ch06/interfaces/InterfacesDemo.java (key parts)
interface Payable {
    double TAX_RATE = 0.2;                          // public static final
    double amount();                                // public abstract
    default double amountAfterTax() { return round(amount() * (1 - TAX_RATE)); }   // Java 8
    static Payable of(double value) { return () -> value; }                        // Java 8, lambda
    private static double round(double v) { return Math.round(v * 100) / 100.0; } // Java 9
}
interface Printable { default String label() { return "printable"; } }
interface Shippable { default String label() { return "shippable"; } }

static class Invoice implements Payable, Printable, Shippable {
    private final double total;
    Invoice(double total) { this.total = total; }
    @Override public double amount() { return total; }
    @Override public String label() {               // conflict: must override
        return Printable.super.label() + "+" + Shippable.super.label();
    }
}

bills.add(new Invoice(100));
bills.add(Payable.of(59.99));
bills.add(new Payable() {                           // anonymous implementation
    public double amount() { return 10; }
    public double amountAfterTax() { return 10; }     // overrides the default
});
```

Output:

```
100.0 -> after tax 80.0
59.99 -> after tax 47.99
10.0 -> after tax 10.0
label: printable+shippable, TAX_RATE = 0.2
Invoice is a Printable? true
```

Real examples: `Comparable`, `Comparator`, `Runnable`, `AutoCloseable`, `Iterable`, `List` / `Set` / `Map` (Chapter 10), `java.util.function.*`. JDBC (the standard database API) is mostly interfaces (`Connection`, `ResultSet`) implemented by each database driver.

Common mistakes

- Implementing an interface method with package access → "attempting to assign weaker access privileges; was public".
- Calling a static interface method via an implementer or instance (`invoice.of(1)`) — not inherited.
- "Constant interface" anti-pattern (an interface only for constants, *Effective Java* Item 22) — use a final class or enum.
- State-like logic in defaults; defaults can't hold state and exist for convenience/evolution.

TL;DR

Interfaces = contracts: abstract + constants + `default` (8) + `static` (8) + `private` (9) methods; no state or constructors; many per class. Conflicts: class wins → more specific interface wins → else override and use `X.super.m()`.

6.7 Sealed classes and interfaces (Java 17)

What it is

A **sealed** class or interface restricts **which** classes may extend/implement it via `permits` (**Java 17, JEP 409**): "a `Shape` is exactly a `Circle`, `Rectangle` or `Polygon` — nothing else".

Why it matters

- Between "anyone may extend" and `final`, sealing gives a **closed, known set** of subtypes — for domain concepts like payment methods, AST nodes, `Success | Failure` results.
- It makes **pattern** `switch` **exhaustive** without `default`; adding a subtype breaks every incomplete `switch` at compile time (§3.6).

How it works

1. `sealed interface Shape permits Circle, Rectangle, Polygon {}` (or `sealed class`).
2. Each permitted subtype directly extends/implements it and is exactly one of `final` (records and enums implicitly are), `sealed` (own `permits`), or `non-sealed` (reopens the hierarchy below it).
3. Permitted subtypes live in the **same module** — or, outside named modules, the **same package**.
4. If all are in the same source file, `permits` may be omitted (inferred).
5. Pairs with records (§6.8), pattern `switch` (Java 21, JEP 441) and record patterns (JEP 440) — syntax in Chapter 3.

```java
// code/ch06/sealed/SealedDemo.java (key parts)
sealed interface Shape permits Circle, Rectangle, Polygon {}
record Circle(double radius) implements Shape {}                // implicitly final
record Rectangle(double w, double h) implements Shape {}
static non-sealed class Polygon implements Shape {               // open again below this point
    final int sides;
    Polygon(int sides) { this.sides = sides; }
}
static final class Triangle extends Polygon { Triangle() { super(3); } }

static String describe(Shape s) {
    return switch (s) {                                          // exhaustive: no default needed
        case Circle c                                  -> "circle r=" + c.radius();
        case Rectangle(double w, double h) when w == h -> "square " + w;          // record pattern + guard
        case Rectangle(double w, double h)             -> "rectangle " + w + "x" + h;
        case Polygon p                                 -> p.sides + "-sided polygon";
    };
}
```

Output:

```
circle r=1.5
square 2.0
rectangle 2.0x5.0
3-sided polygon
Shape is sealed? true, permitted: 3
```

Sealed interfaces + records = **algebraic data types**: records are the "and" (width *and* height), sealed interfaces the "or" (circle *or* rectangle) — *data-oriented programming* (§3.7).

Trade-offs

Use sealed when…	Avoid when…
The variants are part of the domain and change rarely	Building an extension point for other teams/plugins (they can't add subtypes)
You want exhaustive `switch` checks	New variants come often and callers shouldn't need updates — prefer overriding

TL;DR

`sealed … permits A, B, C` = closed set of subtypes (Java 17), each `final` / `sealed` / `non-sealed`. Pair with records and pattern `switch` for exhaustive, compiler-checked handling.

6.8 Records

What it is

A **record** (Java 16, JEP 395) is a class for **transparent, immutable data carriers**. Declare the **components** in the header; `record Point(int x, int y) {}` generates:

1. `private final` fields `x`, `y`;
2. a **canonical constructor** `Point(int x, int y)`;
3. **accessors** `x()`, `y()` (no `get` prefix);
4. `equals` / `hashCode` over all components;
5. `toString()` like `Point[x=3, y=4]`.

~40 lines of hand-written immutable class (like `Money`, §5.9) become one.

Rules

Records can	Records cannot
Implement interfaces	Extend a class (they implicitly extend `java.lang.Record`)
Declare static fields, methods, factories	Declare **instance** fields beyond the components
Declare instance methods	Be extended — implicitly `final`
Add constructors that delegate to the canonical one via `this(...)`	Have setters (fields are `final`)
Override any generated member	Be `abstract`
Be nested (implicitly `static`) or **local** in a method	
Be generic: `record Pair<A, B>(A first, B second) {}`	

How it works — the compact constructor

To validate or normalise, write a **compact canonical constructor**: the record name with **no parameter list**. Parameters are implicit, and fields are assigned automatically **after** its body. You may reassign the *parameters* (e.g. a defensive copy) but never `this.x`.

```
// code/ch06/records/RecordsDemo.java (key parts)
record Point(int x, int y) {
    static final Point ORIGIN = new Point(0, 0);
    Point translate(int dx, int dy) { return new Point(x + dx, y + dy); }
    double distanceTo(Point o) { return Math.hypot(x - o.x, y - o.y); }
}

record Range(int start, int end) {
    Range {                                              // compact canonical constructor
        if (start > end) throw new IllegalArgumentException(start + " > " + end);
    }
    Range(int single) { this(single, single); }          // extra constructors must delegate
    int length() { return end - start; }
}

record Team(String name, List<String> members) {
    Team { members = List.copyOf(members); }             // defensive copy: reassign the PARAMETER
}

record Temperature(double celsius) {
    @Override public String toString() { return celsius + "°C"; }
    static Temperature ofFahrenheit(double f) { return new Temperature((f - 32) * 5 / 9); }
}

// inside main
record Pair(String key, int count) {}                    // local record
```

Output:

```
Point[x=3, y=4] x=3 dist=5.0 moved=Point[x=4, y=5]
equals by value: true, set size: 1
Range[start=2, end=7] length 5, Range[start=5, end=5]
rejected: 9 > 1
Team[name=Core, members=[Ana, Bo]] (unchanged by the caller's add)
100.0°C
local record: [Pair[key=a, count=1]]
```

Uses: DTOs (data transfer objects) for REST APIs (System Design Fundamentals), query results, map keys (`record CacheKey(String user, LocalDate day)`), multiple return values, events/messages, the "and" half of sealed hierarchies. Jackson (≥ 2.12) serialises records; Spring Boot binds configuration to them.

Trade-offs — record vs regular class

Choose a record when…	Choose a class when…
The type *is* its data: equal components are interchangeable	Objects have identity and a lifecycle (`BankAccount` , a session)
Immutable is fine	You need mutable state, or JPA entities (JPA maps objects to tables and needs a no-arg constructor and non-final fields)
All components may be public via accessors	You need to hide the representation

Common mistakes

- Believing records are **deeply** immutable: a `List` or array component stays mutable unless copied (`List.copyOf`) in the compact constructor; array components also break value equality (identity `equals`).
- `this.start = start` in a compact constructor — compile error.
- Writing `getX()` — the accessor is `x()` .

In the interview

> **30-second answer:** "A record is a final, immutable data carrier (Java 16). From the components the compiler generates private final fields, the canonical constructor, accessors, and equals/hashCode/toString over all components. I use a compact constructor for validation and defensive copies. Records can implement interfaces and have static members and methods, but can't extend classes or add instance fields. They pair with sealed interfaces and record patterns."

- *Deeply immutable?* Only if the components are.
- *JPA entity?* Not with standard JPA (needs a no-arg constructor and mutable proxies); records suit DTOs/projections (read-only views of selected columns).

TL;DR

`record R(a, b) {}` = final immutable data class with constructor, accessors, `equals` / `hashCode` / `toString` . Validate and copy in the compact constructor. Not deeply immutable by default.

6.9 Enums

What it is

An **enum** (Java 5) is a class with a **fixed set of named instances** created once at class initialisation: `enum Day { MON, TUE, … }` . Each constant is a full object (a singleton of its kind), so enums can have fields, constructors, methods and per-constant behaviour.

Versus `int` / `String` constants: **type safety** (a `Day` parameter can't receive `42` or `"Mnday"`), namespaced and printable (`Day.MON` prints `MON`), carry data and behaviour, **safe with** `==` (one instance per constant per class loader), and great tooling (`switch` , `EnumSet` , `EnumMap`).

How it works

1. Every enum implicitly extends `java.lang.Enum` — **can't extend a class**, **can implement interfaces**.
2. Constants come first, comma-separated; end with `;` if anything follows.
3. Constructors are implicitly **private**; each constant passes its arguments: `EARTH(5.976e+24, 6.37814e6)` .
4. Built-ins:

Method	Returns
`values()` (static)	A **new** array of all constants in order (cache it if used in a hot loop)
`valueOf(String)` (static)	Constant with that exact name, else `IllegalArgumentException`
`name()`	Declared name — use this for persistence, not the overridable `toString`
`ordinal()`	Zero-based position — never persist or rely on it (*Effective Java* Item 35); reordering changes it
`compareTo(e)`	Order by ordinal (enums are `Comparable`)

5. **Constant-specific bodies** `{ … }` override methods — typically an `abstract` method declared in the enum.
6. `switch` uses bare names (`case SAT, SUN ->`); listing all constants makes a switch expression exhaustive without `default`.
7. `EnumSet` (one bit per constant) and `EnumMap` (array-backed) are the fastest, most compact enum collections.
8. A single-element enum is the most robust **singleton** (*Effective Java* Item 3) — safe against reflection and serialisation attacks (LLD Singleton).

```java
// code/ch06/enums/EnumsDemo.java (key parts)
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN;
    boolean isWeekend() { return this == SAT || this == SUN; }
}
enum Planet {
    MERCURY(3.303e+23, 2.4397e6), EARTH(5.976e+24, 6.37814e6), JUPITER(1.9e+27, 7.1492e7);
    private static final double G = 6.67300E-11;
    private final double mass, radius;
    Planet(double mass, double radius) { this.mass = mass; this.radius = radius; }   // implicitly private
    double surfaceGravity() { return G * mass / (radius * radius); }
}
enum Operation {
    PLUS("+")  { int apply(int a, int b) { return a + b; } },
    TIMES("*") { int apply(int a, int b) { return a * b; } };
    final String symbol;
    Operation(String symbol) { this.symbol = symbol; }
    abstract int apply(int a, int b);
}
static String plan(Day d) {
    return switch (d) {
        case SAT, SUN -> "rest";
        case FRI -> "wrap up";
        case MON, TUE, WED, THU -> "work";
    };
}
EnumSet<Day> weekend = EnumSet.of(Day.SAT, Day.SUN);
EnumSet<Day> weekdays = EnumSet.complementOf(weekend);
Map<Day, Integer> hours = new EnumMap<>(Day.class);
```

Output:

```
MON(0) TUE(1) WED(2) THU(3) FRI(4) SAT(5)* SUN(6)*
FRI -> wrap up, compareTo(MON) = 4, == works: true
valueOf("Friday") -> IllegalArgumentException
MERCURY  g =  3.70 m/s^2
EARTH    g =  9.80 m/s^2
JUPITER  g = 24.81 m/s^2
6 + 7 = 13
6 * 7 = 42
weekdays=[MON, TUE, WED, THU, FRI] hours={MON=8, TUE=8, WED=8, THU=8, FRI=6}
```

Real examples: `DayOfWeek`, `Month`, `ChronoUnit`, `TimeUnit`, `RetentionPolicy`, state machines (`OrderStatus.PLACED → SHIPPED`; LLD state machines), HTTP methods. Design use: LLD §2.10.

Common mistakes

- Persisting `ordinal()` — inserting a constant silently corrupts data. Store `name()` or an explicit code.
- `valueOf` on user input without handling `IllegalArgumentException` and case.
- Mutable fields in enums — shared global state.

TL;DR

Enums = fixed singleton instances with type safety; fields, private constructors, methods, per-constant bodies. Store `name()` not `ordinal()`; `EnumSet` / `EnumMap`; exhaustive `switch` without `default`.

6.10 Nested classes: static nested and inner

What it is

A **nested class** is declared inside another class. Four kinds:

Kind	Declared	Tied to an outer *instance*?	Typical use
Static nested	Member with `static`	No	Helpers belonging to the outer class: `Map.Entry`, builders, a list `Node`
Inner (non-static member)	Member, no `static`	**Yes** — hidden reference to an outer instance	Iterators and views reading the outer object's state
Local	Inside a method/block	Yes in an instance context	One-off helper for that method (§6.11)
Anonymous	Inside an expression, no name	Yes in an instance context	One-off implementation of an interface/abstract class (§6.11)

Nested records, enums and interfaces are implicitly `static`.

How it works

1. Nested and enclosing classes see **all** each other's members, `private` included.
2. Static nested: `new Outer.Node(...)` (or `new Node(...)` inside `Outer`).
3. Inner: created **from an outer instance**: `outer.new ReverseIterator()`; inside an `Outer` instance method, `new ReverseIterator()` uses `this`.
4. In an inner class, `this` is the inner object; **`Outer.this`** is the enclosing one (needed on name clashes).
5. Since Java 16 inner classes may declare `static` members.

```java
// code/ch06/nested/NestedDemo.java (key parts)
static class Node {                                  // static nested
    final String value;
    Node(String value) { this.value = value; }
}

private final List<String> items = new ArrayList<>(List.of("a", "b", "c"));
private final String owner;

class ReverseIterator implements Iterator<String> {   // inner: tied to an outer instance
    private int i = items.size() - 1;                  // reads the outer object's field
    public boolean hasNext() { return i >= 0; }
    public String next() { return items.get(i--) + "@" + NestedDemo.this.owner; }
}

Node n = new Node("static nested");
NestedDemo outer = new NestedDemo("list1");
NestedDemo.ReverseIterator it = outer.new ReverseIterator();
```

Output (first two lines):

```
static nested
c@list1 b@list1 a@list1
```

Common mistakes

- **Inner where static nested would do**: each inner instance keeps its outer object alive — a **memory leak** when inner objects (listeners, tasks) outlive the outer. *Effective Java* Item 24: default to `static`.
- `new Outer.Inner()` from a static context without an outer instance → compile error.

6.11 Local classes, anonymous classes and lambdas

What it is

- A **local class** is declared in a method body and may use the method's **effectively final** locals (never reassigned) — it *captures* their values.
- An **anonymous class** is declared and instantiated in one expression, `new SomeInterface() { … }` or `new SomeClass(args) { … }`; no name, so no constructor, single use.
- A **lambda** (Java 8) implements a **functional interface** compactly and replaces most anonymous classes (Chapter 11).

```java
// code/ch06/nested/NestedDemo.java (continued)
Supplier<String> counterDescription() {
    int size = items.size();                        // effectively final
    class Describer implements Supplier<String> {   // local class
        public String get() { return owner + " has " + size + " items"; }
    }
    return new Describer();
}

Comparator<String> byLengthAnon = new Comparator<>() {            // anonymous class
    @Override public int compare(String a, String b) { return Integer.compare(a.length(), b.length()); }
};
Comparator<String> byLengthLambda = (a, b) -> Integer.compare(a.length(), b.length());   // lambda
```

Output (remaining lines):

```
list1 has 3 items
anonymous: [fig, kiwi, banana]
lambda reversed: [banana, kiwi, fig]
```

Anonymous class vs lambda

	Anonymous class	Lambda
Can implement	Any interface (any number of methods) or extend a class (even abstract)	Only a functional interface
State (fields)	Yes	No
`this` means	The anonymous object	The enclosing instance
Verbosity	High	Low
Use when	Several methods, extending a class (`new Report("Ad hoc") {…}`, §6.5), own state	Single-method callbacks, comparators, stream operations

Common mistakes

- Modifying a captured local (`size++`) → "local variables referenced from an inner class must be final or effectively final".
- Serialising anonymous/inner classes — hidden outer references and synthetic names; avoid.

TL;DR (6.10–6.11)

Static nested = grouping, no outer link (default). Inner = tied to an outer instance (`outer.new Inner()` , `Outer.this`). Local and anonymous classes capture effectively final locals; lambdas replace anonymous classes for functional interfaces.

6.12 Choosing: abstract class, interface, record, enum or plain class

Need	Reach for	Why
A capability many unrelated classes share (`Comparable` , `Closeable`)	**Interface**	Many per class; no state forced on implementers
Contract plus partial implementation with **shared state** or constructors	**Abstract class** (often implementing an interface)	Interfaces can't hold instance state
Evolve a published contract without breaking implementers	Interface **default** method	Java 8's reason for defaults
Fixed, known variants carrying different data	**Sealed interface + records**	Exhaustive, compiler-checked `switch`
Fixed set of instances known at compile time	**Enum**	Type-safe constants with behaviour
Immutable data where equal values are interchangeable	**Record**	Minimal code, correct `equals` / `hashCode`
Mutable object with identity and lifecycle	**Class** (`final` unless designed for extension)	Full encapsulation control

An interface says what a type *can do*; an abstract class says what a family of classes *is* and shares. Modern style is interface first, with an optional skeletal abstract class (`AbstractList` beside `List` , *Effective Java* Item 20). Design discussion incl. ISP (keep interfaces small) and DIP (depend on abstractions): LLD §2.9, LLD §4.9 ISP, LLD §4.10 DIP.

In the interview

Q: "Abstract class or interface?" "An interface defines a contract a class can adopt alongside others; since Java 8 it can carry default and static methods, since 9 private helpers, but never instance state or constructors. An abstract class can have state, constructors and any access level, but a class extends only one. I start with an interface for the abstraction clients depend on, and add an abstract base only if implementations share real state or code."

- *Why multiple interface but not class inheritance?* Classes carry state and constructors, so multiple parents would make state and initialisation ambiguous (the diamond problem). Interfaces have no instance state, and default-method conflicts have explicit rules.

6.13 Object methods: equals, hashCode and toString

What it is

- **`equals(Object o)`** — "logically the same?" Default: identity (`this == o`).

- `hashCode()` — an `int` summary used by hash-based collections to pick a bucket (a slot in their internal array). Default: typically identity-derived.
- `toString()` — readable description for `println`, concatenation, loggers, debuggers. Default: `ClassName@hexHashCode`.

Identity vs equality: `a == b` = same object; `a.equals(b)` = same value. Two `new Book("123", "Dune")` are different objects but should be equal.

The contracts

`equals` (per the `Object` Javadoc) must be:

1. **Reflexive**: `x.equals(x)`.
2. **Symmetric**: `x.equals(y)` ⇔ `y.equals(x)`.
3. **Transitive**: `x.equals(y)` and `y.equals(z)` ⇒ `x.equals(z)`.
4. **Consistent**: same result while unchanged.
5. **Non-null**: `x.equals(null)` is false, never throws.

`hashCode` :

1. **Equal objects ⇒ equal hash codes** (the rule people break).
2. Unequal objects *may* collide — legal, but fewer collisions = faster maps.
3. Consistent within one execution while unchanged.

Override `equals` ⇒ override `hashCode`, from the *same* fields.

How it works — writing them

1. `if (this == o) return true;` — fast path.
2. `if (!(o instanceof Book other)) return false;` — type check + cast (false for `null`).
3. Compare significant fields: `==` for primitives (`Double.compare` / `Float.compare` for floating point — `NaN`, `-0.0`), `equals` for objects, `Objects.equals(a, b)` for nullables (`Objects.equals(null, null)` is `true`), `Arrays.equals` for arrays.
4. `hashCode`: `Objects.hash(f1, f2, …)` (allocates a varargs array; hand-roll `31 * h + …` only in hot paths).
5. `toString`: fields useful for debugging; never secrets (passwords, tokens, personal data).
6. Or use a **record** (§6.8); IDEs also generate them.

```
// code/ch06/objectmethods/EqualsHashCode.java (key parts)
static class EqualsOnly {                                   // BROKEN: equals without hashCode
    final String isbn;
    @Override public boolean equals(Object o) { return o instanceof EqualsOnly other && isbn.equals(other.isbn); }
}

static final class Book {                                   // correct
    private final String isbn;
    private final String title;
    @Override public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Book other)) return false;
        return isbn.equals(other.isbn) && Objects.equals(title, other.title);
    }
    @Override public int hashCode() { return Objects.hash(isbn, title); }
    @Override public String toString() { return "Book[isbn=" + isbn + ", title=" + title + "]"; }
}
```

Output:

```
NoEquals:   equals=false, toString=ch06.objectmethods.EqualsHashCode$NoEquals@<hash>
EqualsOnly: equals=true, set.contains(equal copy)=false  <- hashCode not overridden
Book:       equals=true, same hash=true, set.contains(equal copy)=true, == false
toString:   Book[isbn=123, title=Dune]
equals(null)=false, Objects.equals(null, null)=true
```

Line 2 is the classic bug: equal `EqualsOnly` objects have different default hash codes, so `HashSet` searches the wrong bucket and never calls `equals`. How `HashMap` uses `hashCode` then `equals`: Chapter 10.

`instanceof` or `getClass()` in equals?

	`instanceof` (used above)	`getClass() != o.getClass()`
Subclass equal to parent instance?	Can be	Never
Risk	Breaks symmetry/transitivity if a subclass adds fields to `equals`	Breaks Liskov: a harmless subclass never equals its parent
Common advice	Use it and make the class `final` (as `Book`; records do the equivalent)	Used by some frameworks (e.g. Hibernate-proxy-aware code)

Effective Java Item 10: you cannot extend an instantiable class, add a value component, and keep the `equals` contract — favour composition or final value classes.

Common mistakes

- `equals` without `hashCode`; `equals(Book other)` overload (use `@Override`).
- **Mutable** fields in `equals`/`hashCode`, then mutating a `HashSet` element or `HashMap` key — it becomes unfindable. Prefer immutable keys.
- `==` on `double` fields or on strings (Chapter 7).

In the interview

30-second answer: "`==` compares references; `equals` compares logical value. equals must be reflexive, symmetric, transitive, consistent, and false for null. Equal objects must have equal hashCodes, or hash-based collections break — they bucket by hashCode first. So I override both from the same immutable fields, typically with `Objects.equals` / `Objects.hash`, or use a record."

- *Unequal objects with the same hashCode?* Legal (a collision), but slower.
- *Key's hashCode changes after insertion?* Lookups search the wrong bucket; the entry is effectively lost until the key changes back.
- *Why 31?* An odd prime that spreads values, and `31 * h` optimises to `(h << 5) - h`; convention from *Effective Java* and `String.hashCode`.

TL;DR

`==` = same object; `equals` = same value. Override `equals` and `hashCode` together from the same (ideally immutable) fields; follow the five rules; `@Override`. Records do it for you.

Chapter TL;DR

Topic	Remember
Inheritance	`extends`, one parent, root is `Object`; constructors not inherited, `super(...)` first; prefer composition
Overriding	Same signature, covariant return, same/wider access, no broader checked exceptions, `@Override`; not final/static/private
Hiding	Static methods and fields resolve by declared type
Polymorphism	Declared type decides what you may call; runtime type decides what runs. Downcasts checked at run time; use `instanceof` patterns
Abstract class	Not instantiable; state + constructors + abstract methods; template method
Interface	Contract; abstract + `default` / `static` (8) + `private` (9); no state; many per class; conflicts → override + `X.super.m()`
Sealed (17)	`permits` closed set; subtypes `final` / `sealed` / `non-sealed`; exhaustive `switch`
Records (16)	Final immutable data carriers; compact constructor for validation/copies; not deeply immutable
Enums (5)	Fixed singleton instances; fields, private constructors, per-constant bodies; `EnumSet` / `EnumMap`; don't persist `ordinal()`
Nested	Prefer static nested; inner needs outer instance (`outer.new Inner()`, `Outer.this`); lambdas for functional interfaces
`equals` / `hashCode`	Override together, same fields, contract rules; `toString` for humans

Next: core APIs — `String`, `StringBuilder`, wrappers, `BigDecimal`, `java.time` — in Chapter 7. Design: LLD SOLID, LLD relationships and UML, patterns from LLD Chapter 8. Exam traps: OCP Chapter 6, OCP Chapter 7.

Chapter 7 — Core APIs: Strings, Numbers, Dates and Randomness

Nearly every class touches a few library types: `String` / `StringBuilder` for text, `Math` and the wrapper classes (object versions of primitives: `Integer`, `Double` …) for numbers, `BigDecimal` for money, `java.time` for dates, `Random` for simulations and tests — all in the JDK (`java.lang`, `java.math`, `java.time`, `java.text`, `java.util`), no dependencies needed.

Each API is covered as you'll use it, with the mistakes that cause real bugs (`==` on strings, the `Integer` cache, `new BigDecimal(0.1)`, `YYYY` in date patterns, daylight-saving gaps — the skipped hour when clocks "spring forward"). `String` basics and text blocks are in §2.12, floating-point error in §2.11; this chapter goes deeper.

Code: `code/` (package `ch07.*`), `javac --release 25`, run on JDK 26; every "Output:" is from a real run. Exam traps: `String` / `StringBuilder` chains, `Math` return types, date arithmetic in OCP Chapter 4 — Core APIs; locale formatting and resource bundles in OCP Chapter 11 (sections 5–8).

7.1 Strings: immutability, the pool and memory

What it is

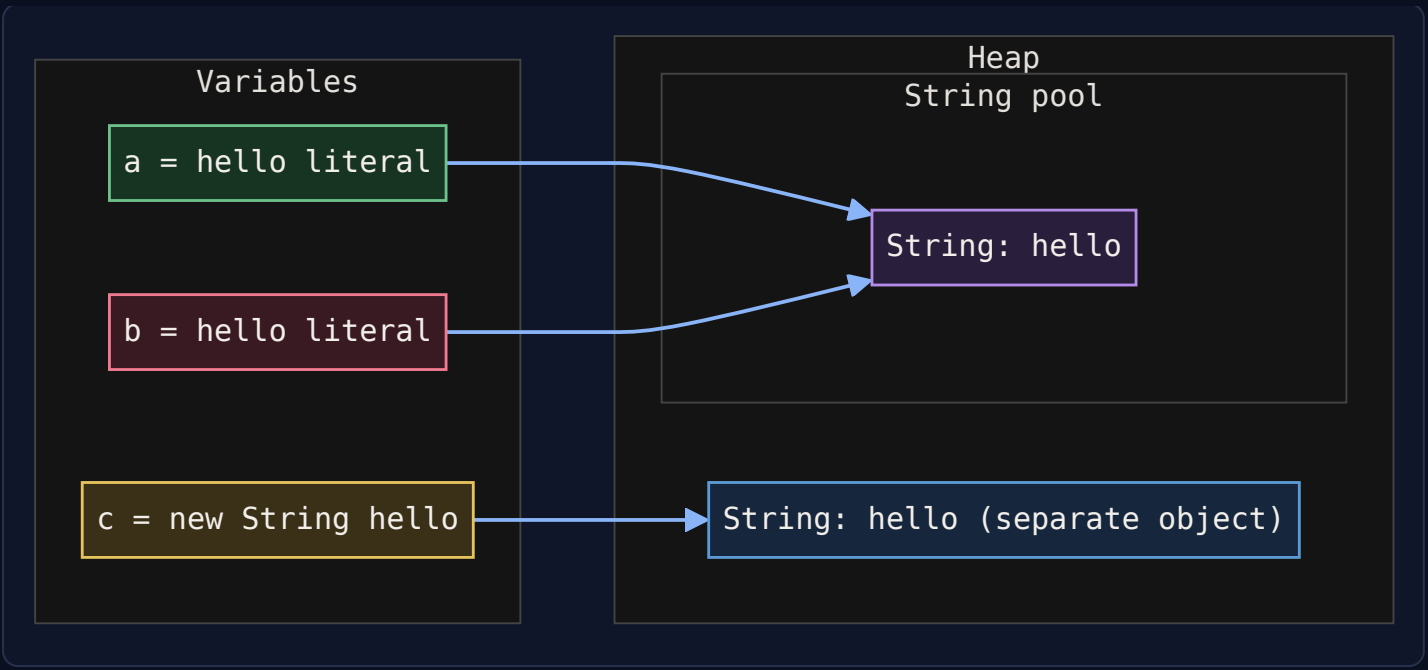
A `String` holds a fixed sequence of characters. **Immutable**: once created its contents never change; every "modifying" method (`toUpperCase`, `replace`, `concat`, `strip` …) returns a **new** `String`. `String` is also `final`, so no subclass can break that guarantee.

Why it matters

Benefit	Why immutability enables it
Safe sharing	Many variables and threads can share one `String` with no locking.
Safe `HashMap` keys	The hash code can't change after insertion; `String` even caches its `hashCode` after the first call.
Security	A path or class name checked by security code can't be altered afterwards.
The string pool	Identical literals can share one object only because nobody can modify it.

How it works — the string pool

1. Every string **literal** (`"hello"`) is stored once in the **string pool** (*intern pool*), a table on the heap (moved there from PermGen in Java 7, so unreachable pooled strings are garbage-collected).
2. Identical literals anywhere refer to the **same** pooled object.
3. **Compile-time constant expressions** (literals joined with `+`, `final` variables initialised with constants) are folded into one literal at compile time, so are pooled too.
4. Strings computed **at run time** (non-final variable concatenation, input, `new String(...)`) are ordinary new heap objects.
5. `s.intern()` returns the pooled copy, adding it if necessary.



code/ch07/strings/StringPool.java:

```
String a = "hello";                    // literal -> string pool
String b = "hello";                    // same pooled object
String c = new String("hello");        // always a new heap object
String d = c.intern();                 // pooled copy
String e = "hel" + "lo";               // constant expression, folded at compile time
String part = "hel";
String f = part + "lo";                // computed at run time -> new object
final String constPart = "hel";
String g = constPart + "lo";           // final + literal initialiser = constant -> pooled
```

Output:

```
a == b: true
a == c: false, a.equals(c): true
a == c.intern(): true
a == "hel" + "lo": true
a == part + "lo": false
a == constPart + "lo": true
s=java, upper=JAVA, same object? false
"x".equals(null)=false, Objects.equals(null,null)=true
```

Lesson: `==` compares references; because of the pool it *sometimes* works on strings — then fails on the first string from user input or a database. Use `equals`, `Objects.equals(a, b)` when either may be `null`, or literal-first `"yes".equals(answer)`.

How strings are stored (Java 9+)

Compact strings (Java 9, JEP 254): characters live in a `byte[]` — one byte each when all fit in Latin-1 (Western European one-byte set), otherwise two (UTF-16) — roughly halving memory for mostly-ASCII text, invisibly. Also since Java 9 (JEP 280) `a + b + c` compiles to an optimised `invokedynamic` call, so a *single* concatenation expression is already efficient; only concatenation *in loops* needs `StringBuilder` (§7.3).

Common mistakes

- Ignoring the result: `s.trim();` / `s.concat("b");` change nothing — assign it.
- `==` comparisons, including `if (cmd == "quit")` chains.
- `intern()` everywhere "to save memory" — the pool is a shared table, and G1 (the default GC) has optional string deduplication. Intern only with measurements.

In the interview

> **30-second answer:** "`String` is immutable and final. That makes it thread-safe, lets it cache its hash code so it's a good map key, keeps security checks valid, and allows the JVM to pool identical literals. Literals and compile-time constants are pooled, runtime strings aren't, so `==` is unreliable — I always use `equals`."

- *Objects created by `new String("hi")`?* Up to two: the pooled literal (if not already pooled at class loading) and the new heap `String`.
- *Is `"ab" == "a" + "b"`?* Yes — a folded constant expression.

7.2 The `String` API toolkit

What it is

`String` has ~100 methods; about 30 matter. Indexes are **0-based**, ranges **half-open**: `substring(begin, end)` includes `begin`, excludes `end` (length `end - begin`).

The methods, by purpose

Job	Methods (version if newer than 8)	Notes
Size & emptiness	`length()`, `isEmpty()`, `isBlank()` (11)	`isBlank`: `""` or whitespace only
Read characters	`charAt(i)`, `toCharArray()`, `chars()` (9), `codePointAt(i)`, `codePoints()` (9)	`charAt` returns a UTF-16 unit (Unicode below)
Search	`indexOf`, `lastIndexOf`, `contains`, `startsWith`, `endsWith`, `indexOf(ch/str, begin, end)` (21)	`indexOf` returns **-1** if absent; `"banana".indexOf("an", 2)` is 3
Extract	`substring(begin)`, `substring(begin, end)`, `subSequence`	`StringIndexOutOfBoundsException` on bad indexes
Whitespace	`trim()`, `strip()` (11), `stripLeading()` / `stripTrailing()` (11), `stripIndent()` (15)	`trim` removes chars ≤ U+0020; `strip` uses Unicode `Character.isWhitespace` (non-breaking/ideographic spaces) — prefer `strip`
Case	`toUpperCase()`, `toLowerCase()`, `Locale` overloads	`toLowerCase(Locale.ROOT)` for identifiers
Replace	`replace(char, char)`, `replace(CharSequence, CharSequence)`, `replaceAll(regex, repl)`, `replaceFirst(regex, repl)`	`replace` is **literal**, `replaceAll` **regex** (regular expression, a pattern language)
Compare	`equals`, `equalsIgnoreCase`, `compareTo`, `compareToIgnoreCase`, `contentEquals`, `regionMatches`, `matches(regex)`	`compareTo` returns <0/0/>0, not necessarily ±1
Split & join	`split(regex)`, `split(regex, limit)`, `splitWithDelimiters` (21), `String.join`, `lines()` (11)	`split` takes a **regex**
Build new text	`concat`, `repeat` (11), `indent` (12), `formatted` (15), `transform` (12), `String.valueOf(x)`, `String.format`	`valueOf(null object)` → `"null"`
Other	`hashCode`, `intern`, `translateEscapes` (15)	

Code

`code/ch07/strings/StringApi.java` (key parts):

```java
String s = "  Hello, Java World  ";
String t = s.strip();                                  // "Hello, Java World"
t.indexOf('o'); t.lastIndexOf('o'); t.indexOf("Kotlin");          // 4, 13, -1
t.substring(7); t.substring(7, 11);                               // "Java World", "Java"
t.replace('o', '0'); t.replaceAll("[aeiou]", "*");                // literal vs regex
"apple".compareTo("banana"); "Z".compareTo("a");                  // -1, -7
"a,b,,c,,".split(",");  "a,b,,c,,".split(",", -1);                // trailing empties dropped vs kept
"1.2.3".split(".");     "1.2.3".split("\\.");                     // regex dot matches everything!
"a+b-c".splitWithDelimiters("[+-]", 0);                           // Java 21
"education".chars().filter(c -> "aeiou".indexOf(c) >= 0).count(); // count vowels
String emoji = "I ❤ Jäva 😀";                  // heart, a-umlaut, grinning face
emoji.length(); emoji.codePointCount(0, emoji.length());
```

Output:

```
length=21 isEmpty=false isBlank("  ")=true
strip -> [Hello, Java World] charAt(7)=J
indexOf('o')=4 lastIndexOf('o')=13 indexOf("Java")=7 indexOf("Kotlin")=-1
contains=true startsWith=true endsWith=true
substring(7)=Java World substring(7, 11)=Java
replace=Hell0, Java W0rld | replaceAll=H*ll*, J*v* W*rld
upper=HELLO, JAVA WORLD lower=hello, java world
repeat=abababab join=a-b-c joinList=usr/local/bin
equals=false equalsIgnoreCase=true compareTo("apple","banana")=-1 compareTo("b","a")=1 compareTo("Z","a")=-7
compareToIgnoreCase=25
split(",")=[a, b, , c]
split(",", -1)=[a, b, , c, , ]
split("\\s+")=[one, two, three]
split(".") on 1.2.3 -> 0 parts; split("\\.") -> [1, 2, 3]
splitWithDelimiters=[a, +, b, -, c]
palindrome(level)=true toCharArray=[a, b, c] vowels=5
lines=[a, b, c] indent=[  x\n]
formatted=Ada is 36 valueOf(3.5)=3.5 valueOf(char[])=hi
I ❤ Jäva 😀 length=11 codePoints=10 charAt(9) is high surrogate? true
```

- **`compareTo`** returns the difference of the first differing characters (`'Z'` 90 − `'a'` 97 = −7), or the length difference if one is a prefix. It is **case-sensitive; uppercase sorts first** (`"Zebra"` before `"apple"`). For human sorting use `String.CASE_INSENSITIVE_ORDER` or a `java.text.Collator`.
- **`split` takes a regex**: `.` means "any character", so `"1.2.3".split(".")` splits everywhere and returns an empty array. Escape (`"\\."`) or `Pattern.quote(".")`; `|`, `+`, `*`, `?`, `(`, `[`, `$`, `^`, `\` are also special.
- **`split` drops trailing empty strings** (`"a:b:".split(":").length` is 2) unless the limit is negative (`split(",", -1)`) — matters for CSV lines with empty last fields.
- **`replace` vs `replaceAll`**: both replace *all* occurrences; literal vs regex. `replaceAll("$", "")` doesn't remove dollar signs; `replace("$", "")` does. For `.` → `-`, use `replace(".", "-")`.

Unicode: `length()` is not "number of characters"

A `char` is a 16-bit **UTF-16 code unit**. Characters beyond the Basic Multilingual Plane (first 65 536) — many emoji, some CJK and historic scripts — need **two** `char`s, a **surrogate pair**. A **code point** is the actual Unicode character number. Above, 10 visible characters give `length()` 11 because 😀 takes two units.

- `charAt`, `substring` and index-based reversal can split a pair (`StringBuilder.reverse` is surrogate-aware; manual loops usually aren't).
- Iterate real characters with `s.codePoints()`; count with `s.codePointCount(0, s.length())`.
- User-perceived characters (flags, family emoji = several code points) need `java.text.BreakIterator`.

Common mistakes

Mistake	Fix
`"TITLE".toLowerCase()` in a Turkish locale gives `"tıtle"` (dotless ı)	`toLowerCase(Locale.ROOT)` for keys, protocols, extensions
`split(".")`, `` `split(" ``	`")` ``
Missing `-1` limit with empty trailing fields	`line.split(",", -1)`
`s.substring(0, 10)` on a shorter string	`s.substring(0, Math.min(10, s.length()))`
`trim` on Unicode spaces	`strip()`
`+=` in a loop	`StringBuilder` (§7.3)

Real use: `line.split("\\s+", 4)` for logs (limit keeps the message whole); `email.strip().toLowerCase(Locale.ROOT)` before lookup; `startsWith("https://")`, `endsWith(".json")` for routing.

TL;DR

Half-open ranges; `indexOf` → -1; `split` / `replaceAll` / `matches` take regexes; `compareTo` is case-sensitive; `length()` counts UTF-16 units. Prefer `strip`, `isBlank`, `repeat`, `lines` (11+).

7.3 `StringBuilder`, `StringBuffer` and `StringJoiner`

What it is

A **`StringBuilder`** is a *mutable* character sequence: append, insert, delete and replace in place, then `toString()` once.

Why it matters

`result += piece` in a loop creates a new `String` each time, copying everything so far — O(n²) for *n* pieces. A `StringBuilder` grows an internal buffer (default capacity 16, or 16 + initial string length; roughly doubles when full), so appends are amortised O(1) (constant on average despite occasional copies) and the build is O(n). A single expression like `"Hello, " + name + "!"` is already efficient (§7.1).

How it works

1. Create: `new StringBuilder()`, `new StringBuilder("start")`, `new StringBuilder(expectedCapacity)`.
2. Mutators **return the same builder** (chainable): `append`, `insert`, `delete`, `deleteCharAt`, `replace`, `reverse`, `repeat` (21). `setCharAt` and `setLength` return `void`.
3. Read: `length()`, `charAt`, `indexOf`, `substring` (returns a `String`; the builder is unchanged).
4. Finish with `toString()`.

code/ch07/strings/BuilderDemo.java (key parts):

```java
StringBuilder sb = new StringBuilder("Java");
sb.append(' ').append(25).append(" LTS");          // methods return this -> chaining
sb.insert(0, ">> ");
sb.replace(3, 7, "JAVA");                          // replace [3, 7) with "JAVA"
sb.delete(0, 3);
sb.deleteCharAt(sb.length() - 1).setCharAt(0, 'j');
new StringBuilder("stressed").reverse();
new StringBuilder().repeat("=-", 5).repeat('!', 2);    // Java 21

StringBuilder x = new StringBuilder("abc"), y = new StringBuilder("abc");
x.equals(y);                                       // false! identity comparison

StringBuilder csv = new StringBuilder();
for (int i = 1; i <= 5; i++) {
    if (!csv.isEmpty()) csv.append(',');
    csv.append(i * i);
}
StringJoiner joiner = new StringJoiner(", ", "[", "]");
```

Output:

```
append:  Java 25 LTS (length 11)
insert:  >> Java 25 LTS
replace: >> JAVA 25 LTS
delete:  JAVA 25 LTS
deleteCharAt+setCharAt: jAVA 25 LT
reverse: desserts
indexOf("25")=5 charAt(1)=A
repeat (21): =-=-=-=-=-!!
x.equals(y)=false x.toString().equals(y.toString())=true x.compareTo(y)=0 "abc".contentEquals(x)=true
loop: 1,4,9,16,25
StringJoiner: [Java, Kotlin, Scala]
capacity=1000 length=0
```

Trade-offs

	`String + +`	`StringBuilder`	`StringBuffer`	`StringJoiner` / `String.join` / `Collectors.joining`
Mutable	No	Yes	Yes	builds internally
Thread-safe	Yes (immutable)	**No**	Yes (every method `synchronized`)	No
Use for	one-off expressions	loops, incremental building	legacy only	joining with delimiter/prefix/suffix

`StringBuffer` (Java 1.0) predates `StringBuilder` (Java 5); its locking is almost never useful — sharing a half-built string between threads is rarely sensible.

Common mistakes

- `sb1.equals(sb2)` — no value-based `equals` / `hashCode` (identity). Compare `toString()`s, `sb1.compareTo(sb2) == 0` (Java 11), or `"abc".contentEquals(sb)`. Never use a builder as a map key or set element.
- `sb.append(a + b + c)` in a loop builds a temporary; chain `.append(a).append(b).append(c)`.
- Sharing one builder between threads.
- `new StringBuilder('x')` — the `char` widens to `int` and becomes the **capacity** (empty builder). Use `"x"`.

In the interview

30-second answer: "Strings are immutable, so `+=` in a loop copies everything each time — $O(n^2)$. `StringBuilder` is a growable mutable buffer with amortised O(1) appends. `StringBuffer` is the synchronized legacy version. A single `+` expression is fine; the compiler optimises it."

- *`new StringBuilder("abc").reverse().insert(1, "-")`*? `"c-ba"`.

7.4 Formatting text and numbers

What it is

Formatting turns values into text with controlled width, alignment, decimals, grouping and symbols. Main tool: the `java.util.Formatter` mini-language behind `String.format`, `"...".formatted(...)` (Java 15) and `printf` (basic specifier table in §2.13). Locale-aware business formatting: `java.text.NumberFormat` / `DecimalFormat`; bytes: `java.util.HexFormat` (Java 17).

How it works — format specifiers

Shape: `%[argument_index$][flags][width][.precision]conversion`.

Part	Examples	Meaning
conversion	`s`, `d`, `f`, `e`, `x` / `X`, `o`, `c`, `b`, `n`, `%%`	value kind
width	`%10s`, `%8d`	minimum characters, right-aligned by default
flags	`-` left-align, `0` zero-pad, `,` grouping, `+` always sign, `(` negatives in parentheses	layout
precision	`%.2f`, `%.2s`	decimals for numbers, max length for strings
argument index	`%2$s`	2nd argument (reuse/reorder)

code/ch07/format/Formatting.java (key parts):

```
String.format("|%s|%10s|%-10s|%.2s|", name, name, name, name);
String.format("|%d|%8d|%-8d|%08d|%,d|%+d|", 42, 42, 42, 42, 1234567, 42);
String.format("|%f|%.2f|%10.3f|%-10.1f|%e|", price, price, price, price, 12345.678);
String.format("|%x|%X|%o|%c|%b|%%|", 255, 255, 8, 'J', null);
String.format("|%2$s %1$s|%s|", "world", "hello");
"%s bought %d items for $%.2f".formatted(name, 3, 9.5);            // Java 15
String.format(Locale.US, "US: %,.2f", 1234567.891);
String.format(Locale.GERMANY, "DE: %,.2f", 1234567.891);
NumberFormat.getCurrencyInstance(Locale.US).format(new BigDecimal("1234.5"));
NumberFormat.getPercentInstance(Locale.US).format(0.256);
NumberFormat.getCompactNumberInstance(Locale.US, NumberFormat.Style.SHORT).format(1_250_000); // Java 12
new DecimalFormat("#,##0.00", DecimalFormatSymbols.getInstance(Locale.US)).format(1234.5);
HexFormat.ofDelimiter(":").withUpperCase().formatHex(new byte[]{10, -1, 127});       // Java 17
```

Output:

```
|Ada|       Ada|Ada       |Ad|
|42|      42|42      |00000042|1,234,567|+42|
|3.141590|3.14|     3.142|3.1       |1.234568e+04|
|ff|FF|10|J|false|%|
|hello world|world|
Ada bought 3 items for $9.50
printf with %n line ending
US: 1,234,567.89
DE: 1.234.567,89
currency=$1,234.50 percent=26% compact=1M
DecimalFormat: 1,234.50 0.50 -7.00
HexFormat: 0A:FF:7F
```

- `%f` defaults to **6** decimals; `%.2f` **rounds** HALF_UP (ties away from zero). Width includes the dot: `"%05.1f"` of 3.14159 is `"003.1"`; `"%-5s|"` of `"ab"` is `"ab |"`.

- `%b` prints `false` for `null` and `true` for *any* other non-Boolean — not a truthiness check.
- In `|%2$s %1$s|%s|` the plain `%s` takes the *first* ordinary argument (`"world"`); don't mix styles.
- Type mismatch (`String.format("%d", 3.5)`) → `IllegalFormatConversionException`; missing arguments → `MissingFormatArgumentException`.
- **Locale matters**: `String.format` uses the JVM default, so a German server prints `1.234.567,89`. For CSV, JSON, parsed logs pass `Locale.ROOT` / `Locale.US`. `Locale.of("de", "DE")` (Java 19) replaces the deprecated `new Locale(...)` constructors.
- `DecimalFormat` patterns: `0` digit always shown, `#` digit hidden if zero, `,` grouping, `.` decimal point, `%` ×100. **Not thread-safe** — one per use or per thread (`DateTimeFormatter`, §7.11, is immutable and safe).
- Compact formatting rounds to whole units by default (`1.25M` → `1M`); with `setMinimumFractionDigits(1)` and its `HALF_EVEN` mode, 1,250,000 prints `1.2M`, 1,260,000 `1.3M`.

When not to use it

- Money *arithmetic* — compute with `BigDecimal` / `long` cents (§7.7), format only at the edge.
- Building SQL/HTML from user input — use prepared statements (placeholders filled safely by the driver) or escaping templates.
- Hot logging — logger placeholders (`log.info("user {} logged in", id)`) skip formatting when the level is off.

TL;DR

`%[index$][flags][width][.precision]conv`. Explicit `Locale` for machine output; `NumberFormat` for currency/percent/compact, `HexFormat` for bytes.

7.5 Math

What it is

`java.lang.Math` is a non-instantiable utility class (private constructor) of `static` methods on primitives; mostly `double`-based; the `*Exact` methods throw on overflow instead of wrapping.

Group	Methods	Return type & behaviour
Absolute, min, max	`abs`, `min`, `max`	argument type after promotion (`min(2.5, -1)` → `-1.0`; §2.7)
Powers & roots	`pow(a, b)`, `sqrt`, `cbrt`, `hypot(x, y)`, `exp`, `log`, `log10`	`double`
Rounding	`round` → `long` (`int` for `float`), `floor` / `ceil` / `rint` → `double`	`round`: ties toward +∞ (`-2.5` → `-2`); `rint`: ties to **even**
Integer division	`floorDiv`, `floorMod` (8), `ceilDiv`, `ceilMod` (18)	round toward −∞ / +∞ instead of 0
Limiting	`clamp(value, min, max)` (21)	keep within a range
Overflow-safe	`addExact`, `subtractExact`, `multiplyExact`, `incrementExact`, `negateExact`, `toIntExact` (8), `absExact` (15), `divideExact` (18), `powExact` (25)	throw `ArithmeticException`
Trig	`sin`, `cos`, `tan`, `atan2`, `toRadians`, `toDegrees`, `PI`, `E`	radians
Misc	`signum`, `random()`	`random()`: `double` in [0.0, 1.0)

code/ch07/math/MathDemo.java output:

```
abs(-7)=7 max(3,9)=9 min(2.5,-1)=-1.0
pow(2,10)=1024.0 sqrt(144)=12.0 cbrt(27)=3.0 hypot(3,4)=5.0
round(2.5)=3 round(-2.5)=-2 rint(2.5)=2.0 floor(-1.1)=-2.0 ceil(-1.1)=-1.0
7/2=3 -7/2=-3 floorDiv(-7,2)=-4 ceilDiv(7,2)=4 floorMod(-7,3)=2
clamp(150,0,100)=100 clamp(-3.5,0.0,1.0)=0.0
signum(-4.2)=-1.0 log10(1000)=3.0 exp(1)=2.7182818284590455
PI=3.141592653589793 sin(PI/2)=1.0 toDegrees(PI)=180.0
multiplyExact overflow -> ArithmeticException: integer overflow
pages=6 (same as (items + perPage - 1) / perPage = 6)
Math.random() in [0,1)? true
```

- `pow` returns `double`; for large integer powers use `BigInteger.pow` or `Math.powExact` (25).
- Integer `/` truncates toward zero (`-7/2 = -3`); `floorDiv` gives `-4`. `floorMod` for wrap-around indexes (`floorMod(-1, 7)` = 6), since `%` keeps the dividend's sign.
- **Paging:** `Math.ceilDiv(items, perPage)` (18) replaces `(items + perPage - 1) / perPage`, which can overflow.
- `Math.abs(Integer.MIN_VALUE)` stays negative (§2.10); `absExact` throws.
- `StrictMath` is bit-identical everywhere; since Java 17 (JEP 306) all floating point is strict anyway, though `Math` may use faster JIT intrinsics for functions like `sin`.

TL;DR

`round` → `long`, ties up; `floor` / `ceil` / `rint` → `double`. `floorDiv` / `floorMod` / `ceilDiv` for negatives and paging, `clamp` (21), `*Exact` for overflow.

7.6 Wrapper classes and autoboxing

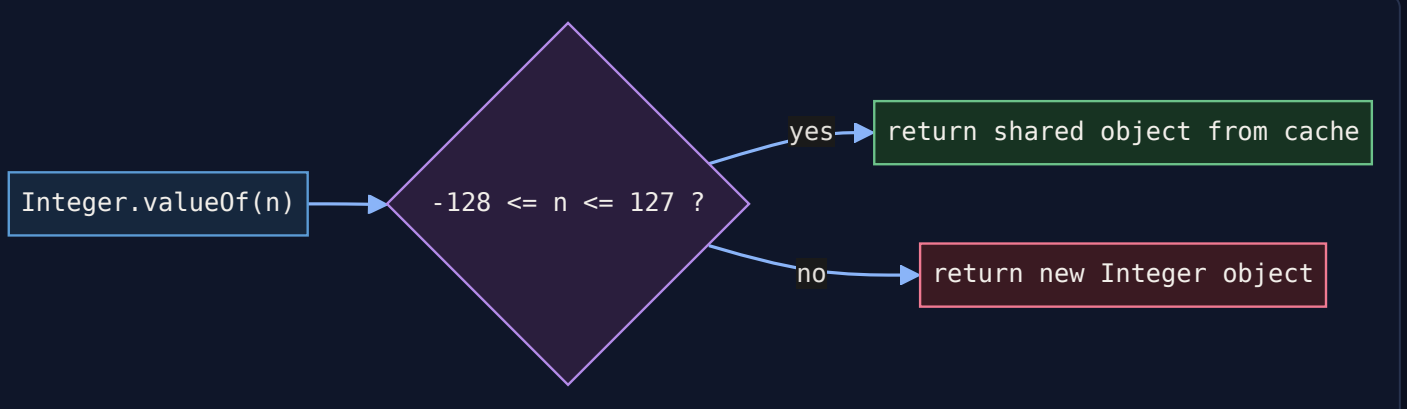
What it is

Each primitive has an immutable **wrapper class**: `byte→Byte`, `short→Short`, `int→Integer`, `long→Long`, `float→Float`, `double→Double`, `char→Character`, `boolean→Boolean`. Needed wherever an *object* is required: collections (`List<Integer>`), generics (Chapter 9), possibly-absent (`null`) values.

Autoboxing (Java 5): the compiler converts primitive → wrapper (`Integer x = 5;` → `Integer.valueOf(5)`); **unboxing** the reverse (`int y = x;` → `x.intValue()`).

How it works

1. **Boxing calls** `valueOf`, which returns **cached shared objects** for `Integer`, `Short`, `Byte`, `Long` (−128…127), `Character` (0…127) and `Boolean`. JLS §5.1.7 guarantees this for boxed `int` / `short` / `byte` in −128…127, `char` ≤ 127 and `true` / `false`; the `Long` cache is a JDK choice. `-XX:AutoBoxCacheMax=<n>` raises `Integer`'s upper bound. `Float` / `Double` have no cache.
2. **Unboxing** calls `xxxValue()` — `NullPointerException` if the reference is `null`.
3. Arithmetic and `<` / `>` unbox first. `==` **between two wrappers compares references**; wrapper `==` primitive unboxes.
4. `new Integer(5)`-style constructors are deprecated since Java 9 (and `new Integer(5) == new Integer(5)` is always false) — use `valueOf` or autobox.



code/ch07/wrappers/WrappersDemo.java (key parts):

```
Integer a = 127, b = 127, c = 128, d = 128;
(a == b); (c == d); c.equals(d);                    // true, false, true

Long big = 42L;
big.equals(42);                                     // false: 42 boxes to Integer, a different class
big == 42;                                          // true: big is unboxed, numeric comparison

Map<String, Integer> stock = new HashMap<>();
int apples = stock.get("apples");                   // get returns null -> NPE while unboxing

List<Integer> list = new ArrayList<>(List.of(10, 20, 30));
list.remove(1);                                     // remove(int index)  -> [10, 30]
list.remove(Integer.valueOf(10));                   // remove(Object o)   -> [30]

Integer maybe = null;
Integer result = true ? maybe : 0;                  // mixed boxed/primitive ternary unboxes -> NPE

Long total = 0L;
for (long i = 0; i < 1_000; i++) total += i;        // unbox, add, re-box: 1,000 allocations
```

Output:

```
boxed then unboxed: 5
127 == 127: true, 128 == 128: false, 128 equals 128: true
parseInt=42 valueOf=42 parseInt("ff",16)=255 toBinaryString(10)=1010 Double.parseDouble("1e3")=1000.0
Boolean.parseBoolean("YES")=false
NumberFormatException: For input string: "4 2"
Integer.MAX_VALUE=2147483647 Integer.compare(3,7)=-1 Integer.sum(2,3)=5 Integer.bitCount(255)=8
Character.isDigit('7')=true Character.toUpperCase('q')=Q Character.getNumericValue('9')=9
Long(42).equals(Integer 42): false, big == 42: true
NPE: Cannot invoke "java.lang.Integer.intValue()" because the return value of "java.util.Map.get(Object)" is null
safe: 0
after remove(1) and remove(Integer.valueOf(10)): [30]
ternary unboxing NPE
total=499500
```

Parsing and converting

You have	You want	Use
`String`	`int`	`Integer.parseInt(s)` (radix overload: `parseInt("ff", 16)`)
`String`	`Integer`	`Integer.valueOf(s)`
`int`	`String`	`String.valueOf(i)`, `Integer.toString(i)`, `toBinaryString` / `toHexString`
`String`	`boolean`	`Boolean.parseBoolean(s)` — `true` **only** for `"true"` (any case, so `"True"` works); `"YES"` is `false`
`double`	`int`	`(int) d` (truncates) or `Math.round`

Bad numeric text throws unchecked **`NumberFormatException`**; parsers don't trim (`" 42"` fails) — validate or catch at the input boundary.

Handy statics: `MAX_VALUE` / `MIN_VALUE`, `Integer.compare(a, b)` (instead of overflow-prone `a - b` in comparators), `Integer.sum/max/min` (as method references, `Integer::sum`), `Character.isDigit/isLetter/isWhitespace/toUpperCase/getNumericValue`.

Trade-offs — primitive vs wrapper

	`int`	`Integer`
Memory	4 bytes	object header + value (typically 16 bytes) + reference
`null`	No (default 0)	Yes — "unknown", dangerous when unboxed
Collections/generics	Not allowed	Required
`==`	values	references
Hot loops	fast	boxing allocates (except cached values)

Use primitives for locals, counters and arithmetic — the `Long total` loop allocates every iteration (*Effective Java* Item 61). For large numeric data prefer `int[]` or `IntStream` over `List<Integer>`.

Common mistakes

1. `==` on wrappers — passes tests with small values, fails at 128.
2. `Long.equals(Integer)` is always false (`Short s = 100; s.equals(100)` too); `map.get(42)` on a `Map<Long, …>` returns `null` — use `42L`.
3. Unboxing `null`: `Map.get`, nullable DB columns, mixed `Integer` / `int` ternaries.
4. `List<Integer>.remove(1)` removes by **index**; `remove(Integer.valueOf(1))` by value.
5. `synchronized (count)` on a wrapper — cached instances are shared JVM-wide; `javac` warns about locking on *value-based* classes (instances meant as plain values, not identities).

In the interview

30-second answer: "Wrappers let primitives live in collections and generics. Autoboxing calls `valueOf`, which caches −128 to 127, so `==` on two `Integer`s is true for 127 and false for 128 — always use `equals`. Unboxing `null` throws NPE, which bites with `Map.get` and mixed ternaries. Primitives for arithmetic, wrappers only where an object or null is needed."

- `Integer i1 = 1000, i2 = 1000; i1 == i2` *guaranteed false?* Not guaranteed (the cache bound is configurable), false by default — it's reference comparison either way.

7.7 `BigInteger` and `BigDecimal`

What it is

Immutable `java.math` classes for numbers primitives can't hold exactly:

- **`BigInteger`** — integers of *any* size (memory-bound), beyond `long` (~9.2×10^{18}): factorials, cryptography, combinatorics.
- **`BigDecimal`** — exact decimals: arbitrary-precision **unscaled value** + **scale** (digits after the point). `12.30` = unscaled `1230`, scale `2`. For money and anything matching human decimal arithmetic.

How it works

1. **Immutable**, no operators: `add`, `subtract`, `multiply`, `divide`, `pow`, `negate`, `abs`, `compareTo` return new objects — assign (`total = total.add(x);`).
2. **Create `BigDecimal` from a `String`** (`new BigDecimal("0.10")`) or `BigDecimal.valueOf(double)` (via `Double.toString`, so `valueOf(1.10).scale()` is 1). `new BigDecimal(0.1)` copies the double's exact binary value — 55 digits of noise.
3. **Division** needs rounding when non-terminating: `divide(divisor, scale, RoundingMode)` or `divide(divisor, MathContext)` (precision + rounding mode). Plain `divide` throws `ArithmeticException` for 10/3.
4. **Rounding** is explicit: `setScale(2, RoundingMode.HALF_EVEN)`. Modes: `UP` / `DOWN` (away from/toward zero), `CEILING` / `FLOOR` (toward +∞/−∞), `HALF_UP` (school), `HALF_DOWN`, `HALF_EVEN` (**banker's rounding**: ties to

the even neighbour, avoiding upward bias — `-1.5` → `-2`), `UNNECESSARY` (throws if rounding is needed).

5. `equals` **compares value** ***and scale*** (`2.0` ≠ `2.00`); `compareTo` **compares value only** — use `compareTo(...) == 0`.

code/ch07/big/BigNumbers.java (key parts):

```
BigInteger f = BigInteger.ONE;
for (int i = 2; i <= 30; i++) f = f.multiply(BigInteger.valueOf(i));      // 30!
BigInteger.TWO.pow(100);
BigInteger.valueOf(3).modPow(BigInteger.valueOf(200), BigInteger.valueOf(1000));
f.longValueExact();                                    // throws: doesn't fit

new BigDecimal("0.1").add(new BigDecimal("0.2"));      // 0.3 exactly
new BigDecimal(0.1);                                   // the double's binary value
new BigDecimal("2.0").equals(new BigDecimal("2.00"));  // false (scale differs)
ten.divide(three);                                     // ArithmeticException
ten.divide(three, 4, RoundingMode.HALF_UP);            // 3.3333

BigDecimal price = new BigDecimal("19.99");
BigDecimal subtotal = price.multiply(BigDecimal.valueOf(3));
BigDecimal tax = subtotal.multiply(new BigDecimal("0.0825")).setScale(2, RoundingMode.HALF_EVEN);
```

Output:

```
30! = 265252859812191058636308480000000 (108 bits)
2^100 = 1267650600228229401496703205376
gcd(84,36)=12 3^200 mod 1000=1 isProbablePrime(97)=true sqrt(10^20)=10000000000
30!.longValueExact(): BigInteger out of long range
double 0.1+0.2=0.30000000000000004 BigDecimal=0.3
new BigDecimal(0.1)=0.1000000000000000055511151231257827021181583404541015625
BigDecimal.valueOf(0.1)=0.1
2.0 equals 2.00? false, compareTo == 0? true, scales 1 vs 2, stripTrailingZeros equal? true
10/3: Non-terminating decimal expansion; no exact representable decimal result.
10/3 scale 4 HALF_UP=3.3333 MathContext.DECIMAL64=3.333333333333333
2.345 -> HALF_UP 2.35, HALF_EVEN 2.34, DOWN 2.34, FLOOR 2.34
2.355 -> HALF_UP 2.36, HALF_EVEN 2.36, DOWN 2.35, FLOOR 2.35
-2.345 -> HALF_UP -2.35, HALF_EVEN -2.34, DOWN -2.34, FLOOR -2.35
subtotal=59.97 tax=4.95 total=64.92
share=33.33 leftover cent(s)=0.01 (divideAndRemainder: 33.00 r 1.00)
```

For `2.345` the tie goes *up* with `HALF_UP` (2.35) but to the *even* digit with `HALF_EVEN` (2.34); for `2.355` both give 2.36. `FLOOR` differs from `DOWN` only for negatives. Last line: $100.00 split three ways leaves **one cent**; real systems allocate the remainder explicitly (e.g. first share 33.34).

Trade-offs

	`double`	`long` cents	`BigDecimal`
Exact decimals	No	Yes (fixed 2 places)	Yes (any scale)
Speed	Fastest	Fast	Much slower, allocates
Range	Huge, approximate	$\pm 9.2 \times 10^{16}$ dollars	Unlimited
Good for	Science, graphics, statistics	Single-currency payments, high throughput	Finance, tax, FX, accounting

Common mistakes

- `new BigDecimal(0.1)`; unassigned `total.add(x);`; unbounded `divide`.
- `equals` for numeric comparison. `HashSet` / `HashMap` **use** `equals` (`2.0`, `2.00` are different keys) while `TreeSet` uses `compareTo` (one key) — normalise with `stripTrailingZeros()` or fixed `setScale`.
- Rounding too early vs at the end — follow domain (often legal) rules; tax is frequently rounded per invoice line.

- `BigInteger` / `BigDecimal` in tight loops where `long` suffices.

Real use: payment platforms store integer minor units *or* `DECIMAL(19,4)` columns (exact SQL type: 19 digits, 4 after the point) mapped to `BigDecimal`; RSA key generation uses `BigInteger.modPow` and `isProbablePrime`.

In the interview

> **30-second answer:** "For money I use `BigDecimal` built from strings, or long minor units — doubles are binary fractions, so 0.1 isn't representable. `BigDecimal` is immutable, needs an explicit scale and `RoundingMode` for division — usually `HALF_EVEN` for finance — and I compare with `compareTo` because `equals` also checks scale."

- *`new BigDecimal("1.0").equals(new BigDecimal("1.00"))`?* False — scale differs; `compareTo` is 0.

TL;DR

`BigInteger` for huge integers, `BigDecimal` for exact decimals. Build from `String`, assign every result, give `divide` a scale + `RoundingMode`, compare with `compareTo`, `HALF_EVEN` for money.

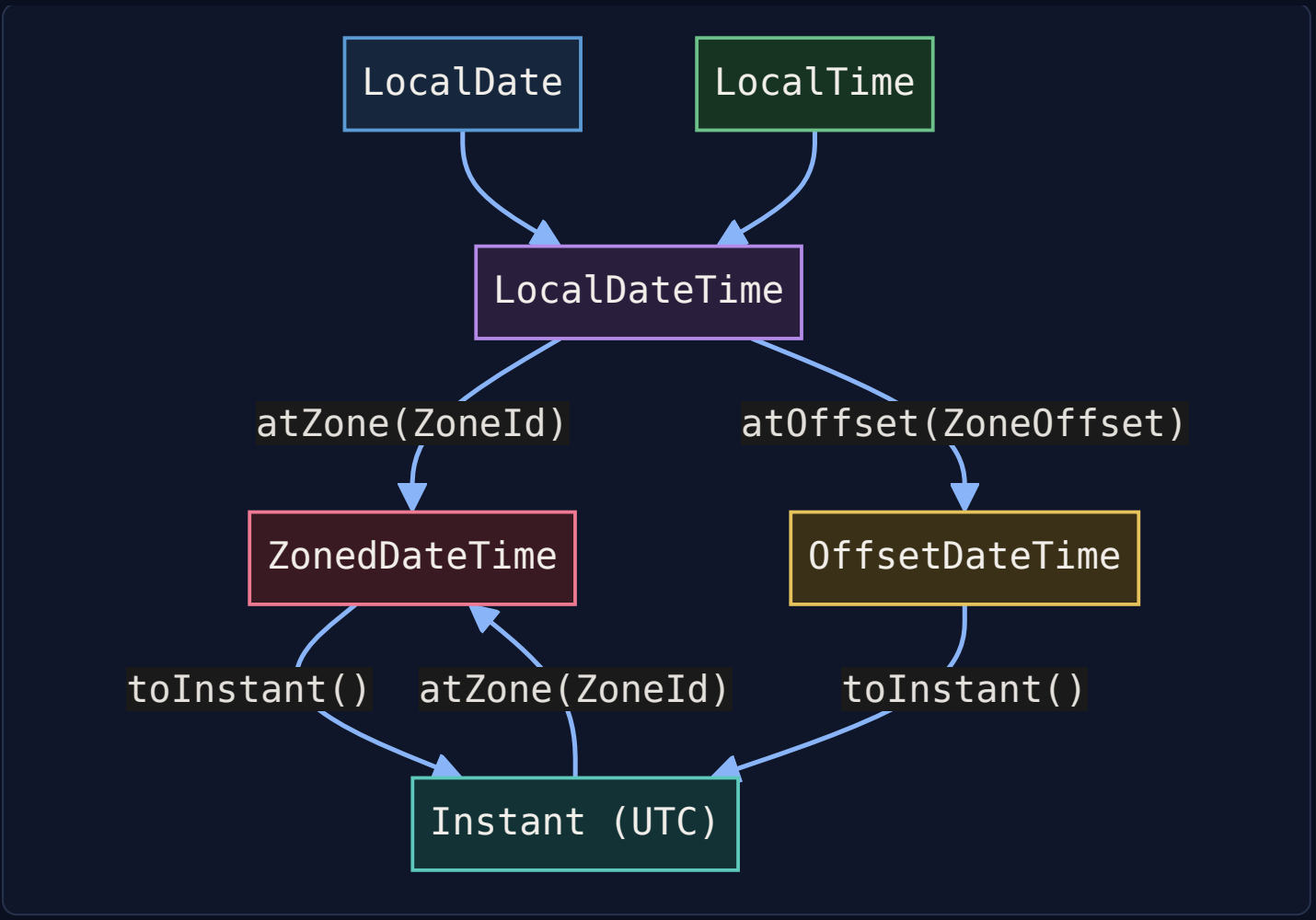
7.8 The `java.time` model

What it is

`java.time` (JSR 310, **Java 8**, designed by Joda-Time's author) replaced `java.util.Date`, `Calendar` and `SimpleDateFormat` — mutable, not thread-safe, 0-based months, conflating "a moment" with "a calendar date". All `java.time` types are **immutable** and **thread-safe**.

Different questions need different types. A birthday has no time zone (a region's rules for its offset from UTC, including daylight saving time — DST — shifts). A payment is an exact moment. "The 9:00 meeting in Tokyo" needs a zone.

Type	Holds	Example	Use for
`LocalDate`	year-month-day	`2025-09-16`	birthdays, holidays, due dates
`LocalTime`	hour-minute-second-nanos	`09:30`	opening hours, alarms
`LocalDateTime`	date + time, **no zone**	`2025-09-16T09:30`	wall-clock time whose zone is implied elsewhere
`ZonedDateTime`	date + time + `ZoneId` with DST rules	`2026-03-20T09:00-04:00[America/New_York]`	scheduling in a region, display
`OffsetDateTime`	date + time + fixed offset	`2026-03-20T09:00-04:00`	APIs, DB `timestamp with time zone`, logs
`Instant`	point on the UTC timeline (seconds + nanos since 1970-01-01T00:00Z)	`2025-09-16T05:20:00Z`	event timestamps, "when"
`Duration`	**time** amount (seconds + nanos)	`PT2H15M30S`	timeouts, elapsed time
`Period`	**calendar** amount (years, months, days)	`P1Y2M3D`	ages, subscriptions, "add one month"
`ZoneId` / `ZoneOffset`	region `Europe/London` / fixed `+05:30`		zone conversion
Also	`Year`, `YearMonth`, `MonthDay`, `DayOfWeek`, `Month` (enums), `Clock`		



How to choose

1. *When something happened* (logs, orders, audit) → `Instant`, stored in UTC, converted for display.
2. Date with no time → `LocalDate` (not midnight UTC, which shifts the day for users west of Greenwich).
3. Future appointment in a place → `LocalDateTime` + `ZoneId`, not `Instant`: if DST rules change before the meeting, it should still be 9:00 local.
4. Machine elapsed time → `System.nanoTime()` or `Duration.between(a, b)`; calendar arithmetic → `Period`.

Common API conventions

Prefix	Meaning	Example
`of`	from parts	`LocalDate.of(2025, 9, 16)`
`now`	current value (optionally from a `Clock`/zone)	`LocalDate.now(clock)`
`parse`	from text	`LocalDate.parse("2025-09-16")`
`get`	read a field	`getDayOfWeek()`
`with`	copy with one field changed	`withDayOfMonth(1)`
`plus` / `minus`	copy shifted	`plusMonths(1)`
`to` / `at`	convert	`toLocalDate()`, `atZone(zone)`
`is`	query	`isBefore`, `isLeapYear`

Months are **1-based** (`Month.JANUARY` = 1), unlike `Calendar`.

Legacy interop

Convert `Date` at the boundary: `Date.from(instant)`, `date.toInstant()`, `GregorianCalendar.from(zdt)`, `java.sql.Timestamp.from(instant)`. JDBC 4.2+ drivers read/write `LocalDate`, `LocalDateTime`, `OffsetDateTime` directly via `getObject(col, LocalDate.class)`.

7.9 Local dates, times, adjusters and `Period`

How it works

1. **Create**: `LocalDate.of(2025, Month.SEPTEMBER, 16)`, `LocalTime.of(9, 30)`, `LocalDateTime.of(date, time)`, `LocalDate.parse("2023-09-19")`; invalid values throw `DateTimeException` immediately.
2. **Read**: `getYear`, `getMonth` / `getMonthValue`, `getDayOfMonth`, `getDayOfWeek`, `getDayOfYear`, `isLeapYear`, `lengthOfMonth`.
3. **Adjust** (new object): `plusDays/Weeks/Months/Years`, `minus…`, `withDayOfMonth`, **`TemporalAdjusters`** ("last day of month", "next Friday").
4. **Compare**: `isBefore`, `isAfter`, `isEqual`, `equals`, `compareTo`.
5. **Measure**: `Period.between(a, b)` → years + months + days; `ChronoUnit.DAYS.between(a, b)` → one total.

`code/ch07/time/LocalDates.java` (key parts):

```
LocalDate release = LocalDate.of(2025, Month.SEPTEMBER, 16);     // Java 25 GA date
LocalDate d = LocalDate.of(2024, 1, 31);
d.plusDays(1);                                                   // result ignored -> d unchanged
d.plusMonths(1);                                                 // 2024-02-29
release.with(TemporalAdjusters.lastDayOfMonth());
release.with(TemporalAdjusters.next(DayOfWeek.FRIDAY));
Period gap = Period.between(LocalDate.of(2023, 9, 19), release);
ChronoUnit.DAYS.between(LocalDate.of(2023, 9, 19), release);
Period.between(LocalDate.of(1990, 2, 28), LocalDate.of(2026, 2, 27)).getYears();   // age
LocalDate.of(2026, 3, 2).datesUntil(LocalDate.of(2026, 3, 9))                    // Java 9
        .filter(x -> x.getDayOfWeek().getValue() <= 5) ...
LocalDate.of(2025, 2, 29);                                       // DateTimeException
```

Output:

```
date=2025-09-16 time=09:30 dateTime=2025-09-16T09:30
dayOfWeek=TUESDAY dayOfYear=259 month=SEPTEMBER monthValue=9 leapYear=false lengthOfMonth=30
d after ignored plusDays: 2024-01-31
2024-01-31 plusMonths(1)=2024-02-29 (clamped to month end, leap year)
2025-01-31 plusMonths(1)=2025-02-28
withDayOfMonth(1)=2025-09-01 lastDayOfMonth=2025-09-30 nextFriday=2025-09-19
isBefore=true isAfter=false equals=true
Period between Java 21 and 25 GA: P1Y11M28D = 1y 11m 28d; total days=728 total months=23
age on 2026-02-27 = 35
weekdays in week of 2026-03-02: 2 3 4 5 6
YearMonth 2028-02 has 29 days
DateTimeException: Invalid date 'February 29' as '2025' is not a leap year
```

- **Month arithmetic clamps** to the last valid day (Jan 31 + 1 month = Feb 28/29; `2024-03-31.minusMonths(1)` = `2024-02-29`), so `plusMonths(1).minusMonths(1)` may not round-trip. Billing must define "monthly on the 31st".
- **`Period` vs totals**: `P1Y11M28D` isn't 728 days re-expressed (months vary). `Period` for display (Jan 15 → Apr 15 is `P3M`), `ChronoUnit.X.between` for one count (90 days).
- The age is 35 the day *before* the 36th birthday — `Period.between` gets this right; subtracting years doesn't.
- `datesUntil` (Java 9) excludes the end, like most Java ranges (2026-03-02 → 03-09 is 7 days).

Common mistakes

- Ignoring return values (`date.plusDays(1);`).
- `LocalDateTime` for event timestamps — no zone, so no instant. Use `Instant` or `OffsetDateTime`.

- `LocalDate.now()` inside business logic — tests become date-dependent. Inject a `Clock` (§7.10).

7.10 Zones, instants, `Duration` and daylight saving

What it is

A **time zone** (`ZoneId` , `America/New_York`) is a region's rules: its UTC offset and when it changes (**DST**). An **offset** (`ZoneOffset` , `-04:00`) is a fixed amount. Rules come from the IANA time-zone database shipped with the JDK and refreshed in JDK updates.

How it works

1. `instant.atZone(zone)` shows one moment locally: the same `Instant` is 01:20 New York, 10:50 Kolkata, 14:20 Tokyo (equal instants, different local readings).
2. `zdt.withZoneSameInstant(otherZone)` converts (same moment, different clock); `withZoneSameLocal` keeps the clock and changes the moment — rarely wanted.
3. **DST gap** (spring forward): nonexistent local times shift forward by the gap. **Overlap** (fall back): repeated times default to the earlier offset (`withLaterOffsetAtOverlap()` for the other).
4. **`Duration` is exact time; `Period` is calendar time.** Across DST, `plus(Period.ofDays(1))` keeps the local clock time (a 23-hour "day"); `plus(Duration.ofDays(1))` adds exactly 24 hours.
5. A **`Clock`** supplies "now": `Clock.systemUTC()` / `systemDefaultZone()` in production, `Clock.fixed(...)` / `Clock.offset` in tests.

`code/ch07/time/ZonesAndInstants.java` (key parts):

```
Instant t = Instant.ofEpochSecond(1_758_000_000L);
t.atZone(ZoneId.of("America/New_York")); // and Asia/Kolkata, Asia/Tokyo
ZonedDateTime call = ZonedDateTime.of(LocalDateTime.of(2026, 3, 20, 9, 0), ny);
call.withZoneSameInstant(kolkata).toLocalTime();
ZonedDateTime gap = ZonedDateTime.of(LocalDateTime.of(2026, 3, 8, 2, 30), ny);     // doesn't exist
ZonedDateTime before = ZonedDateTime.of(LocalDateTime.of(2026, 3, 7, 12, 0), ny);
before.plus(Period.ofDays(1));  before.plus(Duration.ofDays(1));
Duration.between(before, before.plusDays(1)).toHours();
Clock fixed = Clock.fixed(Instant.parse("2026-01-01T00:00:00Z"), ZoneOffset.UTC);
LocalDate.now(fixed);
```

Output:

```
epoch=1970-01-01T00:00:00Z t=2025-09-16T05:20:00Z t.toEpochMilli()=1758000000000
NY:      2025-09-16T01:20-04:00[America/New_York]
Kolkata: 2025-09-16T10:50+05:30[Asia/Kolkata]
Tokyo:   2025-09-16T14:20+09:00[Asia/Tokyo]
9:00 NY call in Kolkata: 18:30; withZoneSameLocal keeps 09:00 but changes the instant: 2026-03-
20T09:00+05:30[Asia/Kolkata]
02:30 in the gap becomes: 2026-03-08T03:30-04:00[America/New_York]
plus(Period.ofDays(1)) = 2026-03-08T12:00-04:00[America/New_York]
plus(Duration.ofDays(1)) = 2026-03-08T13:00-04:00[America/New_York]
hours between noon and noon next day: 23
Duration PT2H15M30S = 2h 15m 30s; Period.of(1,2,3)=P1Y2M3D Period.ofWeeks(2)=P14D
OffsetDateTime=2026-03-20T09:00-04:00 toInstant=2026-03-20T13:00:00Z
LocalDate.now(fixedClock)=2026-01-01 Instant.now(fixedClock)=2026-01-01T00:00:00Z
Date.from(t).getTime()==t.toEpochMilli(): true, back to Instant: 2025-09-16T05:20:00Z
```

US clocks spring forward on Sunday 8 March 2026 at 02:00: 02:30 becomes 03:30 EDT and Saturday noon → Sunday noon is 23 hours (the fall-back day, 1–2 November 2026, is 25). Offsets aren't always whole hours (Kolkata `+05:30`); `Period.ofWeeks(2)` is stored as `P14D` .

Trade-offs — which "now"?

Need	Use
Timestamp to store/send	`Instant.now(clock)`
How long code took	`System.nanoTime()` (monotonic; `Instant.now()` can jump when the clock is adjusted)
Today's date for a user	`LocalDate.now(clock.withZone(userZone))` — "today" depends on the zone
Testable logic	Inject `Clock`; tests use `Clock.fixed` / `Clock.offset`

Common mistakes

- Implicit `ZoneId.systemDefault()` — servers are often UTC, users aren't.
- Abbreviations like `"EST"` / `"IST"` (IST = India, Ireland, Israel) — use region IDs (`Asia/Kolkata`).
- `Duration.ofDays(1)` when you mean "same time tomorrow".
- Future local events stored as `Instant` (break on DST rule changes); past events as `LocalDateTime` (lose the moment).

In the interview

> **30-second answer:** "I store moments as `Instant` in UTC and convert to `ZonedDateTime` with the user's `ZoneId` only for display. Dates without time are `LocalDate`. `Duration` is exact seconds, `Period` calendar units — they differ across DST. java.time is immutable and thread-safe, and I inject a `Clock` so time-dependent code is testable."

- *`ZonedDateTime` vs `OffsetDateTime`?* Zoned knows the region's DST rules; Offset has a fixed offset — better for serialising a moment unambiguously.
- *Why was `SimpleDateFormat` a problem?* Mutable and not thread-safe; a shared static instance corrupted results under load. `DateTimeFormatter` is immutable.

TL;DR

`Instant` for moments, `LocalDate` for dates, `ZonedDateTime` for local time in a region. Convert with `atZone` / `withZoneSameInstant`. DST: gaps shift forward, overlaps take the earlier offset. `Duration` ≠ `Period`. Inject `Clock`.

7.11 Formatting and parsing dates

How it works

1. Every `java.time` `toString()` prints **ISO-8601** (`2025-12-29T14:05:09`) and `parse(text)` reads it back — ideal for JSON, logs, databases.
2. `DateTimeFormatter` offers predefined formatters (`ISO_LOCAL_DATE`, `ISO_OFFSET_DATE_TIME`, `BASIC_ISO_DATE`, `RFC_1123_DATE_TIME` …), localized styles (`ofLocalizedDate(FormatStyle.MEDIUM)`) and patterns (`ofPattern("dd MMM uuuu")`).
3. `value.format(formatter)`; `LocalDate.parse(text, formatter)`; bad text → `DateTimeParseException`.
4. `DateTimeFormatter` is **immutable and thread-safe** — keep it in a `static final` field (building one parses the pattern).

Pattern letters you need

Letter	Meaning	Example (2025-12-29 14:05:09)
`uuuu` / `yyyy`	year / year-of-era	2025
`YYYY`	**week-based year** — almost never wanted	2026 (!)
`MM` / `MMM` / `MMMM`	month number / short / full	12 / Dec / December
`dd`	day of month	29
`D`	day of year	363
`EEE` / `EEEE`	day name short / full	Mon / Monday
`HH` / `hh` + `a`	hour 0–23 / 1–12 + AM/PM	14 / 02 PM
`mm`	**minute** (not month)	05
`ss`, `SSS`	second, fraction	09, 000
`VV`, `z`, `XXX`	zone ID, zone name, offset	Europe/London, GMT, Z
`'text'`	literal	`'T'`

`code/ch07/time/DateFormatting.java` output:

```
toString (ISO-8601): 2025-12-29T14:05:09
ISO_LOCAL_DATE: 2025-12-29
BASIC_ISO_DATE: 20251229
custom: 29 Dec 2025, 02:05 PM
EEEE d MMMM (fr): lundi 29 décembre
MEDIUM (US): Dec 29, 2025
yyyy-MM-dd: 2025-12-29   YYYY-MM-dd: 2026-12-29
MM=12 mm=05
parsed=2026-03-08 SUNDAY
DateTimeParseException: Text '2026-13-01' could not be parsed: Invalid value for MonthOfYear (valid values 1 - 12): 13
ISO_OFFSET_DATE_TIME: 2025-12-29T14:05:09Z | zone pattern: 14:05 Europe/London (GMT)
```

The `YYYY` bug: 29 December 2025 is a Monday in the week containing 1 January 2026, so (US week rules) its *week-based year* is 2026 and `YYYY-MM-dd` prints `2026-12-29` — a year ahead. This has caused real outages (expired certificates, broken bookings) in late December. Use `uuuu` / `yyyy`; `YYYY` only with ISO week numbers (`ww`).

`u` is the proleptic year (extends back through year 0 into negatives); `y` is year-of-era (needs era `G` under `ResolverStyle.STRICT`). With the default smart resolver both work for modern dates; use `uuuu` with strict parsing.

Common mistakes

- `MM` vs `mm`, `HH` vs `hh`, `YYYY` vs `yyyy`, `DD` (day of year) vs `dd`. (`08/03/2026` meaning 3 August is `MM/dd/uuuu`.)
- Parsing names without a `Locale` — `ofPattern("MMM", Locale.US)`, else the default locale decides "Dec" vs "déc.".
- Time fields in a pattern used on a `LocalDate` → `UnsupportedTemporalTypeException`.
- Hand-rolled ISO formats instead of `toString()` / `parse()` / `ISO_*`.

TL;DR

ISO `toString` / `parse` for machines; `DateTimeFormatter.ofPattern(..., Locale)` for humans. `uuuu` / `yyyy` year, `MM` month, `mm` minute, `HH` 24-hour. Never `YYYY`.

7.12 Random numbers

What it is

A **pseudo-random number generator** (PRNG) produces random-looking numbers fully determined by a **seed**: same seed → same sequence — great for reproducible tests and simulations, terrible for security.

Class	Since	Thread-safe?	Use for
`Math.random()`	1.0	yes (one shared generator)	quick scripts; `double` in [0, 1)
`java.util.Random`	1.0	yes, but contended across threads	seeded, reproducible sequences
`ThreadLocalRandom.current()`	7	per-thread instance, no contention	general randomness in concurrent code
`SplittableRandom`	8	no — split per task	parallel computations (Fork/Join)
`SecureRandom`	1.1	yes	**security**: tokens, passwords, keys, API keys, session IDs
`RandomGenerator` + `RandomGeneratorFactory`	17 (JEP 356)	per algorithm	modern algorithms (LXM family) by name

Java 17 made all of these implement `java.util.random.RandomGenerator`, adding `nextInt(origin, bound)` / `nextLong(origin, bound)` everywhere (`nextInt(10, 20)` → 10…19). `Random.from(RandomGenerator)` (Java 19) adapts a new-style generator to `Random`-taking APIs.

`code/ch07/random/RandomDemo.java` (key parts):

```
Random r1 = new Random(42), r2 = new Random(42);          // same seed -> same sequence
r1.nextInt(100);                                          // [0, 100)
new Random(7).nextInt(1, 7);                              // dice: [1, 7), Java 17
Collections.shuffle(deck, new Random(1));                 // reproducible shuffle
new Random(3).ints(5, 0, 10).boxed().toList();            // stream of 5 ints in [0, 10)
RandomGeneratorFactory.of("L64X128MixRandom").create(42L);
ThreadLocalRandom.current().nextInt(1, 7);
new SecureRandom().nextBytes(token);
```

Output:

```
seed 42 twice: 30/30 63/63 48/48 84/84 70/70
nextInt(1,7) dice: 5 3 4 | nextBoolean=false | nextDouble in [0,1)? true
nextGaussian (mean 0, sd 1): 0.5840
shuffle with seed 1: [C, D, B, E, A]
ints stream: [4, 0, 0, 1, 8]
L64X128MixRandom seeded: 98 1
default algorithm: L32X64MixRandom
ThreadLocalRandom roll in 1..6? true
SecureRandom token bytes: 16
```

Seeded lines are identical on every run and JVM (the `Random` algorithm is specified). The default `RandomGenerator` algorithm is an implementation detail — don't rely on `L32X64MixRandom`.

Common mistakes

- `Math.abs(random.nextInt()) % n` — biased, and `Math.abs(Integer.MIN_VALUE)` is negative. Use `nextInt(n)` / `nextInt(origin, bound)`.
- `(int) (Math.random() * 6) + 1` works; `ThreadLocalRandom.current().nextInt(1, 7)` states the range clearly.
- `new Random()` inside a loop — keep one.
- `Random` / `Math.random` for passwords, reset tokens, session IDs — predictable; use `SecureRandom`.
- Upper bounds are **exclusive**: `nextInt(1, 6)` never returns 6.

TL;DR

Seeded `Random` for reproducible tests; `ThreadLocalRandom` in concurrent code; `SecureRandom` for security; upper bounds exclusive.

Chapter TL;DR

Topic	Remember
`String`	Immutable, pooled literals, compare with `equals`; methods return new strings
`String` API	0-based, half-open; `indexOf` → -1; `split` / `replaceAll` use regex; `length()` counts UTF-16 units; prefer `strip` / `isBlank`
`StringBuilder`	Mutable, chainable, for loops; no value `equals`; `StringBuffer` is legacy
Formatting	`%[i$][flags][width][.prec]conv`; explicit `Locale` for machine output; `NumberFormat`, `DecimalFormat` (not thread-safe), `HexFormat` (17)
`Math`	`round` → long, half up; `floorDiv` / `floorMod` / `ceilDiv` (18); `clamp` (21); `*Exact` throws on overflow
Wrappers	Autobox = `valueOf`; cache −128…127 makes `==` unreliable; null unboxing → NPE; `remove(int)` vs `remove(Object)`
`BigDecimal`	Build from `String`; assign results; `divide` needs scale + `RoundingMode`; `compareTo` not `equals`; `HALF_EVEN` for money
`java.time`	Immutable, thread-safe; `Instant` = moment, `LocalDate` = date, `ZonedDateTime` = local time in a region; `Duration` ≠ `Period` across DST; inject `Clock`
Date patterns	`uuuu-MM-dd HH:mm`; never `YYYY`; pass a `Locale` for names
Random	Seeded `Random` for tests, `ThreadLocalRandom` for concurrency, `SecureRandom` for security

Next: when these APIs fail — `NumberFormatException`, `DateTimeParseException`, `ArithmeticException` — and designing error handling: Chapter 8 — Exceptions. Collections: Chapter 10. Exam traps: OCP Chapter 4, OCP Chapter 11.

Chapter 8 — Exceptions

Programs fail: files go missing, users type "forty-two" for a number, networks time out, code has bugs. An **exception** is Java's way to report failure *without* every method returning an error code. This chapter covers the mechanism (how an exception travels up the call stack — the chain of in-progress method calls), the hierarchy (checked vs unchecked vs errors), the syntax (`try` / `catch` / `finally`, multi-catch, try-with-resources, `throw`, `throws`), custom exceptions, and the practices that keep failures visible.

Already met: `NumberFormatException` from `Integer.parseInt`, `ArithmeticException` from `Math.addExact` (Chapter 2), `ArrayIndexOutOfBoundsException` (Chapter 4), and overriding methods not throwing broader checked exceptions (§6.3).

Code: `code/` (package `ch08.*`), `javac --release 25`, run on JDK 26; outputs from real runs. Exam traps (unreachable catches, `finally` returns, resource scope, suppressed exceptions, overriding and `throws`): OCP Chapter 11 (sections 1–4). Service-level strategy (retries — see System Design Fundamentals — error codes at API boundaries, logging): LLD Clean Code §6 and §16.

8.1 What an exception is

What it is

An **exception** is an object describing an abnormal event, plus the act of **throwing** it: the JVM stops the current code and jumps to the nearest matching **handler** (`catch` block). If no method on the call stack handles it, the thread dies and the JVM prints a **stack trace** (the method calls active at the time, §8.9).

Versus C-style error codes, exceptions **separate** normal logic from error handling, **can't be silently ignored** by a forgotten check, **propagate automatically** to the level that knows what to do (a request handler, a UI, a retry loop), and **carry information** (type, message, cause, call path).

How it works

1. Something fails: the JVM detects it (division by zero, `null` dereference, bad index) or code runs `throw new SomeException(...)`.
2. The current block stops; remaining statements are skipped.
3. The JVM looks for an enclosing `try` whose `catch` type matches (the object is an instance of it). That catch runs, then execution continues **after the whole** `try` **statement** — not back at the throw point.
4. If none in this method, the method ends abruptly and the exception is rethrown at the call site in the **caller** — repeating up the stack (**stack unwinding**), running `finally` blocks on the way.
5. At the top of a thread, its *uncaught exception handler* prints the trace and the thread ends. If that was `main` and no non-daemon threads (daemon = background helpers that don't keep the JVM alive) remain, the program exits with status 1 (non-zero = failure).

`code/ch08/basics/FirstException.java` (key parts):

```java
static int average(int[] values) {
    int sum = 0;
    for (int v : values) sum += v;
    return sum / values.length;            // length 0 -> ArithmeticException
}

try {
    System.out.println("before the call");
    int avg = average(new int[0]);         // throws; the rest of the try block is skipped
    System.out.println("never printed " + avg);
} catch (ArithmeticException e) {
    System.out.println("caught " + e.getClass().getName() + ": " + e.getMessage());
}
System.out.println("program continues normally");

static void setAge(int age) {
    if (age < 0) throw new IllegalArgumentException("age must be >= 0, was " + age);
    ...
}
```

Output:

```
average {4, 8} = 6
before the call
caught java.lang.ArithmeticException: / by zero
program continues normally
caught IllegalArgumentException: age must be >= 0, was -5
```

Propagation through the call stack

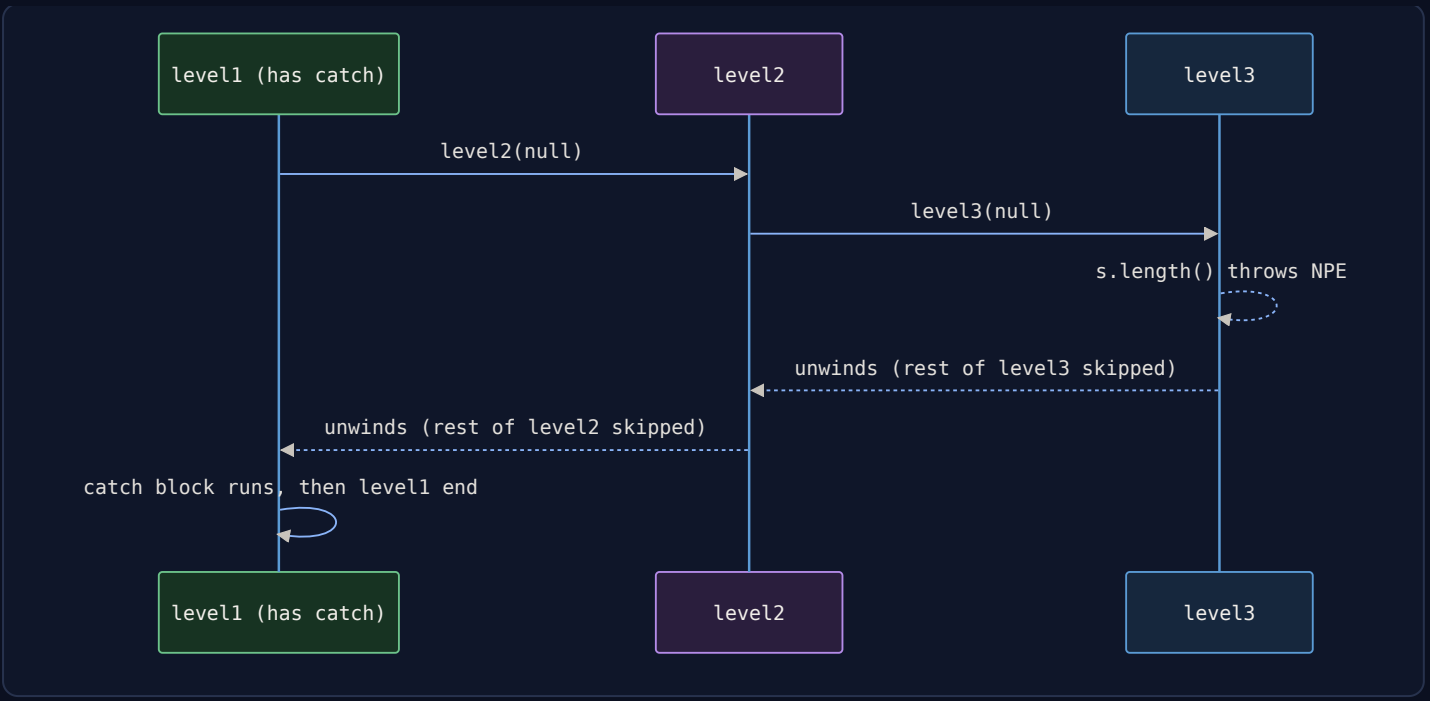
`code/ch08/basics/Propagation.java` calls `level1 → level2 → level3`; only `level1` catches:

```java
static void level3(String s) {
    System.out.println("  level3 start");
    System.out.println("  length = " + s.length());   // NPE when s is null
    System.out.println("  level3 end");
}
static void level2(String s) { System.out.println(" level2 start"); level3(s); System.out.println(" level2 end");
}
static void level1(String s) {
    System.out.println("level1 start");
    try { level2(s); }
    catch (NullPointerException e) { System.out.println("level1 caught: " + e.getMessage()); }
    System.out.println("level1 end");
}
```

Output:

```
level1 start
 level2 start
  level3 start
  length = 2
  level3 end
 level2 end
level1 end
---
level1 start
 level2 start
  level3 start
level1 caught: Cannot invoke "String.length()" because "<parameter1>" is null
level1 end
```

With `null`, "level3 end" and "level2 end" never print — both methods were abandoned as the exception unwound.



`"<parameter1>"` appears because the runner compiles without debug info; with `javac -g` (IDEs and Maven/Gradle default) it reads `because "s" is null`. **Helpful NullPointerException messages**: Java 14 (JEP 358), on by default since 15.

8.2 The `Throwable` hierarchy: checked, unchecked and errors

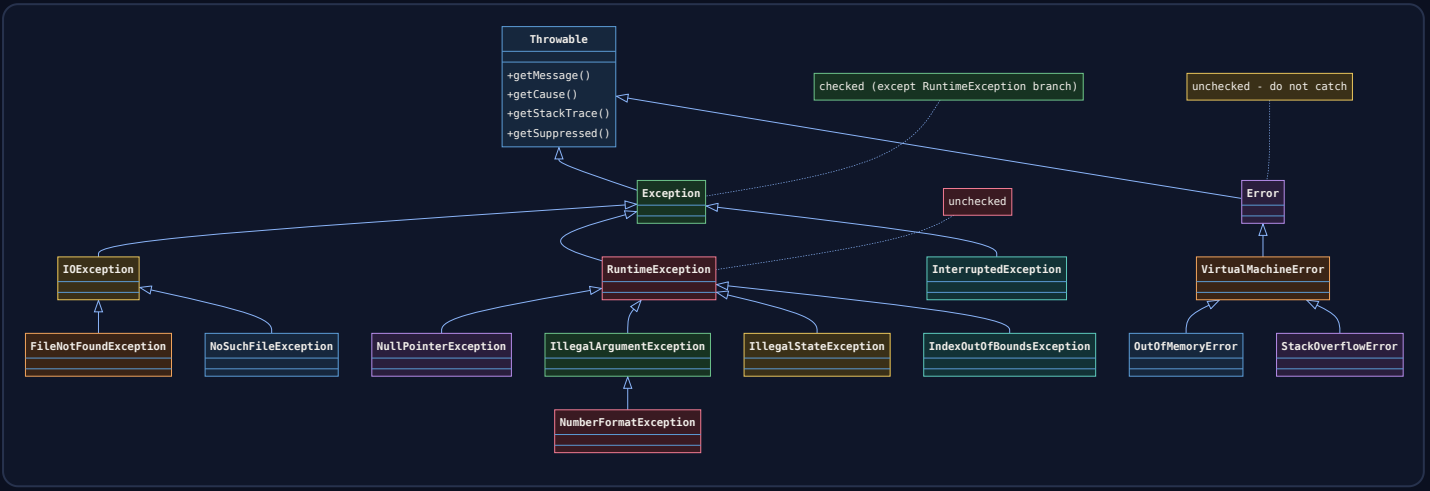
What it is

Only `java.lang.Throwable` and subclasses can be thrown. Two branches:

- `Error` — serious JVM/environment problems applications shouldn't handle: `OutOfMemoryError`, `StackOverflowError`, `NoClassDefFoundError`, `ExceptionInInitializerError`.
- `Exception` — conditions a program may handle; its subclass `RuntimeException` is special.

The compiler sorts them into:

- **Checked**: `Exception` and subclasses *except* the `RuntimeException` branch. The compiler **forces** handle-or-declare (§8.3). They model recoverable external problems: I/O failures, network errors, "file not found", `InterruptedException` (a waiting thread asked to stop).
- **Unchecked**: `RuntimeException` and subclasses **plus** `Error` and subclasses. No enforcement. `RuntimeException`s usually signal **programming errors** — a violated precondition (a condition required before the call): `null` argument, bad index, illegal state.



(`NoSuchFileException`, in `java.nio.file`, actually extends `FileSystemException`, which extends `IOException`.)

`code/ch08/hierarchy/Hierarchy.java` walks `getSuperclass()` for common types (selected lines):

```
NumberFormatException [unchecked]
    extends IllegalArgumentException > RuntimeException > Exception > Throwable
ArrayIndexOutOfBoundsException [unchecked]
    extends IndexOutOfBoundsException > RuntimeException > Exception > Throwable
UncheckedIOException [unchecked]
    extends RuntimeException > Exception > Throwable
FileNotFoundException [CHECKED]
    extends IOException > Exception > Throwable
SQLException [CHECKED]
    extends Exception > Throwable
InterruptedException [CHECKED]
    extends Exception > Throwable
CloneNotSupportedException [CHECKED]
    extends Exception > Throwable
StackOverflowError [error, unchecked]
    extends VirtualMachineError > Error > Throwable
```

The exceptions you'll meet most

Exception	Kind	Typical cause
`NullPointerException`	unchecked	method/field on `null`, unboxing `null`
`IllegalArgumentException`	unchecked	a method rejects an argument value
`NumberFormatException`	unchecked (IAE subclass)	`Integer.parseInt("abc")`
`IllegalStateException`	unchecked	call at the wrong time (`iterator.remove()` twice, using a closed object)
`IndexOutOfBoundsException` (+ `Array…`, `String…`)	unchecked	bad index
`ClassCastException`	unchecked	invalid downcast
`ArithmeticException`	unchecked	integer `/ 0`, `*Exact` overflow, non-terminating `BigDecimal` division
`UnsupportedOperationException`	unchecked	modifying an immutable collection (`List.of(...).add(x)`)
`ConcurrentModificationException`	unchecked	changing a collection while iterating (Chapter 10)
`NoSuchElementException`	unchecked	`iterator.next()` past the end; `Optional.get()` / `orElseThrow()` on empty
`DateTimeException`, `DateTimeParseException`	unchecked	invalid date values/text (Chapter 7)
`IOException` (+ `FileNotFoundException`, `NoSuchFileException` …)	checked	file/network I/O
`InterruptedException`	checked	a blocked thread was interrupted (Chapter 14)
`SQLException`	checked	database errors (JDBC)
`ExecutionException`, `TimeoutException`	checked	a `Future` (handle to a background task's result) failed / waited too long
`ExceptionInInitializerError`	error	exception in a `static` initialiser
`StackOverflowError`, `OutOfMemoryError`	error	runaway recursion; heap exhausted

Trade-offs — checked vs unchecked

	Checked	Unchecked
Compiler enforcement	Catch or declare	None
Signals	Recoverable external condition	Programming error or unrecoverable state
API cost	Every caller must deal with it, even if it can't help	Callers may ignore it; `@throws` docs matter more
Lambdas/streams	Poor — `java.util.function` interfaces declare none (Chapter 11)	Fine
Examples	`IOException`, `SQLException`	`IllegalArgumentException`, `NullPointerException`

Java is unusual in having checked exceptions (C#, Kotlin, Python, JavaScript don't). *Effective Java* Items 70–71: checked for conditions the caller can **recover** from, runtime for programming errors, avoid checked when the caller can't do anything useful. Spring, JPA/Hibernate wrap checked exceptions in unchecked ones for this reason.

In the interview

> **30-second answer:** "Everything thrown is a `Throwable`. `Error`s are JVM-level problems you don't catch. `Exception`s are checked — compiler-enforced catch-or-declare, for recoverable external failures like `IOException` — or unchecked `RuntimeException`s for programming errors like `IllegalArgumentException`. I use unchecked for precondition violations and checked only when the caller can genuinely recover."

- *Is `Error` checked?* No — `Error` and `RuntimeException` (and subclasses) are unchecked.
- *Catch `Throwable`?* Only at the very top of a thread/request to log and fail cleanly — it also catches `OutOfMemoryError`, after which continuing may be unsafe.

TL;DR

`Throwable` → `Error` (don't catch) and `Exception`; `RuntimeException` + `Error` = unchecked; the rest of `Exception` = checked (handle or declare).

8.3 Handle or declare: checked exceptions and `throws`

What it is

The **handle-or-declare rule** ("catch or specify"): if code in a method can throw a **checked** exception, the method must **handle** it (`try` with a matching `catch`) or **declare** it in its **`throws` clause**, passing responsibility to callers. Otherwise: *"unreported exception IOException; must be caught or declared to be thrown"* (e.g. `void m() { Thread.sleep(10); }` doesn't compile).

`throw` is a statement throwing one object now (`throw new IOException("x");`); **`throws`** is part of the method declaration listing exception *types* (`void read() throws IOException, SQLException`).

How it works

code/ch08/checked/CheckedDemo.java :

```java
// "Declare": this method does not handle IOException, it passes it to the caller
static String readConfig(Path path) throws IOException {
    return Files.readString(path);
}

// "Handle": this method catches and decides what to do
static String readConfigOrDefault(Path path) {
    try {
        return readConfig(path);
    } catch (NoSuchFileException e) {            // more specific first
        System.out.println("no such file: " + e.getFile() + " -> using defaults");
        return "mode=default";
    } catch (IOException e) {                    // any other I/O problem
        System.out.println("I/O failure: " + e);
        return "mode=default";
    }
}

public static void main(String[] args) throws IOException {   // main may declare too
    Path tmp = Files.createTempFile("ch08-", ".properties");
    Files.writeString(tmp, "mode=fast");
    System.out.println("existing file: " + readConfigOrDefault(tmp));
    Files.delete(tmp);
    System.out.println("deleted file:  " + readConfigOrDefault(Path.of("missing-config.properties")));
    for (String in : new String[]{"42", "forty-two"}) {
        try { System.out.println("parsed " + Integer.parseInt(in)); }
        catch (NumberFormatException e) { System.out.println("bad number '" + in + "': " + e.getMessage()); }
    }
}
```

Output:

```
existing file: mode=fast
no such file: missing-config.properties -> using defaults
deleted file:  mode=default
parsed 42
bad number 'forty-two': For input string: "forty-two"
```

The rules in one table

Rule	Detail
Checked exceptions must be caught or declared	including those from methods you call
Unchecked *may* be declared	documentation only; the compiler ignores it — prefer Javadoc `@throws`
Can't catch a checked exception the `try` body can't throw	*"exception IOException is never thrown in body of corresponding try statement"* — except `Exception` / `Throwable`, which may hide unchecked ones (so `catch (Exception e)` after an empty `try` compiles)
Overrides may not throw **new or broader** checked exceptions	fewer, narrower or unchecked are fine (§6.3)
Lambdas	may throw a checked exception only if the functional interface method declares it; `Runnable`, `Function` don't (Chapter 11)
Initialiser blocks	`static` initialisers can't throw checked (unchecked are wrapped in `ExceptionInInitializerError`); instance initialisers can only if every constructor declares it

When to handle vs declare

- **Handle** where you can act: retry, fall back, re-ask the user, translate (§8.8).
- **Declare** when this layer can't fix it and the caller has more context. Low-level helpers declare; boundaries (request handler, `main`, UI handler) handle.

- Never "handle" by catching and doing nothing (§8.10).

8.4 `try` / `catch` / `finally` and control flow

What it is

`finally` runs after the `try` (and any `catch` that ran) **however** the `try` ended — normally, by exception (caught or not), or by `return` / `break` / `continue`. It is for cleanup that must always happen (release a lock, reset a flag; for resources §8.6 is better).

How it works — the syntax rules

1. A `try` needs at least one `catch` **or** a `finally` (or resources); `try { } finally { }` is legal.
2. Order: `try`, zero or more `catch`, at most one `finally`. Braces always required.
3. At most one `catch` runs — the **first** match.
4. `finally` runs after that `catch`, or right after `try` if nothing was thrown or nothing matched (then the exception keeps propagating **after** `finally`).

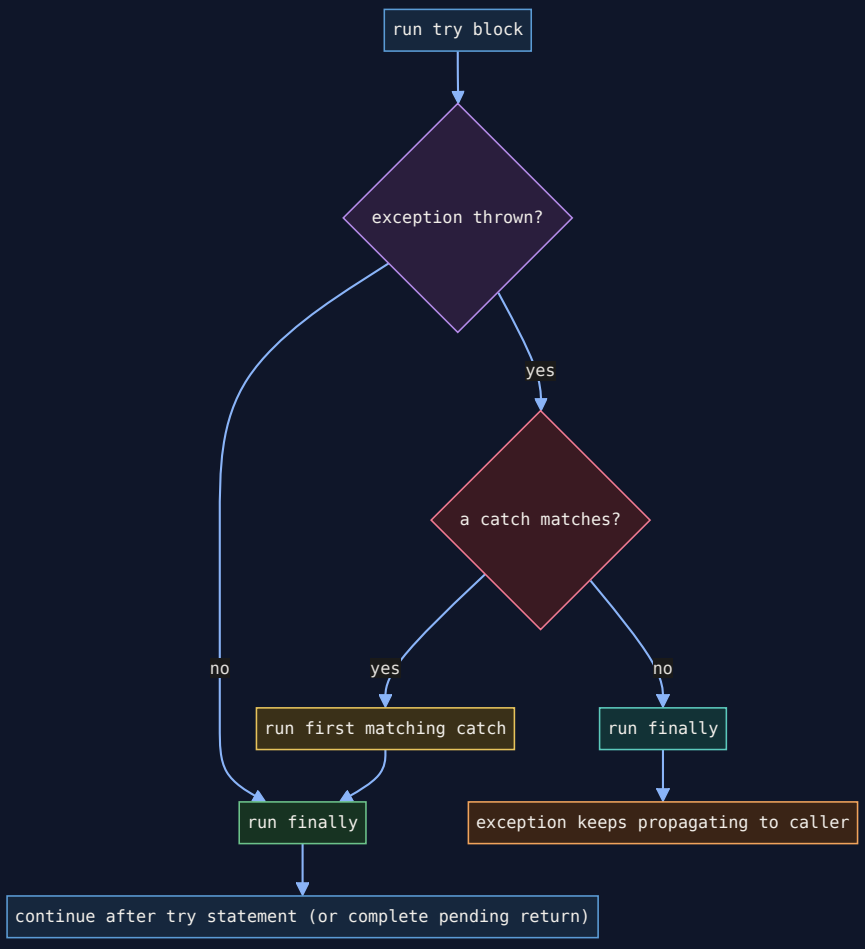
code/ch08/finallyflow/FinallyDemo.java (key parts):

```
static int returnInTry() {
    int x = 1;
    try {
        return x;             // the value 1 is saved here...
    } finally {
        x = 99;               // ...so changing x in finally does not change the returned value
        System.out.println("finally ran before the method returned");
    }
}

static int finallyOverrides() {
    try {
        throw new RuntimeException("lost!");
    } finally {
        return -1;            // BAD: discards the exception and any return value from try
    }
}
```

Output:

```
no failure: try try-end finally
failure:    try catch finally
finally ran before the method returned
returnInTry() = 1
finallyOverrides() = -1 (the RuntimeException vanished)
inner finally runs even though nobody here catches
outer caught: inner
```



The tricky cases

Situation	What happens
`return` in `try`, `finally` changes the variable	The return **value** was already evaluated: `try { return "a"; } finally { print("b"); }` prints `b`, returns `"a"` (a returned object's *contents* can still be mutated)
`return` in `finally`	Overrides the `try`'s return **and silently discards any exception** — never (javac `-Xlint:finally` warns)
`finally` throws	The new exception replaces and loses the original
`catch` throws	`finally` still runs, then the new exception propagates
`System.exit()` in `try`, JVM crash/halt, power loss, `try` never finishes	`finally` does **not** run

Common mistakes

- Returning or throwing from `finally`.
- Manual resource closing in `finally` with nested `try`s — use try-with-resources.
- Confusing `final` (no reassignment), `finally` (always-run block), `finalize()` (deprecated for removal since Java 18, JEP 421).

8.5 Multi-catch, catch order and rethrowing

What it is

- **Multi-catch** (Java 7): one `catch` for several unrelated types: `catch (NumberFormatException | IndexOutOfBoundsException e)`.
- **Catch order**: the first match wins, so a subclass must come **before** its superclass, else the subclass block is unreachable (compile error).
- **Precise rethrow** (Java 7): catch broadly and rethrow the same (effectively final) variable; the compiler knows it can only be the checked types the `try` really throws — so you can log broadly yet keep a narrow `throws`.
- **Unnamed variable** `_` (Java 22, JEP 456): `catch (NumberFormatException _)` when the object isn't used.

`code/ch08/multicatch/MultiCatch.java` (key parts):

```
try {
    risky(i);        // 0: parseInt("x")  1: List.of(1,2).get(5)  2: IOException  3: fine
} catch (NumberFormatException | IndexOutOfBoundsException e) {     // multi-catch
    System.out.println(i + ": bad input -> " + e.getClass().getSimpleName());
} catch (java.io.IOException e) {
    System.out.println(i + ": I/O -> " + e.getMessage());
} finally {
    System.out.println(i + ": finally");
}

try { Integer n = (Integer) (Object) "text"; }
catch (ClassCastException e) { System.out.println("ClassCastException first"); }   // specific first
catch (RuntimeException e)   { System.out.println("RuntimeException"); }           // then general

try { value = Integer.parseInt(input); }
catch (NumberFormatException _) { value = -1; }                                    // Java 22

static void rethrow(boolean io) throws java.io.IOException, java.util.concurrent.TimeoutException {
    try {
        if (io) throw new java.io.IOException("io failed");
        throw new java.util.concurrent.TimeoutException("too slow");
    } catch (Exception e) {
        System.out.println("log and rethrow: " + e);
        throw e;                    // compiler knows e is IOException | TimeoutException
    }
}
```

Output:

```
0: bad input -> NumberFormatException
0: finally
1: bad input -> IndexOutOfBoundsException
1: finally
2: I/O -> disk full
2: finally
  no problem
3: finally
ClassCastException first
value = -1
log and rethrow: java.util.concurrent.TimeoutException: too slow
caller got TimeoutException
```

Compiler rules (real `javac` messages)

You write	Result
`catch (RuntimeException e) {} catch (ClassCastException e) {}`	*exception ClassCastException has already been caught*
`catch (IllegalArgumentException | NumberFormatException e)` (or `IOException | FileNotFoundException`)	*Alternatives in a multi-catch statement cannot be related by subclassing* — catch the parent
`catch (IllegalStateException | ArithmeticException e) { e = null; }`	*multi-catch parameter e may not be assigned* (implicitly `final`)
`catch (java.io.IOException e)` around code that can't throw it	*exception IOException is never thrown in body of corresponding try statement*

A multi-catch variable's type is the alternatives' closest common supertype, so only shared methods (usually `Throwable`'s) are callable.

8.6 try-with-resources

What it is

A **resource** must be closed after use — files, sockets, DB connections, streams, closeable locks. **try-with-resources** (Java 7) declares them in parentheses after `try` and closes them automatically however the block ends.

Any **`java.lang.AutoCloseable`** (`void close() throws Exception`) qualifies. **`java.io.Closeable`** is the older I/O sub-interface whose `close()` throws `IOException` and must be idempotent (a second call has no further effect).

Before Java 7, correct cleanup needed `finally` with a null check and a nested `try` around `close()`, and a `close()` failure could *replace* the real error. *Effective Java* Item 9: "Prefer try-with-resources to try-finally."

How it works

1. Resources initialise left to right; each is implicitly `final` (reassigning: *"auto-closeable resource r may not be assigned"*).
2. The body runs.
3. Resources are **closed in reverse order** (`a, b, c` → `c, b, a`) — **before** any `catch` / `finally` of the same statement. `catch` / `finally` are optional.
4. If an initialisation fails, only already-opened resources are closed; `null` resources are skipped.
5. If the body throws *and* `close()` throws, the body's exception is **primary**; close failures are attached as **suppressed** (`getSuppressed()`; traces print them under "Suppressed:").
6. Java 9: an existing **effectively final** variable can be used directly: `try (reader) { ... }`.

`code/ch08/resources/TryWithResources.java` (key parts):

```java
static class Res implements AutoCloseable {
    Res(String name, boolean failOnClose) { ...; System.out.println("open " + name); }
    void use(boolean fail) { System.out.println("use " + name); if (fail) throw new IllegalStateException("use of
" + name + " failed"); }
    @Override public void close() {
        System.out.println("close " + name);
        if (failOnClose) throw new IllegalStateException("close of " + name + " failed");
    }
}

try (Res a = new Res("A", false); Res b = new Res("B", false)) { a.use(false); b.use(false); }

try (Res a = new Res("A", false)) { a.use(true); }
catch (IllegalStateException e) { System.out.println("catch: " + e.getMessage()); }
finally { System.out.println("finally"); }

try (Res a = new Res("A", true); Res b = new Res("B", true)) { b.use(true); }
catch (IllegalStateException e) {
    System.out.println("primary: " + e.getMessage());
    for (Throwable s : e.getSuppressed()) System.out.println("suppressed: " + s.getMessage());
}

BufferedReader reader = new BufferedReader(new StringReader("line one\nline two"));
try (reader) { System.out.println("first line: " + reader.readLine()); }     // Java 9
```

Output:

```
-- 1. normal: closed in reverse order
open A
open B
use A
use B
close B
close A
-- 2. body fails: resources closed BEFORE catch runs
open A
use A
close A
catch: use of A failed
finally
-- 3. body AND close fail: close exception is suppressed
open A
open B
use B
close B
close A
primary: use of B failed
suppressed: close of B failed
suppressed: close of A failed
-- 4. Java 9: effectively final variable as a resource
first line: line one
after close: Stream closed
```

Real-world use (not run — needs a database; JDBC in Chapter 18):

```java
try (Connection con = dataSource.getConnection();
     PreparedStatement ps = con.prepareStatement("SELECT name FROM users WHERE id = ?")) {
    ps.setLong(1, id);
    try (ResultSet rs = ps.executeQuery()) { ... }
}   // rs, ps, con closed in that order — even if the query throws
```

Common mistakes

- Opening a resource *outside* `try (...)` without passing it in — leaks if something fails in between.
- Returning a resource out of the `try` (e.g. a `Stream` from `Files.lines`) — closed before the caller uses it. `Files.lines` / `Files.walk` must themselves be in try-with-resources (Chapter 13).

- Wrapped streams: `try (var in = new BufferedReader(new FileReader(f)))` closes both, but if the *outer* constructor throws, the inner leaks. For strictness, declare each as its own resource.

In the interview

> **30-second answer:** "try-with-resources closes any `AutoCloseable` automatically, in reverse order, before catch and finally run. If both the body and `close` throw, the body's exception wins and close failures are attached as suppressed exceptions, so nothing is lost. It replaced error-prone `finally { if (x != null) x.close(); }`."

- *`AutoCloseable` vs `Closeable`?* `Closeable` (java.io) extends `AutoCloseable`, throws `IOException`, and should be idempotent.

TL;DR

Anything `AutoCloseable` goes in `try (...)`. Reverse-order close, before catch/finally; secondary failures are suppressed, not lost.

8.7 Throwing exceptions and writing custom ones

Reuse standard exceptions first

Effective Java Item 72: readers already know the standard ones.

Situation	Throw
Invalid argument value (`setAge(-5)`)	`IllegalArgumentException`
Required argument is `null`	`NullPointerException` — `Objects.requireNonNull(x, "x")`
Index out of range	`IndexOutOfBoundsException` — `Objects.checkIndex(i, size)` (Java 9)
Object in the wrong state (`start()` twice)	`IllegalStateException`
Operation not supported by this implementation	`UnsupportedOperationException`
Lookup found nothing and must return a value	`NoSuchElementException` (or return `Optional`, §8.10)
Concurrent modification detected	`ConcurrentModificationException`

Don't throw `Exception`, `RuntimeException`, `Throwable` or `Error` directly — callers can't distinguish your failure.

When to write your own

When callers must **distinguish** it or it should **carry data** (account ID, shortfall, error code):

1. Name it `…Exception`.
2. Extend `RuntimeException` for programming/state errors and most domain rule violations; extend `Exception` only if callers should recover and the compiler should remind them.
3. Provide `(String message)` and `(String message, Throwable cause)` — the cause constructor enables chaining (§8.8).
4. `final` fields with getters, so handlers act on data without parsing messages. Exceptions are `Serializable`; strict linting (`-Xlint:serial`) wants a `serialVersionUID`.
5. Messages carry failure-capture details (*Effective Java* Item 75) — the offending values, not "Error occurred"; never passwords or personal data.

`code/ch08/custom/CustomExceptions.java` (key parts):

```java
// Checked: the caller is expected to recover (e.g. show "insufficient funds" to the user)
static class InsufficientFundsException extends Exception {
    private final BigDecimal shortfall;
    InsufficientFundsException(BigDecimal shortfall) {
        super("insufficient funds: short by " + shortfall);
        this.shortfall = shortfall;
    }
    BigDecimal getShortfall() { return shortfall; }
}

// Unchecked: a programming/state error in our domain
static class AccountFrozenException extends RuntimeException {
    AccountFrozenException(String accountId) { super("account " + accountId + " is frozen"); }
}

void withdraw(BigDecimal amount) throws InsufficientFundsException {
    if (amount.signum() <= 0) throw new IllegalArgumentException("amount must be positive: " + amount);
    if (frozen) throw new AccountFrozenException(id);
    if (amount.compareTo(balance) > 0) throw new InsufficientFundsException(amount.subtract(balance));
    balance = balance.subtract(amount);
}
```

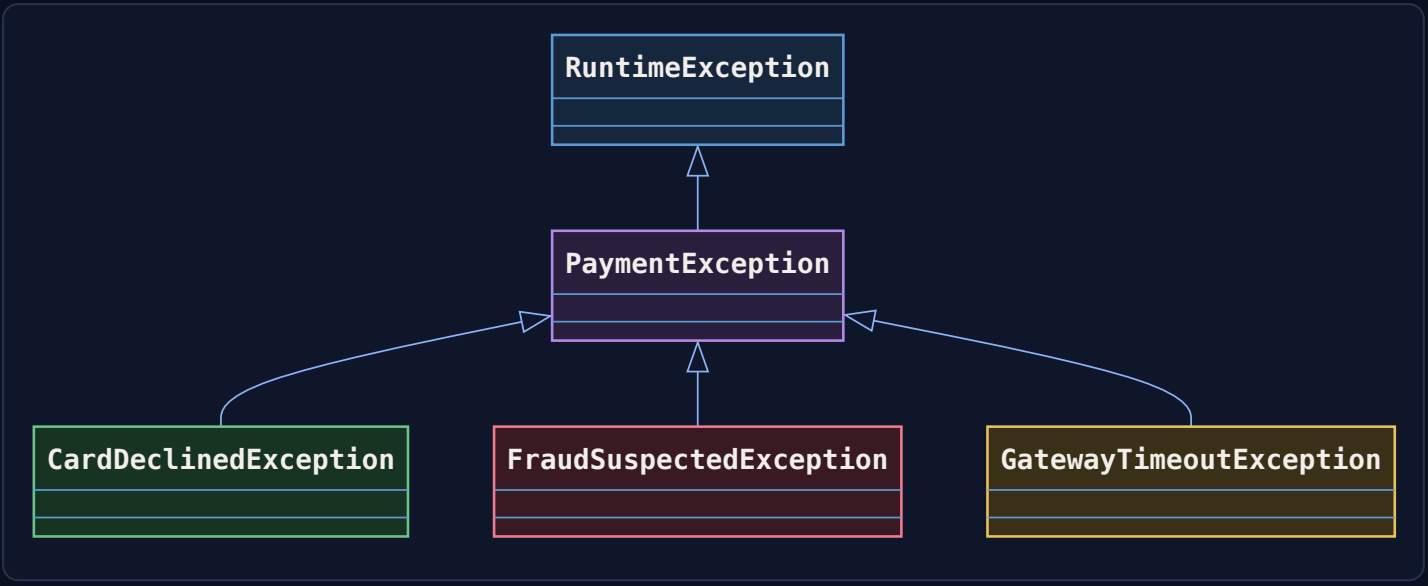
Output (first four lines):

```
withdrew 30.00, balance 70.00
insufficient funds: short by 430.00 (shortfall field = 430.00)
rejected: amount must be positive: -1
rejected: account ACC-1 is frozen
```

`withdraw` checks every condition **before** changing `balance` — **failure atomicity** (*Effective Java* Item 76): a failed call leaves the object as it was.

Custom exception hierarchies

A small hierarchy lets callers catch broadly or narrowly:



Keep it shallow — three or four types callers handle differently beat twenty nobody catches. A sealed interface or an error-code enum field also works.

8.8 Chaining and exception translation

What it is

Chaining stores the original exception as the **cause** of a new one: `new ReportException("could not build report", e)`. **Translation** (*Effective Java* Item 73): a higher layer (web → service → data access) catches low-level exceptions and throws ones meaningful at *its* abstraction — a `ReportService` shouldn't leak `SQLException` / `IOException`. The trace prints the high-level exception, then **"Caused by:"** with the original.

How it works

1. Catch the low-level exception.
2. Throw a new one with the original as `cause` (constructor, or `initCause` for old types without one).
3. Callers use `getCause()` (repeatedly, for the root); logs show the whole chain.
4. To pass a checked exception through code that can't declare it (lambdas, `Runnable`), wrap it in an unchecked one — **`UncheckedIOException`** for `IOException`.

```
static String buildReport(String name) throws ReportException {
    try {
        return loadTemplate(name);                      // throws IOException
    } catch (IOException e) {
        throw new ReportException("could not build report " + name, e);   // translate + chain
    }
}

java.util.List.of("a").forEach(n -> {
    try { loadTemplate(n); }
    catch (IOException e) { throw new UncheckedIOException(e); }        // lambda can't throw IOException
});
```

Output (last three lines of `CustomExceptions`):

```
error: could not build report monthly
cause: java.io.IOException: template 'monthly' not found on disk
unwrapped cause: template 'a' not found on disk
```

Common mistakes

- `throw new ReportException("failed: " + e.getMessage());` — loses the original type and trace. **Always pass `e` as the cause.**
- Log-and-rethrow at every layer — one failure, five log entries. Log once, where handled.
- Bare `RuntimeException` wrapping with no message — say what the operation was.

8.9 Reading stack traces

What it is

A **stack trace** lists the active method calls (**frames**) when the exception was *created*, most recent first, each as `class.method(File.java:line)` — Java's most useful debugging artefact.

code/ch08/stacktrace/StackTraces.java :

```java
static int parsePort(String text) { return Integer.parseInt(text); }

static int loadPort(String text) {
    try {
        return parsePort(text);
    } catch (NumberFormatException e) {
        throw new ServiceException("invalid port setting", e);
    }
}
// in main:
try { loadPort("80a"); }
catch (ServiceException e) { e.printStackTrace(System.out); ... }
```

Output (line numbers in `java.base` frames depend on the JDK build — these are JDK 26):

```
ch08.stacktrace.StackTraces$ServiceException: invalid port setting
    at ch08.stacktrace.StackTraces.loadPort(StackTraces.java:17)
    at ch08.stacktrace.StackTraces.main(StackTraces.java:23)
Caused by: java.lang.NumberFormatException: For input string: "80a"
    at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
    at java.base/java.lang.Integer.parseInt(Integer.java:527)
    at java.base/java.lang.Integer.parseInt(Integer.java:624)
    at ch08.stacktrace.StackTraces.parsePort(StackTraces.java:10)
    at ch08.stacktrace.StackTraces.loadPort(StackTraces.java:15)
    ... 1 more
top frame: ch08.stacktrace.StackTraces.loadPort line 17
caught StackOverflowError (an Error: normally you do NOT catch these)
```

How to read one

1. **First line**: class + message — *what* happened.
2. **Frames**: *where*, most recent first; the first frame in **your** code is usually the place to look (`loadPort`, line 17).
3. **"Caused by:"**: the underlying exception. **Go to the last one** — the root cause (`"80a"` from `parsePort` line 10).
4. `... 1 more`: remaining frames identical to the enclosing trace's, elided.
5. `java.base/`: the module (Chapter 16); `$` marks a nested class.
6. **"Suppressed:"**: exceptions attached by try-with-resources (§8.6).

Tools

API	Use
`e.getMessage()`, `e.toString()`	message; class name + message
`e.printStackTrace()`	to `System.err` — fine for learning; applications use a logger (`log.error("msg", e)`), which records the trace
`e.getStackTrace()`	`StackTraceElement[]` for inspection
`e.getCause()`, `e.getSuppressed()`	navigate the chain
`StackWalker` (Java 9)	lazily inspect the *current* stack without an exception
`Thread.setDefaultUncaughtExceptionHandler`	last-chance logging for dying threads

The trace is captured at **construction**, costing time proportional to stack depth — one reason not to use exceptions for control flow (§8.10). Frameworks sometimes pre-allocate exceptions or disable capture via the protected `Throwable(String, Throwable, boolean enableSuppression, boolean writableStackTrace)` constructor — a niche optimisation.

8.10 Best practices and anti-patterns

The rules (with *Effective Java* item numbers)

#	Practice	Why
1	**Exceptions only for exceptional conditions** (Item 69), never normal control flow	Slower (stack capture), obscures logic, hides bugs
2	**Checked for recoverable, runtime for programming errors** (Item 70); avoid needless checked (Item 71)	Don't force callers to catch what they can't handle
3	**Prefer standard exceptions** (Item 72)	Familiar meaning
4	**Throw exceptions appropriate to the abstraction** (Item 73) — translate and chain	Decoupled layers, cause preserved
5	**Document every exception** with `@throws` (Item 74), including relevant unchecked ones	The compiler can't tell callers
6	**Failure details in the message** (Item 75)	Self-explanatory traces
7	**Failure atomicity** (Item 76): validate before mutating	No half-updated state
8	**Don't ignore exceptions** (Item 77)	Swallowed errors = silent corruption
9	**Fail fast**: validate at the start of public methods/constructors	Error surfaces near its cause
10	**Catch the most specific type you can handle**; broad only at boundaries	Don't catch bugs by accident
11	**Log or rethrow, not both**; log once with the exception object	No duplicates; trace kept
12	**try-with-resources** for anything closeable (Item 9)	Correct cleanup

`code/ch08/practices/BestPractices.java` (key parts):

```
record User(String name, String email) {
    User {
        Objects.requireNonNull(name, "name");                                   // fail fast
        if (email == null || !email.contains("@")) throw new IllegalArgumentException("bad email: " + email);
    }
}

static Optional<String> findUser(int id) { return Optional.ofNullable(USERS.get(id)); }   // absence is normal
static String getUser(int id) {                                                  // absence is a bug
    String u = USERS.get(id);
    if (u == null) throw new NoSuchElementException("no user with id " + id);
    return u;
}

System.out.println("bounds check instead of catching: " + (i < data.length ? data[i] : "out of range"));

try {
    Thread.sleep(1);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();        // restore the interrupt flag
    ...
}
```

The *interrupt flag* is a per-thread "please stop" marker; catching `InterruptedException` clears it, so set it again. Output:

```
findUser(3) -> guest
getUser(3) -> no user with id 3
requireNonNull -> NPE: name
validation -> bad email: not-an-email
helpful NPE: Cannot invoke "String.toUpperCase()" because "<local1>[1]" is null
bounds check instead of catching: out of range
done
```

(With `javac -g` the NPE names the variable: `because "names[1]" is null`.)

Exception or `Optional` or special value?

Situation	Return / throw
"Not found" is normal (search, cache lookup, a `findById` that often misses)	`Optional<T>` (Chapter 11) or an empty collection — never `null` for collections
Caller broke the contract (null argument, bad index)	Unchecked exception
External failure the caller can act on (file missing, remote service down)	Checked, or unchecked with clear docs
User-input validation with many possible errors	A result object listing all errors (throwing on the first is poor UX)

Anti-patterns

```
// 1. Swallowing: the failure disappears
try { save(order); } catch (Exception e) { }

// 2. Pokemon catching ("gotta catch 'em all") deep inside business logic
catch (Throwable t) { ... }                // also catches OutOfMemoryError, StackOverflowError

// 3. Losing the cause
catch (IOException e) { throw new RuntimeException("failed"); }       // pass e!

// 4. Exceptions as control flow
try { int i = 0; while (true) arr[i++].process(); } catch (ArrayIndexOutOfBoundsException e) { }

// 5. Returning from finally (hides exceptions) — see 8.4

// 6. Swallowing InterruptedException without restoring the flag
catch (InterruptedException e) { }         // callers can no longer see the interrupt
```

If ignoring is genuinely right (closing during shutdown), name the variable `ignored` or `_` and comment why.

Assertions — a note

`assert condition : "message";` throws `AssertionError` when false — **only with** `-ea` (off by default). Use for internal invariants during development/testing; never for public-method argument validation or anything with side effects.

Real-world examples

- Spring's `@ControllerAdvice` maps exceptions at the request boundary to HTTP statuses (System Design Fundamentals): `IllegalArgumentException` → 400, custom `NotFoundException` → 404, anything else → 500 with a logged trace.
- Frameworks translate checked JDBC/file exceptions into unchecked hierarchies (Spring's `DataAccessException`) to keep business code clean.
- `Objects.requireNonNull` at the top of constructors throughout the JDK.

In the interview

> **30-second answer:** "I validate early and throw standard unchecked exceptions for contract violations, use checked exceptions only for recoverable external failures, translate low-level exceptions at layer boundaries while chaining the cause, never swallow exceptions or return from `finally`, use try-with-resources for anything closeable, and handle — log once, map to a response — at the boundary. For expected absence I return `Optional`."

- *What's wrong with `catch (Exception e) {}`?* It hides every failure, including bugs, leaving unknown state.
- *`final` vs `finally` vs `finalize`?* Modifier; always-run block; deprecated GC hook (JEP 421, Java 18) — use try-with-resources or `Cleaner`.
- *Why restore the interrupt flag?* So code further up sees cancellation was requested (Chapter 14).

TL;DR

Validate early, standard unchecked exceptions for bugs, checked only when recovery is realistic, translate with a cause at boundaries, never swallow, log once, `Optional` for normal absence.

Chapter TL;DR

Topic	Remember
Mechanism	`throw` stops the block; the stack unwinds to the first matching `catch`; uncaught → thread dies with a stack trace
Hierarchy	`Throwable` → `Error` (don't catch) / `Exception`; `RuntimeException` + `Error` unchecked; rest checked
Handle or declare	Checked exceptions caught or listed in `throws`; overriding can't widen checked exceptions
`finally`	Always runs (except `System.exit`/crash); return value already evaluated; never `return`/`throw` from it
Catch rules	First match wins; subclass first; multi-catch types unrelated, variable final; precise rethrow; `catch (X _)` (22)
try-with-resources	`AutoCloseable`; reverse-order close before catch/finally; close failures suppressed; `try (existingVar)` (9)
Custom exceptions	Standard ones first; custom when callers must distinguish or need data; detailed message; cause constructor
Chaining	Always pass the cause; translate to your layer's abstraction; `UncheckedIOException` for lambdas
Stack traces	Top line, first frame in your code, then the last "Caused by:"; `-g` for variable names in NPE messages
Practices	Exceptions for exceptional cases, fail fast, failure atomicity, don't swallow, log once, `Optional` for normal absence

Next: Chapter 9 — Generics. Checked exceptions in lambdas and `Optional`: Chapter 11. Exceptions in threads, `ExecutionException`: Chapter 15. Exam traps: OCP Chapter 11. Service-level error handling: LLD Clean Code §16.

Chapter 9 — Generics

`List<String>` and `Map<Long, Order>` use **generics** (Java 5): types and methods that take *other types* as parameters, so one piece of code — a list, a cache, a repository (object that saves and loads data), a `max` function — works with many types while the compiler checks you never put an `Integer` where a `String` belongs. They're why modern Java has almost no casts.

Covered: generic classes, interfaces and methods; bounds (limits on allowed types); wildcards (`?` , an unknown type) and **PECS**; **type erasure** — the compiler removing generic type information after checking it — which explains every odd restriction (no `new T()` , no `new List<String>[10]` , no `instanceof List<String>` on an `Object`); and *Effective Java* Items 26–33, including how to read signatures like `<T extends Comparable<? super T>>` . Collections, the main *users* of generics, are Chapter 10; array covariance vs generic invariance was previewed in §4.6.

Code: `code/` (package `ch09.*`), `javac --release 25` , run on JDK 26; quoted compile errors are real `javac` messages. Exam traps (diamond/ `var` , wildcard puzzles, bounds): OCP Chapter 9 (sections 2 and 10). Generics in object design: LLD §2.15.

9.1 Why generics

What it is

A generic class, interface or method has a placeholder — a **type parameter**, conventionally one capital letter like `T` — that each user fills with a concrete **type argument**. `List<E>` is declared once; `List<String>` and `List<Order>` are **parameterized types** made from it.

The problem it solves

Before Java 5 collections held `Object` : nothing stopped a wrong type going in, and every read needed a cast that could fail far from the bug.

`code/ch09/why/WhyGenerics.java` :

```
List raw = new ArrayList();              // raw type: no type argument
raw.add("Ada");
raw.add(42);                             // compiles: nothing stops a wrong type going in
for (Object o : raw) {
    String s = (String) o;               // fails at RUN time on the Integer
    System.out.println("raw: " + s.toUpperCase());
}

List<String> names = new ArrayList<>();  // diamond <> (Java 7) infers <String>
names.add("Ada");
// names.add(42);                        // compile error: incompatible types: int cannot be converted to String
String first = names.get(0);             // no cast
```

Output:

```
raw: ADA
raw list -> ClassCastException: class java.lang.Integer cannot be cast to class java.lang.String
(java.lang.Integer and java.lang.String are in module java.base of loader 'bootstrap')
generic: ADA
```

Generics move the error from **run time** (a `ClassCastException` in production) to **compile time**, and remove the casts.

Vocabulary

Term	Example	Meaning
Generic type	`class Box<T>`	class/interface declared with type parameters
Type parameter	`T` in `Box<T>`	the placeholder
Type argument	`String` in `Box<String>`	the concrete type supplied
Parameterized type	`Box<String>`	generic type + arguments
Raw type	`Box`	generic type **without** arguments — legacy only
Diamond	`new Box<>("x")`	compiler infers the arguments from context (Java 7; with anonymous classes since Java 9)
Generic method	`static <T> T first(List<T> xs)`	method with its own type parameter
Bound	`<T extends Number>`	restriction on the type parameter
Wildcard	`List<?>`, `List<? extends Number>`	unknown type argument at the use site

Raw types

Raw types exist only for pre-2004 compatibility; they switch off generic checking and produce "unchecked" warnings. *Effective Java* Item 26: **don't use raw types**. If the element type doesn't matter, use `List<?>` (§9.6), which stays type-safe. The raw name is required only for class literals (`List.class`, not `List<String>.class`) and `instanceof` against a non-reifiable type (§9.7): write `instanceof List<?>` or `instanceof List`.

9.2 Generic classes and records

How it works

1. Declare type parameters after the class name: `class Box<T>`, `class Pair<K, V>`.
2. Use `T` like any type inside: fields, parameters, return types, locals.
3. Users supply arguments (`Box<String>`); with `var` or the diamond you rarely write them twice.
4. Type arguments must be **reference types** — `Box<int>` gives *"unexpected type"*; use `Box<Integer>` (autoboxing, §7.6).
5. Records (Java 16) and classes enclosing enums can be generic; enums can't declare type parameters.

Naming conventions

Letter	Usually means	JDK example
`T` (then `U`, `V`, `S`)	Type	`Optional<T>`, `BiFunction<T, U, R>`
`E`	Element	`List<E>`, `Set<E>`
`K`, `V`	Key, Value	`Map<K, V>`
`R`	Result	`Function<T, R>`
`N`	Number	
`ID` or longer	when single letters confuse	`Repository<T, ID>` (Spring Data style)

`code/ch09/classes/GenericClasses.java` (key parts):

```java
static final class Box<T> {
    private T value;
    Box(T value) { this.value = value; }
    T get() { return value; }
    void set(T value) { this.value = value; }
    <R> Box<R> map(Function<? super T, ? extends R> f) {     // generic method inside a generic class
        return new Box<>(f.apply(value));
    }
}

record Pair<K, V>(K key, V value) {                          // generic record (Java 16)
    static <K, V> Pair<K, V> of(K k, V v) { return new Pair<>(k, v); }
    Pair<V, K> swap() { return new Pair<>(value, key); }
}

Box<String> s = new Box<>("hello");
Box<Integer> len = s.map(String::length);
Pair<String, Integer> p = Pair.of("age", 36);
var nested = new Box<List<Pair<String, Integer>>>(List.of(p));   // type arguments can nest
```

Output (first three lines):

```
Box[hello] -> Box[5], len.get() + 1 = 6
Pair[key=age, value=36] swapped Pair[key=36, value=age]
nested: age
```

- `len.get() + 1` needs no cast — `get()` returns `Integer`.
- `map` declares its **own** `R` beside the class's `T`; its wildcard parameter types are explained in §9.6.
- Static members can't use the class's type parameters (§9.8), so the factory `Pair.of` redeclares `<K, V>`.

Don't make a class generic "just in case" — if every user passes the same type, a plain class is simpler.

9.3 Generic interfaces

What it is

Interfaces take type parameters like classes: `Comparable<T>`, `Iterable<T>`, `Function<T, R>`, `Repository<T, ID>`. An implementer can:

1. **Fix the argument** — `class StringStack implements Container<String>` (not generic itself);
2. **Pass its own parameter through** — `class Queue<T> implements Container<T>`;
3. Be an **anonymous class or lambda** with the type inferred.

`code/ch09/interfaces/GenericInterfaces.java` (key parts):

```java
@FunctionalInterface
interface Transformer<A, B> {
    B apply(A input);
    default <C> Transformer<A, C> andThen(Transformer<? super B, ? extends C> next) {
        return a -> next.apply(apply(a));
    }
}

record Version(int major, int minor) implements Comparable<Version> {    // concrete type argument
    public int compareTo(Version o) {
        int c = Integer.compare(major, o.major);
        return c != 0 ? c : Integer.compare(minor, o.minor);
    }
}

interface Container<T> { void put(T item); T take(); boolean isEmpty(); }
static class StringStack implements Container<String> { ... }            // 1. fixed
static class Queue<T> implements Container<T> { ... }                    // 2. stays generic
Container<Runnable> tasks = new Container<>() { ... };                   // 3. anonymous + diamond (Java 9)

static <T> List<T> drain(Container<T> c) { ... }                         // works for all three
```

Output:

```
8 chars
natural order: [8.0, 17.2, 21.0, 21.1]
by minor, then descending: [21.0, 8.0, 21.1, 17.2]
stack drains LIFO: [c, b, a], queue drains FIFO: [1, 2, 3]
task ran
```

The repository pattern (from `GenericClasses.java`) — generic interface, generic implementation, subclass fixing the types:

```java
interface Repository<T, ID> {
    void save(ID id, T item);
    Optional<T> findById(ID id);
    List<T> findAll();
}
static class InMemoryRepository<T, ID> implements Repository<T, ID> {
    private final Map<ID, T> store = new HashMap<>();
    ...
}
static class BookRepository extends InMemoryRepository<Book, String> {
    List<String> titles() { return findAll().stream().map(Book::title).sorted().toList(); }
}
```

Output:

```
findById: Effective Java | missing: false | titles: [Effective Java, Java Concurrency in Practice]
numbers: [100]
```

This is Spring Data's `CrudRepository<T, ID>` shape: persistence logic written once, reused for every entity type. Design view (a repository owns persistence so domain classes don't): LLD SRP.

Implementing `Comparable<Version>` (not raw `Comparable`) gives `compareTo(Version o)` — no cast, and comparing to a `String` is a compile error. `Comparable` / `Comparator`: Chapter 10; `Function<T, R>`: Chapter 11.

9.4 Generic methods and type inference

What it is

A **generic method** declares its own type parameters **before the return type**: `static <T> void swap(T[] arr, int i, int j)`, in any class, generic or not (`List.of`, `Collections.sort`, `Arrays.asList`, `Optional.of`,

`Stream.map`). **Type inference** works out `T` from the arguments and the **target type** (what the result is assigned to or passed as).

How it works

1. Declare: `static <T> List<T> repeat(T item, int times)`.
2. Call: `repeat("hi", 3)` — `T` = `String` from the argument.
3. No arguments → target type decides: `List<String> empty = Collections.emptyList();`.
4. Explicit **type witness** if needed: `GenericMethods.<Number>repeat(1, 2)` (a class or `this.` prefix is required before `<…>`).
5. Mixed argument types infer the most specific common supertype, possibly an *intersection type* like `Number & Comparable<…>`.

`code/ch09/methods/GenericMethods.java` (key parts):

```
static <T> void swap(T[] arr, int i, int j) { T tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp; }
static <T> List<T> repeat(T item, int times) { ... }
static <T> T firstOrDefault(List<T> list, T fallback) { return list.isEmpty() ? fallback : list.get(0); }

swap(words, 0, 2);                                  // T inferred as String
swap(nums, 0, 1);                                   // T inferred as Integer
List<String> empty = Collections.emptyList();       // T = String from the assignment
List<Number> mixed = GenericMethods.<Number>repeat(1, 2);   // explicit witness
mixed.add(2.5);
var list = Arrays.asList(1, 2.0, 3L);               // List<Number & Comparable<...>>
```

Output:

```
[c, b, a] [2, 1, 3]
repeat: [hi, hi, hi]
firstOrDefault: none
witness <Number>: [1, 1, 2.5]
Arrays.asList(1, 2.0, 3L) element types: [Integer, Double, Long]
```

Without the witness `repeat(1, 2)` infers `List<Integer>`, and `add(2.5)` fails. (`List<Number> mixed = repeat(1, 2);` also works — the target type drives inference.)

Common mistakes

- `<T>` misplaced: it goes **after modifiers, before the return type** — `public static <T> T pick(...)`.
- A method `<T>` inside `Box<T>` **shadows** the class's `T`; they're unrelated. Use another letter.
- `<T> void print(List<T> list)` with `T` used once — `void print(List<?> list)` is simpler.

9.5 Bounded type parameters

What it is

An unbounded `T` allows only `Object` methods. A **bound** restricts the allowed types and in exchange unlocks the bound's methods:

- **Upper bound** `<T extends Number>`: `T` is `Number` or a subtype, so `t.doubleValue()` compiles (`extends` covers interfaces too).
- **Multiple bounds** `<T extends Number & Comparable<T>>`: all must hold; at most one class, and it comes **first** (`<T extends Runnable & Serializable & Number>` is illegal).
- **Recursive bound** `<T extends Comparable<T>>`: "T comparable with itself" — behind sorting and `max`.

Type *parameters* have only upper bounds; `super` appears only in *wildcards* (§9.6).

`code/ch09/bounds/Bounded.java` (key parts):

```java
static <T extends Number> double sum(List<T> values) {
    double total = 0;
    for (T v : values) total += v.doubleValue();       // allowed because T extends Number
    return total;
}

static <T extends Comparable<? super T>> T max(List<? extends T> list) {
    T best = list.get(0);
    for (T x : list) if (x.compareTo(best) > 0) best = x;
    return best;
}

static <T extends Number & Comparable<T>> T clamp(T value, T lo, T hi) { ... }

static class Stats<T extends Number> {                  // bounded generic class
    double average() { return data.isEmpty() ? 0 : sum(data) / data.size(); }
}
// sum(List.of("a"));            // compile error: String is not within bound Number
```

Output:

```
sum ints=6.0 sum doubles=4.0
max words=zucchini max ints=9
clamp(15, 0, 10)=10 clamp(2.5, 0.0, 1.0)=1.0
Stats average=5.0
```

Reading `<T extends Comparable<? super T>>`

`Collections.max` / `sort`'s (simplified) signature, in pieces:

1. `T extends Comparable<…>` — elements are comparable.
2. `Comparable<? super T>` — comparable to `T` **or a supertype of** `T`.

Why: if `class Fruit implements Comparable<Fruit>` and `class Apple extends Fruit`, `Apple` is `Comparable<Fruit>`, not `Comparable<Apple>`. With `<T extends Comparable<T>>` you couldn't take the max of a `List<Apple>`; with `? super T` you can. Likewise `LocalDate` implements `Comparable<ChronoLocalDate>`.

9.6 Invariance, wildcards and PECS

What it is

Invariance: although `Integer` is a subtype of `Number`, **`List<Integer>` is *not* a `List<Number>`** — otherwise:

```java
List<Integer> ints = new ArrayList<>();
List<Number> nums = ints;        // imagine this compiled...
nums.add(3.14);                  // ...then a Double would sit in a List<Integer>
```

The compiler rejects it: *"incompatible types: ArrayList cannot be converted to List"* (likewise `List<Object> x = new ArrayList<String>();`). Arrays chose **covariance** (`String[]` is an `Object[]`) and pay with a run-time `ArrayStoreException` (§4.6); generics catch the mistake at compile time.

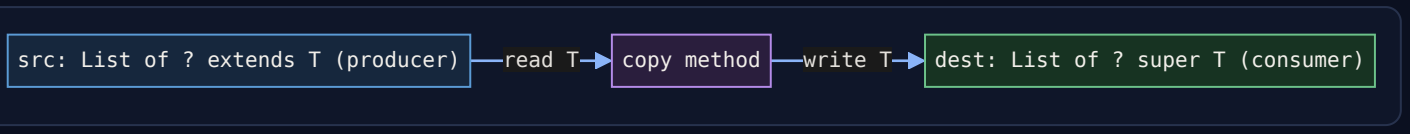
Wildcards restore flexibility safely; `?` means "some specific type I don't know":

Wildcard	Read as	Read elements as	Add	Accepts
`List<? extends Number>`	some subtype of Number	`Number`	nothing (except `null`)	`List<Number>`, `List<Integer>`, `List<Double>`
`List<? super Integer>`	some supertype of Integer	`Object` only	`Integer` (and subtypes)	`List<Integer>`, `List<Number>`, `List<Object>`
`List<?>`	some type	`Object`	nothing (except `null`)	any `List` (`List<?> x = new ArrayList<String>()` is legal)
`List<Number>`	exactly Number	`Number`	any `Number`	only `List<Number>`

PECS: Producer `extends`, Consumer `super`

Joshua Bloch's rule (*Effective Java* Item 31) for **method parameters**:

- Parameter **produces** values you read → `? extends T` (summing any numbers: `Collection<? extends Number>`).
- Parameter **consumes** values you write → `? super T` (adding `Integer`s to a caller's list: `List<? super Integer>`).
- Both → exact `T`.
- Only `Object` methods / `size()` → `?`.



`code/ch09/wildcards/Wildcards.java` (key parts):

```
static double total(Collection<? extends Number> source) {       // producer
    double t = 0;
    for (Number n : source) t += n.doubleValue();
    // source.add(1);                  // compile error: can't add to ? extends (except null)
    return t;
}

static void fillWithSquares(List<? super Integer> sink, int n) {  // consumer
    for (int i = 1; i <= n; i++) sink.add(i * i);
    Object o = sink.get(0);                // reading gives only Object
}

static <T> void copy(List<? super T> dest, List<? extends T> src) {   // both: shape of Collections.copy
    for (T item : src) dest.add(item);
}

static int countNulls(List<?> list) { ... }                       // unbounded

static void swapFirstLast(List<?> list) { swapHelper(list); }     // wildcard capture
private static <T> void swapHelper(List<T> list) {
    T first = list.get(0);
    list.set(0, list.get(list.size() - 1));
    list.set(list.size() - 1, first);
}
```

Output:

```
total(ints)=6.0 total(doubles)=0.75
view.get(0).intValue()=1
numbers=[1, 4, 9] objects=[1, 4]
copy -> [start, 1, 2, 3]
countNulls=2
swapFirstLast=[z, y, x]
arrays: ArrayStoreException java.lang.Integer
```

`total` accepts `List<Integer>` and `List<Double>`; `fillWithSquares` accepts `List<Number>` and `List<Object>`. The commented `source.add(1)` fails with *"incompatible types: int cannot be converted to CAP#1"* — **CAP#1** is the compiler's name for the captured unknown type; it can't prove 1 fits (the list might be `List<Double>`).

Wildcard capture

You can't `set` into a `List<?>`. The **capture helper** passes it to a private generic method where the unknown type gets a name (`T`), so read-then-write of the same `T` is provably safe — how a `Collections.swap(List<?>, int, int)`-style API keeps a parameter-free public signature. (The JDK's own implementation uses a raw type internally; its source comment notes a private capture helper would be the type-safe alternative.)

`List<?>` vs `List<Object>` vs raw `List`

	Accepts a `List<String>`?	Can add a `String`?	Type-safe?
`List<Object>`	No (invariance)	Yes	Yes
`List<?>`	Yes	No (only `null`)	Yes
raw `List`	Yes	Yes (with a warning)	**No**

Never put wildcards in **return types** (`List<? extends Number> getValues()` burdens every caller) — return `List<Number>` / `List<T>`. When a method reads and writes the same parameter, use `T`.

In the interview

> **30-second answer:** "Generics are invariant: `List<Integer>` isn't a `List<Number>`, otherwise you could add a `Double` to a list of integers. Wildcards add flexibility via PECS — a parameter that produces values I read is `? extends T`; one that consumes values I write is `? super T`. `Collections.copy(List<? super T> dest, List<? extends T> src)` is the textbook case. `List<?>` is a read-only-ish list of unknown type, safer than raw `List`."

- *Why can't you add to `List<? extends Number>`?* It might be a `List<Double>`; only `null` is allowed.
- *What can you read from `List<? super Integer>`?* Only `Object` — it could be a `List<Object>`.
- *Arrays vs generics?* Arrays are covariant and reified (run-time `ArrayStoreException`); generics invariant and erased (compile-time checks).

TL;DR

Generics are invariant. Producer → `extends`, consumer → `super`, both → `T`, don't care → `?`. Wildcards on parameters, never return types.

9.7 Type erasure: how generics really work

What it is

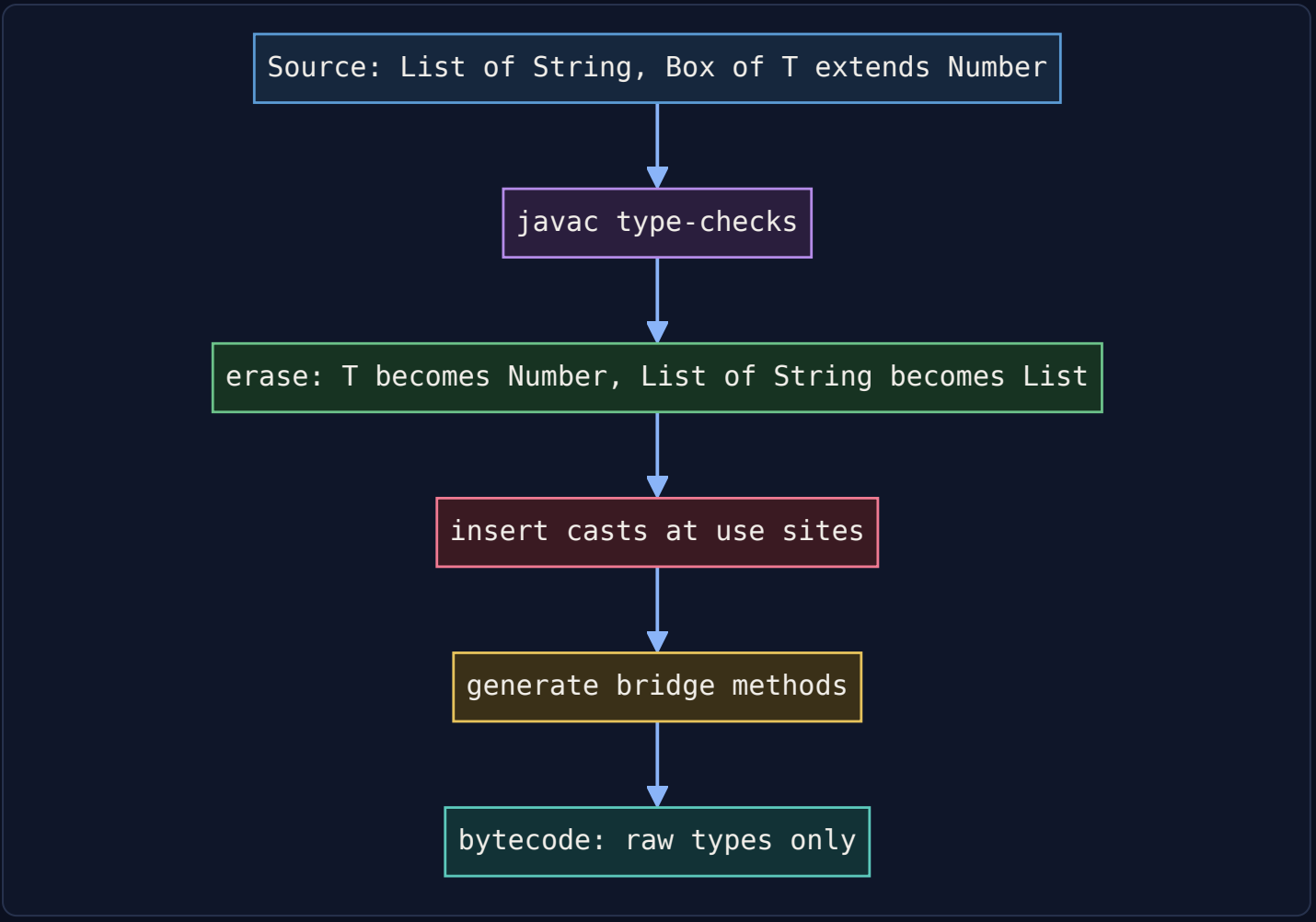
Java implements generics by **type erasure**: the compiler checks generic types, then **removes** them from the bytecode. At run time `List<String>` and `List<Integer>` are both just `ArrayList`. Generic types are therefore **non-reifiable** (full type unavailable at run time), unlike arrays, which *are* reified (`String[]` knows its element type).

Why: **migration compatibility**. In 2004 vast amounts of code and class files used raw collections; erasure let generic and non-generic code interoperate and left the JVM unchanged.

How it works

1. The compiler type-checks every use.

2. Each type parameter becomes its **erasure**: its first bound (`<T extends Comparable<T>>` → `Comparable`), or `Object` if unbounded (`<T extends Number>` → `Number`).
3. Parameterized types become raw (`List<String>` → `List`).
4. The compiler **inserts casts** at use sites (`String s = list.get(0)` → `(String) list.get(0)`); with no unchecked warnings these can never fail.
5. Where erasure would break overriding, it generates a **bridge method**.



code/ch09/erasure/Erasure.java (key parts):

```
List<String> strings = new ArrayList<>();
List<Integer> ints = new ArrayList<>();
strings.getClass() == ints.getClass();             // true: one class at run time

static class Node<T extends Comparable<T>> implements Comparable<Node<T>> {
    public int compareTo(Node<T> o) { return value.compareTo(o.value); }
}
for (Method m : Node.class.getDeclaredMethods())
    if (m.getName().equals("compareTo")) System.out.println(... m.isBridge());
```

Output (relevant lines):

```
same runtime class? true -> java.util.ArrayList
o instanceof List<?>: true
compareTo(Node) bridge=false
compareTo(Object) bridge=true
```

`Comparable<T>` erases to `compareTo(Object)`, which our `compareTo(Node)` doesn't override, so the compiler adds a synthetic **bridge** `compareTo(Object o) { return compareTo((Node) o); }` to keep polymorphism working. Bridges occasionally show up in stack traces.

What survives erasure

Type arguments vanish from *objects and locals*, but generic **declarations** keep their signatures in the class file (the `Signature` attribute). So reflection can see a field declared `List<String>`, and Jackson/Gson use the **super type token** trick (`new TypeReference<List<User>>() {}`) — an anonymous subclass records its generic superclass, which is preserved.

Trade-offs — erasure vs reification (e.g. C#)

	Java (erasure)	C# (reified generics)
Runtime knows `List<String>`?	No	Yes
`new T()`, `T[]`, `is List<string>`	Not allowed	Allowed
Primitives as type arguments	No (boxing)	Yes (`List<int>` without boxing)
Old/new code interoperability	Seamless	Separate generic collection types were added

Project Valhalla (OpenJDK) is working toward value types (identity-free objects stored like primitives) and better generics over primitives; in Java 25 type arguments must still be reference types.

9.8 Restrictions and workarounds (consequences of erasure)

Each follows from "the type argument doesn't exist at run time" (real `javac` messages):

You can't…	javac says	Workaround
Use primitives: `List<int>`	*unexpected type*	Wrappers (`List<Integer>`) or specialisations (`IntStream`, `int[]`)
Instantiate: `new T()`	*unexpected type*	Take a `Supplier<T>` (`StringBuilder::new`) or a `Class<T>` token
Create generic arrays: `new T[10]`, `new List<String>[10]`	*generic array creation*	`List<T>`; `Array.newInstance(type, n)` with a `Class<T>`; or `(T[]) new Object[n]` inside a class that never exposes it (as `ArrayList` does)
Use `T` statically: `static T shared;`	*non-static type variable T cannot be referenced from a static context*	Static generic methods declare their own `<T>`
`instanceof` a parameterized type from an unrelated static type: `o instanceof List<String>`	*Object cannot be safely cast to List*	`o instanceof List<?>`; since Java 16 `instanceof ArrayList<String>` **is** allowed when the static type is already `List<String>` (provably safe)
Overload on type arguments: `f(List<String>)` and `f(List<Integer>)`	*name clash: … have the same erasure*	Different method names
Generic exceptions: `class Bad<X> extends Exception`	*a generic class may not extend java.lang.Throwable*	Concrete-typed field; `catch` needs a runtime type
`.class` on a parameterized type: `List<String>.class`	syntax error	`List.class`

Workarounds in code

```
// Factory instead of new T()
static <T> List<T> fill(int n, Supplier<T> factory) { ... }           // fill(3, StringBuilder::new)

// Class token
static <T> T create(Class<T> type) throws ReflectiveOperationException {
    return type.getDeclaredConstructor().newInstance();
}

// Generic array via reflection
@SuppressWarnings("unchecked")
static <T> T[] newArray(Class<T> type, int n) {
    return (T[]) java.lang.reflect.Array.newInstance(type, n);
}
```

Output:

```
fill: 3 builders, create: ""
newArray component type: String[2]
```

Heap pollution and unchecked warnings

Heap pollution: a parameterized-type variable refers to an object of the wrong type (a `List<Integer>` holding a `String`). Only **unchecked** operations — raw types, unchecked casts, generic varargs — can cause it, which is why the compiler warns. The failure appears later, where an inserted cast runs.

```
List<Object> objs = new ArrayList<>(List.of("text"));
@SuppressWarnings("unchecked")
List<Integer> lie = (List<Integer>) (List<?>) objs;   // compiles with a warning, no check at run time
Integer n = lie.get(0);                               // ClassCastException HERE, far from the lie
```

Output:

```
heap pollution -> ClassCastException on read
```

Effective Java Item 27: **eliminate every unchecked warning** you can; suppress a proven-safe one on the **smallest scope** (one local declaration, not the class) with a comment explaining why.

Generic varargs and `@SafeVarargs`

`T... items` is an array `T[]`, which can't be created safely, so the compiler makes an `Object[]` and warns: *"Possible heap pollution from parameterized vararg type T"*. It's dangerous if the method **stores** or **returns** the array:

```
static <T> T[] toArray(T... items) { return items; }          // returns the varargs array: UNSAFE
static <T> T[] pickTwo(T a, T b) { return toArray(a, b); }    // T erased -> creates Object[]
String[] two = pickTwo("x", "y");                             // ClassCastException: Object[] is not String[]
```

Output:

```
pickTwo -> ClassCastException: generic varargs array escaped
```

A varargs method that only *reads* the elements and never lets the array escape is **safe**; annotate it `@SafeVarargs` to silence call-site warnings:

```
@SafeVarargs
static <T> List<T> listOf(T... items) { return new ArrayList<>(Arrays.asList(items)); }
```

`@SafeVarargs` is allowed only on non-overridable methods: `static`, `final`, constructors, and (Java 9) `private`. `List.of(E...)`, `Arrays.asList`, `Stream.of` use it. *Effective Java* Item 32 (generics + varargs judiciously), Item 28 (prefer lists to arrays).

9.9 Designing generic APIs

Best practices (*Effective Java* Items 26–33)

Item	Rule	In one line
26	Don't use raw types	`List<?>` if you don't care
27	Eliminate unchecked warnings	suppress only proven-safe cases, narrowly, with a comment
28	Prefer lists to arrays	invariant, compile-time checked
29	Favour generic types	generic containers, not `Object`-based
30	Favour generic methods	especially static utilities and factories
31	Bounded wildcards for flexibility	PECS on parameters; never in return types
32	Combine generics and varargs judiciously	`@SafeVarargs` only when truly safe
33	Typesafe heterogeneous containers (values of many types, safely)	`Map<Class<?>, Object>` keyed by `Class<T>`, `<T> T get(Class<T> key)`

- **Type parameter vs wildcard:** if a type parameter appears once, use a wildcard (`void log(List<?> xs)`, not `<T> void log(List<T> xs)`); use a type parameter when it links a parameter to the return type or two parameters.
- **Prefer `List<T>` over `T[]`** in public APIs.
- Three or more type parameters is a smell unless it's a genuine tuple/function type (`BiFunction<T, U, R>`).

Reading JDK signatures

Signature	How to read it
`static <T> void sort(List<T> list, Comparator<? super T> c)`	Sort `T`s with a comparator for `T` or any supertype (a `Comparator<Fruit>` sorts a `List<Apple>`)
`static <T extends Object & Comparable<? super T>> T max(Collection<? extends T> coll)`	Elements comparable to themselves or a supertype; `Object &` keeps the *erasure* of `T` as `Object`, preserving binary compatibility (old compiled code keeps working) with pre-Java-5 code
`<R> Stream<R> map(Function<? super T, ? extends R> mapper)`	The function consumes any supertype of `T` and produces any subtype of `R` — PECS on a function
`static <T, U extends Comparable<? super U>> Comparator<T> comparing(Function<? super T, ? extends U> keyExtractor)`	Build a `Comparator<T>` from a function extracting a comparable key `U`
`boolean addAll(Collection<? extends E> c)`	Any collection of `E` or subtypes

Every `? super` **takes** your values; every `? extends` **gives** you values.

Common mistakes

- Raw types, often by forgetting `<>` (`new ArrayList()`).
- `List<Object>` parameters where `List<?>` or a type parameter was meant — `List<String>` callers can't pass.
- Wildcards in return types; class-level `@SuppressWarnings("unchecked")`.
- Assuming runtime checks: data from reflection, serialization or raw legacy code can still pollute the heap. `Collections.checkedList(list, String.class)` checks insertions at run time to find where bad data enters.

Real use: `Optional<T>`, `CompletableFuture<T>`, `Stream<T>`, `Map<K, V>`; Spring Data `JpaRepository<T, ID>`; Jackson `TypeReference<T>`; JUnit's `assertThrows(Class<T>, Executable)` returning `T` via a `Class<T>` token; event buses and dependency-injection containers keyed by `Class<T>` (Item 33).

In the interview

> **30-second answer:** "Generics give compile-time type safety and remove casts. They're implemented by erasure: type arguments are checked then removed, so at run time a `List<String>` is just a `List`. That's why there's no `new T()`, no generic arrays, no primitive type arguments, no overloading on `List<String>` vs `List<Integer>`. They're invariant, so I use bounded wildcards with PECS for flexible APIs."

- *Bridge method?* A synthetic method the compiler generates so overriding still works after erasure changes a signature.
- *Full generic type at run time?* Super type tokens — an anonymous `TypeReference<List<User>>` subclass keeps its generic superclass in the class file.
- `<T extends Comparable<T>>` *vs* `<T extends Comparable<? super T>>`*?* The second also accepts types inheriting `compareTo` from a supertype (`Apple extends Fruit implements Comparable<Fruit>`).

TL;DR

Generic types/methods for type-safe reuse; bounds to use a type's methods; PECS wildcards on parameters; erasure explains every restriction; no raw types, no unexplained unchecked warnings, lists over arrays.

Chapter TL;DR

Topic	Remember
Purpose	Compile-time type safety, no casts; raw types are legacy — `List<?>` if unsure
Syntax	`class Box<T>`, `record Pair<K, V>`, `interface Repo<T, ID>`, `static <T> T m(...)`; diamond `<>` (7, anonymous classes 9)
Inference	From arguments and target type; explicit witness `Cls.<T>m()`
Bounds	`<T extends Number>`, `<T extends A & I1 & I2>` (class first), recursive `<T extends Comparable<? super T>>`
Invariance	`List<Integer>` is not a `List<Number>`; arrays are covariant (runtime `ArrayStoreException`)
Wildcards	PECS: producer `? extends`, consumer `? super`, both → `T`, don't care → `?`; not in return types
Erasure	Type args removed after checking; `T` → bound or `Object`; casts inserted; bridge methods
Restrictions	No primitives, `new T()`, `new T[]`, static `T`, `instanceof List<String>` (from `Object`), overload by type arg, generic exceptions
Heap pollution	From unchecked operations; fix warnings; `@SafeVarargs` only for non-escaping varargs arrays
Design	*Effective Java* 26–33; lists over arrays; wildcard if a type param appears once

Next: Chapter 10 — Collections, then generic functional interfaces in Chapter 11. Exam traps: OCP Chapter 9.

Chapter 10 — Collections

Real programs hold *groups*: cart items, online users, word counts, pending jobs. Arrays (§4.6) are fixed-length with little behaviour. The **Java Collections Framework** (`java.util`) provides resizable lists, duplicate-free sets, key→value maps, queues (FIFO waiting lines), stacks (LIFO piles) and priority queues behind a small set of interfaces, so implementations can be swapped without rewriting code.

For each collection: what it is, **how it works inside** (so the big-O costs — how running time grows with size, §10.12 — make sense), which to pick, and the traps. Builds on generics (Chapter 9) and `equals` / `hashCode` (§6.13); lambdas like `n -> n % 2 == 0` are "a small function" until Chapter 11.

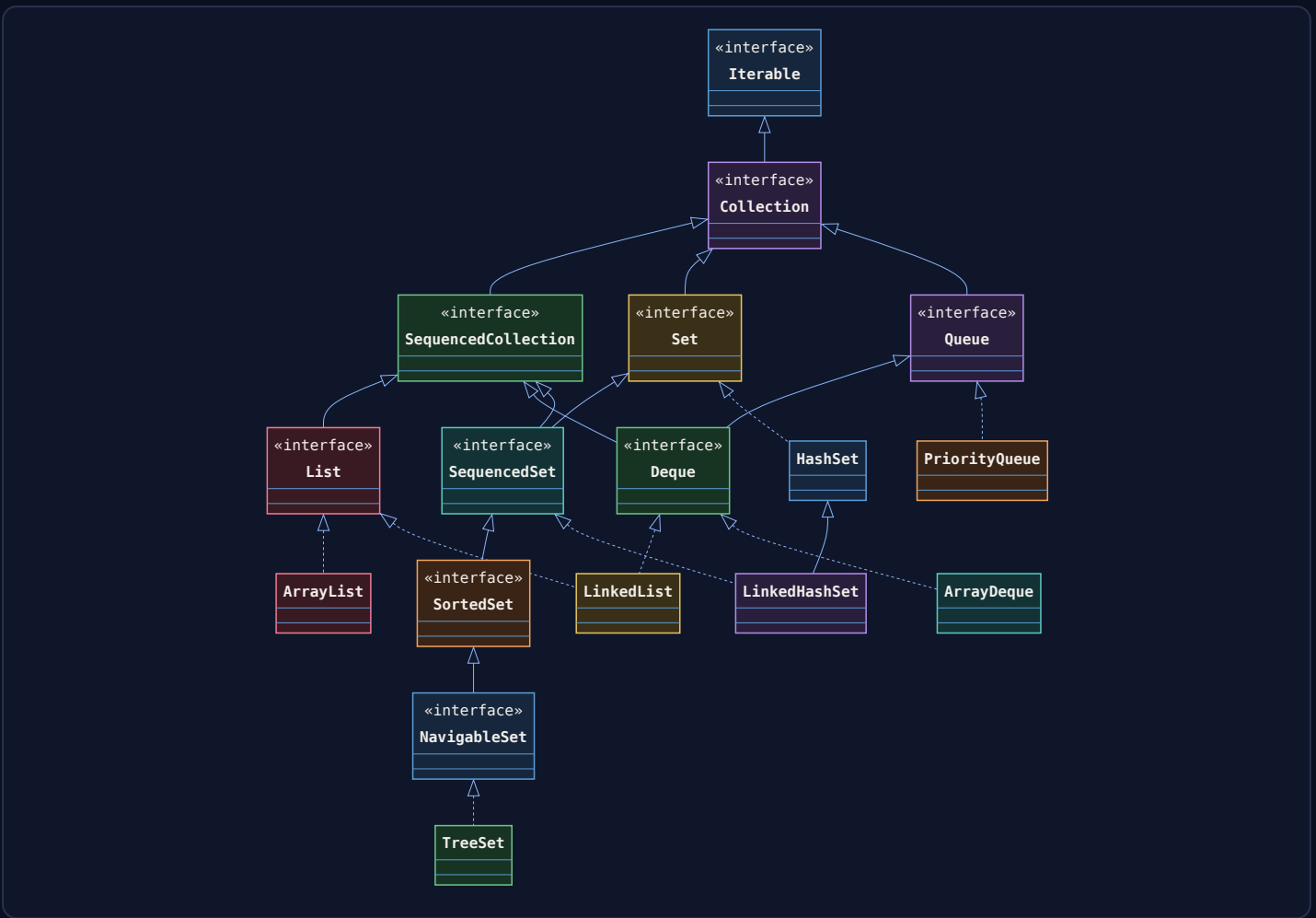
Code: `code/` (package `ch10.*`), `javac --release 25` , run on JDK 26. Exam traps (`List.of` vs `Arrays.asList` , `remove(int)` vs `remove(Object)` , `TreeSet` / `TreeMap` ordering, `Comparator` chaining, wildcards): OCP Chapter 9.

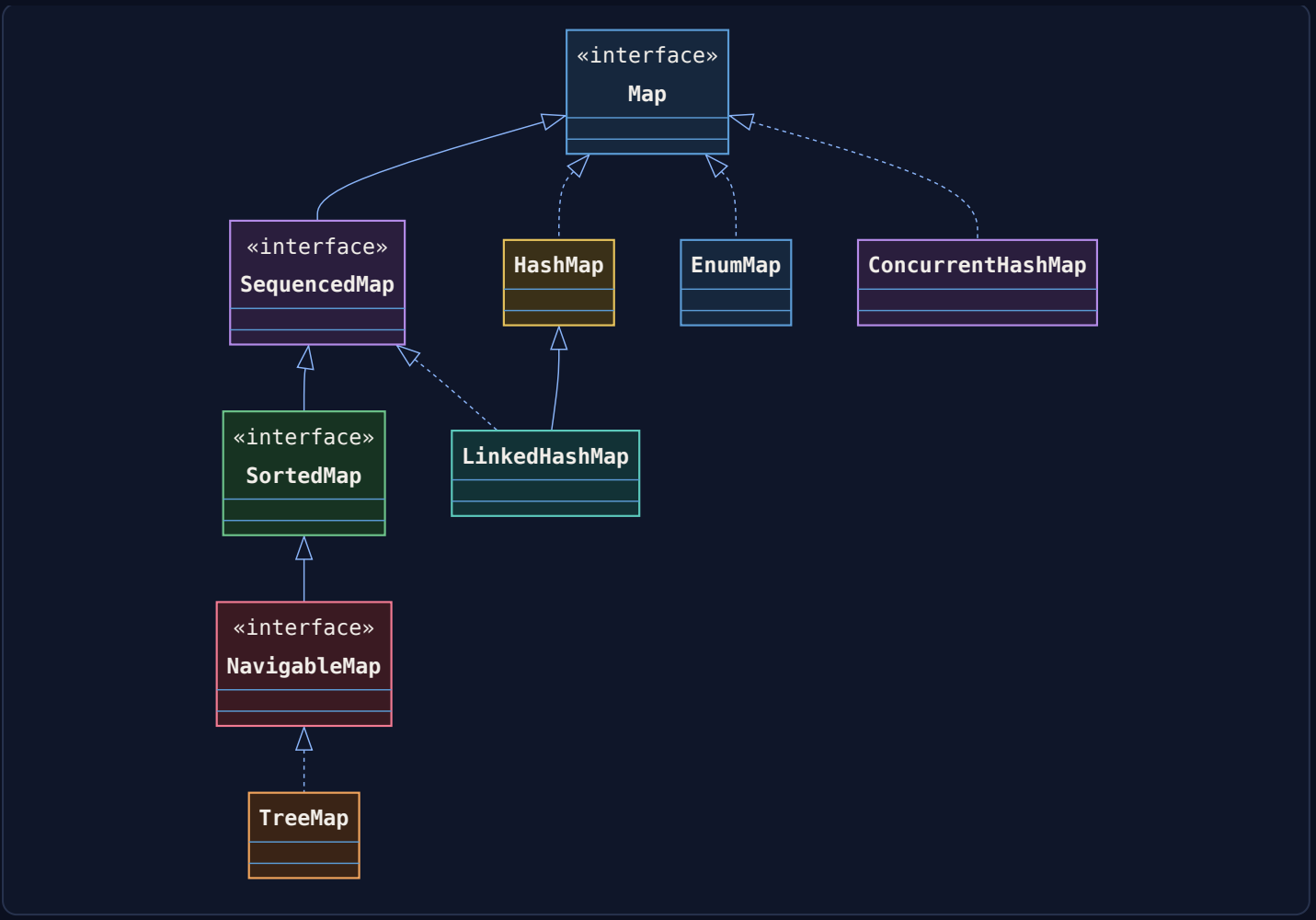
10.1 The Collections Framework and its hierarchy

What it is

A **collection** groups other objects (its **elements**). The framework is **interfaces** (contracts such as `List` — ordered, indexable, duplicates allowed — and `Set` — no duplicates), **implementations** (`ArrayList` , `HashSet` … each a particular data structure) and **algorithms** (static sorting/searching/shuffling in `Collections` and `Arrays`).

How it is organised





Interface	Promise	Main implementations
`Iterable<T>`	Usable in for-each	every collection
`Collection<E>`	`add`, `remove`, `contains`, `size`, `iterator`, `stream`	root of element-based types
`List<E>`	Ordered by index, duplicates allowed	`ArrayList`, `LinkedList`
`Set<E>`	No duplicates (by `equals`)	`HashSet`, `LinkedHashSet`, `TreeSet`, `EnumSet`
`Queue<E>`	Elements wait for processing, usually FIFO	`ArrayDeque`, `LinkedList`, `PriorityQueue`
`Deque<E>`	Double-ended queue; also a stack	`ArrayDeque`, `LinkedList`
`Map<K,V>`	Unique keys, one value each	`HashMap`, `LinkedHashMap`, `TreeMap`, `EnumMap`
`SequencedCollection` / `SequencedMap` (21)	Defined first and last element	`List`, `Deque`, `LinkedHashSet`, `TreeSet`, `LinkedHashMap`, `TreeMap` (§10.11)

Terms:

- **Encounter order** — iteration follows a defined sequence (index, insertion or sorted order).
- **Sorted** — ordered by comparison (natural order or a `Comparator`, §10.8).
- **Structural modification** — a size change (add/remove), unlike replacing via `set`.
- **Optional operation** — a method an implementation may reject with `UnsupportedOperationException` (`add` on an unmodifiable list).

`Map` **is not a** `Collection` (`add(E)` makes no sense for pairs). It offers three live **views** that are collections: `keySet()`, `values()`, `entrySet()`.

Program to the interface

```
// code/ch10/hierarchy/HierarchyDemo.java (key parts)
Collection<String> langs = new ArrayList<>();          // declared type = interface
langs.add("Java"); langs.add("Go"); langs.add("Java"); // lists allow duplicates
langs.addAll(List.of("Rust", "Kotlin"));
langs.remove("Java");                                  // removes the FIRST match only
langs.removeIf(s -> s.length() <= 2);                  // bulk remove by condition (Java 8)

Collection<String> unique = new HashSet<>(List.of("Java", "Go", "Rust"));
System.out.println("add duplicate to set -> " + unique.add("Java"));   // add returns false

String[] arr = langs.toArray(String[]::new);           // typed array (Java 11 form)
Map<String, Integer> ages = new TreeMap<>(Map.of("ana", 31, "bo", 25));
System.out.println("keys=" + ages.keySet() + " values=" + ages.values() + " entries=" + ages.entrySet());
```

Output:

```
list-backed: [Java, Go, Java, Rust, Kotlin] size=5
contains Go? true
after removes: [Java, Rust, Kotlin]
add duplicate to set -> false
array: [Java, Rust, Kotlin] isEmpty=false
keys=[ana, bo] values=[31, 25] entries=[ana=31, bo=25]
Map is a Collection? false
```

Declaring `List`, `Set`, `Map` (not `ArrayList`…) lets code work with any implementation and change it in one place. Use the concrete type only for its extra methods (`TreeMap.floorKey`, `LinkedList.addFirst` before Java 21).

Legacy classes

`Vector` (synchronized `ArrayList`), `Stack` (extends `Vector`), `Hashtable` (synchronized map, no nulls), `Enumeration` — Java 1.0. Every method takes a lock, and `Stack` inherits index methods a stack shouldn't have. Use `ArrayList`, `ArrayDeque`, `HashMap`, and `java.util.concurrent` classes when threads share data (Chapter 14).

TL;DR

Interfaces (`List`, `Set`, `Queue`, `Deque`, `Map`) describe behaviour; classes pick the data structure. Program to the interface. Avoid `Vector`, `Stack`, `Hashtable`.

10.2 Lists: `ArrayList` and `LinkedList`

What it is

A `List` is an ordered, indexable sequence (`get(i)`), allowing duplicates and (mostly) `null`.

- `ArrayList` — a resizable array: instant jump to index 500, but inserting at 0 shifts everything.
- `LinkedList` — a **doubly linked list**: each element in a *node* pointing to previous and next; reaching index 500 means following 500 links.

How `ArrayList` works

1. Internal `Object[] elementData` plus `int size`.
2. `new ArrayList<>()` starts with a shared empty array; the first `add` allocates capacity **10**.
3. `add(e)` stores at `elementData[size++]`; when full it allocates **~1.5×** (`old + old >> 1`) and copies. Geometric growth keeps total copying proportional to n, so append is **amortised O(1)**.
4. `get(i)` / `set(i, e)`: **O(1)**.

5. `add(i, e)` / `remove(i)` shift the tail with `System.arraycopy`: **O(n − i)** (so `add(0, x)` is O(n)).
6. `contains` / `indexOf` scan with `equals`: **O(n)**.
7. Never shrinks itself; `trimToSize()` releases spare capacity; `new ArrayList<>(expectedSize)` / `ensureCapacity(n)` avoid repeated growth.

How `LinkedList` works

1. Each element is a `Node { item, prev, next }`; the list tracks `first`, `last`, `size`.
2. Add/remove at either end rewires two pointers: **O(1)**.
3. `get(i)` walks from the nearer end: **O(n)**.
4. Mid-list insert is O(1) *only once you're there* (via a `ListIterator`, §10.7); getting there is O(n).
5. Each node is a separate object (~24 bytes overhead on a typical 64-bit JVM with compressed pointers), scattered in memory, defeating the CPU cache (fast memory that favours data stored side by side).

`LinkedList` also implements `Deque`, but `ArrayDeque` does that faster (§10.6). **`ArrayList` is the right default** (measured in §10.12).

Code

```
// code/ch10/lists/ListDemo.java (key parts)
List<String> todo = new ArrayList<>(List.of("write", "test", "ship"));
todo.add(1, "review");                // insert at index 1, shifts the rest right
todo.set(0, "design");                // replace; returns the old value

List<Integer> nums = new ArrayList<>(List.of(10, 20, 30, 1));
nums.remove(1);                       // int argument -> remove(int index): removes 20
nums.remove(Integer.valueOf(1));      // object argument -> remove(Object): removes the value 1

List<String> firstTwo = todo.subList(0, 2);
firstTwo.clear();                     // subList is a VIEW: clears those elements from todo

String[] arr = {"a", "b", "c"};
List<String> view = Arrays.asList(arr); // fixed-size view backed by the array
view.set(0, "z");                      // writes through to arr
view.add("d");                         // UnsupportedOperationException

LinkedList<String> ll = new LinkedList<>(List.of("b", "c"));
ll.addFirst("a"); ll.addLast("d");

List<String> names = new ArrayList<>(List.of("mia", "Al", "zoe", "Bob", "ann"));
names.sort(null);                      // null comparator = natural order
names.sort(String.CASE_INSENSITIVE_ORDER);
```

Output:

```
[design, review, test, ship] get(2)=test indexOf(ship)=3
nums=[10, 30]
after subList.clear(): [test, ship]
array now [z, b, c]
asList.add -> UnsupportedOperationException
linked=[a, b, c, d] removeFirst=a peekLast=d
natural (uppercase first): [Al, Bob, ann, mia, zoe]
case-insensitive: [Al, ann, Bob, mia, zoe]
ArrayList equals LinkedList? true
```

- `remove(1)` on a `List<Integer>` picks **`remove(int index)`** — `1` is an `int`, no boxing needed (§4.3). Box explicitly to remove by value.
- Natural `String` order compares UTF-16 code units: all uppercase before lowercase.
- `List.equals`: same elements, same order, regardless of implementation.

Trade-offs

Operation	ArrayList	LinkedList
`get(i)` / `set(i, e)`	**O(1)**	O(n)
`add(e)` at end	amortised **O(1)**	O(1)
`add(0, e)` / `remove(0)`	O(n) (shift)	**O(1)**
insert/remove in middle	O(n) shift	O(n) to find + O(1) to link
`contains` / `indexOf`	O(n)	O(n)
Memory per element	one reference (+ spare capacity)	node object ≈ 24 B + reference
Cache friendliness	high (contiguous)	low (pointer chasing)

Frequent membership checks → a `Set`; queue/stack work → `ArrayDeque`, not `ArrayList.remove(0)`; fixed-size numeric data in hot loops → `int[]` (no boxing).

Common mistakes

- `remove(int)` vs `remove(Object)` on `List<Integer>`.
- Structurally modifying the parent while holding a `subList` → the sublist throws `ConcurrentModificationException` on next use.
- Expecting `Arrays.asList(...)` to grow — wrap: `new ArrayList<>(Arrays.asList(...))`. `Arrays.asList(intArray)` gives a one-element `List<int[]>`.
- `list.get(i)` in a loop over a `LinkedList` — O(n²); use for-each.

In the interview

> **30-second answer:** "ArrayList is a growable array: O(1) random access, amortised O(1) append, O(n) middle inserts/removes because it shifts. LinkedList is doubly linked: O(1) at the ends, O(n) to reach an index, more memory and poor cache locality. I default to ArrayList, and use ArrayDeque for queues and stacks."

- *How does ArrayList grow?* 10 on first add, then ~1.5× with an array copy; geometric growth makes append amortised O(1).
- *When does LinkedList win?* Many inserts/removes at positions you're already at via a `ListIterator` in a very long list — rare; measure.
- *`List.of` vs `Arrays.asList`?* `List.of` is fully unmodifiable and rejects nulls; `Arrays.asList` is a fixed-size writable view allowing nulls (§10.10).

TL;DR

`ArrayList` = array + 1.5× growth: fast index and append, slow front/middle edits. `LinkedList` = linked nodes: fast ends, slow indexing, heavy. Default to `ArrayList`.

10.3 Sets: `HashSet`, `LinkedHashSet`, `TreeSet`

What it is

A `Set` holds each element at most once — no two elements `equals`.

Class	Built on	Order	add / contains / remove	Nulls
`HashSet`	a `HashMap` (elements = keys)	none guaranteed	**O(1)** average	one `null`
`LinkedHashSet`	a `LinkedHashMap`	insertion order	O(1) average	one `null`
`TreeSet`	a `TreeMap` (red-black tree)	sorted	**O(log n)**	no `null` with natural order
`EnumSet`	bit vector (one bit per constant)	declaration order	O(1), very fast	no `null` (§6.9)

How it works

- **`HashSet`** stores each element as a key of an internal `HashMap` with a shared dummy value, so §10.5 applies: `hashCode()` picks a bucket, `equals()` confirms.
- **`LinkedHashSet`** threads a doubly linked list through entries to remember insertion order (re-adding doesn't move an element).
- **`TreeSet`** uses a **red-black tree** — a self-balancing binary search tree (left subtree smaller, right larger) of height O(log n). It never calls `equals` / `hashCode`: elements are "the same" when `compareTo` / `compare` returns 0.

Code

```
// code/ch10/sets/SetDemo.java (key parts)
List<String> visits = List.of("home", "cart", "home", "search", "cart", "about");
new HashSet<>(visits);  new LinkedHashSet<>(visits);  new TreeSet<>(visits);

Set<String> union = new TreeSet<>(a); union.addAll(b);      // A ∪ B
Set<String> inter = new TreeSet<>(a); inter.retainAll(b);   // A ∩ B
Set<String> diff  = new TreeSet<>(a); diff.removeAll(b);    // A − B

TreeSet<Integer> prices = new TreeSet<>(List.of(5, 12, 20, 35, 50));
prices.floor(19); prices.ceiling(19); prices.lower(20); prices.higher(20);
prices.headSet(20); prices.tailSet(20); prices.subSet(10, 40);

List<String> key = new ArrayList<>(List.of("x"));
Set<List<String>> set = new HashSet<>(); set.add(key);
key.add("y");                                               // mutates a key already in the set
```

Output:

```
HashSet       [search, about, cart, home]  (order not guaranteed)
LinkedHashSet [home, cart, search, about]  (insertion order)
TreeSet       [about, cart, home, search]  (sorted)
union=[docker, git, java, sql] intersection=[git] difference=[java, sql]
floor(19)=12 ceiling(19)=20 lower(20)=12 higher(20)=35
headSet(20)=[5, 12] tailSet(20)=[20, 35, 50] subSet(10,40)=[12, 20, 35]
first=5 pollLast=50 descending=[35, 20, 12, 5]
contains mutated key? false, size=1
HashSet null ok=true, TreeSet.add(null) -> NPE
```

Navigation (`NavigableSet`): `floor(x)` = greatest ≤ x, `ceiling(x)` = least ≥ x, `lower` / `higher` = strictly < / >. `headSet(x)` excludes x, `tailSet(x)` includes it, `subSet(from, to)` = `[from, to)`; these range views are live.

The mutated-key line is the key bug: after `key.add("y")` its `hashCode` changed, so `contains` searches another bucket and says `false` though the object is inside. **Never mutate an object while it is a set element or map key** — use immutable keys.

Common mistakes

- `equals` without `hashCode` in a `HashSet` → duplicates (§6.13).
- A `TreeSet` comparator returning 0 for different elements silently drops them (§10.8).

- Relying on `HashSet` print order — it can change between JDKs or after a resize.

Uses: unique visitors (`HashSet<UserId>`), de-dup keeping first-seen order (`LinkedHashSet`), "next event after t" (`TreeSet.ceiling`), feature flags (`EnumSet`).

TL;DR

`HashSet` for fast membership, `LinkedHashSet` to keep insertion order, `TreeSet` for sorted order and nearest/range queries (O(log n)). Elements must not change while inside.

10.4 Maps: `HashMap` , `LinkedHashMap` , `TreeMap`

What it is

A `Map<K,V>` associates each unique **key** with one **value**; putting under an existing key *replaces* the value.

The everyday API

Method	Does	Returns
`put(k, v)`	insert or replace	previous value or `null`
`get(k)` / `getOrDefault(k, d)`	look up	value, or `null` / `d`
`containsKey(k)` / `containsValue(v)`	test	`boolean` (`containsValue` is O(n))
`remove(k)`	delete	removed value or `null`
`putIfAbsent(k, v)`	insert only if missing (or mapped to `null`)	existing value or `null`
`merge(k, v, fn)`	absent → put `v` , else put `fn(old, v)` ; `null` result removes	new value
`compute(k, fn)`	`fn(k, oldOrNull)` ; `null` result removes	new value
`computeIfAbsent(k, fn)`	if missing, compute and store `fn(k)`	current value
`computeIfPresent(k, fn)`	if present, recompute; `null` removes	new value
`forEach` , `replaceAll`	iterate / transform all values	—
`keySet()` , `values()` , `entrySet()`	live views	collections

The Java 8 `merge` / `compute*` methods replace most "get, null-check, put" boilerplate.

Code

```java
// code/ch10/maps/MapDemo.java (key parts)
Map<String, Integer> stock = new HashMap<>();
stock.put("apple", 5);
Integer old = stock.put("apple", 7);                    // replaces, returns 5
stock.putIfAbsent("pear", 3);
stock.merge("apple", 10, Integer::sum);                 // 7 + 10
stock.merge("plum", 1, Integer::sum);                   // absent -> puts 1
stock.compute("pear", (k, v) -> v == null ? 1 : v * 2);
stock.computeIfPresent("plum", (k, v) -> null);         // null result REMOVES plum

Map<String, Integer> counts = new TreeMap<>();          // word count idiom
for (String w : "to be or not to be".split(" ")) counts.merge(w, 1, Integer::sum);

Map<Character, List<String>> byInitial = new TreeMap<>(); // multimap idiom
for (String n : List.of("ana", "bo", "al", "cy", "ben"))
    byInitial.computeIfAbsent(n.charAt(0), c -> new ArrayList<>()).add(n);

for (Map.Entry<String, Integer> e : counts.entrySet())  // iterate entries, not keys + get
    if (e.getValue() > 1) System.out.print(e.getKey() + "x" + e.getValue() + " ");

TreeMap<Integer, String> bracket = new TreeMap<>(Map.of(0, "10%", 11_000, "12%", 44_725, "22%", 95_375, "24%"));
bracket.floorEntry(50_000).getValue();                  // bracket whose start <= 50,000

Map<String, String> lru = new LinkedHashMap<>(16, 0.75f, true) {   // true = access order
    @Override protected boolean removeEldestEntry(Map.Entry<String, String> eldest) { return size() > 3; }
};
lru.put("a", "A"); lru.put("b", "B"); lru.put("c", "C");
lru.get("a");       // a becomes most recently used
lru.put("d", "D");  // evicts b
```

Output:

```
old=5 get=7 missing=null default=0
stock={apple=17, pear=6}
counts={be=2, not=1, or=1, to=2}
byInitial={a=[ana, al], b=[bo, ben], c=[cy]}
bex2 tox2
income 50000 -> 22%, first bracket above 50000 starts at 95375, headMap(44725)={0=10%, 11000=12%}
LRU keys=[c, a, d]
HashMap null key -> 0
```

(Brackets are illustrative progressive-tax numbers, not current law.) The `LinkedHashMap` with access order + `removeEldestEntry` is an **LRU cache** (evicts the Least Recently Used entry when full; System Design Fundamentals).

The three main implementations

	`HashMap`	`LinkedHashMap`	`TreeMap`
Structure	array of buckets (§10.5)	`HashMap` + doubly linked entry list	red-black tree
Iteration order	unspecified	insertion (or access) order	sorted by key
`get` / `put` / `remove`	**O(1)** average	O(1) average	**O(log n)**
`null` key	one	one	not with natural ordering
Extras	—	`removeEldestEntry` → LRU	`floorKey`, `ceilingEntry`, `headMap`, `tailMap`, `subMap`, `firstKey`, `descendingMap`

Also: **`EnumMap`** (enum keys, array indexed by `ordinal()` — very fast), **`IdentityHashMap`** (keys compared with `==`, for object-graph algorithms like serialization), **`WeakHashMap`** (weakly held keys — an entry

vanishes once nothing else references its key; metadata caches), `ConcurrentHashMap` (thread-safe, no null keys/values — Chapter 14; simplified build in LLD §13.7).

Common mistakes

- `if (map.get(k) != null)` to test presence — wrong with `null` values; use `containsKey`.
- `for (K k : map.keySet()) use(map.get(k))` — two lookups per entry; iterate `entrySet()` or `forEach`.
- `map.get(k) + 1` for an absent key → NPE from unboxing; use `merge` / `getOrDefault`.
- A `computeIfAbsent` function that modifies the same map (for `HashMap` throws `ConcurrentModificationException` since Java 9) — bites recursive memoisation (§11.8).

Uses: HTTP headers and configuration (`Map<String,String>`), ID-keyed caches, counts (`merge`), grouping (`computeIfAbsent`), LRU thumbnail cache (`LinkedHashMap`), time-series and rate tables (`TreeMap.floorEntry`).

TL;DR

Unique keys → values. `HashMap` default, `LinkedHashMap` for predictable order or LRU, `TreeMap` for sorted keys and ranges. `merge` , `computeIfAbsent` , `getOrDefault` instead of null-check boilerplate.

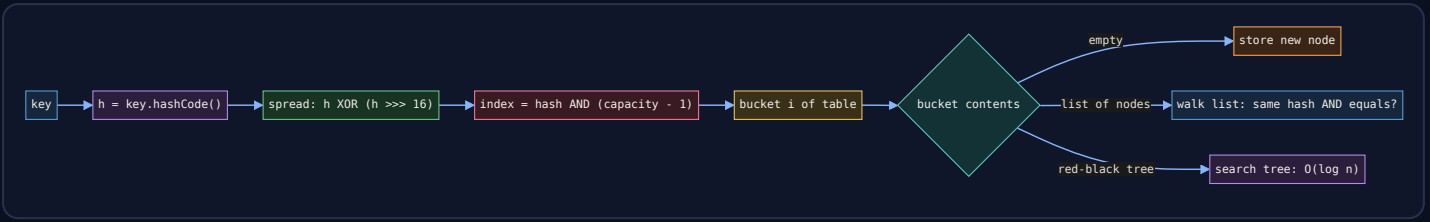
10.5 `HashMap` internals

What it is

`HashMap` is a **hash table**: an array of **buckets** (*bins*); a key's **hash code** (`int` from `hashCode()`) decides its bucket, so lookups go straight there instead of searching. Keys landing in the same bucket are kept together and distinguished by `equals` .

Average **O(1)** `get` / `put` makes `HashMap` / `HashSet` the most-used collections and an interview favourite; the internals explain real bugs (slow maps from bad `hashCode` , lost keys from mutation, odd iteration order, wasted resizes).

How it works (JDK 8 and later)



1. **Hash.** `put(key, value)` calls `key.hashCode()` ; a `null` key gets hash 0 (bucket 0).
2. **Spread.** `HashMap.hash()` XORs the high 16 bits into the low 16: `h ^ (h >>> 16)` . Only low bits pick the bucket, so without this, hashes differing only in high bits would all collide.
3. **Index.** Table length `n` is always a **power of two** (default **16**), so `hash & (n − 1)` equals `hash mod n` in one cheap AND.
4. **Store.** A bucket holds a chain of `Node{hash, key, value, next}` . Empty → the node goes in. Otherwise walk the chain: a node matches when **hashes are equal and** (`==` or `equals`) — its value is replaced and `put` returns the old one; no match → append at the tail.
5. **Load factor and resize.** The **load factor** (default **0.75**) is the fullness that triggers growth; **threshold** = capacity × load factor (16 × 0.75 = **12**). When `size` exceeds it, the table **doubles** and nodes are redistributed. Because capacity is a power of two, each node stays at `i` or moves to `i + oldCapacity` (bucket 5 of 16 → 5 or 21), decided by one hash bit (`hash & oldCapacity`) — a cheap, order-preserving split.
6. **Treeification (Java 8).** If adding to a bucket that already holds **8** nodes (`TREEIFY_THRESHOLD`), the chain becomes a **red-black tree** of `TreeNode` s — worst case O(n) → **O(log n)**. But if the table has fewer than **64** buckets (`MIN_TREEIFY_CAPACITY`) it **resizes instead**. A tree bucket split down to **≤ 6** nodes (`UNTREEIFY_THRESHOLD`) reverts to a list. With a decent hash and 0.75 load, a bucket of 8 is

astronomically unlikely (the JDK source estimates about 6 in 100 million) — trees defend against bad or malicious hash codes.

7. **Inside a tree bucket**, nodes order by hash, then by `compareTo` **if keys are** `Comparable` **and of the same class**; otherwise lookups may search both subtrees (the slow case below).
8. **Get** repeats steps 1–3 and searches one bucket.

Constants from `java/util/HashMap.java` (JDK 26):

Constant	Value	Meaning
`DEFAULT_INITIAL_CAPACITY`	16	buckets allocated on first `put` (table created lazily)
`DEFAULT_LOAD_FACTOR`	0.75	resize when size > capacity × 0.75
`TREEIFY_THRESHOLD`	8	chain length that triggers list → tree
`UNTREEIFY_THRESHOLD`	6	tree → list when a resize split leaves this few
`MIN_TREEIFY_CAPACITY`	64	below this table size, resize instead of treeifying
`MAXIMUM_CAPACITY`	2^30	table never grows beyond this

Why 0.75? Time/space trade-off: lower = fewer collisions but more empty buckets; higher = tighter packing but longer chains. 0.75 keeps chains very short while wasting at most about a quarter of the table.

Watching it happen: a teaching-size hash map

`TinyHashMap` implements steps 1–5 in ~60 lines (no trees, no null keys), starting with 4 buckets to show two resizes.

```
// code/ch10/maps/TinyHashMap.java (key parts)
static int spread(Object key) { int h = key.hashCode(); return h ^ (h >>> 16); }

void put(K key, V value) {
    int h = spread(key), i = h & (table.length - 1);          // bucket index
    for (Node<K, V> n = table[i]; n != null; n = n.next())
        if (n.hash() == h && n.key().equals(key)) { table[i] = replace(table[i], key, value); return; }
    table[i] = new Node<>(h, key, value, table[i]);           // new node in bucket i
    if (++size > table.length * loadFactor) resize();         // threshold = capacity * 0.75
}

private void resize() {                                       // double and re-bucket every node
    Node<K, V>[] old = table;
    table = (Node<K, V>[]) new Node[old.length * 2];
    for (Node<K, V> head : old)
        for (Node<K, V> n = head; n != null; n = n.next()) {
            int i = n.hash() & (table.length - 1);            // stays at i or moves to i + oldCapacity
            table[i] = new Node<>(n.hash(), n.key(), n.value(), table[i]);
        }
}
```

Output:

```
put apple   hashCode=   93029210 spread=   93030097 index(n=4)=1
put banana  hashCode=-1396355227 spread=-1396317280 index(n=4)=0
put cherry  hashCode=-1361513063 spread=-1361552575 index(n=4)=1
put date    hashCode=    3076014 spread=    3075968 index(n=4)=0
  resize: size=4 exceeded threshold 3 -> capacity 4 -> 8
put elder   hashCode=   96592394 spread=   96593867 index(n=8)=3
put fig     hashCode=     101380 spread=     101381 index(n=8)=5
put grape   hashCode=   98615627 spread=   98616491 index(n=8)=3
  resize: size=7 exceeded threshold 6 -> capacity 8 -> 16
final table (capacity 16):
  bucket  0: date -> banana -> null
  bucket  1: cherry -> apple -> null
  bucket  5: fig -> null
  bucket 11: elder -> grape -> null
get(fig)=99 get(kiwi)=null size=7
```

With 4 buckets `apple` and `cherry` collide in bucket 1; after the 4th key, 4 > 4 × 0.75 = 3, so the table doubles. At 16 buckets, `elder` and `grape` still share bucket 11 (spread hashes agree in the low 4 bits). `put("fig", 99)` replaced a value, so size stays 7. (`String.hashCode` is the documented formula `s[0]·31^(n−1) + … + s[n−1]`, identical on every JVM.)

What bad hash codes cost (measured)

`HashCollisionDemo` puts 20,000 keys whose `hashCode()` is the constant 42 (one bucket) and times 60,000 lookups. **Timings are machine-dependent** (Apple M4 Pro, JDK 26); compare ratios.

```java
// code/ch10/maps/HashCollisionDemo.java (key parts)
record CmpKey(int id) implements Comparable<CmpKey> {          // constant hash, but Comparable
    @Override public int hashCode() { return 42; }
    @Override public int compareTo(CmpKey o) { return Integer.compare(id, o.id); }
}
record PlainKey(int id) { @Override public int hashCode() { return 42; } }  // constant hash, not Comparable
record GoodKey(int id) {}                                       // record's generated hashCode
```

Output:

```
good hashCode (GoodKey)            n=20,000  3n lookups:      0.7 ms
constant hash, Comparable (tree)   n=20,000  3n lookups:      6.5 ms
constant hash, not Comparable      n=20,000  3n lookups:   1648.2 ms
```

- **Good hash:** spread across buckets → O(1).
- **Constant hash + `Comparable`:** one treeified bucket ordered by `compareTo` → O(log n); ~10× slower, still fast.
- **Constant hash, not `Comparable`:** the tree can't order equal-hash keys → lookups near O(n), **thousands of times slower**. Hash-flooding denial-of-service attacks (many colliding keys sent to grind a server down) exploit this; it's why Java 8 added treeification, and why `String` keys (`Comparable`) are well protected.

Sizing a map correctly

`new HashMap<>(100)` is an initial *capacity*, rounded to 128; threshold 96, so the 97th `put` still resizes. To size for an expected number of **entries**, use the Java 19 factories:

```java
Map<String, User> users = HashMap.newHashMap(100);       // Java 19: capacity for 100 mappings without resizing
Set<String> seen        = HashSet.newHashSet(100);       // also LinkedHashMap.newLinkedHashMap,
LinkedHashSet.newLinkedHashSet
```

Thread safety

`HashMap` is **not** thread-safe: concurrent `put`s can lose entries or corrupt a bucket (in Java 7 and earlier a concurrent resize could even create a chain cycle so `get` spun forever). Use `ConcurrentHashMap` (Chapter 14); `Collections.synchronizedMap` works but locks the whole map per call.

Trade-offs

	HashMap	TreeMap
Needs	good `hashCode` + consistent `equals`	`Comparable` keys or a `Comparator`
Average `get` / `put`	O(1)	O(log n)
Worst case	O(log n) per bucket if keys `Comparable`, else O(n)	O(log n) guaranteed
Order	none	sorted
Memory	table + a node per entry	a tree node per entry (more pointers)

Common mistakes

- `equals` without `hashCode` (or different fields) → equal keys in different buckets.
- Constant or low-variety `hashCode` (e.g. a boolean field) → every lookup scans.
- Relying on iteration order — it follows buckets and can change after resizes or across JDKs; use `LinkedHashMap` / `TreeMap`.

In the interview

30-second answer: "HashMap is an array of buckets. It takes the key's hashCode, spreads high bits into low bits, and ANDs with capacity minus one to pick a bucket. Collisions chain in a list; since Java 8 a bucket past 8 entries becomes a red-black tree if the table has at least 64 buckets, so the worst case is O(log n), not O(n). When size passes capacity × 0.75 the table doubles and each entry moves to index i or i + oldCapacity. Lookups compare hash, then equals — so equals and hashCode must agree and keys shouldn't be mutable."

- *Why a power-of-two capacity?* `hash & (n−1)` replaces modulo, and resizing splits each bucket into two predictable positions.
- *Null key?* Yes, one, hash 0; `Hashtable` and `ConcurrentHashMap` reject nulls.
- *Same hashCode, not equal?* Same bucket; `equals` tells them apart. Correct, slower.

TL;DR

hashCode → spread → `& (n−1)` → bucket; match by hash then `equals`. Load 0.75 → double; buckets over 8 become trees (table ≥ 64). Good, stable hash codes = O(1). Pre-size with `HashMap.newHashMap(n)`. Not thread-safe.

10.6 Queues, deques and priority queues

What it is

- A **queue** holds elements awaiting processing, normally **FIFO**.
- A **deque** ("deck", double-ended queue) adds/removes at both ends — a FIFO queue *or* a **LIFO stack**.
- A **priority queue** always hands out the *smallest* element next (natural order or comparator).

Two method families

Operation	Throws exception	Returns special value
Insert at tail	`add(e)`	`offer(e)` → `false` if full (bounded queues)
Remove head	`remove()`	`poll()` → `null` if empty
Examine head	`element()`	`peek()` → `null` if empty

Prefer the special-value versions for ordinary flow. `Deque` adds `First` / `Last` variants (`offerFirst`, `pollLast`, `peekFirst` …) and stack methods `push` (= `addFirst`), `pop` (= `removeFirst`), `peek`.

How the implementations work

- **`ArrayDeque`** — a resizable **circular array** with wrapping `head` / `tail` indexes: amortised **O(1)** at both ends, no node allocations. Rejects `null` (it's how `poll` signals "empty"). Per its Javadoc, likely faster than `Stack` as a stack and `LinkedList` as a queue — **use it for both**.
- **`PriorityQueue`** — a **binary min-heap** in an array: a complete binary tree (levels full except the last, filled left to right) where each parent ≤ its children; element *i*'s children are at *2i+1* and *2i+2*. `offer` appends and sifts up; `poll` moves the last element to the root and sifts down — both **O(log n)**; `peek` O(1); `contains` / `remove(Object)` O(n); building from a collection (heapify) O(n). Only the root is guaranteed smallest, so `toString()` and iteration show heap order, not sorted order. ("Heap" here is unrelated to the JVM memory heap.)

Code

```
// code/ch10/queues/QueueDemo.java (key parts)
Queue<String> printer = new ArrayDeque<>();
printer.offer("doc1"); printer.offer("doc2"); printer.offer("doc3");

Deque<Character> stack = new ArrayDeque<>();                // balanced-brackets check
for (char c : "{[()]}(".toCharArray()) {
    if ("([{".indexOf(c) >= 0) stack.push(c);
    else if (!stack.isEmpty() && "([{".indexOf(stack.peek()) == ")]}".indexOf(c)) stack.pop();
    else { stack.push('!'); break; }
}

PriorityQueue<Integer> heap = new PriorityQueue<>(List.of(7, 3, 9, 1, 5));
while (!heap.isEmpty()) out.append(heap.poll()).append(' ');

PriorityQueue<Task> jobs = new PriorityQueue<>(
        Comparator.comparingInt(Task::priority).reversed().thenComparing(Task::name));

int k = 3; PriorityQueue<Integer> topK = new PriorityQueue<>();  // top-k largest with a min-heap
for (int x : new int[]{15, 2, 99, 42, 8, 77, 23}) { topK.offer(x); if (topK.size() > k) topK.poll(); }
```

Output:

```
peek=doc1 poll=doc1 left=[doc2, doc3]
empty.poll()=null empty.peek()=null
empty.remove() -> NoSuchElementException
{[()]}( balanced? false (left on stack: [(])
ArrayDeque.push(null) -> NPE
heap toString (NOT sorted): [1, 3, 9, 7, 5]
poll order: 1 3 5 7 9
backup deploy lint email
top-3 = [99, 77, 42]
```

The top-k trick keeps only *k* elements in a min-heap: O(log k) per element, evicting the smallest of the current top k — O(n log k) overall vs O(n log n) sorting; an interview staple (Chapter 19).

When not to use them

- Producer–consumer threads (some add work, others take it) → a `BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`) that waits instead of returning `null` (Chapter 14).
- Sorted output once → just sort a list; heaps shine when elements keep arriving.
- Changing a queued element's priority isn't supported — remove and re-add (O(n)), or insert anew and skip stale entries when polled.

Common mistakes

- `Stack` / `LinkedList` where `ArrayDeque` fits.
- Printing a `PriorityQueue` expecting sorted output — drain with `poll()`.

- Mixing `add` / `remove` (throwing) with `offer` / `poll` expectations.
- `push` then `pollLast` — `push` adds at the **front**.

TL;DR

`ArrayDeque` for FIFO queues and LIFO stacks; `PriorityQueue` (binary heap) when the next item is "the smallest": O(log n) add/poll, O(1) peek, unsorted iteration.

10.7 Iterators and fail-fast behaviour

What it is

An `Iterator<E>` is a cursor with `hasNext()`, `next()` and optional `remove()`; for-each is shorthand for one. `ListIterator` adds `hasPrevious` / `previous`, index queries, `set` and `add`.

How fail-fast works

Most `java.util` collections keep a **modification counter** (`modCount`) bumped on every structural change. An iterator records it at creation and re-checks on each `next()`; if anything *other than the iterator itself* changed the collection, `next()` throws `ConcurrentModificationException` (CME) — in single-threaded code too. It's best-effort bug detection, not a thread-safety guarantee.

Fail-safe iterators (`java.util.concurrent`) walk a snapshot or a weakly consistent view (may or may not show concurrent changes, never fails) and never throw CME. `CopyOnWriteArrayList` copies its array on every write — suited to read-mostly lists like event listeners.

Code

```
// code/ch10/iteration/IteratorDemo.java (key parts)
for (Integer n : nums) if (n % 2 == 0) nums.remove(n);          // WRONG: CME

for (Iterator<Integer> it = nums.iterator(); it.hasNext(); )    // RIGHT: iterator's own remove
    if (it.next() % 2 == 0) it.remove();

nums.removeIf(n -> n % 2 == 0);                                 // SIMPLEST (Java 8)

for (ListIterator<String> li = words.listIterator(); li.hasNext(); ) {
    String w = li.next();
    li.set(w.toUpperCase());
    if (w.equals("b")) li.add("b2");                            // inserted after the cursor
}
scores.entrySet().removeIf(e -> e.getValue() < 50);             // remove map entries by condition

List<String> listeners = new CopyOnWriteArrayList<>(List.of("L1", "L2"));
for (String l : listeners) listeners.add(l + "'");              // iterates the old snapshot
```

Output:

```
for-each + list.remove -> ConcurrentModificationException, list now [1, 3, 4, 5, 6]
iterator.remove -> [1, 3, 5]
removeIf        -> [1, 3, 5]
listIterator set/add -> [A, B, b2, C]
backwards -> Cb2BA
passed -> {bo=90, cy=55}
CopyOnWriteArrayList -> [L1, L2, L1', L2']
```

The first line shows `2` was already removed before the exception — fail-fast stops you *after* the first bad change, so never rely on catching CME.

Common mistakes

- Removing inside for-each; calling `it.remove()` twice per `next()` or before the first `next()` → `IllegalStateException`.

- Thinking CME means a threading problem — usually a loop modifying its own collection.
- Removing by index in a forward `for (int i…)` loop skips the element that shifted into `i`; loop backwards or `removeIf`.

TL;DR

Don't structurally modify a collection during for-each. Use `removeIf`, `Iterator.remove`, or apply changes afterwards. Fail-fast = bug detector, not a lock.

10.8 Ordering: `Comparable` and `Comparator`

What it is

- **`Comparable<T>`** — the class defines its **natural order** via `int compareTo(T other)` (`String`, `Integer`, `LocalDate`, enums in declaration order).
- **`Comparator<T>`** — a separate object, `int compare(T a, T b)`, defining *any* order without changing the class (an external strategy).

Both return **negative** if the first comes first, **zero** for a tie, **positive** otherwise; only the sign matters.

The contract

Must be **antisymmetric** (`a.compareTo(b)` and `b.compareTo(a)` have opposite signs), **transitive**, and ideally **consistent with equals** (`compareTo == 0` exactly when `equals`). `TreeSet` / `TreeMap` use comparison *instead of* `equals`, so inconsistency drops or merges elements. (`BigDecimal` is the famous JDK exception: `2.0` and `2.00` compare equal but aren't `equals` .)

Building comparators (Java 8+)

Factory / method	Meaning
`Comparator.comparing(Employee::name)`	by a `Comparable` key
`comparingInt/Long/Double(keyFn)`	by a primitive key (no boxing)
`comparing(keyFn, keyComparator)`	by a key using a given comparator
`.thenComparing(...)`	tie-breaker
`.reversed()`	flip the whole comparator so far
`Comparator.naturalOrder()` / `reverseOrder()`	natural / reverse natural
`nullsFirst(c)` / `nullsLast(c)`	tolerate nulls
`String.CASE_INSENSITIVE_ORDER`	ready-made

Code

```
// code/ch10/sorting/SortingDemo.java (key parts)
record Employee(int id, String name, String dept, double salary) implements Comparable<Employee> {
    @Override public int compareTo(Employee o) { return Integer.compare(id, o.id); }   // natural order: id
}
Collections.sort(staff);                                          // uses compareTo
staff.sort(Comparator.comparing(Employee::name).reversed());
staff.sort(Comparator.comparing(Employee::dept)
        .thenComparing(Employee::salary, Comparator.reverseOrder())   // reverse just this key
        .thenComparing(Employee::name));
staff.sort(Comparator.comparingDouble(Employee::salary));
cities.sort(Comparator.nullsLast(String.CASE_INSENSITIVE_ORDER));

Comparator<Integer> bad = (a, b) -> a - b;                        // overflow bug
Set<String> byLength = new TreeSet<>(Comparator.comparingInt(String::length));
byLength.addAll(List.of("kiwi", "pear", "fig", "plum", "banana"));
```

Output:

```
by id:      [Ana, Bo, Cy, Dee]
name desc:  [Dee, Cy, Bo, Ana]
dept,salary desc,name: [Bo, Cy, Dee, Ana]
salary asc: [Ana, Cy, Dee, Bo]
nullsLast, case-insensitive: [Accra, lima, Oslo, null]
bad.compare(MIN_VALUE, 1) = 2147483647 (positive, so MIN_VALUE looks BIGGER), Integer.compare = -1
TreeSet by length keeps only [fig, kiwi, banana]  (pear and plum were dropped!)
stable (dept, then previous name order): [Bo, Cy, Dee, Ana]
```

- `reversed()` flips **everything chained before it**; to reverse one key, pass `Comparator.reverseOrder()` as that key's comparator (as for salary).
- `a - b` overflows (`MIN_VALUE − 1` wraps to `MAX_VALUE`); use `Integer.compare` , `Long.compare` , `Double.compare` (also handles `NaN`) or `comparingInt` .
- The length-ordered `TreeSet` treats `"kiwi"` , `"pear"` , `"plum"` (all length 4) as duplicates. Add a tie-breaker: `comparingInt(String::length).thenComparing(Comparator.naturalOrder())` .
- **Stability**: `List.sort` / `Collections.sort` (TimSort, merge-sort-based, O(n log n)) keep equal elements' relative order — so sorting by name, then dept, leaves each dept sorted by name.

Common mistakes

- Subtraction comparators; `<` / `>` on doubles ignoring `NaN` .
- `Comparable` inconsistent with `equals` used as `TreeMap` keys.
- Non- `Comparable` objects in a `TreeSet` without a comparator → `ClassCastException` on the first add.

In the interview

> **30-second answer:** "Comparable is the class's single natural order via compareTo; Comparator is an external strategy for any number of orders. I build comparators with comparing, thenComparing and reversed, use primitive variants to avoid boxing, and never subtract. Sorted collections treat compare == 0 as equality, so the order should be consistent with equals."

- *Is `Collections.sort` stable?* Yes (TimSort for objects).
- *Descending by one key, ascending by another?* `comparing(a).thenComparing(b, reverseOrder())` .

TL;DR

`Comparable` = one built-in order; `Comparator` = any order, composed with `comparing` / `thenComparing` / `reversed` / `nullsLast` . Consistent with `equals` for sorted collections. Object sorts are stable.

10.9 The `Collections` utility class

`java.util.Collections` (plural) holds static algorithms and wrappers — the collection counterpart of `Arrays` (§4.8).

```
// code/ch10/utilities/CollectionsUtilDemo.java (key parts)
Collections.max(xs); Collections.min(xs); Collections.frequency(xs, 3);
Collections.sort(xs); Collections.binarySearch(xs, 8);    // binarySearch needs a sorted list
Collections.reverse(xs); Collections.swap(xs, 0, xs.size() - 1);
Collections.rotate(xs, 2); Collections.shuffle(xs, new Random(42));   // seeded -> repeatable
Collections.nCopies(3, "ab"); Collections.disjoint(List.of(1, 2), List.of(3));
Collections.addAll(letters, "d", "b");

List<String> view = Collections.unmodifiableList(source);  // read-only VIEW
List<String> copy = List.copyOf(source);                   // immutable COPY
source.add("y");

List<Integer> sync = Collections.synchronizedList(new ArrayList<>(List.of(1, 2)));
synchronized (sync) { for (int v : sync) System.out.print("sync " + v + " "); }   // lock while iterating
```

Output:

```
max=9 min=1 frequency(3)=2
sorted=[1, 3, 3, 5, 8, 9] binarySearch(8)=4 binarySearch(4)=-4 (negative: -(insertionPoint)-1)
reverse=[9, 8, 5, 3, 3, 1]
swap ends=[1, 8, 5, 3, 3, 9]
rotate 2=[3, 9, 1, 8, 5, 3]
shuffle(seed 42)=[9, 3, 5, 3, 8, 1]
nCopies=[ab, ab, ab] disjoint=true
addAll varargs=[c, a, d, b]
view sees change: [x, y]  copy does not: [x]
view.add -> UnsupportedOperationException
emptyList=[] singleton=[only]
sync 1 sync 2
```

Group	Methods
Algorithms	`sort`, `binarySearch`, `reverse`, `shuffle`, `swap`, `rotate`, `fill`, `copy`, `min`, `max`, `frequency`, `disjoint`, `addAll`, `nCopies`
Read-only wrappers (views)	`unmodifiableList/Set/Map/Collection`, `unmodifiableSequencedCollection/Set/Map` (21), sorted/navigable variants
Synchronized wrappers	`synchronizedList/Set/Map/...` — each call locks; lock manually around iteration
Constants and singletons	`emptyList/Set/Map`, `singletonList/singleton/singletonMap` (immutable)
Type-checked wrappers	`checkedList(list, String.class)` — catches heap pollution from raw types at insertion (Chapter 9)
Other	`reverseOrder()`, `newSetFromMap(map)`, `list(enumeration)`, `enumeration(coll)`

`binarySearch` (sorted lists only — unsorted gives undefined results) returns the index, or `-(insertionPoint) - 1` — always negative, so "not found" can't be index 0. Here 4 would go at index 3 → −4. `unmodifiableList(l)` is a view reflecting later changes; `List.copyOf(l)` an independent snapshot.

10.10 Unmodifiable and immutable collections

What it is

- **Unmodifiable** — the object rejects changes (`add`, `set`, `remove` throw `UnsupportedOperationException`).
- **Immutable** — unmodifiable *and* nothing else can change it (no backing collection).

- A **view** wraps another collection; it can be unmodifiable yet still change when the underlying one does.

Returning immutable collections (or storing them in records) stops callers corrupting your state, makes sharing across threads safe, and states intent — *Effective Java* Item 17 ("minimise mutability") and Item 50 ("make defensive copies when needed").

The factory methods

Factory	Since	Nulls	Duplicates	Order
`List.of(...)`	9	NPE	allowed	as given
`Set.of(...)`	9	NPE	**IllegalArgumentException**	unspecified (randomised per JVM run)
`Map.of(k1, v1, … up to 10 pairs)` / `Map.ofEntries(Map.entry(k, v), …)`	9	NPE	duplicate key → IAE	unspecified
`List.copyOf` / `Set.copyOf` / `Map.copyOf`	10	NPE	`Set.copyOf` drops duplicates	—
`Stream.toList()`	16	allowed	allowed	encounter order
`Collectors.toUnmodifiableList/Set/Map`	10	NPE	—	—
`Arrays.asList(...)`	1.2	allowed	allowed	fixed-size, **`set` works**, write-through to the array
`Collections.unmodifiableX(c)`	1.2	as `c`	as `c`	read-only **view** of `c`

Code

```
// code/ch10/immutable/ImmutableDemo.java (key parts)
List<String> days = List.of("Mon", "Tue", "Wed");
attempt("List.of(...).add", () -> days.add("Thu"));
attempt("List.of with null", () -> List.of("a", null));
attempt("List.of(...).contains(null)", () -> days.contains(null));
attempt("Set.of duplicate", () -> Set.of("a", "a"));
attempt("Map.of duplicate key", () -> Map.of("k", 1, "k", 2));
List<String> snap = List.copyOf(src); src.add("b");
List<Integer> t = Stream.of(3, 1, 2).toList();
List<StringBuilder> sbs = List.of(new StringBuilder("hi")); sbs.get(0).append("!");
```

Output:

```
List.of(...).add -> UnsupportedOperationException
List.of(...).set -> UnsupportedOperationException
List.of with null -> NullPointerException
List.of(...).contains(null) -> NullPointerException
Set.of duplicate -> IllegalArgumentException
Map.of duplicate key -> IllegalArgumentException
Map.ofEntries get = 404
Arrays.asList set -> ok
Arrays.asList add -> UnsupportedOperationException
copyOf snapshot=[a] src=[a, b] copyOf(List.of) same object? true
Stream.toList().add -> UnsupportedOperationException
element mutated inside an 'immutable' list: [hi!]
Set.of size=3 (print order may differ between runs)
```

- **Shallowly** immutable: the list can't change, but mutable elements (a `StringBuilder`) can. Store immutable elements for true immutability.
- `List.of(...).contains(null)` throws NPE — surprising with nullable arguments.

- `copyOf` returns the same object when given an unmodifiable `List.of`-style list.
- `Collectors.toList()` promises nothing about mutability (currently an `ArrayList`); ask for unmodifiable with `Stream.toList()` (Chapter 12).
- `List.of` is safe to read from many threads (immutable), though its elements may not be.

Defensive copy in a record (records are shallowly immutable, §6.8):

```
record Team(String name, List<String> members) {
    Team { members = List.copyOf(members); }   // callers can't change our list later, and we reject nulls
}
```

TL;DR

`List.of` / `Set.of` / `Map.of` for constants, `List.copyOf` for defensive copies — null-rejecting and shallowly immutable. `unmodifiableList` is only a view. `Arrays.asList` is fixed-size but writable.

10.11 Sequenced collections (Java 21)

The problem it solves

Before Java 21 "first/last element" and "reverse iteration" were spelled differently per type — `list.get(0)` / `list.get(list.size() - 1)`, `deque.getFirst()`, `sortedSet.first()` / `last()`, and no direct way to get a `LinkedHashSet`'s last element — and no common type meant "ordered collection".

What it is

JEP 431 (final in Java 21) added three interfaces for "well-defined encounter order":

Interface	New methods	Retrofitted onto
`SequencedCollection<E>`	`addFirst`, `addLast`, `getFirst`, `getLast`, `removeFirst`, `removeLast`, `reversed()`	`List`, `Deque`
`SequencedSet<E>`	`reversed()` returns a `SequencedSet`	`LinkedHashSet`, `SortedSet` (so `TreeSet`)
`SequencedMap<K,V>`	`firstEntry`, `lastEntry`, `pollFirstEntry`, `pollLastEntry`, `putFirst`, `putLast`, `reversed()`, `sequencedKeySet()`, `sequencedValues()`, `sequencedEntrySet()`	`LinkedHashMap`, `SortedMap` (so `TreeMap`)

Plus `Collections.unmodifiableSequencedCollection/Set/Map`. `HashSet` / `HashMap` are **not** sequenced.

How it behaves

1. `reversed()` is a **live reverse view**, O(1) to create — later changes show through.
2. `getFirst` / `removeFirst` etc. on an empty collection throw `NoSuchElementException`; the `SequencedMap` entry methods (`firstEntry`…) return `null` on an empty map.
3. Sorted collections (`TreeSet`, `TreeMap`) throw `UnsupportedOperationException` from `addFirst` / `addLast` / `putFirst` / `putLast` — the comparator decides position.
4. In `LinkedHashSet` / `LinkedHashMap`, `addFirst` / `putFirst` on an existing element **moves** it to the front.
5. Unmodifiable collections reject the mutators as usual.

Code

```
// code/ch10/sequenced/SequencedDemo.java (key parts)
List<String> list = new ArrayList<>(List.of("b", "c"));
list.addFirst("a"); list.addLast("d");
list.getFirst(); list.getLast(); list.reversed();

LinkedHashSet<String> tags = new LinkedHashSet<>(List.of("java", "sql", "git"));
tags.addFirst("git");                 // already present -> moved to the front

TreeSet<Integer> sorted = new TreeSet<>(List.of(30, 10, 20));
sorted.addFirst(5);                   // UnsupportedOperationException

List<String> rev = list.reversed(); list.add("e");   // the view sees the change

LinkedHashMap<String, Integer> steps = new LinkedHashMap<>();
steps.put("build", 1); steps.put("test", 2); steps.put("deploy", 3);
steps.putFirst("checkout", 0);
steps.firstEntry(); steps.lastEntry(); steps.reversed().keySet(); steps.pollFirstEntry();

static <E> E lastOf(SequencedCollection<E> c) { return c.getLast(); }   // one method for any ordered collection
```

Output:

```
List        first=a last=d reversed=[d, c, b, a]
LinkedHashSet [git, java, sql] removeLast=sql -> [git, java]
Deque       reversed=[3, 2, 1] getLast=3
TreeSet     first=10 last=30 reversed=[30, 20, 10]
TreeSet.addFirst -> UnsupportedOperationException (order comes from the comparator)
view after list.add(e): [e, d, c, b, a]
SequencedMap first=checkout=0 last=deploy=3 reversed keys=[deploy, test, build, checkout]
pollFirstEntry=checkout=0 -> [build, test, deploy]
lastOf(list)=e lastOf(dq)=3 lastOf(tags)=java
empty getFirst -> NoSuchElementException
HashSet is sequenced? false
```

Common mistakes

- Assuming `reversed()` copies.
- `addFirst` on a `TreeSet` or `List.of(...)`.
- Source compatibility: a custom `List` / `Deque` implementation already declaring `reversed()` or `getFirst()` with a different return type stops compiling on Java 21 (a known JEP 431 risk).

TL;DR

Java 21 gives ordered collections one API — `getFirst/getLast/addFirst/addLast/removeFirst/removeLast/reversed` (maps: `firstEntry/lastEntry/pollFirstEntry/putFirst/...`). `reversed()` is a view; sorted collections refuse positional adds.

10.12 Big-O cheat sheet and a measured demo

Big-O describes how cost grows with *n*: O(1) constant, O(log n) very slow growth (tree height), O(n) proportional (scan). "Amortised" = averaged over many calls; "average" for hash structures assumes a reasonable `hashCode`.

Collection	add (end)	add/remove front	get(i)	contains / get(key)	remove(element/key)	Iteration order	Nulls
`ArrayList`	O(1)*	O(n)	O(1)	O(n)	O(n)	index	yes
`LinkedList`	O(1)	O(1)	O(n)	O(n)	O(n)	index	yes
`ArrayDeque`	O(1)*	O(1)*	—	O(n)	O(n)	head→tail	**no**
`PriorityQueue`	O(log n)	poll O(log n)	—	O(n)	O(n)	heap (unsorted)	no
`HashSet` / `HashMap`	O(1)†	—	—	O(1)†	O(1)†	none	one null key
`LinkedHashSet` / `LinkedHashMap`	O(1)†	—	—	O(1)†	O(1)†	insertion/access	one null key
`TreeSet` / `TreeMap`	O(log n)	—	—	O(log n)	O(log n)	sorted	no (natural order)
`EnumSet` / `EnumMap`	O(1)	—	—	O(1)	O(1)	enum order	no
`CopyOnWriteArrayList`	O(n)	O(n)	O(1)	O(n)	O(n)	index	yes

* amortised. † average; worst case O(log n) for a treeified bucket (O(n) for non-`Comparable` colliding keys).

Measured demo

`CollectionBenchmark` times operations once after a warm-up run (so the JIT has optimised). A *demonstration*, not a rigorous benchmark — use JMH for real measurements (Chapter 16). **Timings are machine-dependent** (Apple M4 Pro, JDK 26); compare ratios within a row.

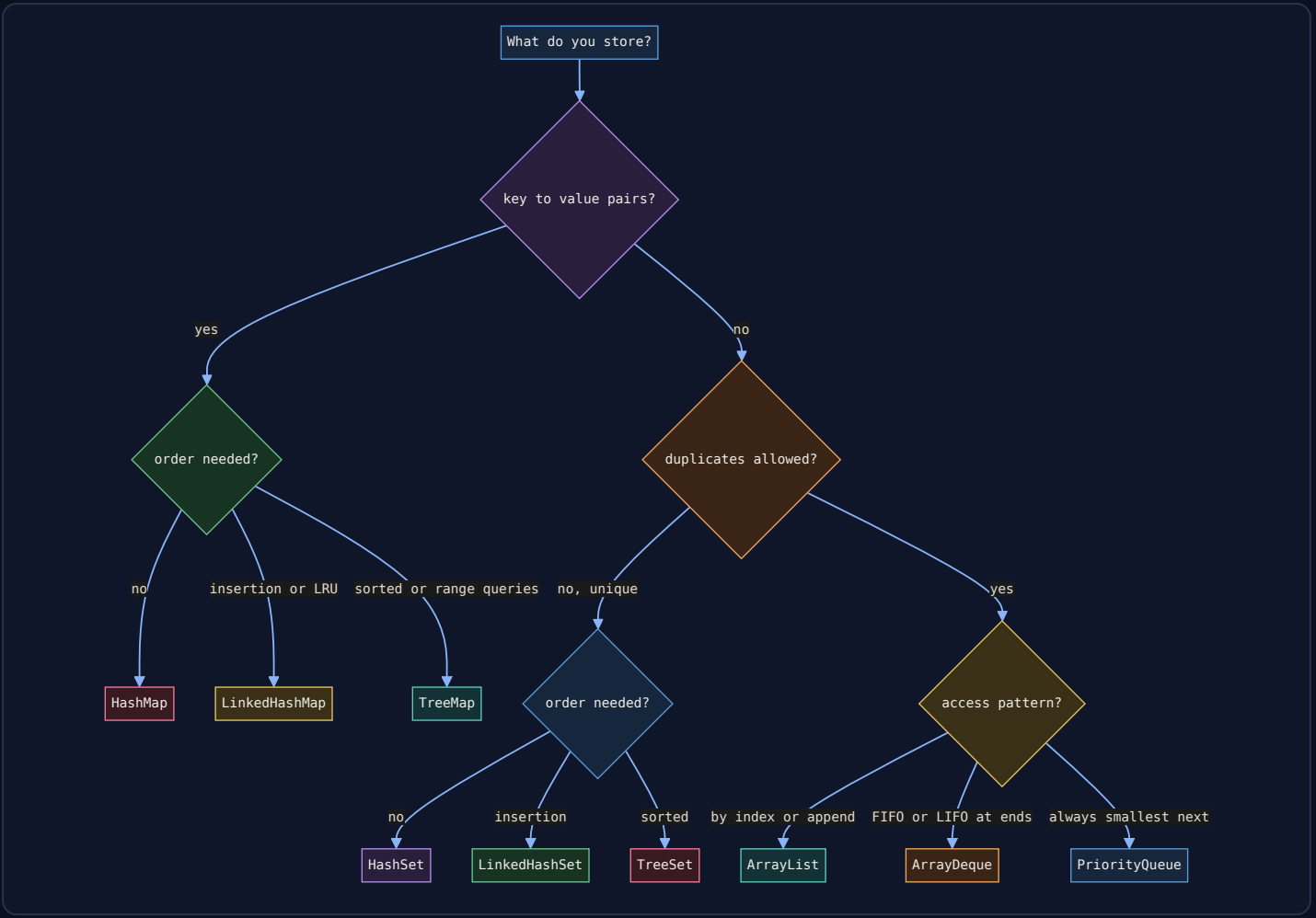
```
// code/ch10/perf/CollectionBenchmark.java (key parts)
Runnable getA = () -> { for (int i = 0; i < 20_000; i++) sink += array.get((i * 7919) % n); };
Runnable getL = () -> { for (int i = 0; i < 20_000; i++) sink += linked.get((i * 7919) % n); };
Runnable frontA = () -> { List<Integer> l = new ArrayList<>(); for (int i = 0; i < 50_000; i++) l.add(0, i); };
Runnable frontD = () -> { Deque<Integer> d = new ArrayDeque<>(); for (int i = 0; i < 50_000; i++) d.addFirst(i);
};
Runnable cL = () -> { for (int i = 0; i < 2_000; i++) sink += array.contains(n - 1 - i) ? 1 : 0; };
Runnable cH = () -> { for (int i = 0; i < 2_000; i++) sink += hash.contains(n - 1 - i) ? 1 : 0; };
// ... TreeSet contains, full iteration, presized vs default ArrayList
```

Output:

```
20k get(i) n=100k        ArrayList     0.6 ms | LinkedList   870.1 ms
50k add-at-front          ArrayList    83.6 ms | ArrayDeque     1.0 ms
2k contains n=100k         ArrayList    65.7 ms | HashSet 0.1 ms | TreeSet 0.3 ms
20 full iterations        ArrayList     4.2 ms | LinkedList     4.1 ms
2M adds                    default    42.7 ms | presized     7.7 ms
```

1. **Indexing** — `LinkedList.get(i)` walks up to 50,000 nodes per call: >1000× slower.
2. **Front inserts** — `ArrayList.add(0, x)` shifts everything (O(n²) total); `ArrayDeque.addFirst` is O(1).
3. **Membership** — list scan vs hashing vs a ~17-level tree walk.
4. **Iteration** — both O(n); freshly allocated nodes sit close together here, so the gap is small. In long-running apps with fragmented heaps, linked structures usually iterate noticeably slower.
5. **Pre-sizing** — skipping ~30 grow-and-copy steps (and their garbage) is several times faster.

10.13 Choosing a collection



- Enum keys/elements → `EnumMap` / `EnumSet`.
- Fixed compile-time data → `List.of` / `Set.of` / `Map.of`.
- Shared between threads → `ConcurrentHashMap`, `CopyOnWriteArrayList` (read-mostly), `BlockingQueue` (Chapter 14).
- Millions of primitives in a hot path → primitive arrays or a primitive-collections library (Eclipse Collections, fastutil).
- In doubt: `ArrayList` and `HashMap`, then measure.

Python readers: same ideas as `list`, `set`, `dict`, `deque`, `heapq` in Python Chapter 6.

In the interview — collections quick round

- *HashSet vs TreeSet?* O(1) unordered vs O(log n) sorted with range queries; `TreeSet` needs `Comparable` / `Comparator` and uses compare-to-zero for equality.
- *Why `ArrayDeque` over `Stack`?* `Stack` extends synchronized `Vector` (locking, exposed index methods); `ArrayDeque` is a lean circular array.
- *LRU cache?* `LinkedHashMap` with `accessOrder = true` and `removeEldestEntry` (or `HashMap` + hand-written doubly linked list — LLD case studies).
- *Is `List.of` thread-safe?* For reading, yes — it's immutable; its elements may not be.

Chapter TL;DR

Topic	Remember
Hierarchy	`Iterable` → `Collection` → `List` / `Set` / `Queue` / `Deque`; `Map` separate with `keySet` / `values` / `entrySet` views; program to interfaces
`ArrayList`	growable array (10, then ×1.5): O(1) get/append, O(n) middle edits; the default list
`LinkedList`	doubly linked nodes: O(1) ends, O(n) get; rarely the best choice
Sets	`HashSet` O(1), `LinkedHashSet` insertion order, `TreeSet` sorted O(log n) + `floor` / `ceiling`; never mutate elements
Map API	`getOrDefault`, `putIfAbsent`, `merge`, `compute*`; iterate `entrySet`; `TreeMap` for ranges; `LinkedHashMap` for LRU
`HashMap` internals	hash spread → `& (n−1)`; chain then tree (>8 in bucket, table ≥ 64); load factor 0.75 → double; equal hash then `equals`; `newHashMap(n)` (19); not thread-safe
Queues	`ArrayDeque` for queue and stack; `PriorityQueue` = binary heap (O(log n), unsorted iteration); `offer/poll/peek` return special values
Iteration	fail-fast CME; remove with `removeIf` or `Iterator.remove`
Ordering	`Comparable` natural vs `Comparator` composed; never subtract; consistent with equals for `TreeSet` / `TreeMap`; sorts are stable
Immutability	`List/Set/Map.of` (9), `copyOf` (10): no nulls, shallow; `Stream.toList` (16) unmodifiable but allows nulls; `unmodifiableX` = view; `Arrays.asList` fixed-size
Sequenced (21)	`getFirst/getLast/addFirst/addLast/removeFirst/removeLast/reversed` on lists, deques, `LinkedHashSet`, sorted sets; maps get `firstEntry/pollFirstEntry/putFirst/...`

Next: lambdas and method references in Chapter 11, then Chapter 12 — Streams. Exam traps: OCP Chapter 9. Thread-safe collections: Chapter 14.

Chapter 11 — Lambdas and Functional Programming

Before Java 8, handing a piece of *behaviour* to a method — "sort by length", "run this later", "keep only the big orders" — meant wrapping it in an object, usually an anonymous class with five lines of ceremony around one line of logic. **Lambdas** (Java 8) let you write that behaviour directly as a value: `s -> s.length()`. With **functional interfaces**, **method references** and `Optional` they brought a **functional style** (programming by passing and combining functions rather than changing variables step by step) that collections (Chapter 10) and streams (Chapter 12) are built on.

All runnable code is in `code/` (package `ch11.*`), compiled with `javac --release 25` (target the Java 25 language and library level) and run on JDK 26.

> **Exam depth.** Which lambda forms compile, `var` in parameters, method-reference puzzles, the full built-in interface table with method names (`test`, `apply`, `accept`, `get`, `applyAsInt` …) and "effectively final" traps are drilled in OCP Chapter 8 — Lambdas and functional interfaces (see *Gotchas & exam traps*). `Optional` exam questions are in OCP Chapter 10.

11.1 Why lambdas: passing behaviour

The problem it solves

Many methods need a small piece of logic from the caller: `sort` needs "how to compare", a thread needs "what to run", a filter needs "what to keep". Making behaviour an argument, just like data, is **behaviour parameterisation**. Before Java 8 the only way was an **anonymous class** (defined and instantiated in one expression, §6.11):

```
names.sort(new Comparator<String>() {
    @Override public int compare(String a, String b) { return Integer.compare(a.length(), b.length()); }
});
```

Five lines, one idea. A **lambda expression** is an anonymous function — parameters, an arrow, a body — and the receiving method decides when (and whether) to run it:

```
names.sort((a, b) -> Integer.compare(a.length(), b.length()));
```

Why it matters

- **Less noise, more intent** — *Effective Java* (Joshua Bloch's book of numbered best-practice "Items") Item 42: "prefer lambdas to anonymous classes".
- Function-taking APIs: `Map.merge`, `List.removeIf`, `Comparator.comparing`, `CompletableFuture`, Streams.
- Deferred and conditional execution: pass a `Supplier` and the work happens only if needed (§11.8).

11.2 Functional interfaces and lambda syntax

What it is

A **functional interface** has **exactly one abstract method** (a "single abstract method", or **SAM**, type); a lambda is a compact way to create an instance of one. Classic examples: `Runnable` (`void run()`), `Comparator<T>` (`int compare(T, T)`), `Callable<V>`.

Rules:

1. Only *abstract* methods count. `default` and `static` methods are fine, and so are abstract re-declarations of `Object`'s public methods (that's why `Comparator` can declare `equals` and still be functional).

2. The optional `@FunctionalInterface` annotation makes the compiler verify the "exactly one" rule — a guard against someone later adding a second abstract method.
3. A lambda has **no type of its own**. The compiler infers it (**type inference**) from the **target type** of its context (assignment, method parameter, return, cast), so `s -> s.isEmpty()` can become a `Predicate<String>` or a `Function<String, Boolean>`. That's why `var f = x -> x;` (or `var f = () -> 42;`) doesn't compile: there is no target type.

Syntax

Form	Example	Notes
No parameters	`() -> System.out.println("hi")`	parentheses required
One parameter, inferred type	`s -> s.length()`	parentheses optional
Several parameters	`(a, b) -> a + b`	parentheses required
Explicit types	`(Integer a, Integer b) -> a + b`	all or none typed
`var` parameters (Java 11)	`(var a, var b) -> a * b`	lets you add annotations: `(@NonNull var s) -> …`; all or none
Unnamed parameter (Java 22)	`(key, _) -> key`	`_` for a parameter you don't use
Expression body	`x -> x * 2`	value of the expression is returned
Block body	`n -> { if (n < 0) return "neg"; return "ok"; }`	braces, semicolons and `return` required

Code

```java
// code/ch11/lambdas/LambdaBasics.java (key parts)
@FunctionalInterface
interface Discount {
    double apply(double price);                                    // the single abstract method
    default Discount then(Discount next) { return p -> next.apply(apply(p)); }   // defaults allowed
}

Runnable noArgs                 = () -> System.out.println("no parameters");
Function<String, Integer> oneArg = s -> s.length();
BinaryOperator<Integer> typed    = (Integer a, Integer b) -> a + b;
BiFunction<Integer, Integer, Integer> withVar = (var a, var b) -> a * b;
Function<Integer, String> block  = n -> { if (n < 0) return "negative"; return "non-negative"; };

Predicate<String> isEmpty1          = s -> s.isEmpty();   // same text,
Function<String, Boolean> isEmpty2  = s -> s.isEmpty();   // different target types

Discount tenOff = p -> p - 10, twentyPercent = p -> p * 0.8;
tenOff.then(twentyPercent).apply(100);                     // (100 - 10) * 0.8

Map<String, BinaryOperator<Integer>> ops = Map.of("+", Integer::sum, "*", (a, b) -> a * b, "max", Math::max);
```

Output:

```
anonymous class: [al, Bea, Charlie]
lambda:          [Charlie, Bea, al]
no parameters
6 5 20 negative
Predicate.test=true Function.apply=false
100 -> tenOff then 20% = 72.0
+=13 *=42 max=7
```

The last line shows lambdas are *values*: stored in a map, looked up, called.

How it works under the hood

1. The compiler turns the lambda body into a private method of the enclosing class (named like `lambda$main$0`).
2. At the lambda's location it emits an **`invokedynamic`** bytecode instruction (§1.6), whose target is decided at run time on first use. That first time, the JVM's `LambdaMetafactory` generates a small class implementing the functional interface that calls the private method.
3. No `.class` file per lambda appears on disk (anonymous classes produce `Outer$1.class`).
4. A non-capturing lambda is typically created once and reused; a capturing one gets a new instance holding the values. (Identity is unspecified either way — don't rely on it.)

Lambda vs anonymous class

	Lambda	Anonymous class
Implements	only functional interfaces	any interface or class (can extend an abstract class)
Number of methods	one	any; can have fields and state
`this` refers to	the **enclosing** instance	the anonymous object itself
Shadowing (redeclaring an outer name)	can't redeclare a local variable of the enclosing scope	can shadow
Compiled as	`invokedynamic` + private method	a separate class file

Use an anonymous class only when you need state, several methods, or to extend a class.

Common mistakes

- A block body that forgets `return`, or an expression body followed by a semicolon inside: `x -> { x * 2 }` ✗ vs `x -> x * 2` ✓.
- Mixing typed and untyped parameters: `(String a, b) -> …` ✗.
- Naming a lambda parameter the same as an existing local variable → "variable is already defined".
- Long multi-statement lambdas — extract a named method and use a method reference; stack traces and readers both benefit.

TL;DR

A lambda is an anonymous function that implements a functional interface (one abstract method). Its type comes from context. Syntax: `params -> expression` or `params -> { statements; }`. Prefer it over anonymous classes unless you need state or `this` to mean the object itself.

11.3 The built-in functional interfaces (`java.util.function`)

`java.util.function` has **43** interfaces covering the common shapes, so you rarely write your own. *Effective Java* Item 44: favour the standard ones — callers know them and they come with useful **default methods** (interface methods with a body).

The core shapes

Interface	Shape	Method	Typical use
`Supplier<T>`	`() → T`	`get()`	factories, lazy values, defaults
`Consumer<T>`	`T → void`	`accept(t)`	`forEach`, logging, sending
`BiConsumer<T,U>`	`(T, U) → void`	`accept(t, u)`	`Map.forEach`
`Predicate<T>`	`T → boolean`	`test(t)`	`filter`, `removeIf`
`BiPredicate<T,U>`	`(T, U) → boolean`	`test(t, u)`	two-argument checks
`Function<T,R>`	`T → R`	`apply(t)`	`map`, key extractors
`BiFunction<T,U,R>`	`(T, U) → R`	`apply(t, u)`	`Map.merge`, `compute`
`UnaryOperator<T>`	`T → T`	`apply(t)`	`List.replaceAll`, transformations
`BinaryOperator<T>`	`(T, T) → T`	`apply(a, b)`	`reduce`, `merge` functions

Older JDK types are functional interfaces too: `Runnable` (`() → void`), `Callable<V>` (`() → V`, may throw), `Comparator<T>`. `UnaryOperator<T>` extends `Function<T, T>` and `BinaryOperator<T>` extends `BiFunction<T, T, T>`, so they inherit `apply`. Typical targets: `list.removeIf(...)` takes a `Predicate<? super E>`; `map.merge(k, 1, ...)` takes a `BiFunction<? super V, ? super V, ? extends V>` (commonly `Integer::sum`).

Primitive specialisations

Generics (Chapter 9) only work with objects, so a `Function<Integer, Integer>` **boxes** every `int` into an `Integer` (allocating outside the -128..127 `Integer` cache, §7.6). The primitive versions avoid that. Their names follow a pattern:

Pattern	Meaning	Examples
`IntX`, `LongX`, `DoubleX`	the **input** is that primitive	`IntPredicate`, `IntFunction<R>`, `IntConsumer`, `IntUnaryOperator`, `IntBinaryOperator`
`ToIntX`, `ToLongX`, `ToDoubleX`	the **output** is that primitive	`ToIntFunction<T>`, `ToDoubleBiFunction<T,U>`
`IntToLongFunction`, `DoubleToIntFunction`, …	primitive to primitive	
`XSupplier`	supplies a primitive	`IntSupplier` (`getAsInt`), `BooleanSupplier` (`getAsBoolean`)
`ObjIntConsumer<T>` etc.	object + primitive, no result	`(s, n) -> …`

The method names change accordingly: `applyAsInt`, `applyAsDouble`, `getAsInt`, `test`, `accept`. Only `int`, `long` and `double` get full sets (plus `BooleanSupplier`) — there is no `CharFunction` or `FloatPredicate`.

Code

```
// code/ch11/functional/BuiltInInterfaces.java (key parts)
Supplier<List<String>> newList = ArrayList::new;
Consumer<String> log = s -> System.out.println("log: " + s);
BiConsumer<String, Integer> put = (k, v) -> System.out.println(k + "=" + v);
Predicate<Order> big = o -> o.amount() > 100;
Function<Order, String> toId = Order::id;
BiFunction<Double, Double, Double> pow = Math::pow;
UnaryOperator<String> shout = s -> s.toUpperCase() + "!";
BinaryOperator<Integer> max = BinaryOperator.maxBy(Comparator.naturalOrder());

IntPredicate even = n -> n % 2 == 0;           IntUnaryOperator square = n -> n * n;
ToIntFunction<String> len = String::length;    IntFunction<String> stars = "*"::repeat;
ObjIntConsumer<String> repeatPrint = (s, n) -> System.out.println(s.repeat(n));

Function<Integer, Integer> boxedInc = x -> x + 1;   // boxes on every call
IntUnaryOperator primInc = x -> x + 1;              // no boxing
```

Output:

```
log: supplier made [x]
retries=3
big? true, longerThan("java",3)? true, id=A-1, 2^10=1024.0, HEY!, max=9
even(6)=true square(7)=49 add=5 len=5 stars=*** dice=4 ratio=0.25
abababab
20M calls: Function<Integer,Integer> 57 ms vs IntUnaryOperator 8 ms (machine-dependent) results equal=true
```

The last line is a rough single-run timing (Apple M4 Pro, JDK 26 — **machine-dependent**; the JIT compiler can sometimes remove boxing on its own). The point stands: in hot loops over numbers, pick the primitive interface.

When to write your own

- The shape doesn't exist (three parameters, or it throws a checked exception — §11.8).
- A domain name adds clarity and you'll attach default methods (`Discount.then` in §11.2).
- Annotate it with `@FunctionalInterface`.

TL;DR

Supplier (get), Consumer (accept), Predicate (test), Function (apply) plus `Bi` / `Unary` / `Binary` variants cover almost everything. Use `Int` / `Long` / `Double` versions for primitives to avoid boxing.

11.4 Method references

What it is

A **method reference** is shorthand for a lambda that only calls one existing method: `String::length` means `s -> s.length()`. It also needs a target functional interface. *Effective Java* Item 43: prefer them when clearer than the lambda — and keep the lambda when not.

The four kinds

Kind	Syntax	Equivalent lambda	Example
Static method	`Class::staticMethod`	`(args) -> Class.staticMethod(args)`	`Integer::parseInt`
Bound instance (specific object)	`obj::method`	`(args) -> obj.method(args)`	`System.out::println`, `prefix::concat`
Unbound instance (any object of a type)	`Class::instanceMethod`	`(obj, args) -> obj.method(args)`	`String::toUpperCase`, `String::startsWith`
Constructor	`Class::new`	`(args) -> new Class(args)`	`ArrayList::new`, `User::new`, `int[]::new`

The **unbound** form confuses people: the *first* parameter of the functional interface becomes the receiver. So `BiFunction<String, String, Boolean> f = String::startsWith` means `(s, p) -> s.startsWith(p)`, and `Comparator<String> c = String::compareToIgnoreCase` means `(a, b) -> a.compareToIgnoreCase(b)`.

Code

```
// code/ch11/methodrefs/MethodRefDemo.java (key parts)
record User(String name, int age) {
    boolean isAdult() { return age >= 18; }
    static int byAge(User a, User b) { return Integer.compare(a.age, b.age); }
}
Function<String, Integer> parse = Integer::parseInt;          // static
Comparator<User> ageOrder = User::byAge;                      // static, shape (User, User) -> int
Function<String, String> addPrefix = prefix::concat;          // bound
Consumer<Object> printer = System.out::println;               // bound
Function<String, String> upper = String::toUpperCase;         // unbound
Predicate<User> adult = User::isAdult;                        // unbound
BiFunction<String, String, Boolean> startsWith = String::startsWith;   // unbound, 2 params
Supplier<List<User>> listMaker = ArrayList::new;              // constructor
BiFunction<String, Integer, User> makeUser = User::new;       // constructor with 2 args
IntFunction<int[]> arrayMaker = int[]::new;                   // array constructor

StringBuilder sb = new StringBuilder("v1");
Supplier<String> snap = sb::toString;                         // receiver sb fixed now
sb.append("+v2");
```

Output:

```
[User[name=Zed, age=17], User[name=Kim, age=25], User[name=Amy, age=34]]
43 ID-7 OK adult(Zed)=false startsWith=true array length=5
4 5
bound receiver is the same object: v1+v2
Amy Kim Zed
```

Notes:

- `parse.apply("42") + 1` printed `43`: the `Integer` result was unboxed and added before string concatenation started.
- `String::length` fits both `ToIntFunction<String>` and `Function<String, Integer>` — the target type decides.
- A **bound** reference evaluates its receiver expression **once**, when the reference is created (throwing `NullPointerException` right then if it's `null`). It holds that *object*, so later mutations are visible (`v1+v2`), but reassigning the variable afterwards would not change the reference.
- Overloads can make a reference ambiguous (e.g. `Integer::toString` fits both static `toString(int)` and instance `toString()` for a `Function<Integer, String>`) — use a lambda then.

TL;DR

Four kinds: `Class::static`, `obj::method` (bound), `Class::method` (unbound: first argument is the receiver), `Class::new`. Use when it reads better than the lambda.

11.5 Variable capture and "effectively final"

What it is

A lambda can use variables from the surrounding code — **capturing**; a function carrying captured variables is a **closure**. Java's rule: a captured **local variable or parameter** must be **final or effectively final** — never reassigned after initialisation (whether or not it says `final`).

Why the rule exists

1. Locals live on the method's **stack frame**, which disappears when the method returns, but a lambda can outlive it (stored, or passed to another thread, Chapter 14). So Java **copies the value** into the lambda object.
2. If the variable could change, copy and original would silently disagree; forbidding reassignment keeps the copy correct.
3. It also rules out **data races** on locals (several threads touching one variable, at least one writing, uncoordinated).

Fields are different. Instance fields are reached through the captured `this` (static fields directly), so lambdas may read and modify them freely — which is exactly why shared **mutable** fields are an easy source of thread-safety bugs.

Code

```
// code/ch11/capture/CaptureDemo.java (key parts)
int threshold = 10;                          // effectively final
Predicate<Integer> above = x -> x > threshold;
// threshold = 11;                           // would make the lambda above a compile error

for (String s : List.of("a", "b", "c")) suppliers.add(() -> s.toUpperCase());   // new 's' each iteration: fine
for (int i = 0; i < 3; i++) { int copy = i; byIndex.add(() -> copy * 10); }    // i is reassigned: copy it

AtomicInteger count = new AtomicInteger();                  // mutable holder: legal but a smell
List.of("x", "yy", "zzz").forEach(w -> count.addAndGet(w.length()));
int better = List.of("x", "yy", "zzz").stream().mapToInt(String::length).sum();   // return a result instead

Runnable lambda = () -> System.out.println("lambda this -> " + this.name);        // enclosing instance
Runnable anon = new Runnable() { public void run() { System.out.println(this.getClass().getName()); } };
Runnable inc = () -> counterField++;                        // fields may be modified
```

Output:

```
above(12)=true
ABC
0 10 20
holder count=6 vs stream sum=6
lambda this      -> CaptureDemo instance
anonymous this   -> ch11.capture.CaptureDemo$1
field modified from lambda: 2
```

Common mistakes

- Capturing the classic `for (int i…)` index — it's reassigned by `i++`. Copy to a local or use `IntStream.range(0, n).forEach(i -> …)`. (The enhanced-for variable is fine: each iteration declares a fresh, never-reassigned variable.)
- Working around the rule with one-element arrays (`int[] c = {0}; list.forEach(x -> c[0]++)`) or an `AtomicInteger` just to accumulate. It compiles, but re-creates shared mutable state that breaks if the code later runs in parallel. Compute a result instead (`sum` , `count` , `collect`).
- Assuming a captured *object* is frozen — only the variable is effectively final; the object can still be mutated.

TL;DR

Captured locals must be effectively final (values are copied into the lambda). Fields aren't restricted. Don't smuggle mutation through arrays or atomics — return results instead.

11.6 Composing functions

What it is

Composition builds a new function from existing ones, like joining pipes. The functional interfaces ship default and static methods for it, so behaviour is assembled from small, tested, named pieces.

Interface	Composition methods
`Function`	`f.andThen(g)` = g(f(x)) (f first); `f.compose(g)` = f(g(x)) (g first); `Function.identity()` returns its input unchanged (handy in `toMap(identity(), …)`)
`Predicate`	`and`, `or`, `negate`, `Predicate.not(p)` (11), `Predicate.isEqual(x)`
`Consumer` / `BiConsumer`	`andThen` — run one after the other
`BiFunction`	`andThen(Function)`
`BinaryOperator`	`minBy(comparator)`, `maxBy(comparator)`
`UnaryOperator`	`identity()`
`Comparator`	`comparing`, `thenComparing`, `reversed`, `nullsFirst` (§10.8)

Code

```
// code/ch11/composition/CompositionDemo.java (key parts)
Function<Integer, Integer> plus2 = x -> x + 2, times3 = x -> x * 3;
plus2.andThen(times3).apply(5);     // (5 + 2) * 3
plus2.compose(times3).apply(5);     // 5 * 3 + 2

UnaryOperator<String> trim = String::strip, lower = String::toLowerCase;
UnaryOperator<String> collapse = s -> s.replaceAll("\\s+", " ");
Function<String, String> clean = trim.andThen(lower).andThen(collapse);

Predicate<String> nonBlank = Predicate.not(String::isBlank);
Predicate<String> shortWord = s -> s.length() <= 4, startsJ = s -> s.startsWith("J");
filter(words, nonBlank.and(shortWord));   filter(words, startsJ.or(shortWord));
filter(words, startsJ.negate().and(nonBlank));   filter(words, Predicate.isEqual("Go"));

Consumer<String> audit = s -> System.out.print("[audit " + s + "] ");
audit.andThen(send).accept("msg-1");

Function<Double, UnaryOperator<Double>> taxAdder = rate -> price -> price * (1 + rate);   // function factory
UnaryOperator<Double> vat20 = taxAdder.apply(0.20);
```

Output:

```
plus2.andThen(times3)(5) = 21, plus2.compose(times3)(5) = 17
identity = same
clean -> [hello world]
nonBlank&short: [Java, Go, JVM]  startsJ|short: [Java, , Jakarta, Go,   , JVM]  !startsJ: [Go]  isEqual(Go): [Go]
[audit msg-1] [send msg-1]
area.andThen(format) = 12 m2, minBy length = fig
vat20(50.0) = 60.0
length, then reverse alpha: [a, cc, bb, aa, ddd]
```

Notes:

- `and` / `or` **short-circuit** like `&&` / `||`: the second predicate isn't evaluated when the first decides.

- `startsJ.or(shortWord)` kept `""` and `" "` because they are short — combine with `nonBlank` when that matters.
- `Predicate.not(String::isBlank)` exists because you can't write `String::isBlank.negate()` — a method reference has no type until it's assigned.
- `trim.andThen(lower)` returns a `Function`, not a `UnaryOperator` (the default method is inherited from `Function`).

Real-world examples

Validation rules (`Predicate<Order>` combined per market), request-processing **middleware** (a chain of `UnaryOperator<Request>` adding logging or auth before the handler), text-normalisation pipelines for search indexing, configurable pricing rules (function factories).

TL;DR

Build big behaviour from small functions: `andThen` / `compose` for functions, `and` / `or` / `negate` / `not` for predicates, `andThen` for consumers, and functions that return functions for configurable logic.

11.7 Optional

What it is

`Optional<T>` (Java 8) is a container holding either one non-null value or nothing. It is meant for **method return types** where "no result" is normal — `findById`, `max`, `findFirst` — so the signature tells the caller to handle absence.

Why it matters

A returned `null` says nothing in the signature, and a forgotten check causes a `NullPointerException` far from the cause. `Optional` makes absence explicit and replaces nested `if (x != null)` chains with fluent operations.

The API

Group	Methods
Create	`Optional.of(v)` (NPE if `v` is null), `Optional.ofNullable(v)`, `Optional.empty()`
Test	`isPresent()`, `isEmpty()` (11)
Transform	`map(f)` (wraps the result; a `null` result becomes empty), `flatMap(f)` (when `f` already returns an `Optional`), `filter(p)`
Extract	`orElse(default)`, `orElseGet(supplier)`, `orElseThrow()` (10), `orElseThrow(exceptionSupplier)`, `get()` (throws if empty)
Act	`ifPresent(consumer)`, `ifPresentOrElse(consumer, runnable)` (9)
Combine	`or(() -> otherOptional)` (9), `stream()` (9) → 0 or 1 elements
Primitive versions	`OptionalInt`, `OptionalLong`, `OptionalDouble` (`getAsInt()` …)

Code

```
// code/ch11/optional/OptionalDemo.java (key parts)
record Address(String city, String zip) {}
record Customer(String name, Address address, String email) {
    Optional<Address> addressOpt() { return Optional.ofNullable(address); }
    Optional<String> emailOpt() { return Optional.ofNullable(email); }
}
static Optional<Customer> findById(int id) { return Optional.ofNullable(DB.get(id)); }

String city = findById(1).flatMap(Customer::addressOpt).map(Address::city).orElse("no city");

findById(1).map(Customer::name).orElse(expensiveDefault());            // default computed ANYWAY
findById(1).map(Customer::name).orElseGet(OptionalDemo::expensiveDefault); // computed only if empty

findById(9).orElseThrow(() -> new IllegalArgumentException("customer 9 not found"));
findById(2).flatMap(Customer::emailOpt).ifPresentOrElse(e -> ..., () -> System.out.println("Bo has no email ->
send SMS"));
findById(2).flatMap(Customer::emailOpt).or(() -> Optional.of("sms:+000"));
List.of(1, 2, 9).stream().map(OptionalDemo::findById).flatMap(Optional::stream)...   // drop empties
```

Output:

```
Optional[hi] Optional.empty Optional.empty isPresent=true isEmpty=true
Optional.of(null) -> NPE
cities: Lisbon, no city, no city
gmail? false
orElse on present:    (computing default) Ana
orElseGet on present: Ana
orElseThrow() -> No value present
orElseThrow(supplier) -> customer 9 not found
Bo has no email -> send SMS
fallback Optional via or(): sms:+000
all known emails: [ana@example.com]
OptionalInt max=3, empty average=OptionalDouble.empty
anti-pattern isPresent+get: Ana
better: Ana
```

Read `findById(id).flatMap(Customer::addressOpt).map(Address::city).orElse("no city")` as: find the customer → get the address (itself optional, so `flatMap` avoids `Optional<Optional<Address>>`) → take the city → else "no city". Three "nothing" points, zero `if`s.

The `orElse` line proves that **`orElse`'s argument is always evaluated** — it's an ordinary method argument. Use `orElseGet` when the default is expensive or has side effects (§11.8).

Right uses and wrong uses

✅ Do	❌ Don't
Return `Optional<T>` from a method when "no result" is normal	Return `null` from a method declared to return `Optional`
Chain `map` / `flatMap` / `filter` and finish with `orElse*` / `orElseThrow` / `ifPresent*`	`if (opt.isPresent()) opt.get()` — a verbose null check
Use `orElseThrow()` (10) when absence is a bug — its name says it can throw	Call `get()` without a check (`orElseThrow()` is the same thing with an honest name)
Use `OptionalInt` / `OptionalDouble` for primitives	`Optional<Integer>` in numeric code
Return an **empty collection** for "no items"	`Optional<List<T>>` — an empty list already means "none"
Use nullable fields internally and expose `Optional` from getters if helpful	`Optional` **fields** (not `Serializable` ; extra object per field)
Overload or use a default for optional inputs	`Optional` **parameters** (callers must wrap; they can still pass `null`)
Use it as a return value	Use it as a map key/value or in collections

These follow *Effective Java* Item 55 ("return optionals judiciously") and the JDK designers' intent: `Optional` is primarily for return types. It is also a *value-based* class (instances are pure values with no meaningful identity) — don't compare with `==` or synchronise on them (Chapter 14).

In the interview

> **30-second answer:** "Optional is a return-type container for 'maybe a value'. It makes absence explicit in the signature and lets me chain map, flatMap and filter and end with orElse, orElseGet or orElseThrow instead of null checks. I don't use it for fields, parameters or collections, I never return null from an Optional method, and I use orElseGet when the default is expensive because orElse evaluates its argument eagerly."

- *Follow-up:* `map` *vs* `flatMap` *?* → Use `flatMap` when the function itself returns an `Optional`, to avoid nesting.
- *Follow-up:* `Optional.of` *vs* `ofNullable` *?* → `of` throws NPE on null (use when null is a bug); `ofNullable` returns empty.
- *Follow-up: Is* `Optional` *serializable?* → No — one reason not to use it for fields.

TL;DR

`Optional` = explicit "maybe" for return values. Transform with `map` / `flatMap` / `filter`, end with `orElse` / `orElseGet` / `orElseThrow` / `ifPresentOrElse`. Not for fields, parameters or collections; never `null` itself.

11.8 Functional style in everyday Java

What it is

Functional programming (FP) builds programs from **functions** that compute results from inputs, avoiding shared mutable state. Java is not a pure functional language, but several FP ideas make Java code simpler and safer. Key terms:

Term	Meaning
Pure function	Result depends only on its arguments; no side effects (no I/O, no mutation of outside state). Same input → same output, every time.
Side effect	Anything a function does besides returning a value: printing, writing a field, changing an argument, I/O.
Immutability	Data that can't change after creation (records, `List.of`, `String`).
Higher-order function	Takes a function as a parameter and/or returns one (`retryUntil`, `memoize`, `Comparator.comparing`).
Currying	Turning `f(a, b)` into `a -> b -> …`, so you can fix `a` first (**partial application**).
Laziness	Delaying a computation until its result is needed (`Supplier`, streams in Chapter 12).
Memoisation	Caching a pure function's results by input.
Declarative	Describe *what* result you want (`filter`, `map`) rather than *how* to loop — the style Chapter 12 is built on.

Code

```
// code/ch11/fp/FunctionalStyle.java (key parts)
static int total = 0;
static int addToTotal(int x) { total += x; return total; }    // impure: result depends on history
static int add(int a, int b) { return a + b; }                // pure

static <T> List<T> retryUntil(Supplier<T> attempt, Predicate<T> ok, int max) { ... }   // higher-order

Function<Integer, Function<Integer, Integer>> curriedAdd = a -> b -> a + b;
Function<Integer, Integer> add10 = curriedAdd.apply(10);      // partial application

static void logIf(boolean enabled, Supplier<String> message) { if (enabled) System.out.println(message.get()); }
logIf(debug, () -> "state dump " + "x".repeat(1_000_000).length());   // string never built when debug=false

static <T, R> Function<T, R> memoize(Function<T, R> f) {
    Map<T, R> cache = new HashMap<>();
    return x -> { R r = cache.get(x); if (r == null) { r = f.apply(x); cache.put(x, r); } return r; };
}

Map<String, DoubleUnaryOperator> shipping = Map.of(           // strategy pattern as data
        "standard", w -> 4.99, "express", w -> 9.99 + 1.5 * w, "free", w -> 0.0);

@FunctionalInterface interface ThrowingFunction<T, R> { R apply(T t) throws Exception; }
static <T, R> Function<T, R> unchecked(ThrowingFunction<T, R> f) {
    return t -> {
        try { return f.apply(t); }
        catch (RuntimeException e) { throw e; }
        catch (Exception e) { throw new RuntimeException(e); }
    };
}
```

Output:

```
impure twice: 5, 10   pure twice: 5, 5
retryUntil six: [2, 5, 6]
add10(5)=15
lazy log: supplier not called because debug=false
memoized: 5 calls, underlying function ran 2 times
standard(2kg)=4.99  express(2kg)=12.99  free(2kg)=0.00
app.properties=ok
wrapped: java.io.IOException: no such file: missing.properties
original [10, 20, 30] untouched; new [9, 18, 27]
```

What each idea buys you:

- **Pure functions** are trivially testable (no setup, no mocks), safe to cache and safe to run in parallel.
- **Higher-order functions** remove duplicated control flow: the retry loop is written once; callers pass *what* to try and *when* it succeeded.
- **Laziness via `Supplier`** is how logging frameworks avoid building expensive messages (SLF4J's fluent API and `java.util.logging.Logger.fine(Supplier<String>)` accept suppliers).
- **Strategies as lambdas** replace a hierarchy of one-method classes; see the Strategy pattern in LLD Chapter 10.
- **Immutability**: transform into a new list instead of editing in place; the original stays valid for anyone else holding it.

Memoisation note. The obvious `cache.computeIfAbsent(x, f)` is wrong when `f` is *recursive* (e.g. memoised Fibonacci calling back into the same map): a `HashMap` must not be modified by its own mapping function and throws `ConcurrentModificationException` (since Java 9); `ConcurrentHashMap` can **deadlock** or throw `IllegalStateException`. The get-then-put version above is safe single-threaded.

Checked exceptions (Chapter 8): `java.util.function` methods don't declare `throws`, so a lambda calling `Files.readString` (throws `IOException`) won't compile as a `Function<Path, String>`. Options: handle it inside the lambda, wrap and rethrow unchecked as above (`UncheckedIOException` is the JDK's own choice for I/O), or define a throwing interface. Never silently swallow it.

When not to use functional style

- The loop is simpler: a `for` with early `return` or several interacting variables is often clearer than a clever chain.
- Deeply nested lambdas (a function returning a function returning a function) — name intermediate pieces.
- Performance-critical inner loops where boxing or **megamorphic** calls (one call site seeing many implementing classes, which stops the JIT inlining it) matter — measure (Chapter 16).
- Stack traces through lambdas include **synthetic** (compiler-generated) frames like `lambda$main$0`; debugging a long chain is harder than a named method.

Real-world examples

Spring's `RestClient` / `WebClient` builders and Security configuration **DSLs** (domain-specific languages: mini-APIs that read like a small language) take lambdas; `CompletableFuture` chains functions over async results (Chapter 15); UI listeners and callbacks; retry/backoff helpers (Resilience4j decorates `Supplier`s and `Function`s); JUnit's `assertThrows(Class, Executable)` (Chapter 18).

In the interview

> **30-second answer:** "In Java I use functional ideas where they reduce bugs: pure functions and immutable data for anything shared or tested, higher-order functions to remove repeated control flow, Suppliers for laziness, and lambdas as strategies. I stop when a plain loop is clearer, and I'm careful with checked exceptions and mutable captured state."

- *Follow-up: Is Java a functional language?* → No — it's object-oriented with functional features (functions as storable, passable values via functional interfaces, lambdas, streams, records, pattern matching). Functions are objects implementing interfaces, not a separate function type.
- *Follow-up: What's a pure function and why care?* → Output depends only on input, no side effects (so `x -> { System.out.println(x); return x; }` is impure); easy to test, cache, and parallelise.

TL;DR

Pure functions + immutable data + higher-order functions = code that's easier to test and parallelise. Use `Supplier` for laziness, lambdas for strategies, wrap checked exceptions deliberately, and fall back to loops when they're clearer.

Chapter TL;DR

Topic	Remember
Why	Pass behaviour as a value; lambdas replace anonymous classes (EJ Item 42)
Functional interface	Exactly one abstract method; `@FunctionalInterface`; lambda's type comes from the target
Syntax	`x -> expr`, `(a, b) -> { …; return r; }`, `(var a, var b)` (11), `_` for unused (22)
Built-ins	`Supplier.get`, `Consumer.accept`, `Predicate.test`, `Function.apply`, `Bi*`, `UnaryOperator`, `BinaryOperator`; `IntX` / `ToIntX` to avoid boxing
Method refs	`Class::static`, `obj::method`, `Class::method` (first arg is receiver), `Class::new`
Capture	Locals must be effectively final (value copied); fields free; `this` = enclosing instance
Composition	`andThen` / `compose` / `identity`, `and` / `or` / `negate` / `Predicate.not`, functions returning functions
Optional	Return type only; `map` / `flatMap` / `filter` → `orElse` / `orElseGet` (lazy)/ `orElseThrow`; no fields/params/collections; never null
FP style	Pure, immutable, higher-order, lazy; wrap checked exceptions; loops when clearer

Where next: put all of this to work on collections with Chapter 12 — Streams. Exam traps: OCP Chapter 8. Python equivalents (lambdas, closures, `functools`): Python Chapter 7.

Chapter 12 — Streams

The **Streams API** (`java.util.stream` , Java 8) lets you describe *what* to compute from a sequence of values — "names of customers who spent over 450, upper-cased and sorted" — as a **pipeline** (a chain of steps, each feeding the next) instead of hand-writing *how* to loop, test, accumulate and sort. It builds on lambdas and functional interfaces (Chapter 11) and usually starts from a collection (Chapter 10). Covered here: lazy execution, every everyday operation, collectors, primitive streams, parallel streams (and when not to use them), **stream gatherers** (final in Java 24), and common mistakes.

All runnable code is in `code/` (package `ch12.*`), compiled with `javac --release 25` and run on JDK 26.

> **Exam depth.** Output-prediction puzzles (laziness, `peek` , infinite streams that hang), the exact return types of every terminal operation, `Optional` vs primitive optionals, `Spliterator` , and collector signatures are drilled in OCP Chapter 10 — Streams (see *Gotchas & exam traps*).

12.1 What a stream is

What it is

A **stream** is a sequence of elements supporting a chain of operations computed on demand: elements come from a **source** (collection, array, file, generator), pass through **intermediate operations** that inspect, reshape or discard them, and end at one **terminal operation** that produces the result. Nothing moves until the terminal operation runs.



A stream is **not** a collection:

	Collection	Stream
Purpose	stores elements	computes over elements
Elements	all exist in memory	produced on demand; may be infinite
Traversal	as often as you like	**once** (then it's consumed)
Iteration	external (you write the loop)	internal (the library runs the loop)
Changes source?	`add` / `remove` modify it	never modifies its source

Why it matters

- **Readability**: each step names its intent (`filter` , `map` , `groupingBy`); multi-step processing reads top to bottom.
- **Fewer bugs**: no index variables or accumulators to forget to reset.
- **Optimisation room**: the library runs the loop, so it can fuse steps into a single pass, stop early, and optionally split work across CPU cores.

Code: loop vs pipeline

```
// code/ch12/basics/PipelineDemo.java (key parts)
record Order(String customer, String product, int qty, double price) { double total() { return qty * price; } }

List<String> loop = new ArrayList<>();                          // imperative: HOW
for (Order o : ORDERS) if (o.total() >= 450) loop.add(o.customer().toUpperCase());
Collections.sort(loop);

List<String> stream = ORDERS.stream()                           // declarative: WHAT
        .filter(o -> o.total() >= 450)
        .map(o -> o.customer().toUpperCase())
        .sorted()
        .toList();                                              // Java 16
```

The loop is **imperative** (step-by-step instructions saying *how*); the pipeline is **declarative** (a description of *what* result you want).

Output (first two lines):

```
loop:   [ANA, ANA, BO, CY]
stream: [ANA, ANA, BO, CY]
```

Creating streams

Source	Code	Notes
Any collection	`list.stream()`, `set.parallelStream()`	the usual start
Values	`Stream.of("a", "b")`	varargs
Array	`Arrays.stream(arr)`, `Arrays.stream(arr, from, to)`	`int[]` → `IntStream`
Number ranges	`IntStream.range(0, n)` (exclusive), `rangeClosed(1, n)`	
Infinite, by rule	`Stream.iterate(seed, f)`; `Stream.iterate(seed, hasNext, next)` (9)	use `limit` or the 3-arg form
Infinite, by supplier	`Stream.generate(supplier)`	unordered, needs `limit`
Maybe one value	`Stream.ofNullable(x)` (9), `optional.stream()` (9)	0 or 1 elements
Joining two	`Stream.concat(a, b)`	
Characters / lines	`str.chars()`, `str.lines()` (11)	
Files	`Files.lines(path)`, `bufferedReader.lines()`	lazy; close it — Chapter 13
Map	`map.entrySet().stream()`	maps aren't streams themselves
Random numbers	`new Random(seed).ints(n, lo, hi)`	same seed, same numbers
Builder	`Stream.<T>builder().add(x).build()`	

```
Stream.iterate(1, x -> x * 2).limit(8).toList();
Stream.iterate(1, x -> x < 100, x -> x * 3).toList();          // like a for loop
Stream.generate(() -> "ab").limit(3).collect(Collectors.joining("-"));
Stream<String> s = Stream.of("once"); s.forEach(x -> {}); s.count();   // IllegalStateException
```

Output (excerpt):

```
iterate(seed,f)  [1, 2, 4, 8, 16, 32, 64, 128]
iterate(s,has,f) [1, 3, 9, 27, 81]
generate         ab-ab-ab
reuse -> stream has already been operated upon or closed
source [3, 1, 2] unchanged, result [1, 2, 3]
```

A stream supports **one** terminal operation; a second throws `IllegalStateException` — create a new stream from the source. And `list.stream().sorted()` does not sort `list`: streams never modify their source.

TL;DR

Stream = a one-shot, lazy pipeline over a source: source → intermediate ops → terminal op → result. It describes a computation; it doesn't store data.

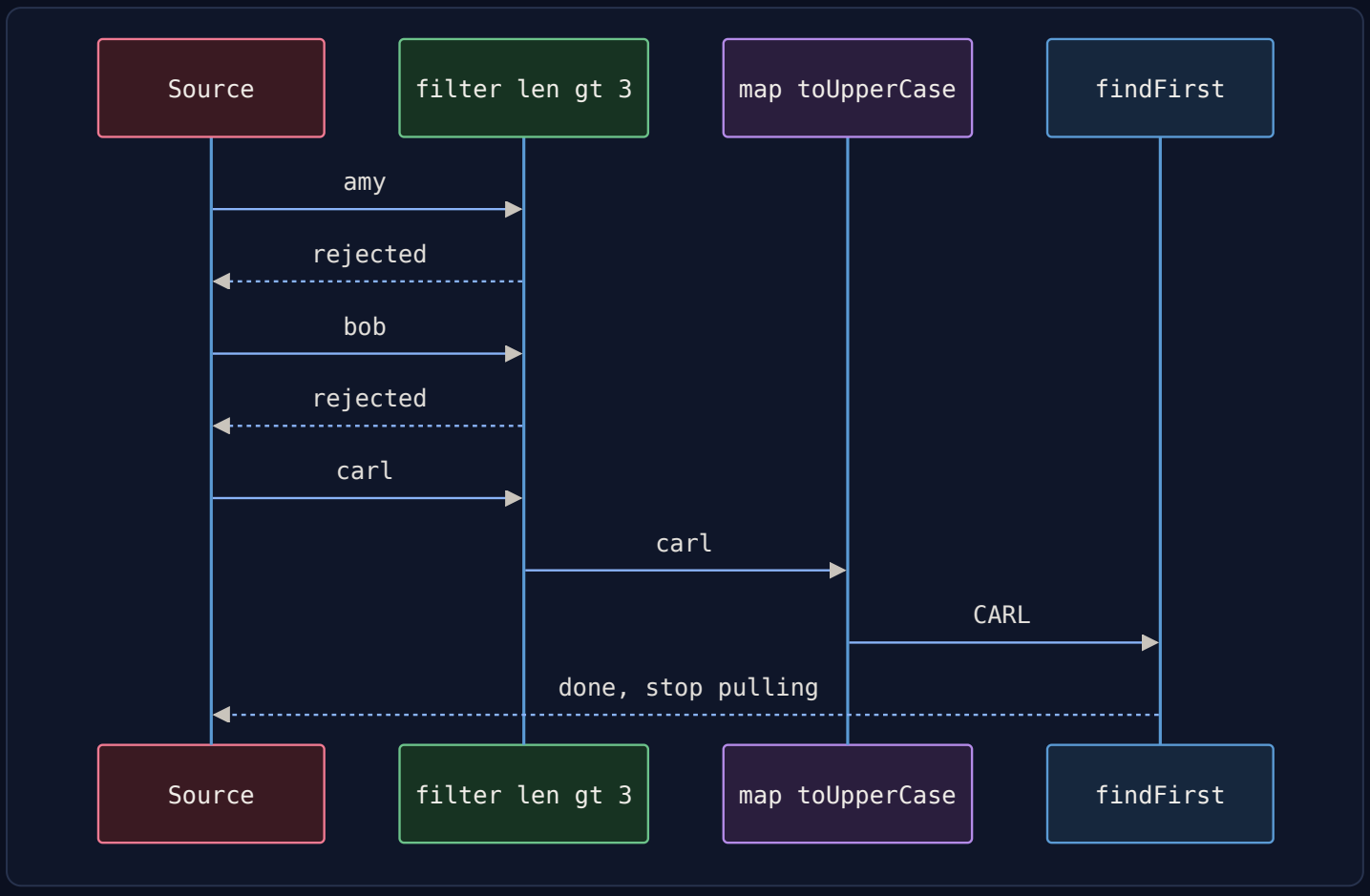
12.2 Laziness: how a pipeline really executes

What it is

Intermediate operations are lazy (work is postponed until a result is asked for): `filter` or `map` only *records* a step and returns a new stream. Work happens when a **terminal operation** pulls elements through — **element by element through all stages** ("vertical" processing), not stage by stage over the whole list ("horizontal"). So `Stream.of(1, 2, 3).map(x -> { System.out.println(x); return x; });` prints nothing.

How it works

1. Each intermediate call adds a *stage* to a linked pipeline description.
2. The terminal operation asks the source for elements — via a `Spliterator` (a "splittable iterator" that can also split its range in two for parallel work).
3. Each element travels through every stage in order; stages are fused into one pass, so there are no intermediate lists.
4. **Short-circuiting** operations (`findFirst`, `findAny`, `anyMatch`, `allMatch`, `noneMatch`, `limit`, `takeWhile`) stop pulling as soon as the answer is known — which is what makes infinite streams usable.
5. **Stateless** operations (`filter`, `map`, `flatMap`, `peek`) handle each element independently. **Stateful** ones remember other elements: `distinct` (a set of seen items), `sorted` (**buffers everything** before emitting anything — a *barrier*), `limit` / `skip` (counters), `takeWhile` / `dropWhile`.



Code

```
// code/ch12/laziness/LazinessDemo.java (key parts)
Stream<String> pipeline = names.stream().filter(n -> { System.out.println("filter " + n); return true; });
System.out.println("pipeline built, nothing printed yet");   // no terminal op -> nothing ran

Optional<String> first = names.stream()
        .filter(n -> { System.out.println("filter " + n); return n.length() > 3; })
        .map(n -> { System.out.println("map    " + n); return n.toUpperCase(); })
        .findFirst();

List<Integer> squares = Stream.iterate(1, i -> i + 1)          // infinite
        .peek(i -> System.out.print("gen " + i + " | "))
        .map(i -> i * i).filter(sq -> sq % 2 == 1).limit(3).toList();

Stream.of("c", "a", "b").peek(x -> System.out.println("before sort " + x))
      .sorted().forEach(x -> System.out.println("after sort  " + x));
```

Output:

```
pipeline built, nothing printed yet
--- vertical processing + short-circuit (findFirst)
filter amy
filter bob
filter carl
map    carl
result CARL  (dave and eve were never touched)
--- limit on an infinite stream
gen 1 | gen 2 | gen 3 | gen 4 | gen 5 |
odd squares [1, 9, 25]
--- sorted() is a barrier
before sort c
before sort a
before sort b
after sort  a
after sort  b
after sort  c
map-then-filter: 1000 map calls; filter-then-map: 5 map calls (same count 5=5)
```

What to notice:

- The first pipeline never printed anything — it was never run.
- `map` ran **once**, not five times: `findFirst` stopped the pipeline at `carl`.
- The infinite `iterate` generated exactly 5 numbers, because `limit(3)` stopped pulling after the third odd square.
- `sorted()` consumed all three elements before passing any on.
- **Order of operations affects cost**: filtering first meant 5 `map` calls instead of 1,000. Put cheap, selective `filter`s early and expensive or stateful steps (`sorted`, `distinct`) late.

Common mistakes

- Expecting a side effect in `map` / `peek` to happen without a terminal operation.
- Putting `sorted()` or `distinct()` *before* `limit` on an infinite stream → it never finishes (the barrier waits forever).
- Relying on `peek` for real work — the library may skip it entirely (§12.10).

TL;DR

Nothing runs until a terminal operation. Elements flow one at a time through all fused stages; short-circuiting ops stop early; stateful ops (`sorted`, `distinct`) buffer. Filter early, sort late.

12.3 Intermediate operations

Operation	Does	Stateful?	Notes
`filter(Predicate)`	keep matching elements	no	
`map(Function)`	transform each element (1 → 1)	no	may change type
`mapToInt/Long/Double`, `mapToObj`, `boxed`	switch between object and primitive streams	no	§12.6
`flatMap(Function<T, Stream<R>>)`	each element → 0..n elements, flattened	no	
`mapMulti(BiConsumer<T, Consumer<R>>)` (16)	push 0..n results imperatively	no	avoids creating tiny streams
`distinct()`	drop duplicates (by `equals` / `hashCode`)	**yes**	keeps first occurrence
`sorted()` / `sorted(Comparator)`	sort	**yes (barrier)**	stable (equal items keep their order) for ordered streams
`limit(n)` / `skip(n)`	first n / all but first n	yes (counter)	`limit` short-circuits
`takeWhile(p)` / `dropWhile(p)` (9)	take / drop the **prefix** while `p` holds	yes	stop at the first failure
`peek(Consumer)`	look at elements passing by	no	debugging only
`parallel()`, `sequential()`, `unordered()`, `onClose(Runnable)`	change execution mode / characteristics	—	§12.8

Code

```
// code/ch12/ops/IntermediateOps.java (key parts)
List<Integer> nums = List.of(5, 3, 8, 3, 1, 9, 8, 2);
nums.stream().distinct().toList();
nums.stream().sorted(Comparator.reverseOrder()).toList();
nums.stream().skip(2).limit(3).toList();
nums.stream().takeWhile(n -> n < 8).toList();
nums.stream().dropWhile(n -> n < 8).toList();

books.stream().map(Book::tags).toList();                                    // List<List<String>>
books.stream().flatMap(b -> b.tags().stream()).distinct().sorted().toList(); // List<String>
Stream.of("to be", "or not").flatMap(line -> Arrays.stream(line.split(" "))).toList();

books.stream().<String>mapMulti((b, sink) -> {
    for (String t : b.tags()) if (t.startsWith("d")) sink.accept(b.title() + ":" + t);
}).toList();
```

Output:

```
filter even      [8, 8, 2]
map x10          [50, 30, 80, 30, 10, 90, 80, 20]
distinct         [5, 3, 8, 1, 9, 2]
sorted           [1, 2, 3, 3, 5, 8, 8, 9]
sorted desc      [9, 8, 8, 5, 3, 3, 2, 1]
skip 2 limit 3   [8, 3, 1]
takeWhile <8     [5, 3]   (Java 9: stops at first failure)
dropWhile <8     [8, 3, 1, 9, 8, 2]
filter <8        [5, 3, 3, 1, 2]   (compare: looks at every element)
map (nested)     [[java, best-practices], [java, streams, fp], [databases, distributed]]
flatMap          [best-practices, databases, distributed, fp, java, streams]
flatMap words    [to, be, or, not]
mapMulti         [DDIA:databases, DDIA:distributed]
mapToInt sum     6053
peek saw 3 titles, kept [Effective Java, DDIA]
sorted by length [a, bb, ccc]
boxed / asLong   [1, 2]
unordered/parallel/sequential/onClose also exist (see 12.8)
```

`map` vs `flatMap` — the one to really understand

- `map` is **one in, one out**. If your function returns a list (or array, stream, `Optional`), you get a stream *of lists*: `Stream<List<String>>`; `Stream.of("a b", "c").map(s -> s.split(" "))` is a `Stream<String[]>`.
- `flatMap` is **one in, many out**: the function returns a *stream* per element, and `flatMap` splices them into one `Stream<String>` (`flatMap(s -> Arrays.stream(s.split(" ")))`).
- Uses: all words from all lines, all order lines from all orders, all present values from `Optional`s (`flatMap(Optional::stream)`).
- **`mapMulti`** (Java 16) does the same imperatively — call `sink.accept(x)` per output. Handy when each element yields very few results (no stream allocated per element) or the logic reads better as a loop. Inference often needs the explicit **type witness** `<String>mapMulti(...)` (spelling out the generic type yourself).

`takeWhile` / `dropWhile` vs `filter`

`takeWhile(n -> n < 8)` stopped at the first 8 and returned `[5, 3]`, even though later `3, 1, 2` are also < 8; likewise `Stream.of(1, 5, 2, 7).dropWhile(x -> x < 3)` gives `[5, 2, 7]`. They are meant for **sorted or naturally ordered** data ("log lines until the first error"). `filter` examines every element.

TL;DR

`filter` keeps, `map` transforms 1→1, `flatMap` / `mapMulti` 1→many, `distinct` / `sorted` need state, `limit` / `skip` / `takeWhile` / `dropWhile` cut the sequence, `peek` is for debugging.

12.4 Terminal operations and `reduce`

Operation	Returns	Short-circuits?	Notes
`forEach(c)` / `forEachOrdered(c)`	`void`	no	`forEachOrdered` keeps **encounter order** (the order the source delivers elements) in parallel
`toList()` (16)	unmodifiable `List<T>` (`add` / `remove` throw)	no	allows `null` elements
`collect(collector)`	anything	no	§12.5
`toArray()` / `toArray(String[]::new)`	`Object[]` / `T[]`	no	
`count()`	`long`	no	may skip the pipeline if size is known
`min(c)` / `max(c)`	`Optional<T>`	no	
`findFirst()` / `findAny()`	`Optional<T>`	**yes**	`findAny` is cheaper in parallel
`anyMatch` / `allMatch` / `noneMatch`	`boolean`	**yes**	on an empty stream (any stream type): `false` / `true` / `true`
`reduce(...)`	`T` or `Optional<T>` or `U`	no	three forms below
`iterator()` / `spliterator()`	pull elements manually	–	escape hatch out of the stream API

`reduce` — folding a stream into one value

Reduction repeatedly combines elements until one value remains — summing, multiplying, finding the longest.

Form	Signature	Use
Identity + accumulator	`T reduce(T identity, BinaryOperator<T> acc)`	`reduce(0, Integer::sum)`
Accumulator only	`Optional<T> reduce(BinaryOperator<T> acc)`	empty stream → `Optional.empty()`
Identity + accumulator + combiner	`U reduce(U identity, BiFunction<U,T,U> acc, BinaryOperator<U> combiner)`	result type differs from element type, e.g. total length of strings; the combiner merges partial results in parallel

Rules that keep `reduce` correct (they matter the moment you go parallel):

1. The **identity** must be neutral: `combine(identity, x) == x` (0 for +, 1 for ×, `""` for concatenation). `reduce(5, Integer::sum)` is wrong: in parallel 5 is added once per chunk (§12.8).
2. The function must be **associative** (grouping doesn't change the answer): `(a ⊕ b) ⊕ c == a ⊕ (b ⊕ c)`. Subtraction and averaging are not — `reduce(0, (a, b) -> a - b)` is unsafe in parallel.
3. It must be **stateless** and side-effect free.

For summing numbers, prefer `mapToInt(...).sum()`; for building collections use `collect`, not `reduce` (reducing into a new list at every step is $O(n^2)$).

Code

```
// code/ch12/ops/TerminalOps.java (key parts)
words.stream().max(Comparator.comparingInt(String::length)).orElseThrow();
words.stream().filter(w -> w.startsWith("z")).findAny();              // Optional.empty
Stream.<String>empty().allMatch(w -> false);                          // true (vacuous truth)
Stream.of(1, 2, 3, 4).reduce(0, Integer::sum);
Stream.of(1, 2, 3, 4).reduce((a, b) -> a * b);                        // Optional
words.stream().reduce(0, (acc, w) -> acc + w.length(), Integer::sum);  // String -> int
words.stream().collect(Collectors.toCollection(ArrayList::new));       // when you need a mutable list
words.parallelStream().forEachOrdered(w -> System.out.print(w + " "));
```

Output:

```
count            3
min / max        java / stream
findFirst        Optional[record]
findAny          Optional.empty
anyMatch         true
allMatch         true
noneMatch        true
allMatch on EMPTY true (vacuous truth)
reduce           sum=10 product=24 totalLength=25 emptyReduce=Optional.empty
reduce longest   stream
toList [STREAM, LAMBDA, JAVA, RECORD, VAR] | toCollection+add 6 | toArray 5
forEachOrdered (parallel): stream lambda java record var
iterator first two: java, lambda
```

`allMatch` on an empty stream is `true` by **vacuous truth** ("every element matches" can't be false when there are none). `max` by length returned `stream`, the first of three 6-letter words, but the Javadoc doesn't promise which tied element you get — add a tie-breaker (`thenComparing`) when it matters.

`toList()` vs `collect(Collectors.toList())`

	`stream.toList()` (16)	`collect(Collectors.toList())`	`collect(Collectors.toUnmodifiableList())` (10)
Mutability	unmodifiable	unspecified (currently `ArrayList`)	unmodifiable
Nulls	allowed	allowed	**NPE**

Use `toList()` by default; `toCollection(ArrayList::new)` when you genuinely need to modify the result.

TL;DR

Terminal ops run the pipeline: `toList`, `collect`, `count`, `min` / `max`, `find*`, `*Match`, `reduce`, `forEach`. `reduce` needs a true identity and an associative, stateless function. `toList()` is unmodifiable.

12.5 Collectors

What it is

A `Collector` is a recipe for **mutable reduction**: accumulate elements into a container (list, map, string builder, statistics) and optionally finish it. `Collectors` (with an *s*) provides ready-made ones for `collect`; `import static java.util.stream.Collectors.*;` keeps pipelines readable.

The toolbox

Collector	Result	Notes
`toList()`, `toSet()`, `toUnmodifiableList/Set()` (10)	`List`, `Set`	
`toCollection(TreeSet::new)`	any collection	choose the implementation
`toMap(keyFn, valueFn)`	`Map`	**duplicate key** → **`IllegalStateException`**; `null` value → NPE
`toMap(keyFn, valueFn, mergeFn)`	`Map`	merge function resolves duplicates
`toMap(keyFn, valueFn, mergeFn, TreeMap::new)`	chosen map type	
`joining()`, `joining(", ")`, `joining(", ", "[", "]")`	`String`	elements must be `CharSequence`
`groupingBy(classifier)`	`Map<K, List<T>>`	the SQL `GROUP BY` of streams
`groupingBy(classifier, downstream)` / `(classifier, mapFactory, downstream)`	`Map<K, D>`	downstream collector per group
`partitioningBy(predicate[, downstream])`	`Map<Boolean, …>`	always both keys `false` and `true`
`counting()`, `summingInt/Long/Double`, `averagingX`, `minBy`, `maxBy`	numbers / `Optional`	usually as downstream
`summarizingInt/Long/Double`	`IntSummaryStatistics` etc.	count, sum, min, max, average in one pass
`mapping(fn, downstream)`, `filtering(p, downstream)` (9), `flatMapping(fn, downstream)` (9)	adapt elements inside a group	
`collectingAndThen(downstream, finisher)`	transformed result	e.g. unwrap an `Optional`, freeze a list
`teeing(c1, c2, merger)` (12)	anything	two collectors in one pass, then combine
`reducing(identity, op)`	`T`	reduction as a downstream collector
`groupingByConcurrent`, `toConcurrentMap`	`ConcurrentMap`	for parallel streams, unordered

Code

```java
// code/ch12/collectors/CollectorsDemo.java (key parts)
record Employee(String name, String dept, String city, double salary, List<String> skills) {}

STAFF.stream().map(Employee::name).collect(joining(", ", "[", "]"));
STAFF.stream().collect(toMap(Employee::dept, Employee::name));                  // throws: duplicate ENG
STAFF.stream().collect(toMap(Employee::dept, Employee::name, (a, b) -> a + "+" + b, TreeMap::new));

STAFF.stream().collect(groupingBy(Employee::dept, TreeMap::new, mapping(Employee::name, toList())));
STAFF.stream().collect(groupingBy(Employee::dept, TreeMap::new, counting()));
STAFF.stream().collect(groupingBy(Employee::dept, TreeMap::new,
        collectingAndThen(maxBy(Comparator.comparingDouble(Employee::salary)), o -> o.orElseThrow().name())));
STAFF.stream().collect(groupingBy(Employee::city, TreeMap::new, groupingBy(Employee::dept, TreeMap::new,
counting())));
STAFF.stream().collect(groupingBy(Employee::dept, TreeMap::new,
        filtering(e -> e.salary() > 100_000, mapping(Employee::name, toList()))));
STAFF.stream().collect(groupingBy(Employee::dept, TreeMap::new,
        flatMapping(e -> e.skills().stream(), toCollection(TreeSet::new))));

STAFF.stream().collect(partitioningBy(e -> e.salary() >= 110_000, mapping(Employee::name, toList())));
STAFF.stream().collect(summarizingDouble(Employee::salary));
STAFF.stream().collect(teeing(minBy(bySalary), maxBy(bySalary),
        (min, max) -> min.orElseThrow().name() + ".." + max.orElseThrow().name()));

Collector<String, StringJoiner, String> pipes = Collector.of(
        () -> new StringJoiner("|"),   // supplier: new container
        StringJoiner::add,             // accumulator: add one element
        StringJoiner::merge,           // combiner: merge two containers (parallel)
        StringJoiner::toString);       // finisher: container -> result
```

Output:

```
joining          [Ana, Bo, Cy, Dee, Eli]
toSet(TreeSet)   [Berlin, Lisbon]
toMap            {Ana=120000.0, Bo=150000.0, Cy=90000.0, Dee=80000.0, Eli=110000.0}
toMap dup key -> IllegalStateException: Duplicate key ENG (attempted merging values Ana and Bo)
toMap merge      {ENG=Ana+Bo+Eli, OPS=Cy, SALES=Dee}
toMap identity   Bo -> ENG
groupingBy       {ENG=[Ana, Bo, Eli], OPS=[Cy], SALES=[Dee]}
  counting       {ENG=3, OPS=1, SALES=1}
  averaging      {ENG=126666.66666666667, OPS=90000.0, SALES=80000.0}
  summing        {Berlin=230000.0, Lisbon=320000.0}
  maxBy          {ENG=Bo, OPS=Cy, SALES=Dee}
  multi-level    {Berlin={ENG=1, SALES=1}, Lisbon={ENG=2, OPS=1}}
  filtering(9)   {ENG=[Ana, Bo, Eli], OPS=[], SALES=[]}
  flatMapping(9) {ENG=[go, java, k8s, sql], OPS=[bash, k8s], SALES=[excel]}
partitioningBy   {false=[Cy, Dee], true=[Ana, Bo, Eli]}
partition empty  {false=[], true=[]}
summarizing      count=5 min=80000 max=150000 avg=110000 sum=550000
teeing           lowest..highest paid = Dee..Bo
collectingAndThen UnmodifiableRandomAccessList
reducing         550000.0
Collector.of     Ana|Bo|Cy|Dee|Eli
```

How to read the grouping results:

- `groupingBy` = SQL `GROUP BY dept`; the **downstream** collector is the aggregate (`COUNT(*)`, `AVG(salary)`, `MAX(...)`).
- `groupingBy` with no map factory returns a `HashMap` (no order); pass `TreeMap::new` for sorted keys — e.g. "names per department, sorted by department" is `groupingBy(Employee::dept, TreeMap::new, mapping(Employee::name, toList()))`.
- `filtering` inside the group kept the `OPS` and `SALES` keys with empty lists. A `filter` *before* `groupingBy` would have removed those groups entirely — choose based on whether you want empty groups reported.
- `partitioningBy` always contains both `false` and `true` keys, even for an empty stream (`{false=[], true=[]}`); `groupingBy` only creates keys that occur.

- The `maxBy` downstream yields `Optional<Employee>` per group; `collectingAndThen` unwraps it.

How a collector works (for writing your own)

A `Collector<T, A, R>` bundles four functions — the same shape as the `Collector.of` call above: a **supplier** creates an empty container `A`, an **accumulator** adds one `T`, a **combiner** merges two containers (used by parallel streams), and a **finisher** turns `A` into the result `R`. Optional **characteristics** (`IDENTITY_FINISH`, `UNORDERED`, `CONCURRENT`) let the library optimise. Write one when the same aggregation appears in many places.

Common mistakes

- `toMap` without a merge function on keys that can repeat → `IllegalStateException` in production data.
- `toMap` when a value can be `null` → NPE (use `groupingBy` / `collect(HashMap::new, (m, e) -> m.put(...), Map::putAll)` if nulls are legitimate).
- Expecting `groupingBy` to preserve insertion or sorted order without a map factory.
- Nesting four collectors deep — at that point, a named helper method or a small loop is clearer.

In the interview

> **30-second answer:** "collect performs a mutable reduction via a Collector: supplier, accumulator, combiner, finisher. The workhorses are toList, toMap with a merge function, joining, and groupingBy with a downstream collector — counting, summing, mapping, maxBy — which is basically SQL GROUP BY. partitioningBy is a boolean groupingBy that always has both keys, and teeing runs two collectors in one pass."

- *Follow-up: How do you count word frequencies with streams?* → `words.stream().collect(groupingBy(Function.identity(), counting()))`.
- *Follow-up: What happens on duplicate keys in `toMap`?* → `IllegalStateException` unless you pass a merge function.

TL;DR

`collect` + `Collectors`: `toList` / `toSet` / `toCollection`, `toMap` (+ merge fn!), `joining`, `groupingBy` + downstream (`counting`, `summingX`, `averagingX`, `mapping`, `filtering`, `flatMapping`, `maxBy`), `partitioningBy`, `summarizingX`, `teeing`, `collectingAndThen`. Custom: `Collector.of(supplier, accumulator, combiner, finisher)`.

12.6 Primitive streams

What it is

`IntStream`, `LongStream` and `DoubleStream` hold primitive values directly, avoiding the **boxing** a `Stream<Integer>` performs on every element (§7.6). They add numeric operations — `sum()`, `average()`, `summaryStatistics()`, `range` / `rangeClosed` (int and long only) — and return `OptionalInt` / `OptionalLong` / `OptionalDouble` for possibly-empty results.

From → To	Method
`Stream<T>` → `IntStream`	`mapToInt(ToIntFunction)` (also `mapToLong`, `mapToDouble`)
`IntStream` → `Stream<Integer>`	`boxed()`
`IntStream` → `Stream<R>`	`mapToObj(IntFunction)`
`IntStream` → `LongStream` / `DoubleStream`	`asLongStream()`, `asDoubleStream()`, `mapToLong`, `mapToDouble`

Operation on empty stream	Result
`sum()`	`0`
`average()`	`OptionalDouble.empty` (average is always `OptionalDouble`, even for `IntStream`)
`min()` / `max()`	`OptionalInt.empty` etc.
`count()`	`0`

Code

```
// code/ch12/primitive/PrimitiveStreams.java (key parts)
int[] scores = {72, 95, 88, 61, 79};
IntStream.of(scores).sum(); IntStream.of(scores).max().getAsInt(); Arrays.stream(scores).average();
IntStream.of(scores).summaryStatistics();
words.stream().mapToInt(String::length).sum();
IntStream.rangeClosed(1, 5).boxed().toList();
IntStream.range(0, names.size()).mapToObj(i -> i + ":" + names.get(i));      // index-based loop
text.toLowerCase().chars().filter(c -> "aeiou".indexOf(c) >= 0).count();
LongStream.rangeClosed(1, 20).reduce(1, (a, b) -> a * b);                    // 20!
new Random(7).ints(5, 1, 7).boxed().toList();                                // 5 dice rolls, seeded

long a = IntStream.range(0, n).asLongStream().sum();                         // primitive
long b = Stream.iterate(0L, i -> i + 1).limit(n).reduce(0L, Long::sum);      // boxed
```

Output:

```
sum=395 max=95 average=79.0
empty average -> OptionalDouble.empty, empty sum -> 0
stats IntSummaryStatistics{count=5, sum=395, min=61, average=79.000000, max=95}
totalChars=12 boxed=[1, 2, 3, 4, 5] labels=[row0, row1, row2] halves=[0.5, 1.0, 1.5]
indexed: 0:x 1:y 2:z
vowels in 'Hello World' = 3
20! = 2432902008176640000 (21! overflows long)
dice: [5, 3, 4, 5, 5]
sum 0..10M: LongStream 6 ms vs Stream<Long> 76 ms (same result: true)
```

The last line is a single rough timing (Apple M4 Pro, JDK 26 — **machine-dependent**): the boxed version allocates up to ten million `Long` objects and is roughly an order of magnitude slower here. For numeric work, stay primitive until you need objects — `mapToInt(...).sum()` beats `map(...).reduce(0, Integer::sum)` on both speed and readability.

`sum()` wraps silently on **overflow** like `+` does (§2.10); use `LongStream` or `mapToObj(BigInteger::valueOf)` for large totals.

> **TL;DR**
>
> Use `IntStream` / `LongStream` / `DoubleStream` for numbers: `sum`, `average` (→ `OptionalDouble`), `summaryStatistics`, `range`. Convert with `mapToInt` / `boxed` / `mapToObj`. Avoid boxing in **hot paths** (code that runs extremely often).

12.7 Streams, `Optional` and nulls

Streams and `Optional` (§11.7) are designed to work together:

- `findFirst`, `findAny`, `min`, `max` and single-argument `reduce` return `Optional` because the stream might be empty.
- `optional.stream()` (Java 9) turns an `Optional` into a 0- or 1-element stream, so `flatMap(Optional::stream)` drops the empties from a `Stream<Optional<T>>` — used in the sales report below to skip unparseable lines.
- `Stream.ofNullable(x)` (Java 9) turns a possibly-`null` value into a 0- or 1-element stream.

- Streams tolerate `null` elements, but many operations don't (`toMap` values, `Collectors.toUnmodifiableList`, `max` on a `null`, method references like `String::length`). Filter them out early: `.filter(Objects::nonNull)`.

12.8 Parallel streams

What it is

`.parallel()` on a stream (or `collection.parallelStream()`) lets the library split the source into chunks, process them on several threads, and merge the partial results. You change one word; correctness and performance are your responsibility.

How it works

1. The source's `Spliterator` splits its range recursively (like halving an array) into chunks.
2. Chunks run as tasks on the **common** `ForkJoinPool` (a JVM-wide **thread pool** — a set of reusable worker threads — designed for split-and-join work; default parallelism = cores minus one, and the calling thread also helps; §15.2).
3. Partial results are merged with the reduction's **combiner** (or the collector's combiner).
4. For ordered streams, `findFirst`, `limit`, `forEachOrdered` and `sorted` must respect **encounter order**, which costs coordination; `unordered()`, `findAny` (cheaper — any element will do) and `forEach` relax it.

When it helps — measured

`ParallelDemo` runs each case twice and times the second run. **Timings are machine-dependent** (this run: 14-core Apple M4 Pro, JDK 26); some lines, such as the `forEach` order and the wrong `reduce` result, vary between runs by design.

```
// code/ch12/parallel/ParallelDemo.java (key parts)
LongStream.rangeClosed(1, 3_000_000).filter(ParallelDemo::isPrime).count();              // heavy per element
LongStream.rangeClosed(1, 3_000_000).parallel().filter(ParallelDemo::isPrime).count();
IntStream.range(0, 1_000).parallel().map(i -> i * 2).sum();                              // tiny work
Stream.iterate(1L, i -> i + 1).limit(2_000_000).parallel().reduce(0L, Long::sum);       // unsplittable source

IntStream.range(0, 10_000).parallel().boxed().toList();                                  // safe: collector merges
IntStream.range(0, 10_000).parallel().boxed()
        .collect(Collectors.groupingByConcurrent(i -> i % 3, ConcurrentHashMap::new, Collectors.counting()));
IntStream.rangeClosed(1, 100).parallel().reduce(5, Integer::sum);                        // WRONG identity
```

Output (one run):

```
cores available: 14 (common pool parallelism 13)
count primes <= 3M      sequential  227.2 ms | parallel   33.5 ms
sum 1k doubled          sequential   0.07 ms | parallel   0.29 ms
iterate+limit 2M sum    sequential   13.7 ms | parallel   29.2 ms
safe toList size=10000, in encounter order? true, groupingByConcurrent={0=3334, 1=3333, 2=3333}
reduce identity 0 -> 5050, identity 5 -> 5550 (5 added once per chunk; the exact value depends on how the work was split)
forEach (parallel, order not guaranteed):   4 3 5 7 2 1 6 0
forEachOrdered (parallel, encounter order): 0 1 2 3 4 5 6 7
```

- **Prime counting** (CPU-bound, independent elements, perfectly splittable range): ~7× faster on 14 cores.
- **1,000 tiny operations**: splitting and thread hand-offs cost more than the work — parallel is slower. Brian Goetz's "NQ model": parallelism pays when *N* (elements) × *Q* (work per element) is large; for trivial per-element work you need well over ten thousand elements.
- `Stream.iterate` + `limit`: the source can only produce elements one after another and `limit` must respect order, so parallel is *slower*. Sources split well when they know their size and can be cut

anywhere: arrays, `ArrayList`, `IntStream.range`. They split badly: `LinkedList`, `Stream.iterate`, `BufferedReader.lines()`, most `Iterator`-based sources.

- **Wrong identity**: `reduce(5, Integer::sum)` added 5 once per chunk (in this run the extra 500 means the 100 numbers were split into 100 chunks). Sequentially it would be 5055. Both are wrong — 5 was never an identity.

Correctness rules

1. **No shared mutable state** in lambdas. Adding to an `ArrayList` from `forEach` in parallel loses elements or throws `ArrayIndexOutOfBoundsException` (it isn't thread-safe); let `toList()` / `collect` do the merging.
2. **Reduction functions must be associative, stateless, with a true identity** (§12.4).
3. **Don't block** (on I/O, locks or `Thread.sleep`) inside parallel streams — you tie up the JVM-wide common pool, starving every other parallel stream and `CompletableFuture` using it. For concurrent I/O use **virtual threads** (Chapter 15) or the `mapConcurrent` gatherer (§12.9).
4. **Order costs**: if you don't need order, `unordered()`, `findAny()` and `forEach` are cheaper; if you do, `forEachOrdered`.

When not to use it

Most of the time. *Effective Java* Item 48: "use caution when making streams parallel" — measure first with a proper **benchmark** (JMH, Chapter 16). Avoid it in web servers (requests already run in parallel), for small inputs, poorly splittable sources, **I/O-bound** work (mostly waiting on disk or network), or pipelines with `limit` / `sorted` / `findFirst` on ordered data.

In the interview

> **30-second answer:** "Parallel streams split the source with a Spliterator and run chunks on the common ForkJoinPool, combining partial results. They help only for large, CPU-bound, independent work on splittable sources like arrays or ranges. They hurt for small inputs, LinkedList or iterate sources, ordered operations like limit, and anything blocking — which also starves the shared pool. Lambdas must be stateless and reductions associative with a real identity. I measure before switching."

- *Follow-up: Which thread pool do parallel streams use?* → The common `ForkJoinPool` (plus the caller thread).
- *Follow-up: Why might `parallel().forEach(list::add)` fail?* → `ArrayList` isn't thread-safe; concurrent adds lose data or throw. Use `collect` / `toList`.

TL;DR

Parallel = split + common ForkJoinPool + combine. Worth it only for big, CPU-heavy, splittable, stateless work — measure. Never share mutable state, never block, and respect reduction rules.

12.9 Stream gatherers (Java 24)

The problem it solves

The built-in intermediate operations are a fixed set. Operations needing **state across elements** — sliding windows, running totals, distinct by a key, groups of consecutive equal items — meant breaking out into a loop or writing a custom `Spliterator`. Collectors keep state, but only at the *end* of a pipeline.

What it is

JEP 485 (previewed in Java 22 and 23, **final in Java 24**, so available in Java 25 **LTS**; JEPs and LTS: §1.12) adds `Stream.gather(Gatherer)`: **custom intermediate operations**. A `Gatherer<T, A, R>` consumes `T` elements, keeps private state `A`, and pushes zero or more `R` elements **downstream** (to the next stage).

It can be one-to-one, one-to-many, many-to-one or many-to-many, and can stop the stream early — "a collector, but in the middle of the pipeline".

A gatherer has up to four functions (the same idea as a collector's):

Part	Role
initializer	creates the private state (e.g. a `HashSet` of seen keys)
integrator	called per element with (state, element, downstream); may `push` results; returns `false` to **short-circuit** the stream
combiner	merges states when running in parallel (optional)
finisher	called after the last element; may push final results (e.g. a last partial window)

Factory methods: `Gatherer.ofSequential(...)` (no combiner, sequential only) and `Gatherer.of(...)` (parallel-capable). `Gatherer.Integrator.ofGreedy(...)` marks an integrator that never short-circuits, allowing optimisations. Gatherers compose with `a.andThen(b)`.

Built-in gatherers (`java.util.stream.Gatherers`)

Gatherer	Does	Example result
`windowFixed(n)`	non-overlapping lists of n (last may be shorter)	batches of 3
`windowSliding(n)`	overlapping windows, each shifted by one	moving averages
`scan(initial, f)`	running accumulation, emits every intermediate value	running totals
`fold(initial, f)`	like reduce, but as an intermediate op emitting one final value	single aggregate
`mapConcurrent(maxConcurrency, f)`	applies `f` using **virtual threads**, at most n at once, preserving encounter order	parallel I/O calls

Code

```
// code/ch12/gatherers/GatherersDemo.java (key parts)
temps.stream().gather(Gatherers.windowFixed(3)).toList();
temps.stream().gather(Gatherers.windowSliding(3))
     .map(w -> w.stream().mapToInt(Integer::intValue).average().orElseThrow()).toList();   // moving average
Stream.of(5, 10, 20).gather(Gatherers.scan(() -> 0, Integer::sum)).toList();
Stream.of("a", "b", "c").gather(Gatherers.fold(() -> "", (acc, s) -> acc + s)).toList();
Stream.of("u1", "u2", "u3", "u4")
      .gather(Gatherers.mapConcurrent(2, url -> { sleep(20); return url.toUpperCase(); })).toList();

static <T, K> Gatherer<T, ?, T> distinctBy(Function<? super T, ? extends K> key) {
    return Gatherer.ofSequential(
            HashSet<K>::new,                                          // initializer
            Gatherer.Integrator.ofGreedy((seen, element, downstream) -> {  // integrator
                if (seen.add(key.apply(element))) return downstream.push(element);
                return true;
            }));
}

static Gatherer<Integer, ?, Integer> takeWithinBudget(int budget) {
    return Gatherer.ofSequential(() -> new int[1], (spent, cost, downstream) -> {
        if (spent[0] + cost > budget) return false;                   // stop the whole stream
        spent[0] += cost;
        return downstream.push(cost);
    });
}

static <T> Gatherer<T, ?, List<T>> runs() {                           // with a finisher
    class State { List<T> current = new ArrayList<>(); }
    return Gatherer.ofSequential(State::new,
            Gatherer.Integrator.ofGreedy((st, e, down) -> {
                if (!st.current.isEmpty() && !st.current.getLast().equals(e)) { down.push(st.current); st.current
= new ArrayList<>(); }
                st.current.add(e);
                return true;
            }),
            (st, down) -> { if (!st.current.isEmpty()) down.push(st.current); });   // flush the last run
}
```

Output:

```
windowFixed(3)    [[21, 23, 22], [25, 27, 26], [24]]
windowSliding(3)  [[21, 23, 22], [23, 22, 25], [22, 25, 27], [25, 27, 26], [27, 26, 24]]
3-day moving avg  [22.0, 23.3, 24.7, 26.0, 25.7]
scan (running sum)[5, 15, 35]
fold (one result) [abc]
mapConcurrent(2)  [U1, U2, U3, U4]
distinctBy user   [/home, /cart, /home]
within budget 50  [10, 15, 20, 5]
runs              [[a, a], [b], [c, c, c], [a]]
andThen           [[1, 2], [3, 4], [5]]
```

Notes:

- `downstream.push(x)` returns `false` when downstream no longer wants elements (e.g. a later `limit` is satisfied); returning that value propagates the short-circuit.
- `takeWithinBudget` stopped at 30 even though a later small element might fit — that's the difference from `filter`, the same as `takeWhile`.
- `runs()` needs the finisher: without it the final `[a]` group would be lost.
- `mapConcurrent` is the idiomatic way to call a slow service per element with bounded **concurrency** (a cap on calls in flight) — a job parallel streams do badly (§12.8).

When not to use it

If a built-in operation or a collector already does the job (`distinct` , `groupingBy`), use that. Gatherers are for stateful *mid-pipeline* logic. Keep state private to the gatherer — the initializer

creates a fresh state object for each evaluation of the stream.

> **TL;DR**
>
> `stream.gather(g)` (Java 24) = custom intermediate operations with private state. Built-ins: `windowFixed`, `windowSliding`, `scan`, `fold`, `mapConcurrent`. Write your own with `Gatherer.ofSequential(initializer, integrator[, finisher])`; return `false` to short-circuit.

12.10 Common mistakes

```
// code/ch12/mistakes/StreamMistakes.java (key parts)
Stream.of(arr).count();                                  // int[] -> Stream<int[]> with ONE element
Stream.of("a", "b").map(String::toUpperCase).forEach(out::add);   // side effect: use toList()
src.stream().forEach(x -> { if (x == 2) src.add(99); });  // modifying the source
Stream.of("apple", "avocado").collect(Collectors.toMap(s -> s.charAt(0), s -> s));   // duplicate key
Stream.of("x").collect(Collectors.toMap(s -> s, s -> (String) null));               // null value
Stream.of(1).toList().add(2);                            // unmodifiable
List.of("a", "b", "c").stream().peek(log::add).count();   // peek skipped!
```

Output:

```
1. Stream.of(int[]).count() = 1  vs Arrays.stream(arr).count() = 3
2. forEach(out::add) [A, B] -> prefer toList() [A, B]
3. modifying source -> ConcurrentModificationException
4. toMap duplicate -> IllegalStateException
5. toMap null value -> NullPointerException
6. Stream.toList().add -> UnsupportedOperationException
7. min() = 2 (same answer as sorted().findFirst(), less work)
8. [3, 2, 1] (limit first)
10. count()=3 but peek ran 0 times -- never put logic in peek
12. loop=2 stream=2 (both fine -- choose the clearer one)
```

#	Mistake	Fix
1	`Stream.of(intArray)`	`Arrays.stream(intArray)` or `IntStream.of(...)`
2	Side effects in `forEach` / `map` to build results (*Effective Java* Item 46: use side-effect-free functions)	`toList()` , `collect(...)`
3	Modifying the source during the pipeline (→ `ConcurrentModificationException`)	Collect first, then modify; or `removeIf`
4	`toMap` on non-unique keys	pass a merge function or use `groupingBy`
5	`toMap` with `null` values	filter nulls or collect manually
6	Mutating `toList()` results	`toCollection(ArrayList::new)`
7	`sorted().findFirst()` for a minimum	`min(comparator)` — O(n), no buffering
8	Stateful op (`sorted` , `distinct`) before `limit` on an infinite stream	`limit` first, or a bounded source
9	`Stream.generate(() -> 1).distinct().limit(2)`	hangs: only one distinct value exists
10	Logic in `peek`	Since Java 9, `count()` may compute the size without running the pipeline — `peek` never ran. Use `peek` only for debugging
11	Checked exceptions in lambdas	wrap (§11.8) or use a loop
12	Streams everywhere	a loop with `break` , several accumulators or complex control flow is often clearer (*Effective Java* Item 45: use streams judiciously)
13	Returning `Stream` from APIs that callers will iterate twice	return a `Collection` (EJ Item 47) — a stream can be traversed only once
14	Not closing I/O-backed streams	`try (Stream<String> lines = Files.lines(p)) { … }` — `BaseStream` is `AutoCloseable`
15	Reusing a stream variable	create a new stream each time, or keep a `Supplier<Stream<T>>`

12.11 Streams vs loops, and a real-world pipeline

Choosing

Prefer a stream when…	Prefer a loop when…
The work is a clear sequence of transform / filter / aggregate steps	You need `break` / `continue` / `return` from the middle in complex ways
You'd otherwise write grouping or counting boilerplate	You update several variables per iteration
Readers benefit from named steps (`groupingBy` , `partitioningBy`)	You need checked exceptions to propagate naturally
You may later want `parallel()` for heavy CPU work	You're in a very hot loop over primitives and need every nanosecond (measure)
Data is already in a collection	You need the index and neighbours constantly (or use a gatherer)

Performance: for typical business code a sequential stream costs about the same as a loop — the JIT optimises both. Overhead shows with tiny collections in hot paths or boxing; readability usually

matters more.

A realistic pipeline: a sales report

```
// code/ch12/realworld/SalesReport.java (key parts)
record Sale(LocalDate date, String region, String product, int units, double unitPrice) {
    double revenue() { return units * unitPrice; }
    static Optional<Sale> parse(String line) { ... }        // Optional.empty() for malformed lines
}
List<Sale> sales = csv.lines().map(Sale::parse).flatMap(Optional::stream).toList();

sales.stream().collect(groupingBy(Sale::region, TreeMap::new, summingDouble(Sale::revenue)));
sales.stream().collect(groupingBy(s -> s.date().getMonthValue(), TreeMap::new, summingLong(Sale::units)));
sales.stream().collect(groupingBy(Sale::product, summingDouble(Sale::revenue)))
     .entrySet().stream()
     .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
     .limit(2)
     .map(e -> e.getKey() + "=" + String.format("%.2f", e.getValue()))
     .toList();
sales.stream().collect(partitioningBy(s -> s.revenue() >= 500, counting()));
```

Output:

```
parsed 6 of 8 lines
  APAC    1050.00
  EU      1445.40
  US       925.00
units by month {1=19, 2=30}
top-2 products by revenue [monitor=2225.00, keyboard=707.90]
orders >= 500: 2, smaller: 4
```

The **CSV** input is parsed, bad lines dropped, then totals by region and month, top-2 products and big-order counts — the pipeline reads like the requirements. Two malformed lines (`bad line` and a non-numeric unit count) were skipped by `parse` returning `Optional.empty()` plus `flatMap(Optional::stream)`; in production you'd also log or count rejects.

Real-world uses: report generation and **ETL** (Extract, Transform, Load) transforms in back-end services, request validation (`allMatch`), building lookup maps from query results (`toMap`), batch processing of log lines (`Files.lines` + `filter` + `groupingBy`), and processing Spring Data / JPA result streams inside a transaction.

In the interview

30-second answer: "A stream is a lazy, single-use pipeline: source, intermediate operations, terminal operation. Nothing runs until the terminal op, elements flow one at a time, and short-circuiting ops can stop early, so infinite streams work with limit. I use filter/map/flatMap, then collect with toList, toMap with a merge function, or groupingBy with a downstream collector. I keep lambdas side-effect free, use primitive streams for numbers, and only go parallel for large CPU-bound work after measuring. Since Java 24, gatherers let me write custom stateful intermediate ops like sliding windows."

- *Follow-up: Difference between `map` and `flatMap`?* → 1→1 vs 1→many (flattening nested streams).
- *Follow-up: Intermediate vs terminal?* → Intermediate returns a new lazy `Stream`; terminal produces a result or side effect and consumes the stream.
- *Follow-up: Are streams faster than loops?* → Not inherently; similar for typical code, sometimes slower for tiny inputs. Choose for clarity; parallel can win on big CPU-bound work.
- *Follow-up: What's a stateful intermediate operation?* → One that needs to see other elements — `sorted`, `distinct`, `limit`, `skip` — and may buffer.

Chapter TL;DR

Topic	Remember
Pipeline	source → lazy intermediate ops → one terminal op; single use; never modifies the source
Laziness	nothing runs without a terminal op; vertical, fused processing; short-circuit ops stop early; `sorted` / `distinct` buffer
Intermediate	`filter`, `map`, `flatMap` (1→many), `mapMulti` (16), `distinct`, `sorted`, `limit`, `skip`, `takeWhile` / `dropWhile` (9), `peek` (debug only)
Terminal	`toList` (16, unmodifiable), `collect`, `count`, `min` / `max`, `findFirst` / `findAny`, `any/all/noneMatch` (empty → false/true/true), `reduce`
`reduce`	real identity + associative + stateless; 3-arg form with combiner for type change
Collectors	`toMap` (merge fn!), `joining`, `groupingBy` + downstream, `partitioningBy` (both keys), `teeing` (12), `filtering` / `flatMapping` (9), `Collector.of`
Primitive	`IntStream` / `LongStream` / `DoubleStream`; `sum`, `average` → `OptionalDouble`, `summaryStatistics`, `boxed`, `mapToObj`
Parallel	common ForkJoinPool; big CPU-bound splittable work only; no shared state, no blocking; measure
Gatherers (24)	`gather(...)` : `windowFixed`, `windowSliding`, `scan`, `fold`, `mapConcurrent`; custom via `Gatherer.ofSequential`
Style	side-effect-free lambdas; filter early; loops when clearer; return collections, not streams

Where next: stream file contents with `Files.lines` and `Files.walk` in Chapter 13 — I/O and NIO; `CompletableFuture` and virtual threads for concurrent work in Chapter 15; gatherers and other recent features in context in Chapter 17 — Modern Java. Exam traps: OCP Chapter 10. Python comparison (generators, comprehensions, `itertools`): Python Chapter 11.